

Fortifying Data Availability and Synchronization in Decentralized Storage Systems

Hein Meling

Distributed Systems, March 2026

Thanks for Coming to my Talk!

Racin Nygaard, Vero Estrada-Galiñanes, and Hein Meling. *Snarl: entangled merkle trees for improved file availability and storage utilization.* In ACM Middleware '21.

Racin Nygaard and Hein Meling. *SNIPS: Succinct Proof of Storage for Efficient Data Synchronization in Decentralized Storage Systems.* In arXiv 2304.04891. 2023.

Racin Nygaard, Hein Meling, and John Ingve Olsen. *Cost-effective Data Upkeep in Decentralized Storage Systems.* In ACM SAC '23.



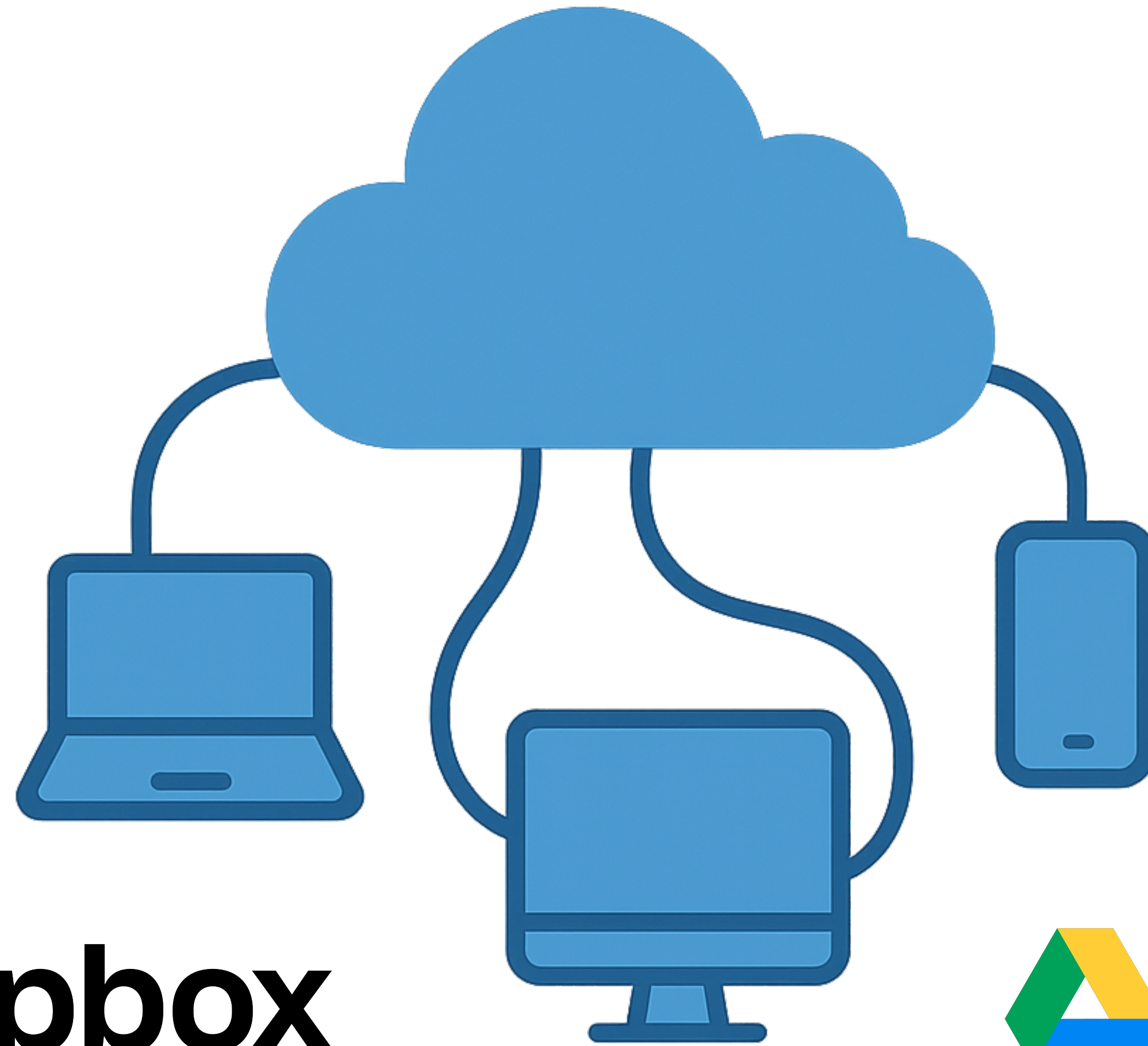
Racin Nygaard



Vero Estrada-Galiñanes

Introduction

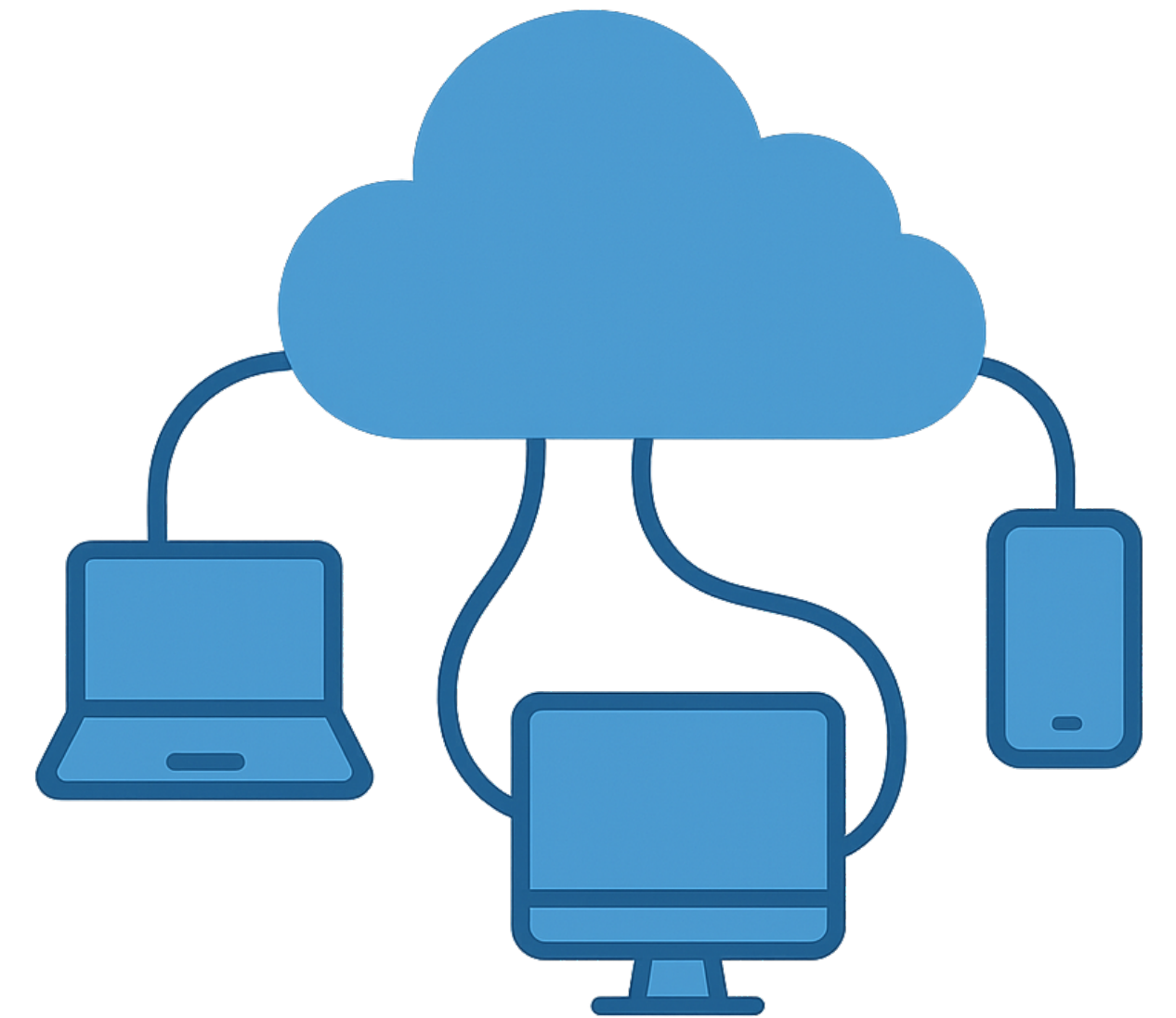
Centralized Storage



Centralized Storage

Benefits

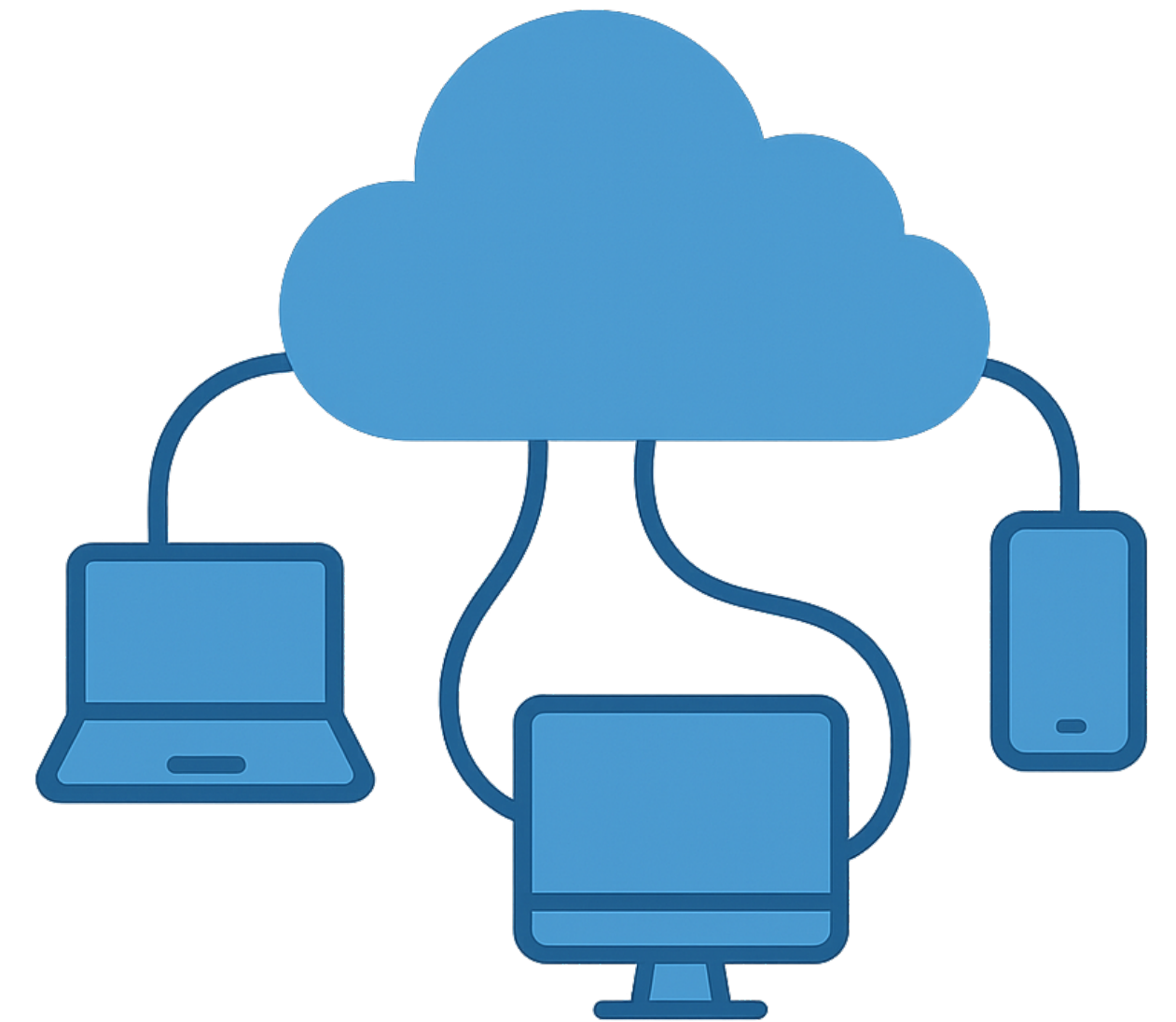
- Low-Cost
- Easy to Use
- Great Features: Sync Devices



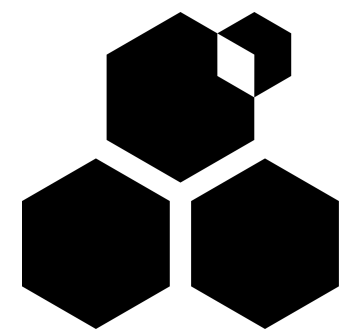
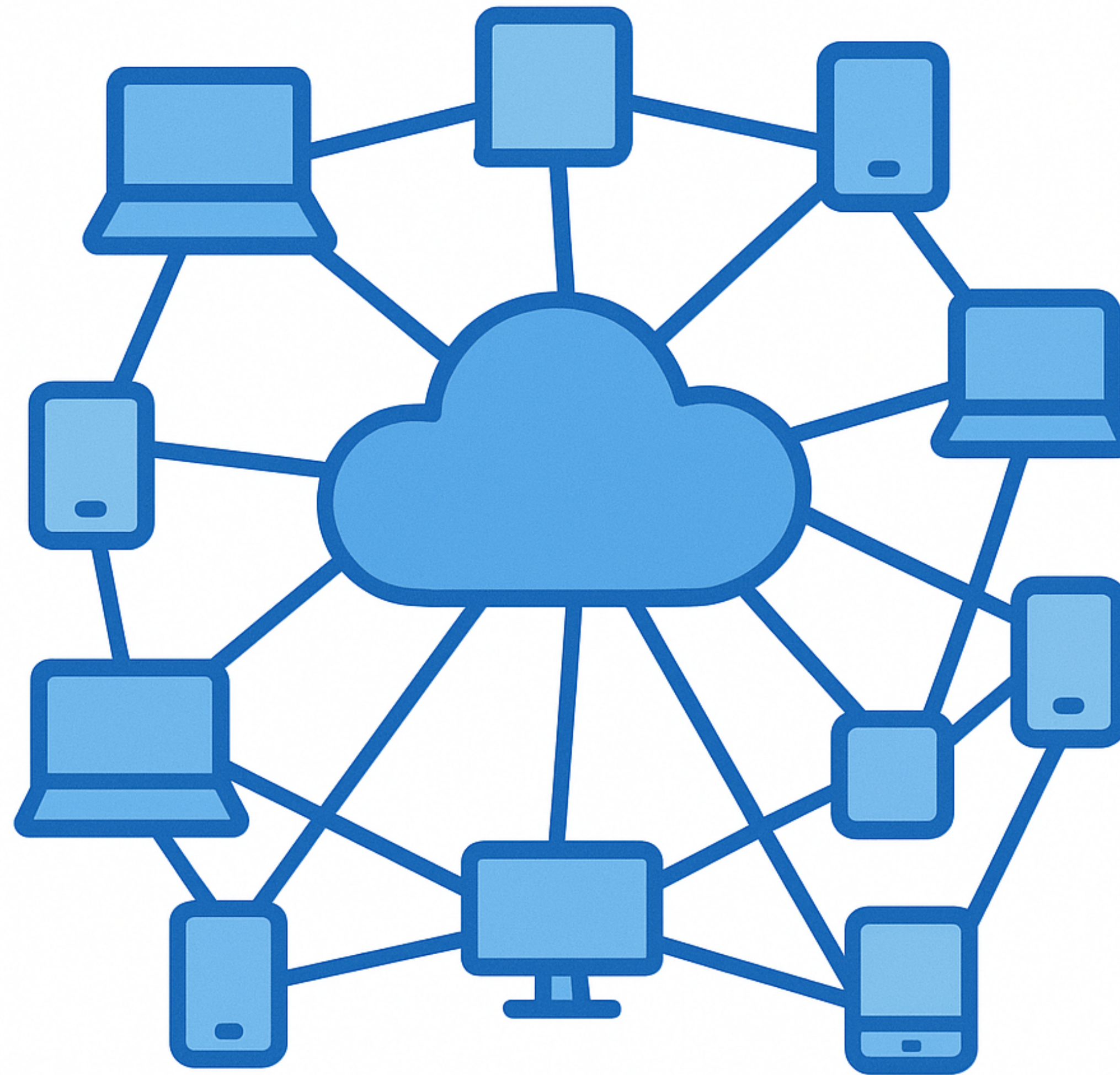
Centralized Storage

Concerns

- Trust the Provider to
 - Not Lose or Corrupt **Your Data**
 - Keep it Private
 - Make it Available to you
- Fair Cost Allocation



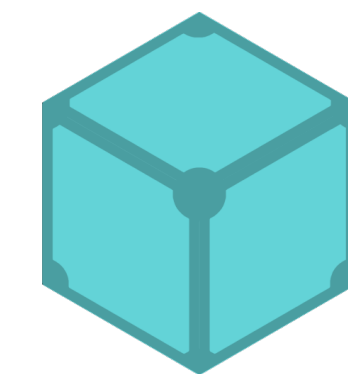
Decentralized Storage



swarm

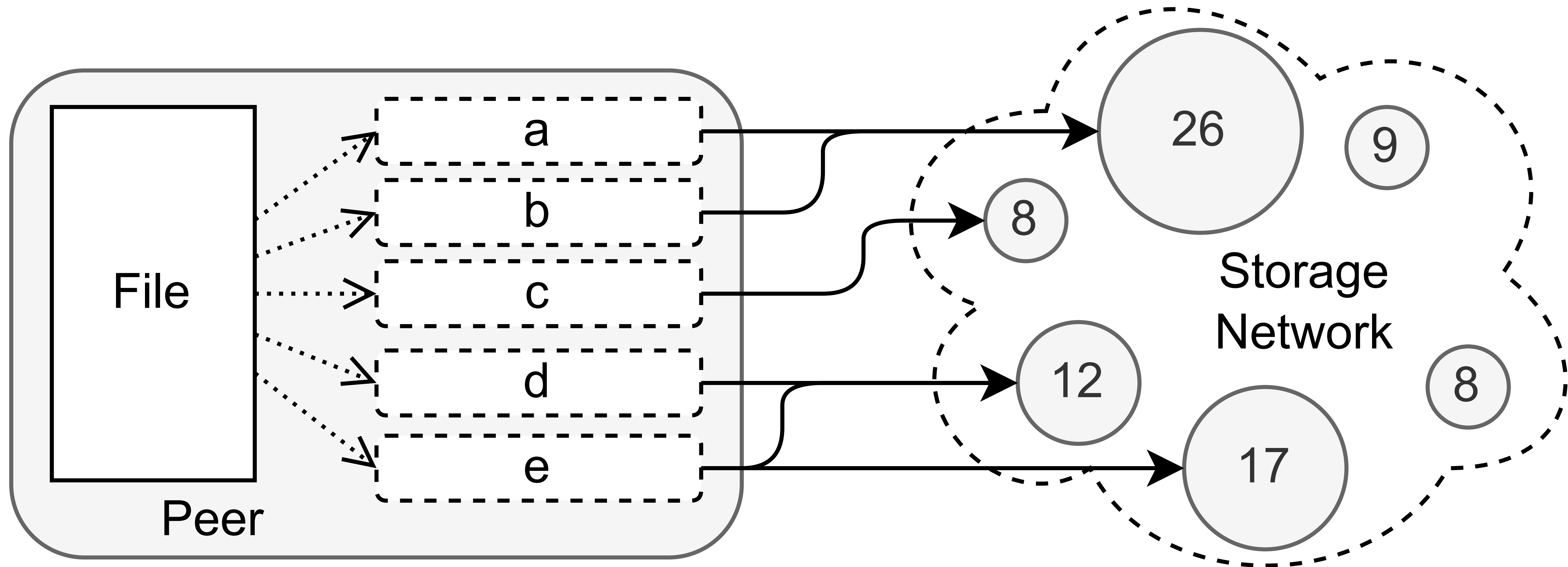


Filecoin



IPFS

Decentralized Storage

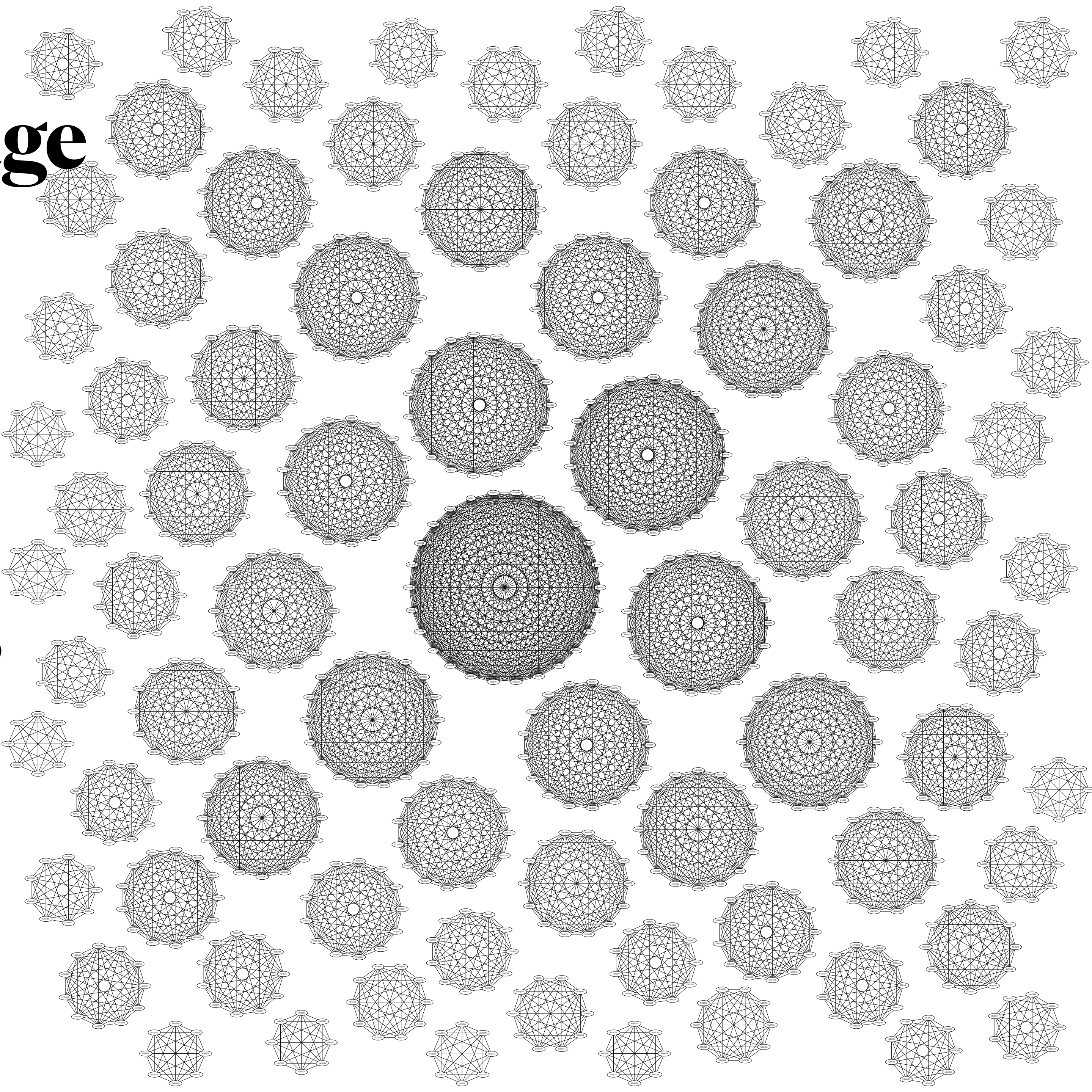
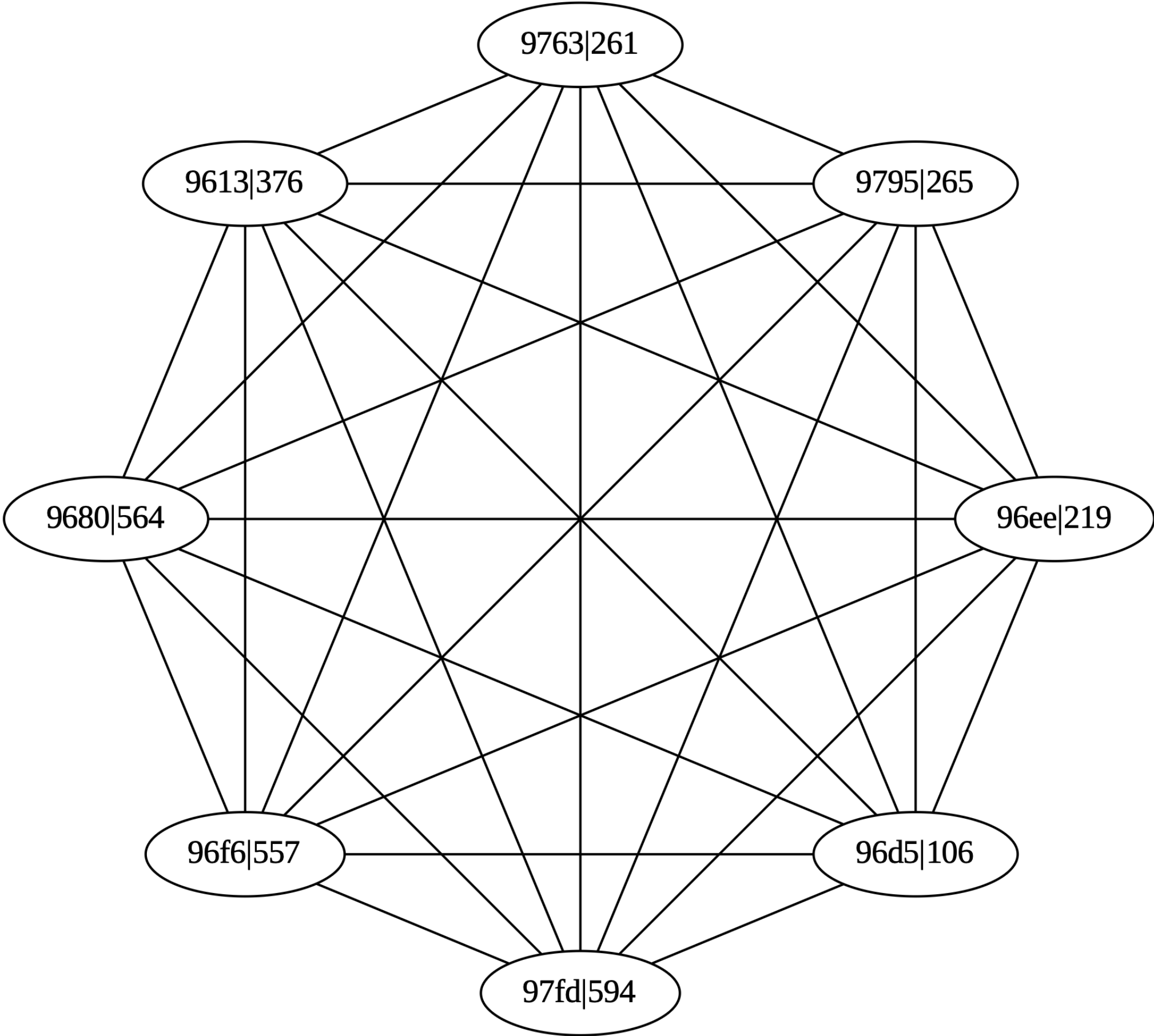


Decentralized Storage

- Content-Address Storage
- Peer-to-Peer Network
- Untrusted Peers
- *Storage Incentives via Blockchains*

Decentralized Storage

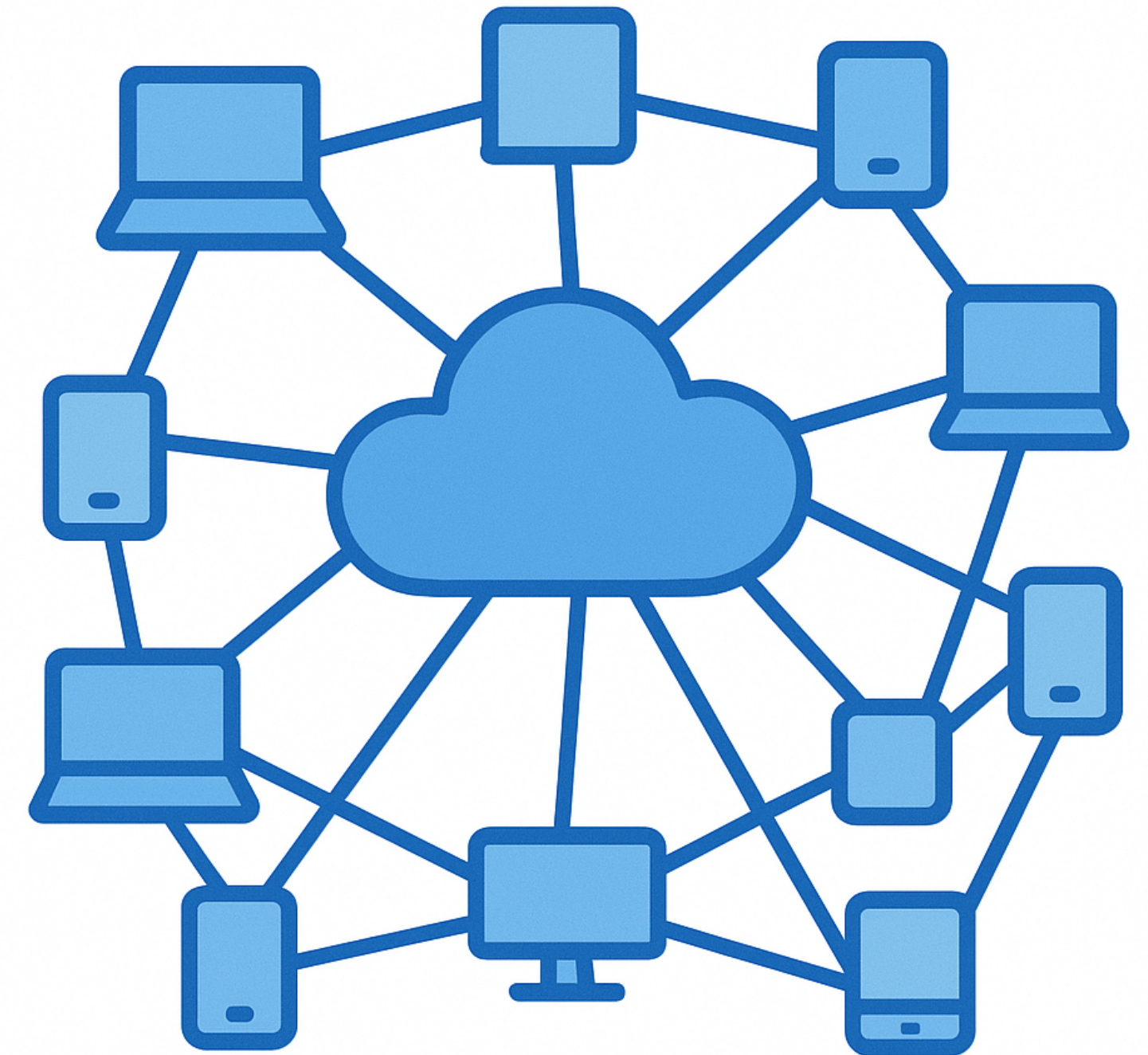
Peer Neighborhoods



Decentralized Storage

Promises

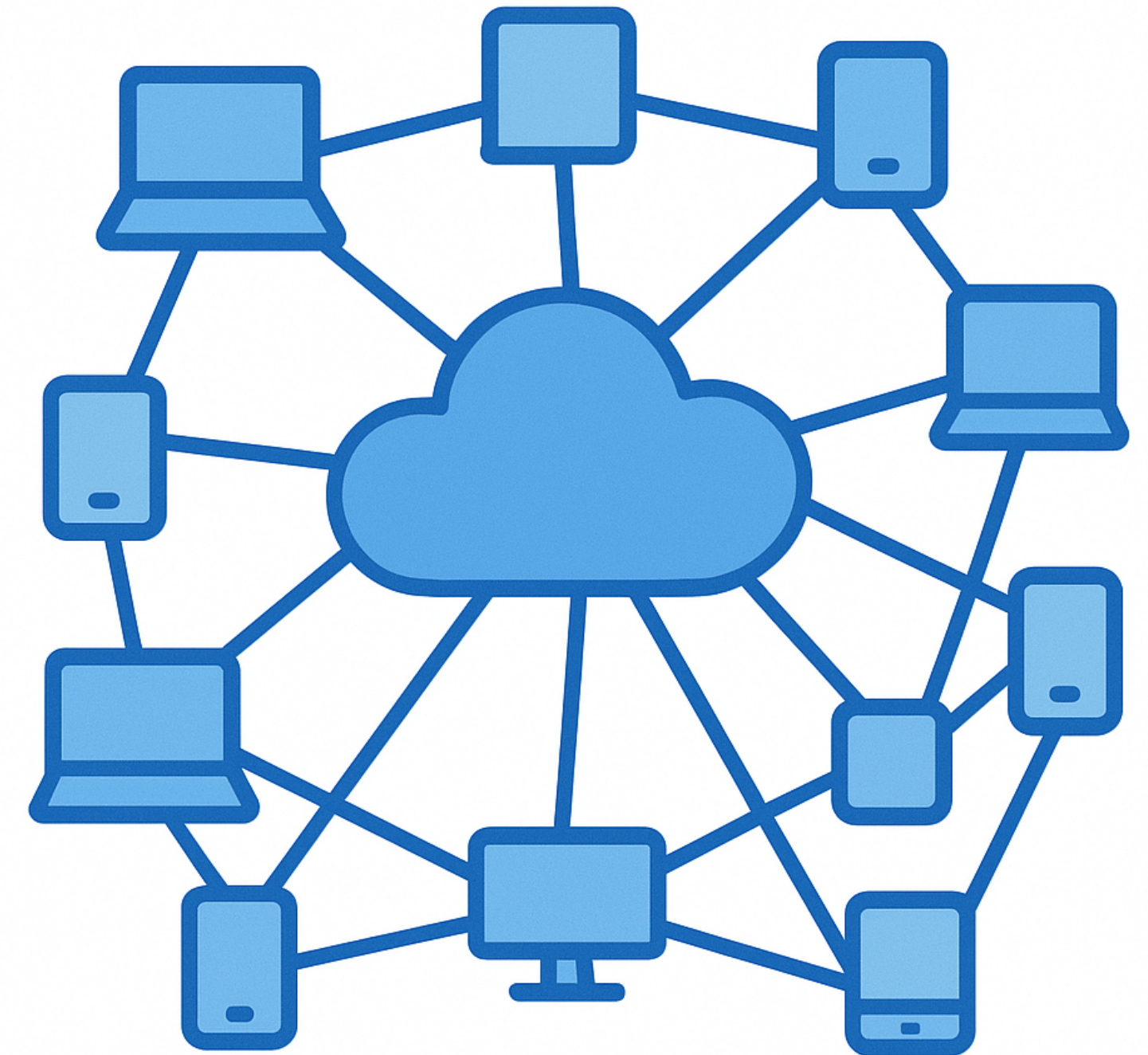
- Censorship Resistance
- High Availability and Data Integrity
- Low-Cost
- Fair Cost Allocation



Decentralized Storage

Concerns

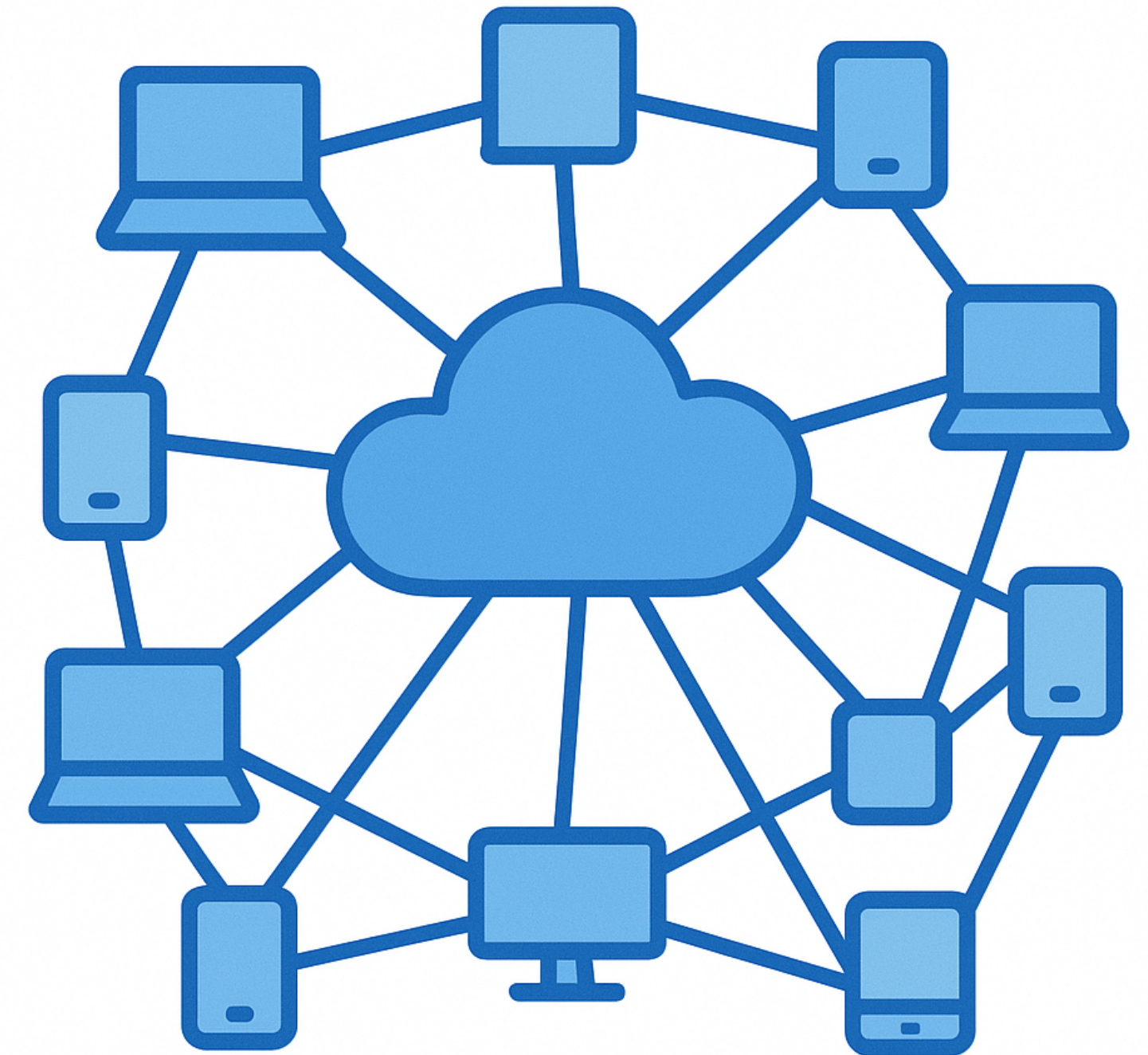
- Incentive-based Pricing
- Performance Issues
- Complex to Use
- Legal Ambiguities



Decentralized Storage

Concerns

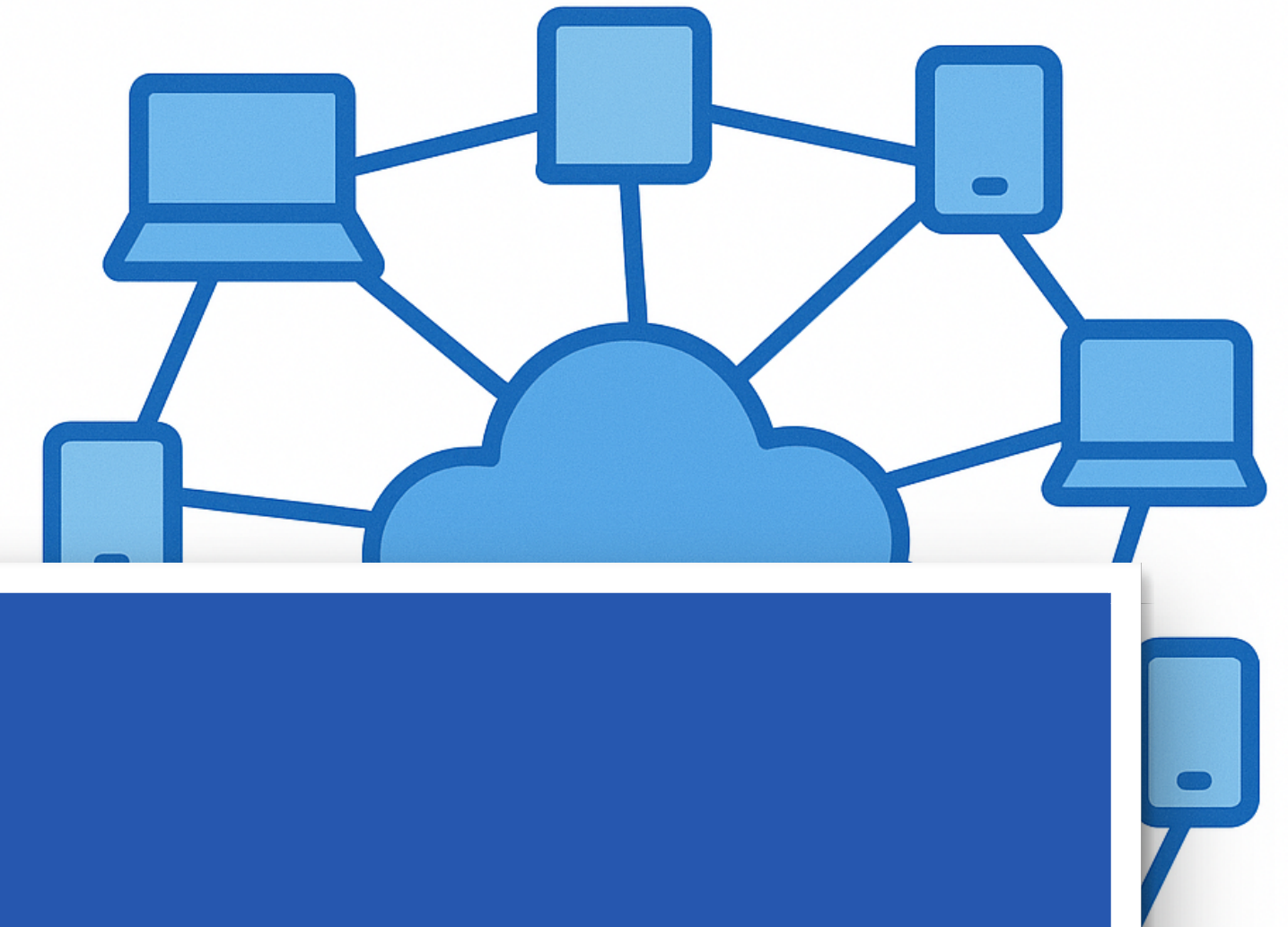
- Incentive-based Pricing
- **Performance Issues**
- Complex to Use
- Legal Ambiguities



Decentralized Storage

Concerns

- Incentive-based Pricing
- **Performance Issues**



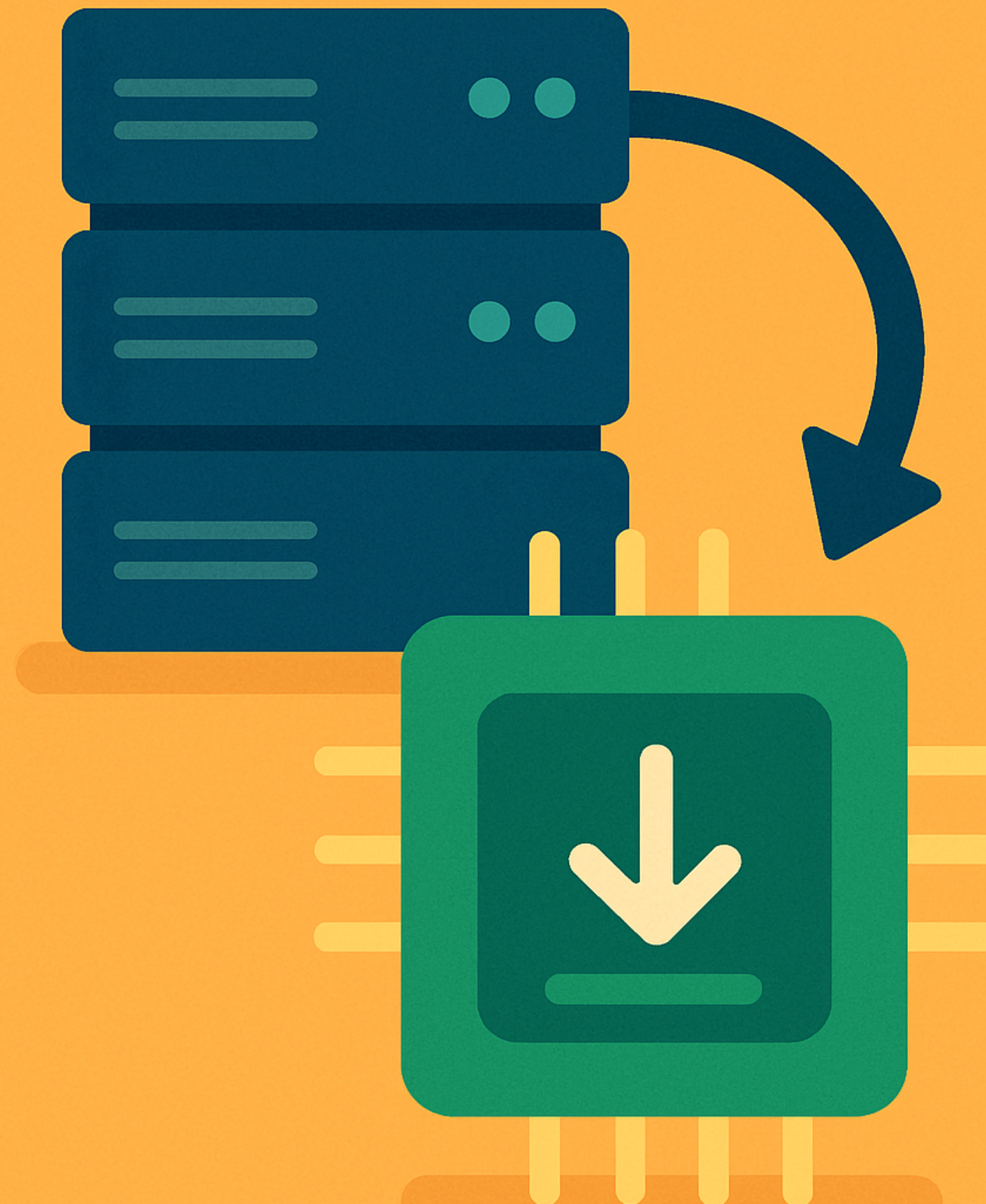
Storage Utilization and Synchronization Efficiency

Data Availability

**Does the data
exist?**



Can it be retrieved?



**Has it been
tampered with?**



Talk Overview

Brief Primer

Snarl: Improving File Availability and Storage Utilization

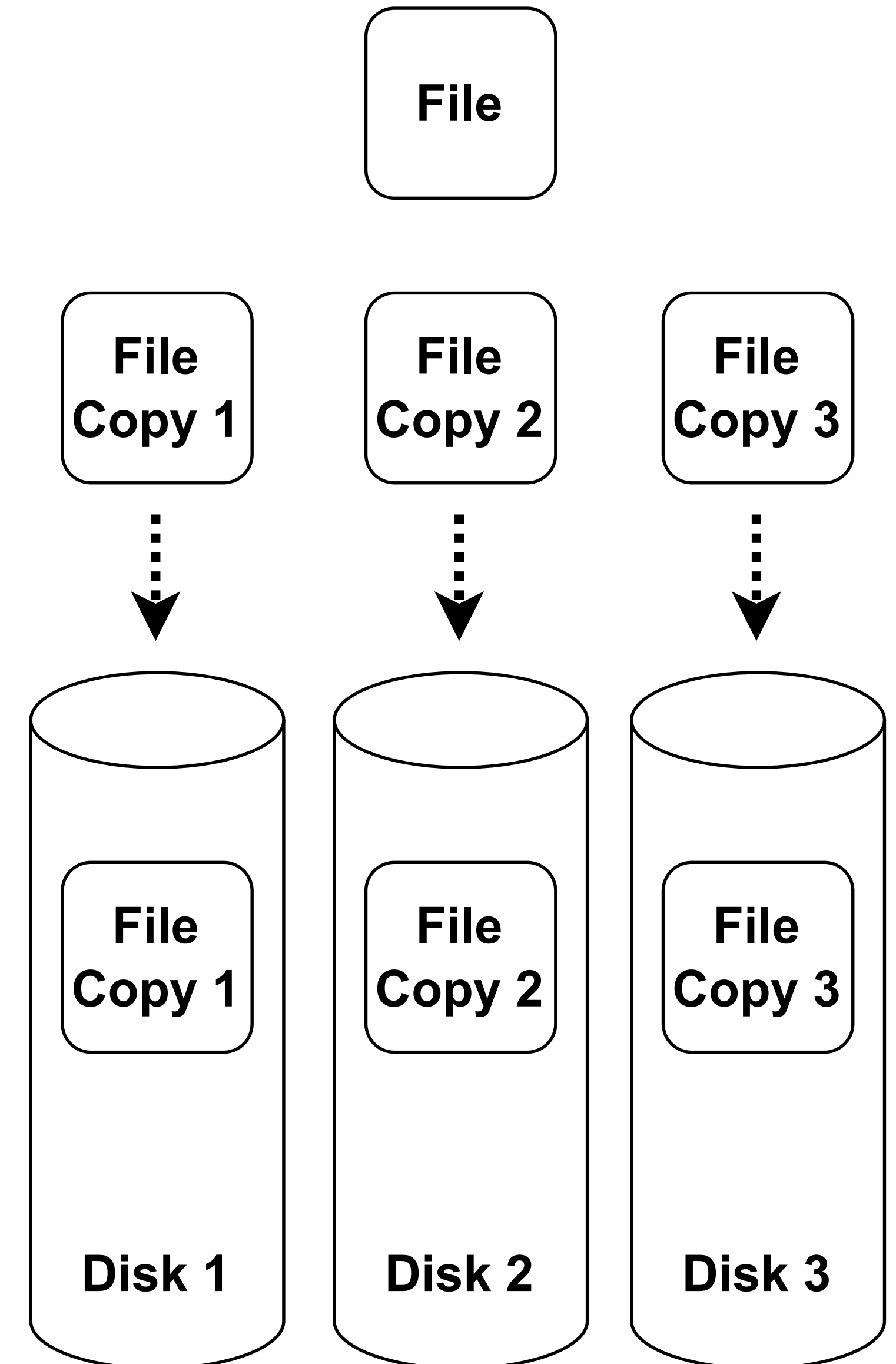
SNIPS: Synchronizing Decentralized Storage Peers

Brief Primer

File Replication

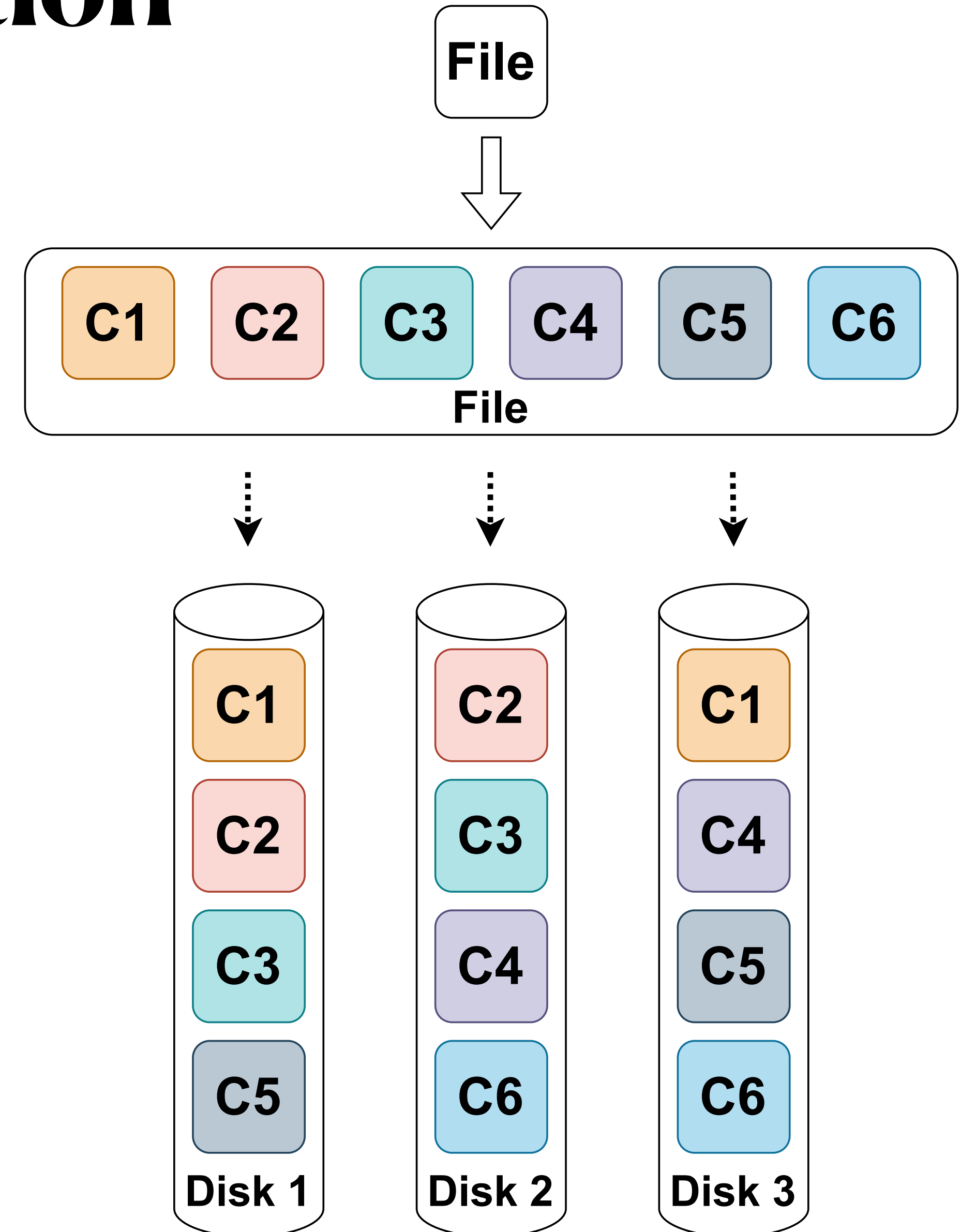
- R copies

- Storage Consumption: $R \times |File|$
- Can recover the *File* from any one disk



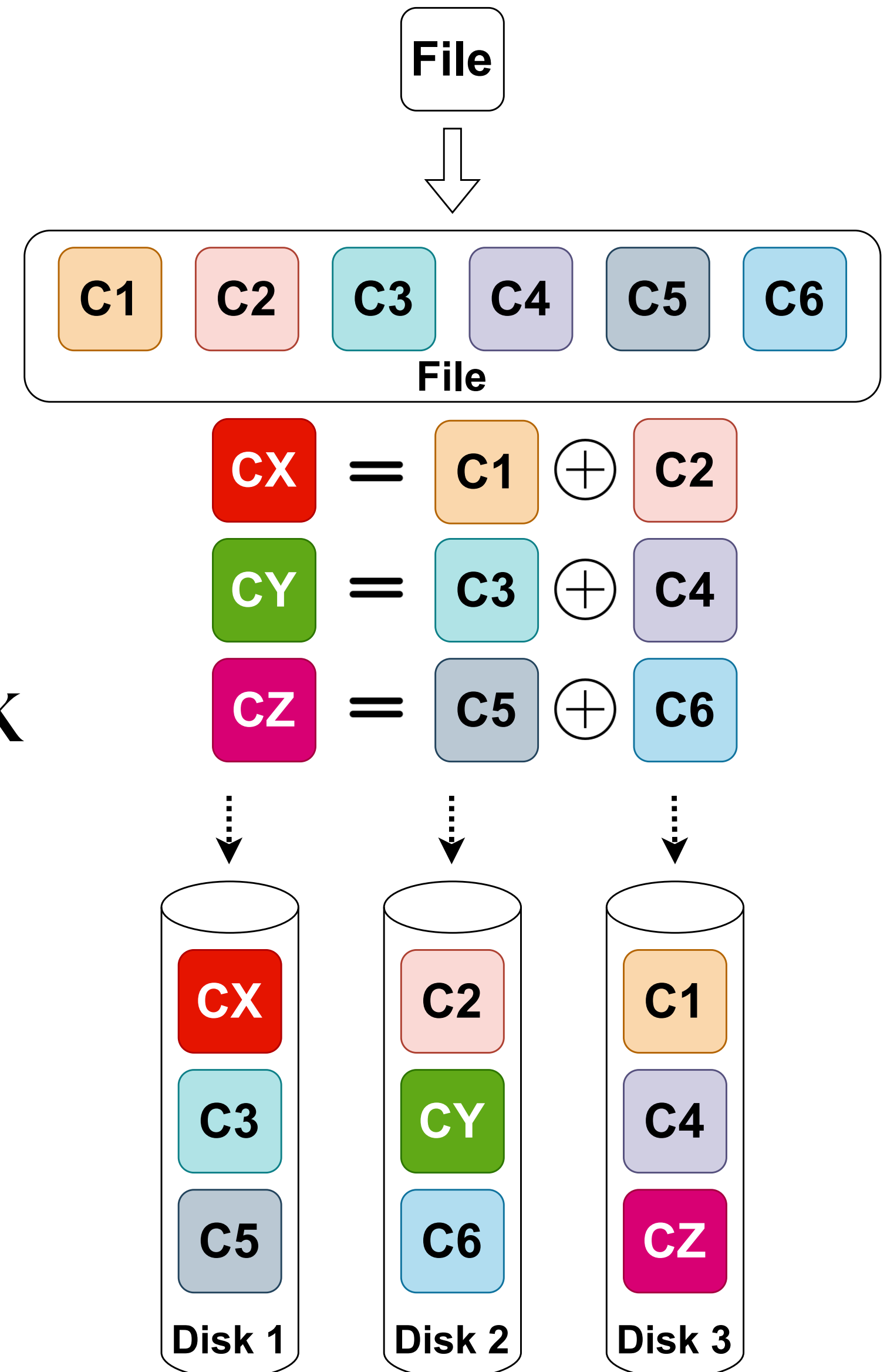
Chunk Replication

- Split File Into Chunks
- $|File| = 6 \times ChunkSize$
- Four Chunks per Disk
- Storage Consumption: $2 \times |File|$
- Can recover the *File* from *any two* disks



Erasure Codes

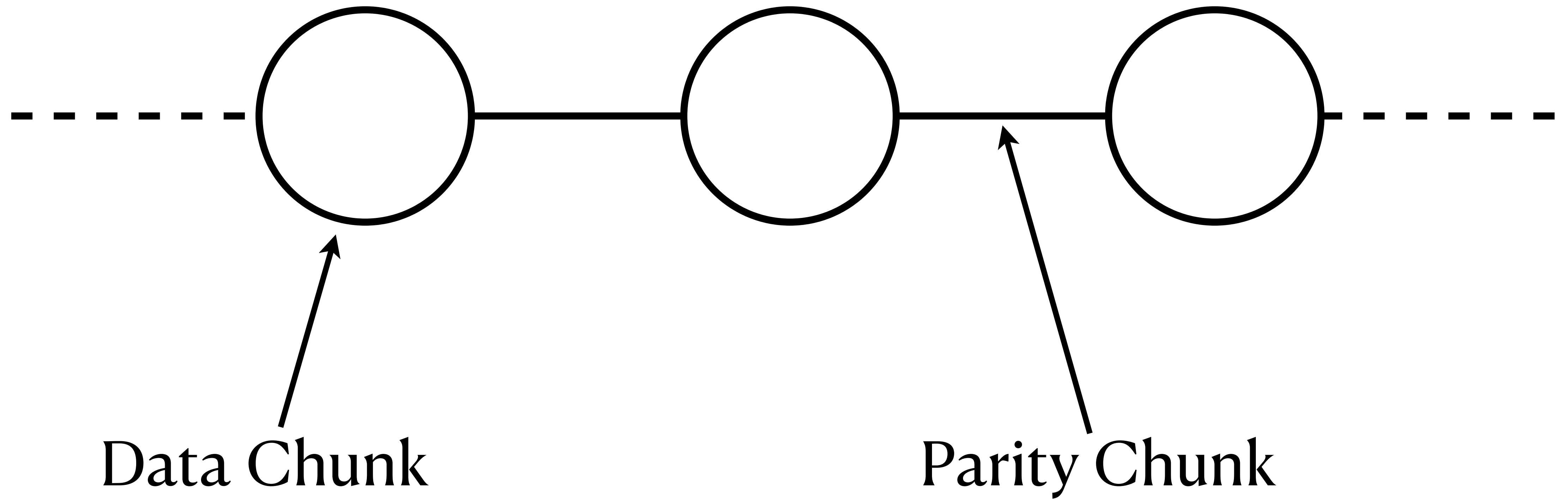
- Split File Into Chunks
- Create 3 Parity Chunks
- Two Data and One Parity Chunks per Disk
- Storage Consumption: $1.5 \times |File|$
- Can recover the *File* from *any two* disks



Entanglement Codes

- Split File Into Chunks
- Chain Data Chunks with Parity Chunks
- Storage Consumption: $(\alpha + 1) \times |File|$
- Can *repair* any chunk using two neighboring chunks

Entanglement Codes



Exclusive OR

$$X = \{0111\}$$

$$Y = \{1100\}$$

$$Z = X \oplus Y = \{1011\}$$

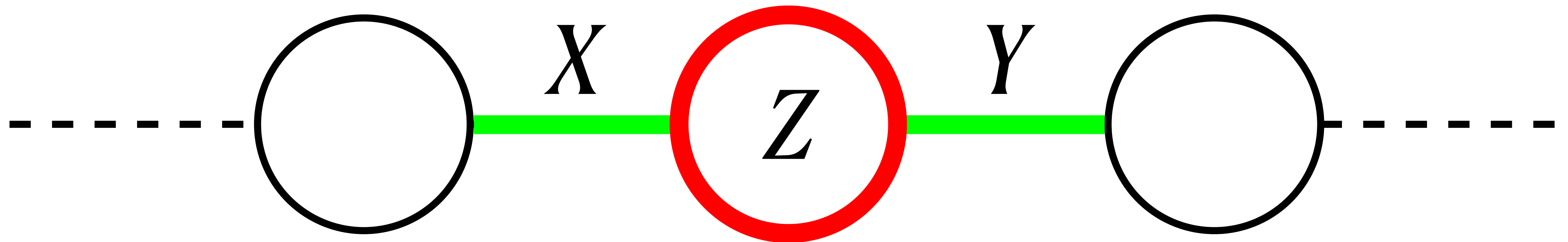
$$X = Z \oplus Y = X \oplus Y \oplus Y = \{0111\}$$

$$Y = Z \oplus X = X \oplus Y \oplus X = \{1100\}$$

A	B	
0	0	0
0	1	1
1	0	1
1	1	0

Entanglement Codes

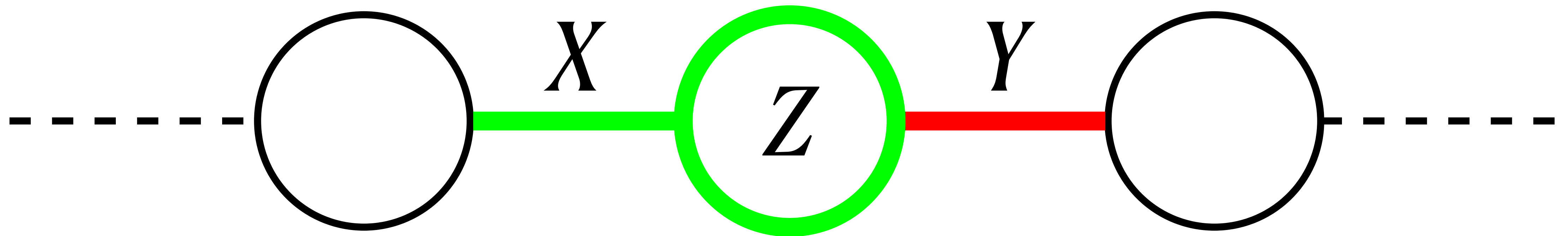
Repair



$$Z = X \oplus Y$$

Entanglement Codes

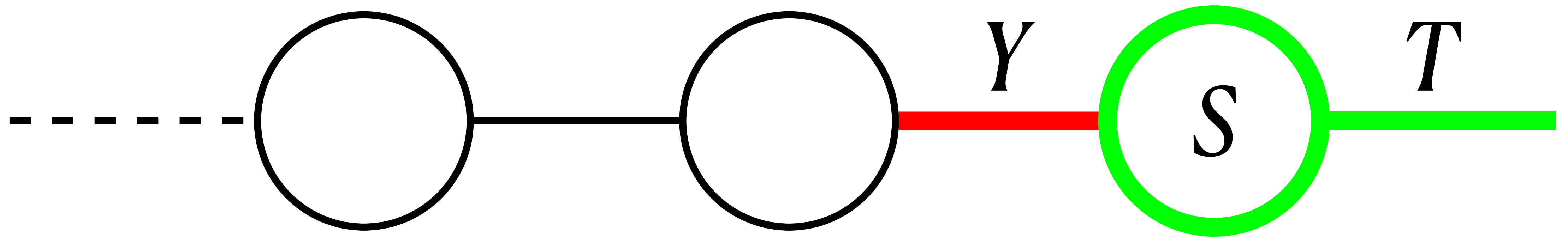
Repair



$$Y = Z \oplus X$$

Entanglement Codes

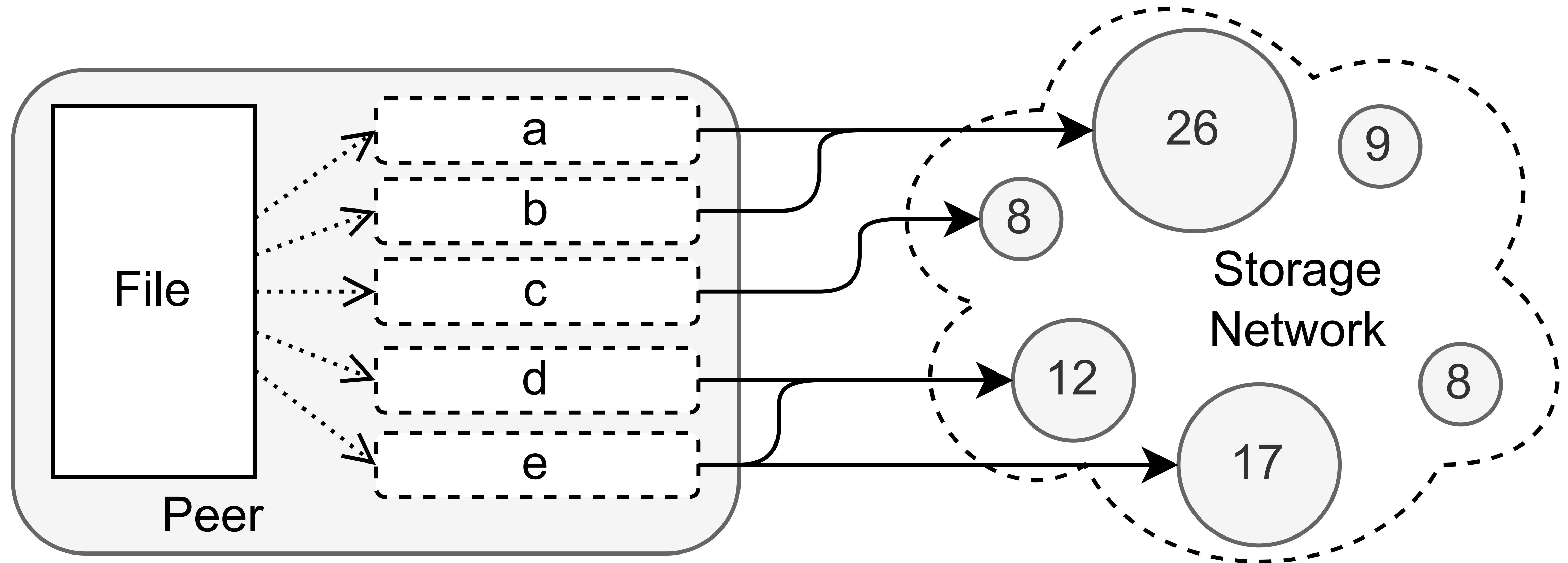
Repair



$$Y = S \oplus T$$

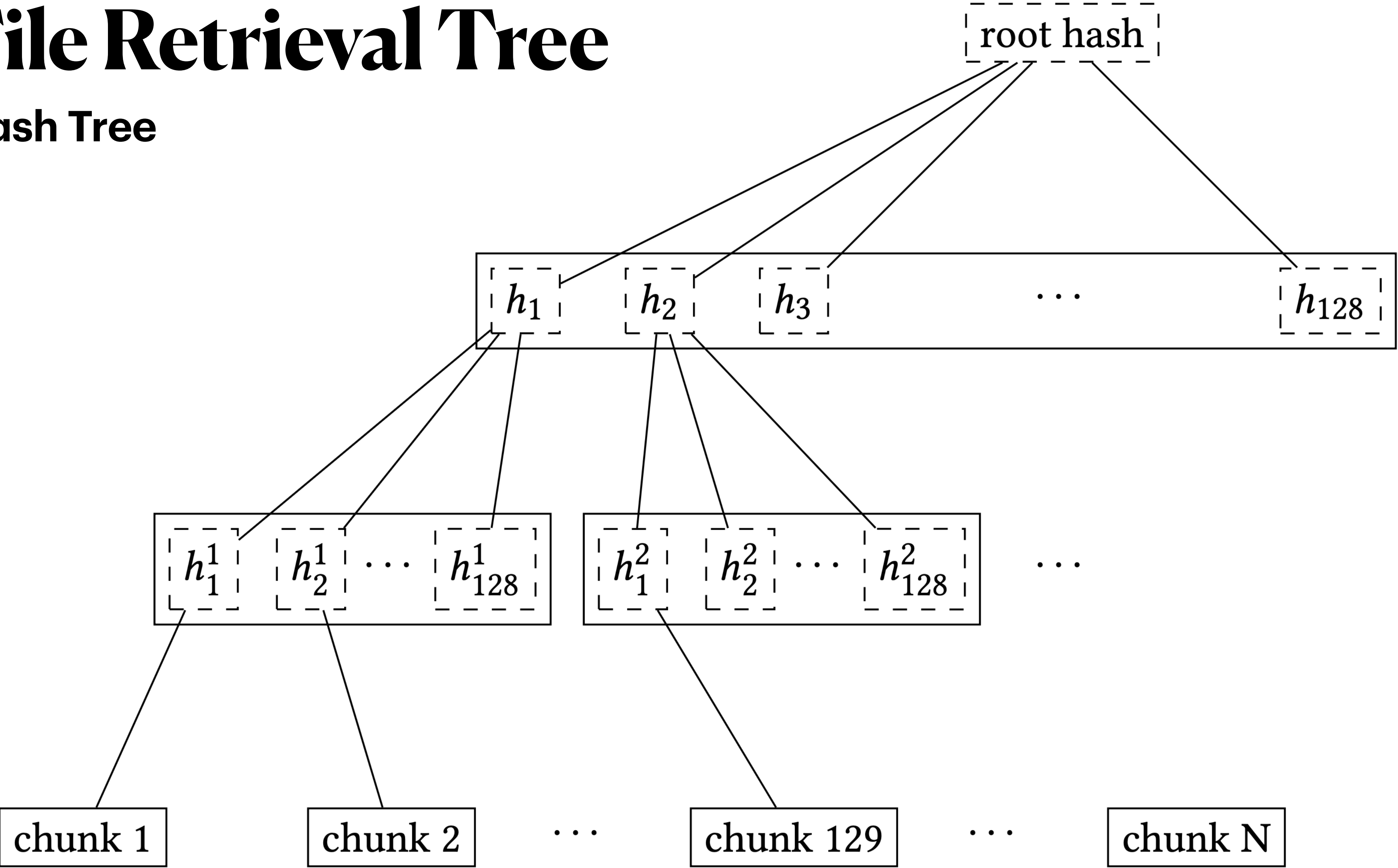
Data Structures for Decentralized Storage

Distribution of Chunks



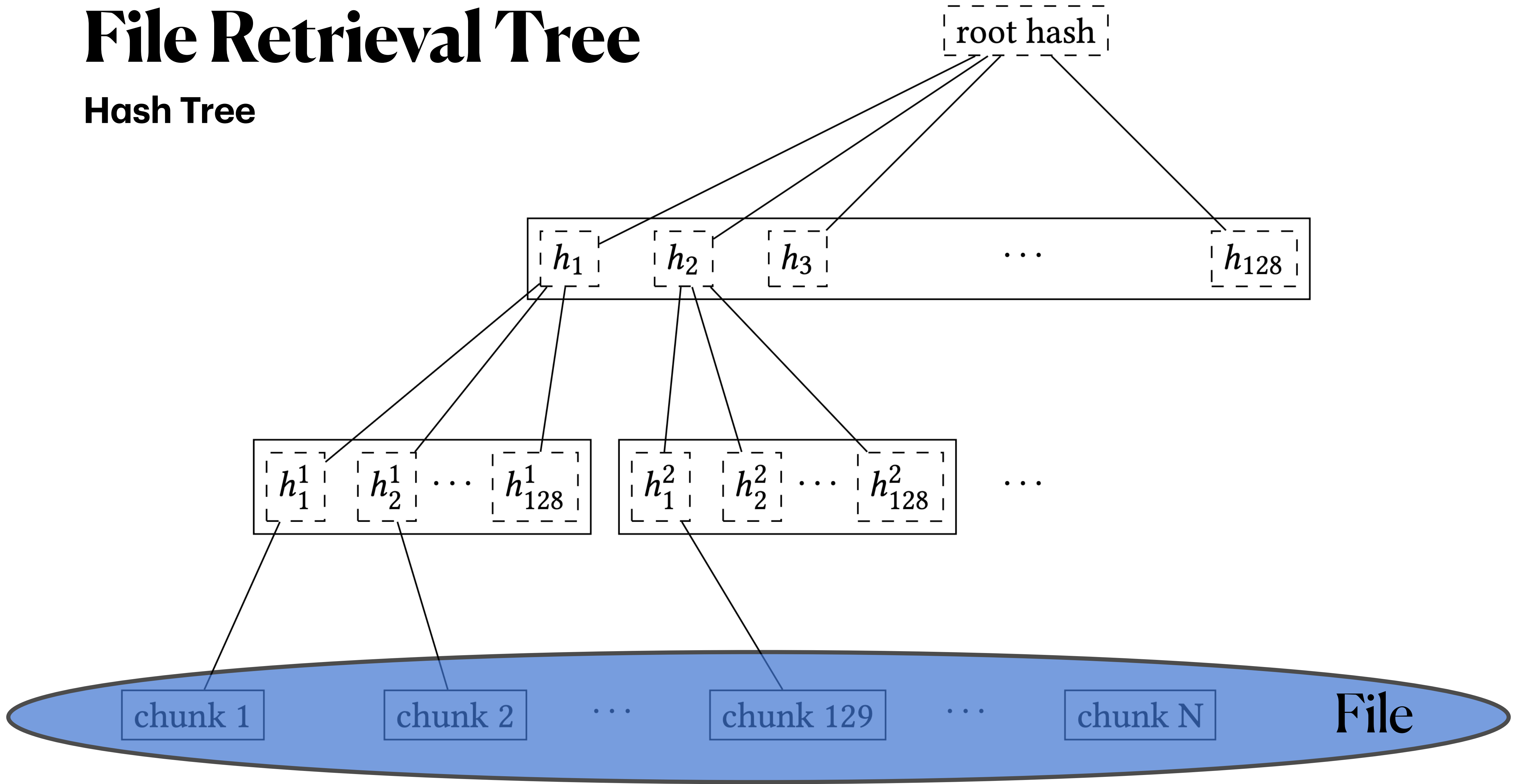
File Retrieval Tree

Hash Tree



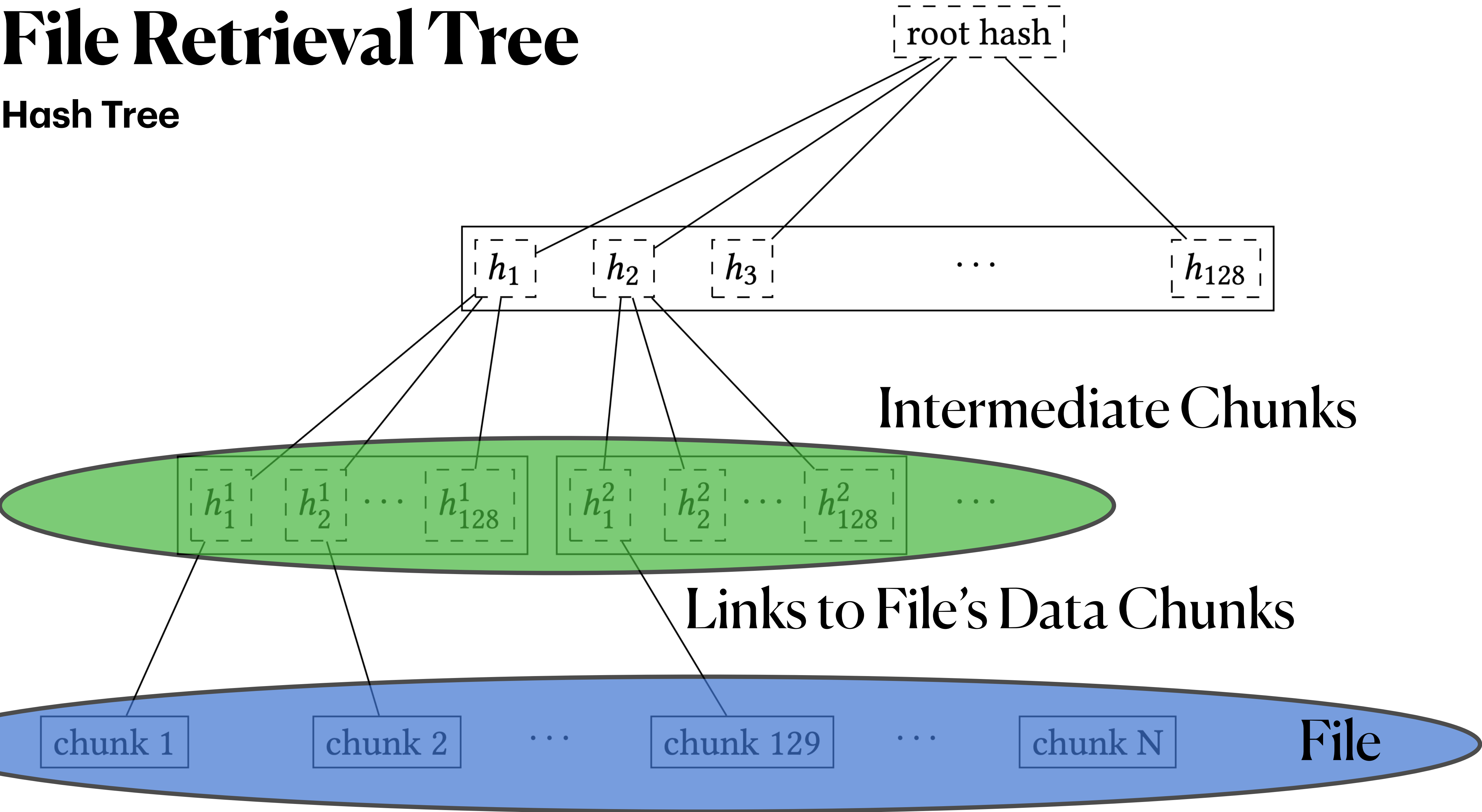
File Retrieval Tree

Hash Tree



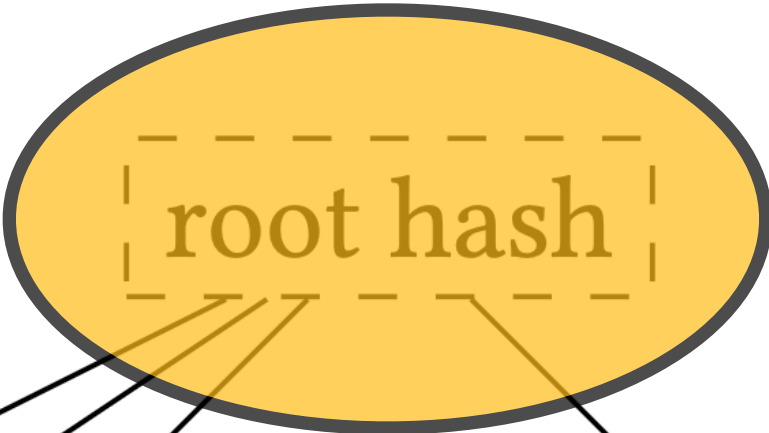
File Retrieval Tree

Hash Tree



File Retrieval Tree

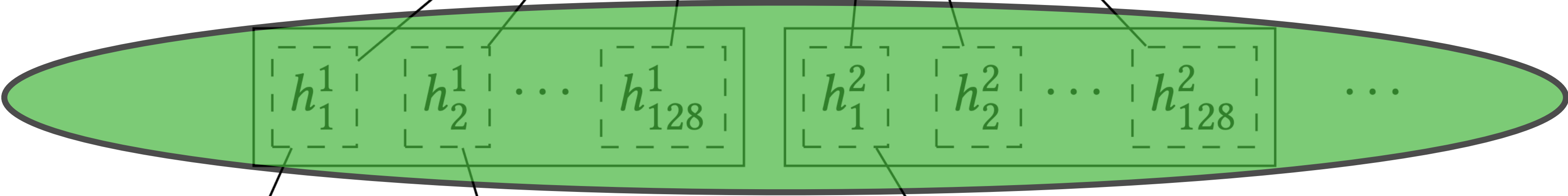
Hash Tree



Needed to
Download File



Intermediate Chunks



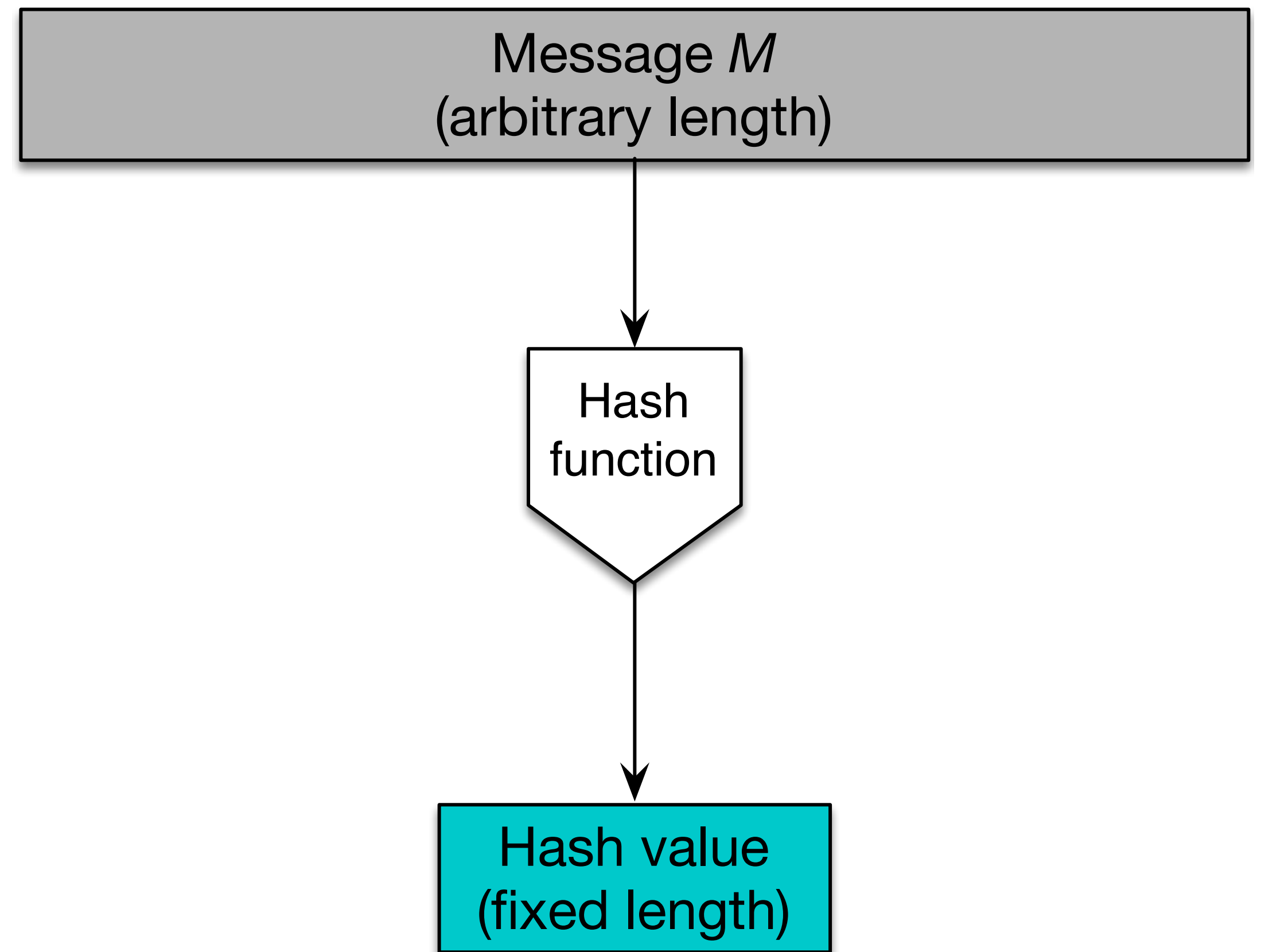
Links to File's Data Chunks



File

Cryptographic Hash Functions

- Collision-Free
- ...



Cryptographic Hash Functions



EASY
→

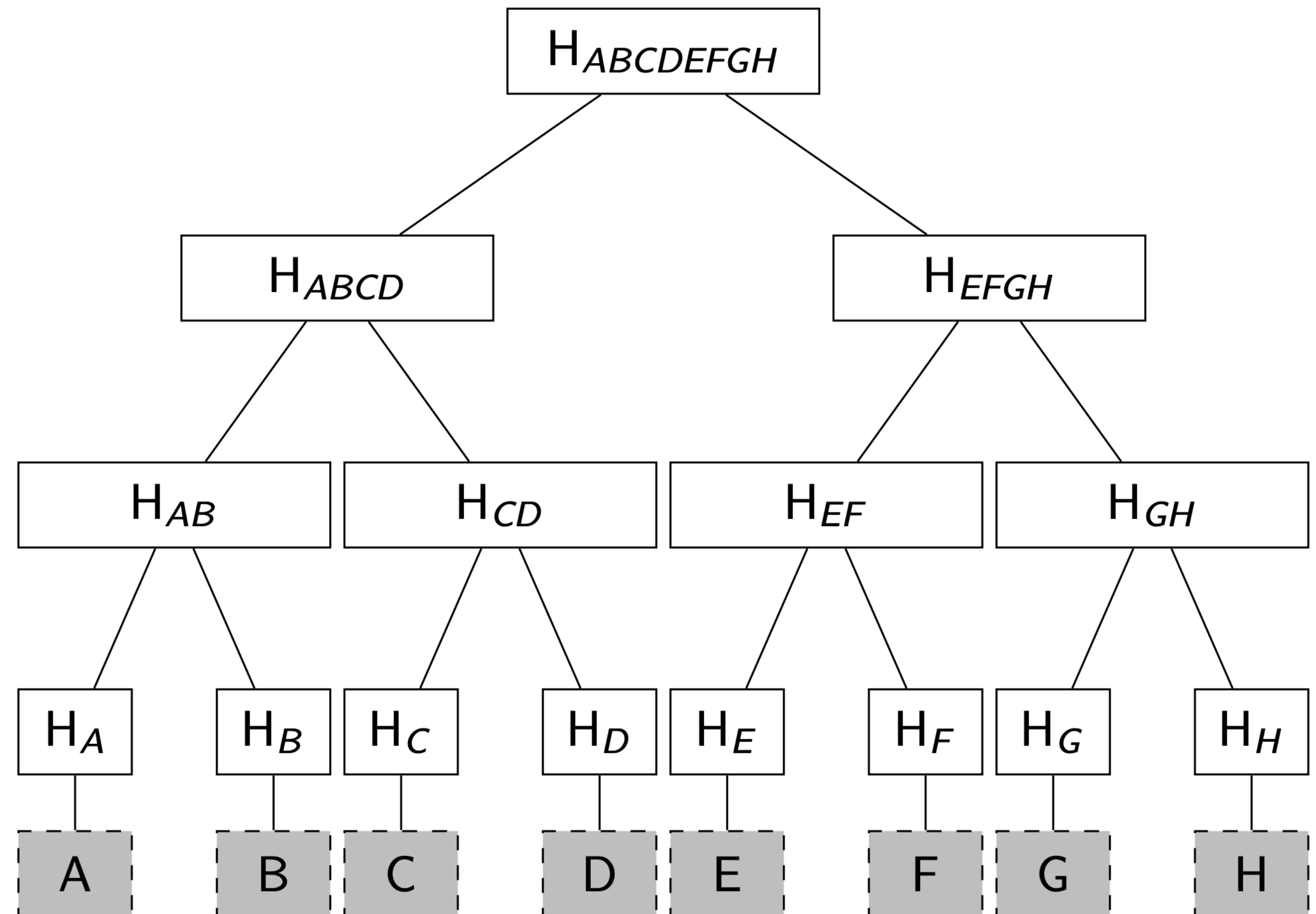


←
HARD

2-ary Merkle Tree

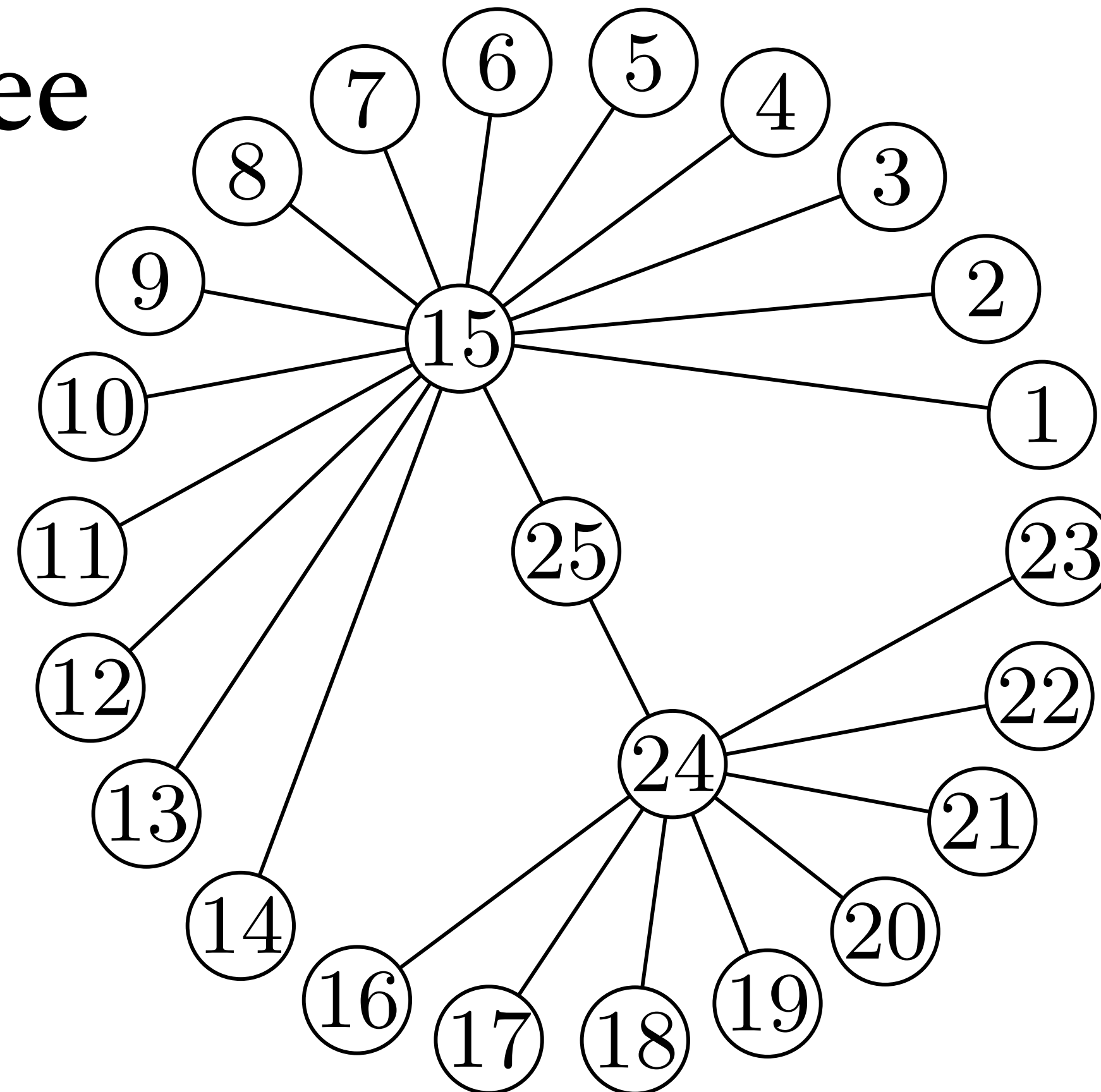
aka Hash Tree

- Leaves store data
- Parent: hash of children
- Compact Proofs
- Tamper Detection



K-ary Merkle Tree

- 14-ary Merkle Tree



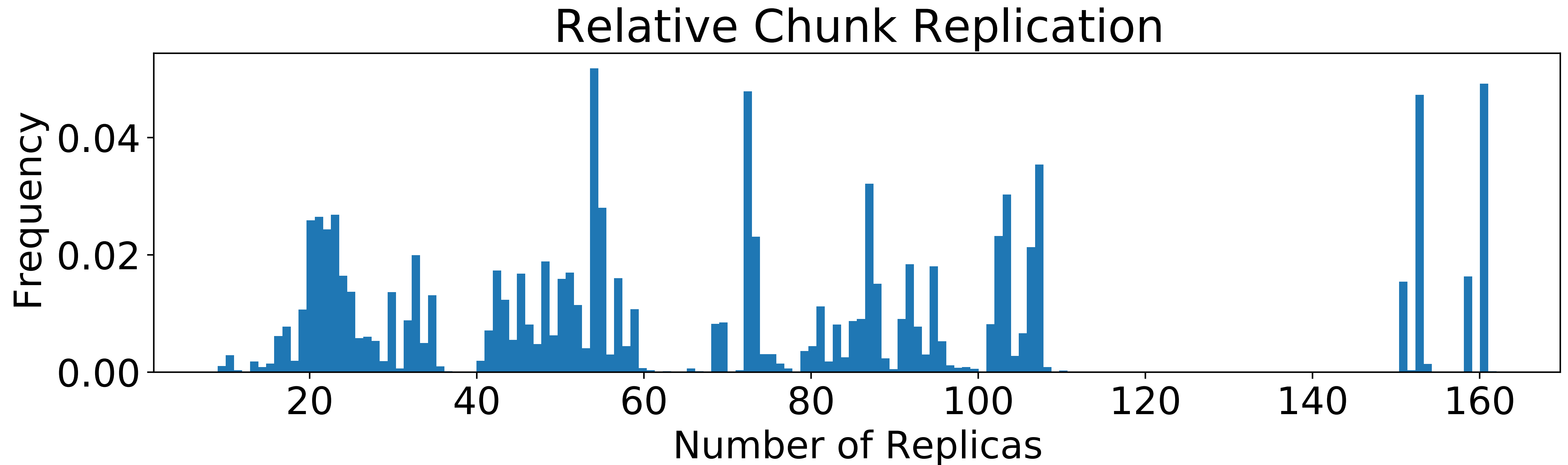
Snarl

**Improving File Availability
and Storage Utilization**

Chunk Replication in Ethereum Swarm

100 MB File

- Chunk Replication is not Uniform

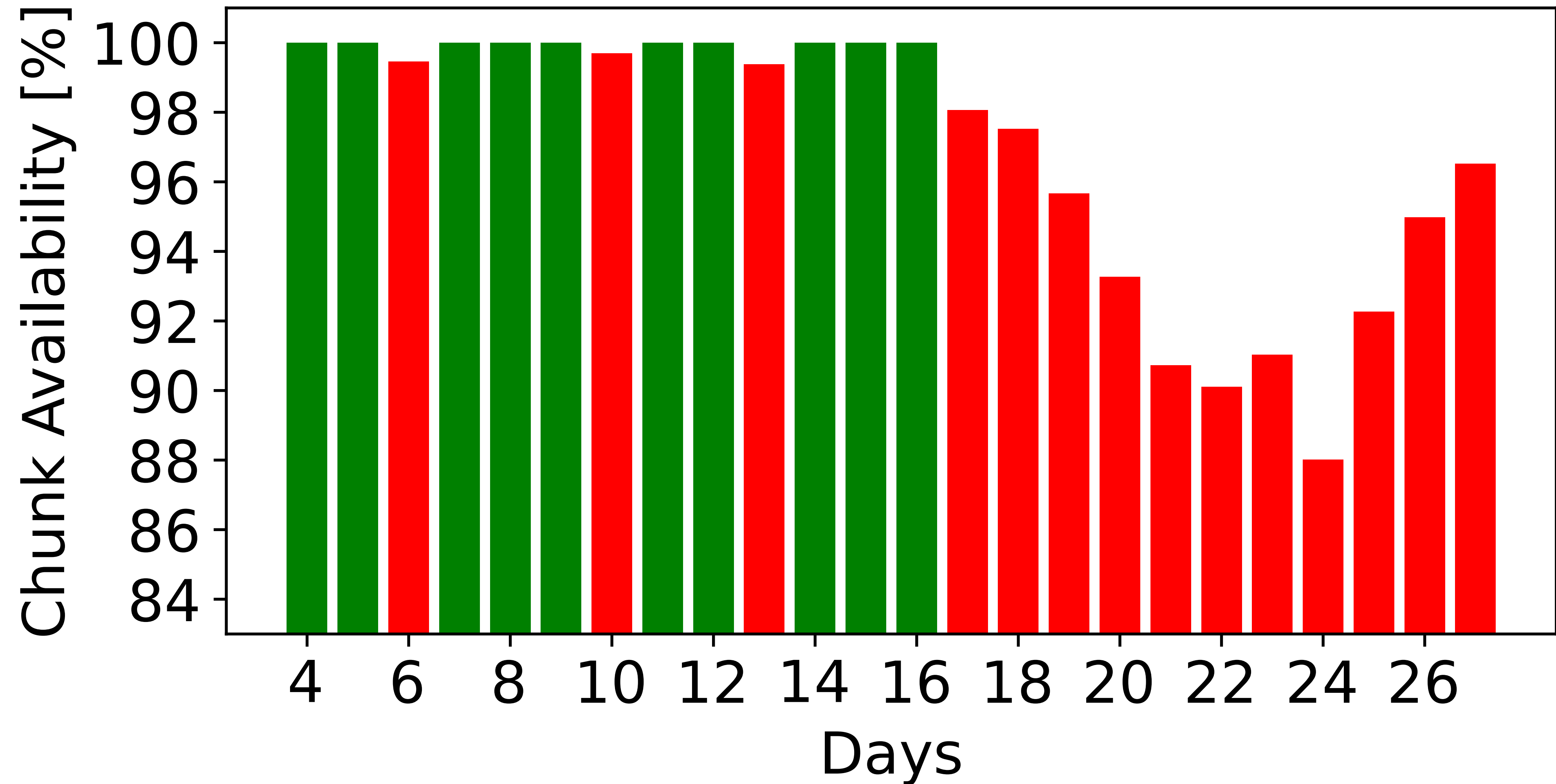


Chunk Replication in Ethereum Swarm

- Chunk Replication is not Uniform
- Average Replication Factor: ~ 72
- Storage Consumption for 100 MB File: 7.2 GB

File Availability in Ethereum Swarm

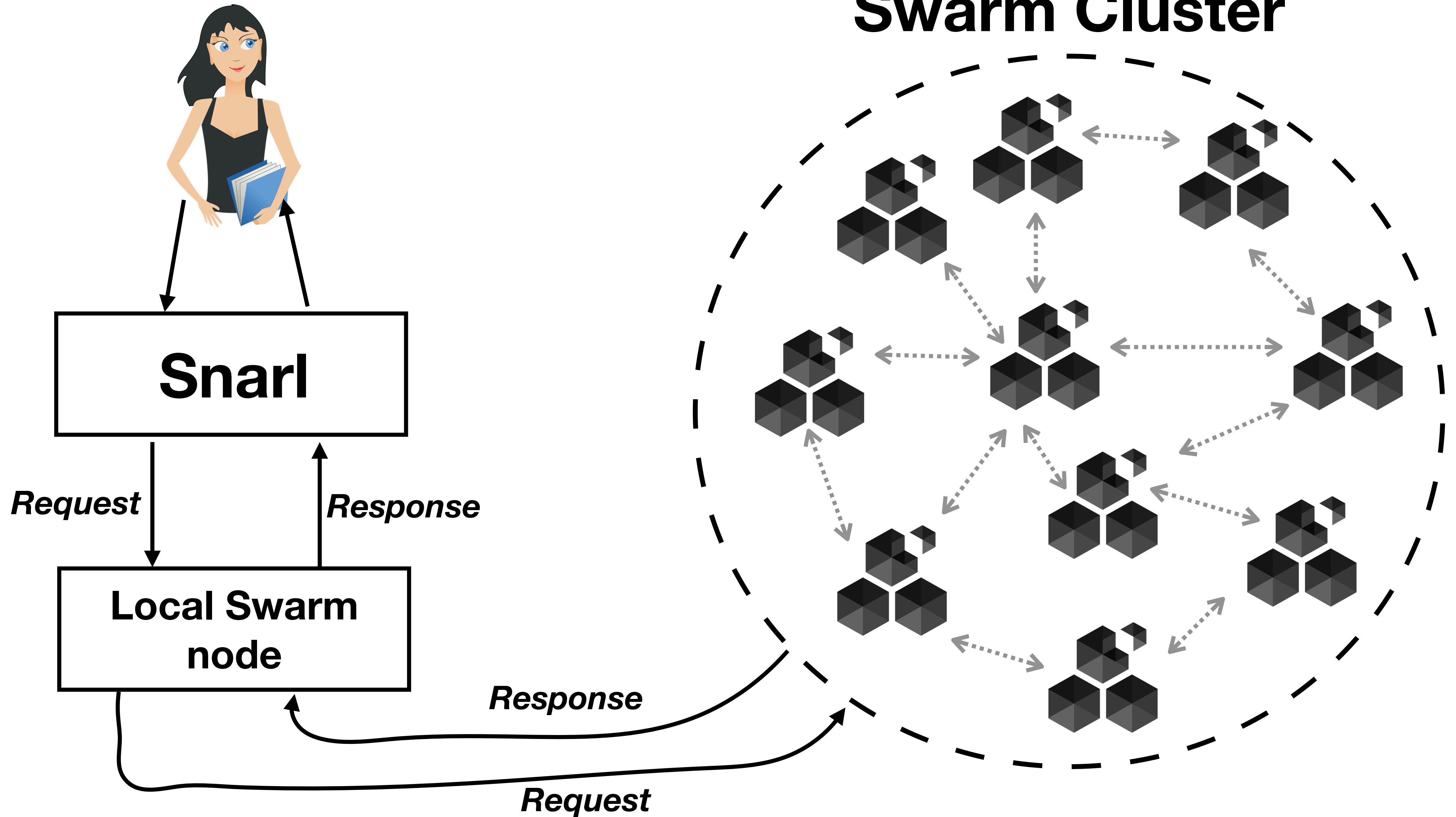
4 Week Experiment — 5 MB File



Key Idea: Entangled Merkle Trees

- Novel Data Structure
 - Entanglement Codes Mapped onto
 - Existing File Retrieval Tree Structure
- Algorithms
 - Lookup Parity Chunks in Retrieval Tree
 - Repair Lost Chunks

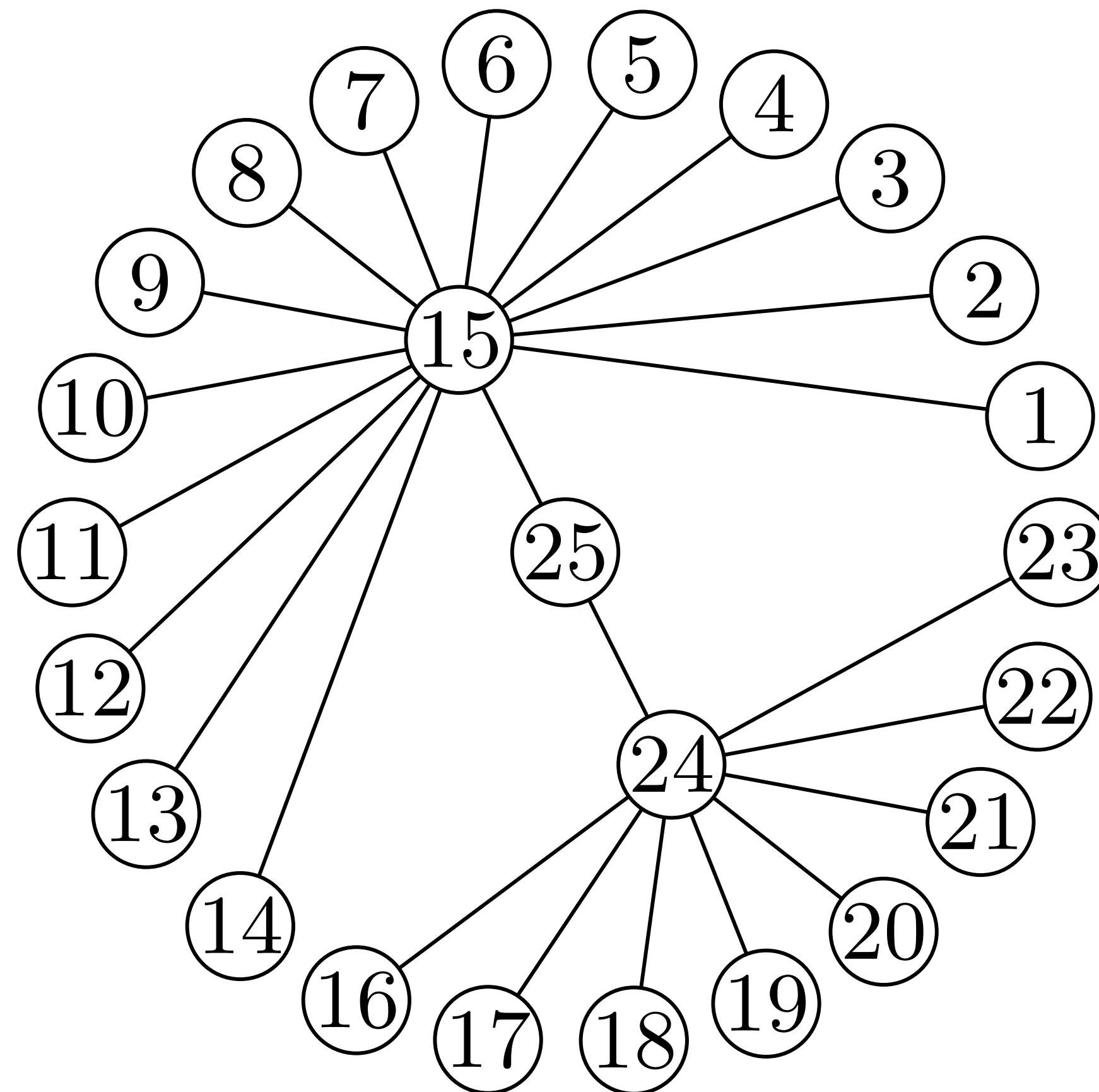
Swarm Cluster



Key Performance Result

- Improve Storage Utilization by $5\times$
- Without Modifying the File Retrieval Structure
- Encoding Speed is Linear: 1 GB in less than 4 seconds

Data Chunks in a *K*-ary Merkle Tree



Alpha Entanglement Codes

Data Chunks

1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24
5	10	15	20	25

Alpha Entanglement Codes

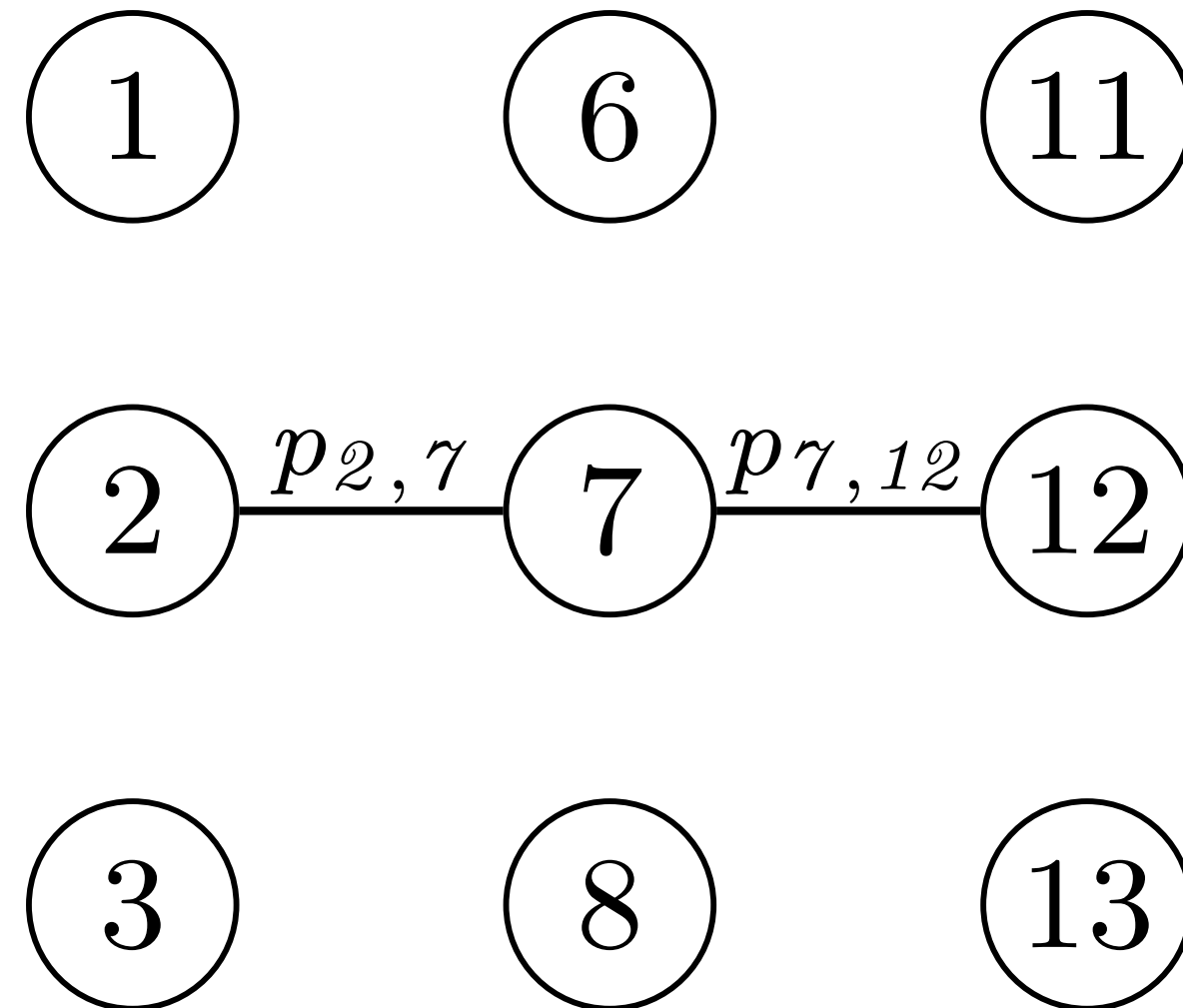
Data Chunks

1	6	11
2	7	12
3	8	13

Alpha Entanglement Codes

Data Chunk 7's Repair Pairs

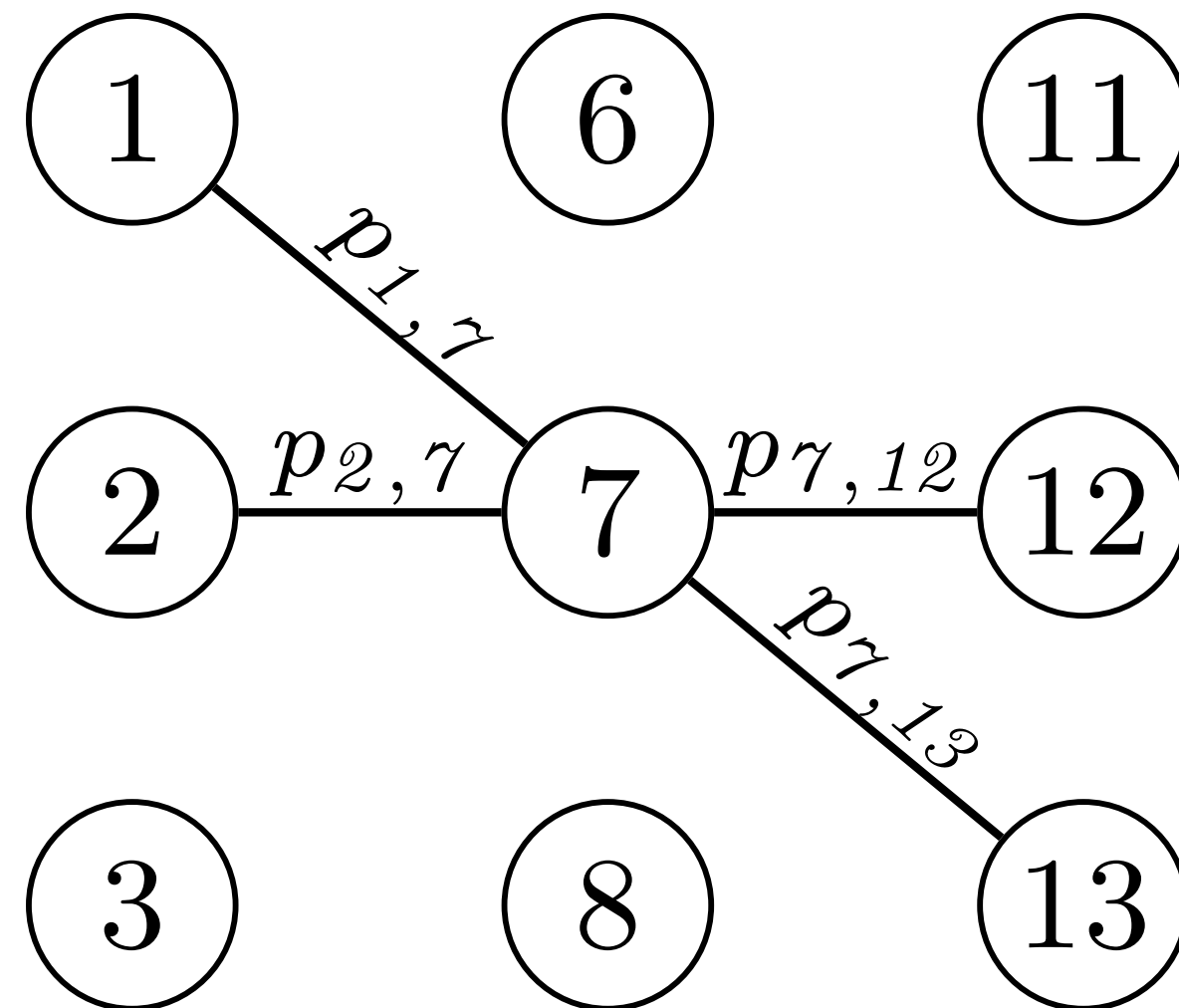
- $\alpha = 1$



Alpha Entanglement Codes

Data Chunk 7's Repair Pairs

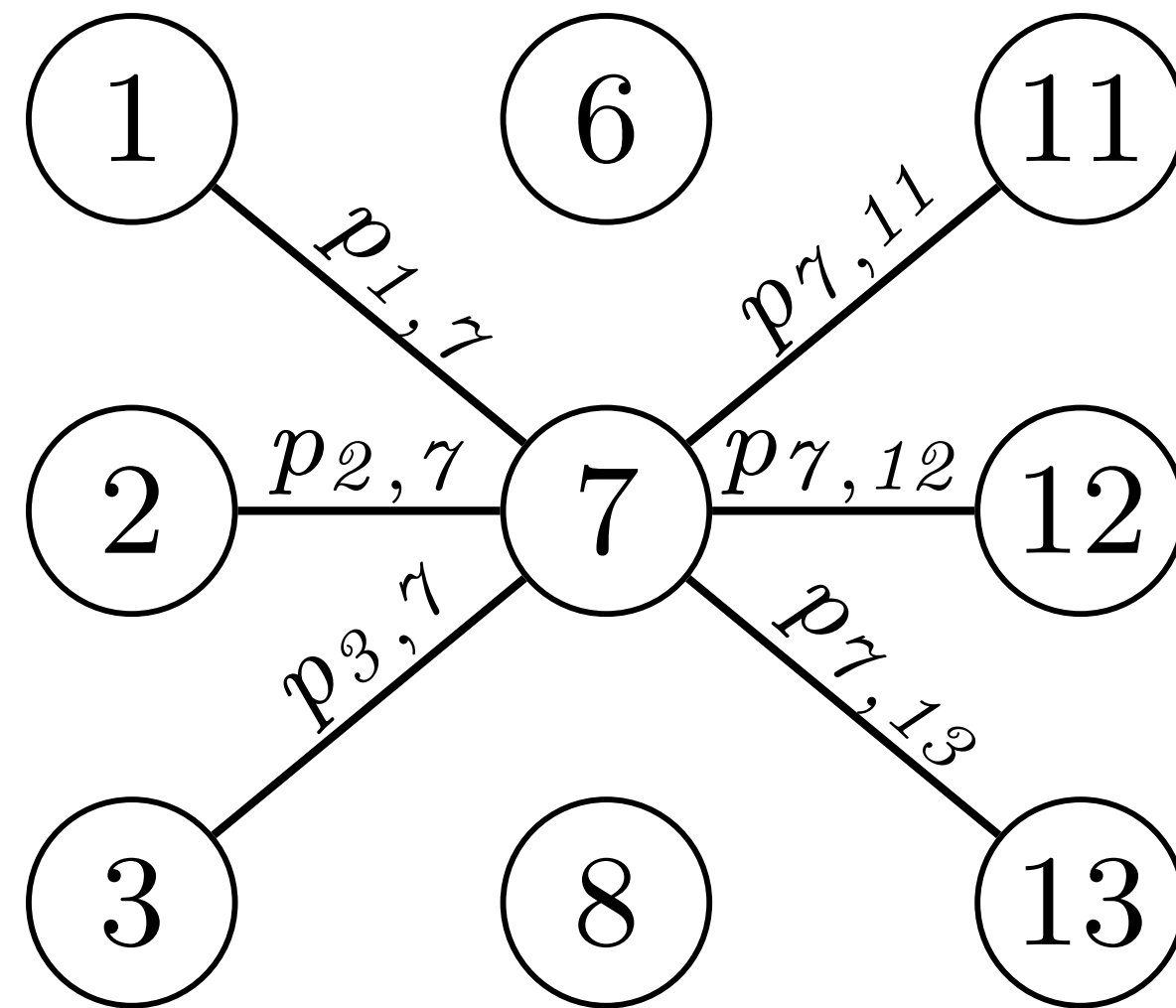
• $\alpha = 2$



Alpha Entanglement Codes

Data Chunk 7's Repair Pairs

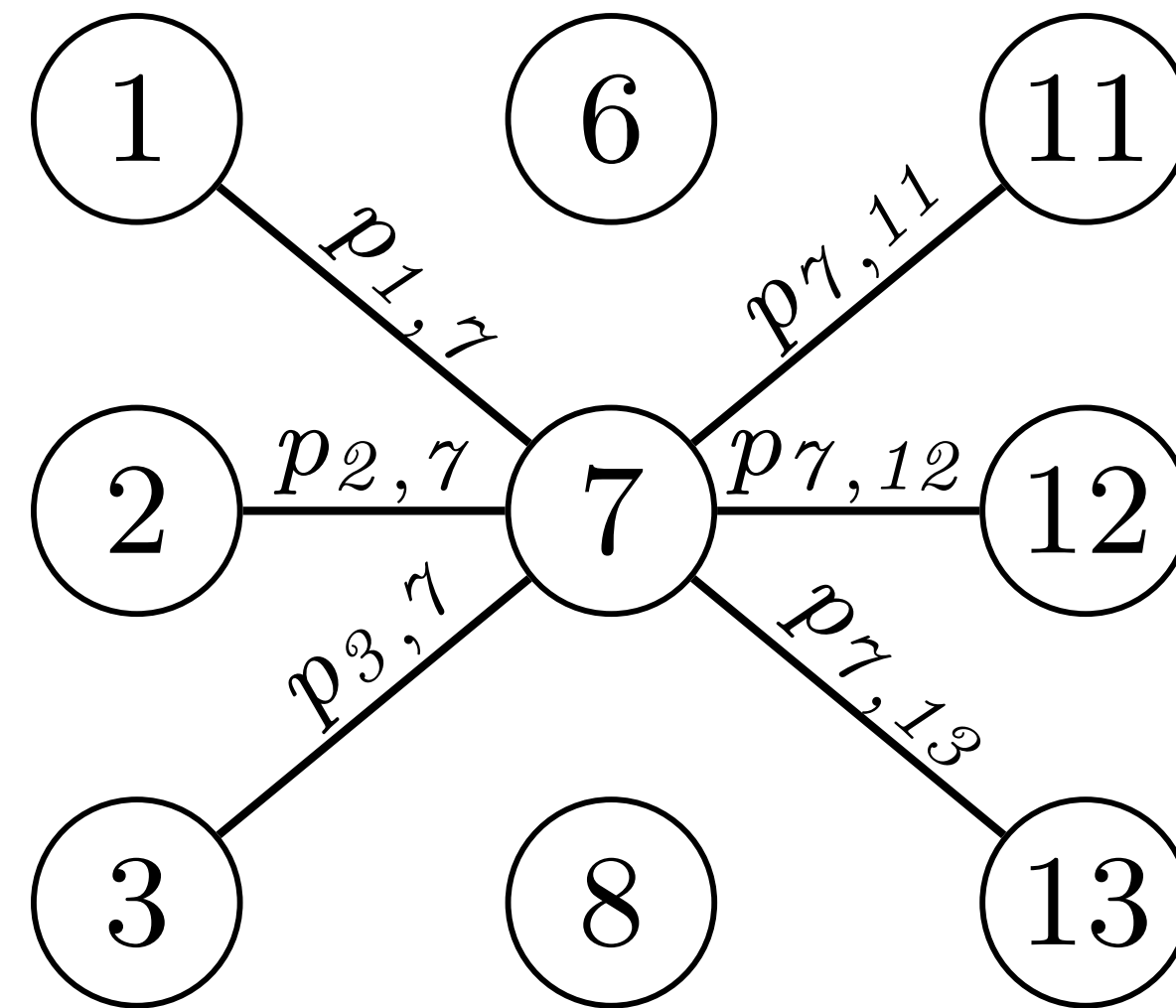
- $\alpha = 3$



Alpha Entanglement Codes

Data Chunk 7's Repair Pairs

• $\alpha = 3$



$$d_7 = p_{2,7} \oplus p_{7,12} \quad (\text{Horizontal})$$

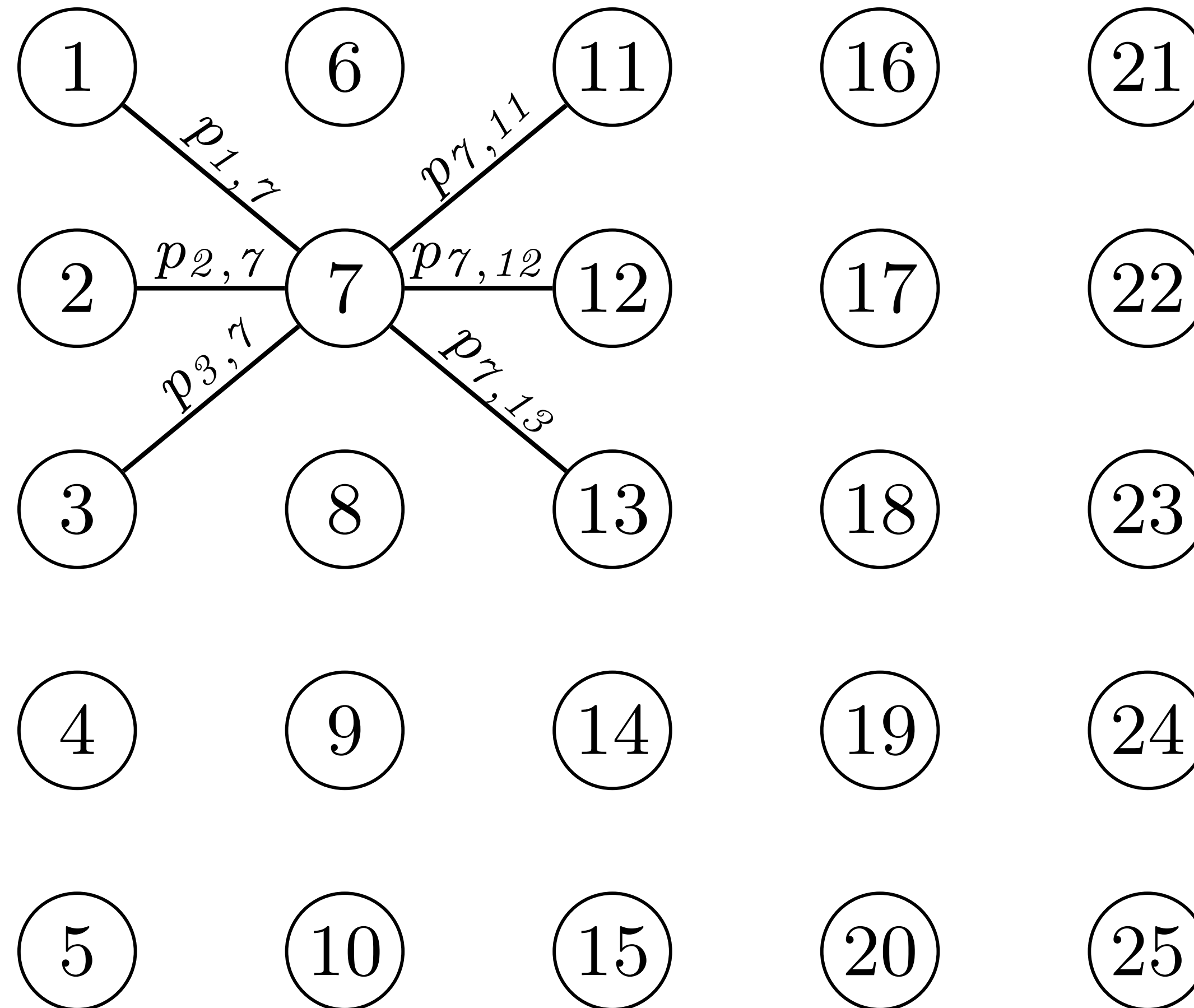
$$d_7 = p_{1,7} \oplus p_{7,13} \quad (\text{Right})$$

$$d_7 = p_{3,7} \oplus p_{7,11} \quad (\text{Left})$$

Alpha Entanglement Codes

Data Chunk 7's Repair Pairs

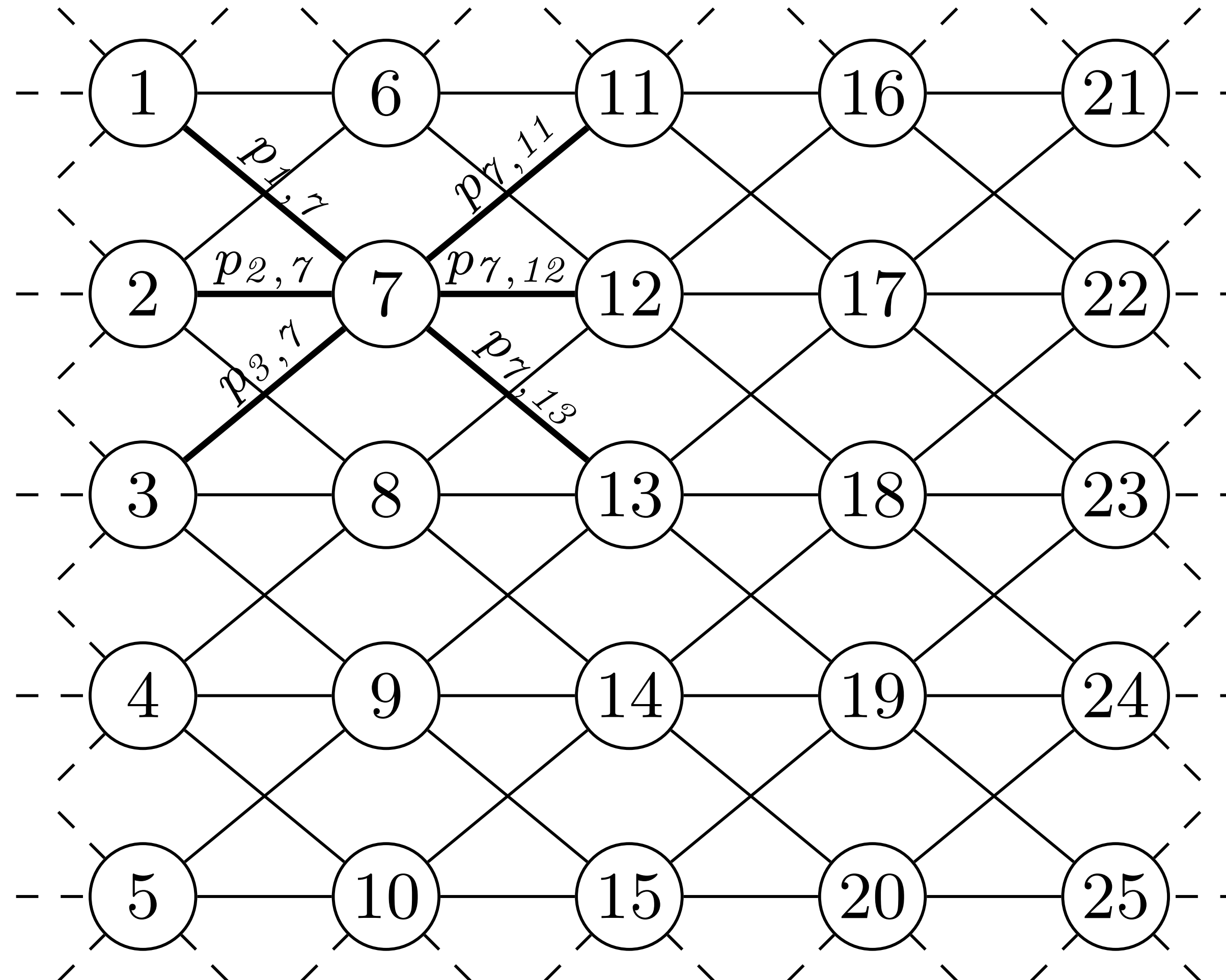
• $\alpha = 3$



Alpha Entanglement Codes

The Remaining Parity Strands

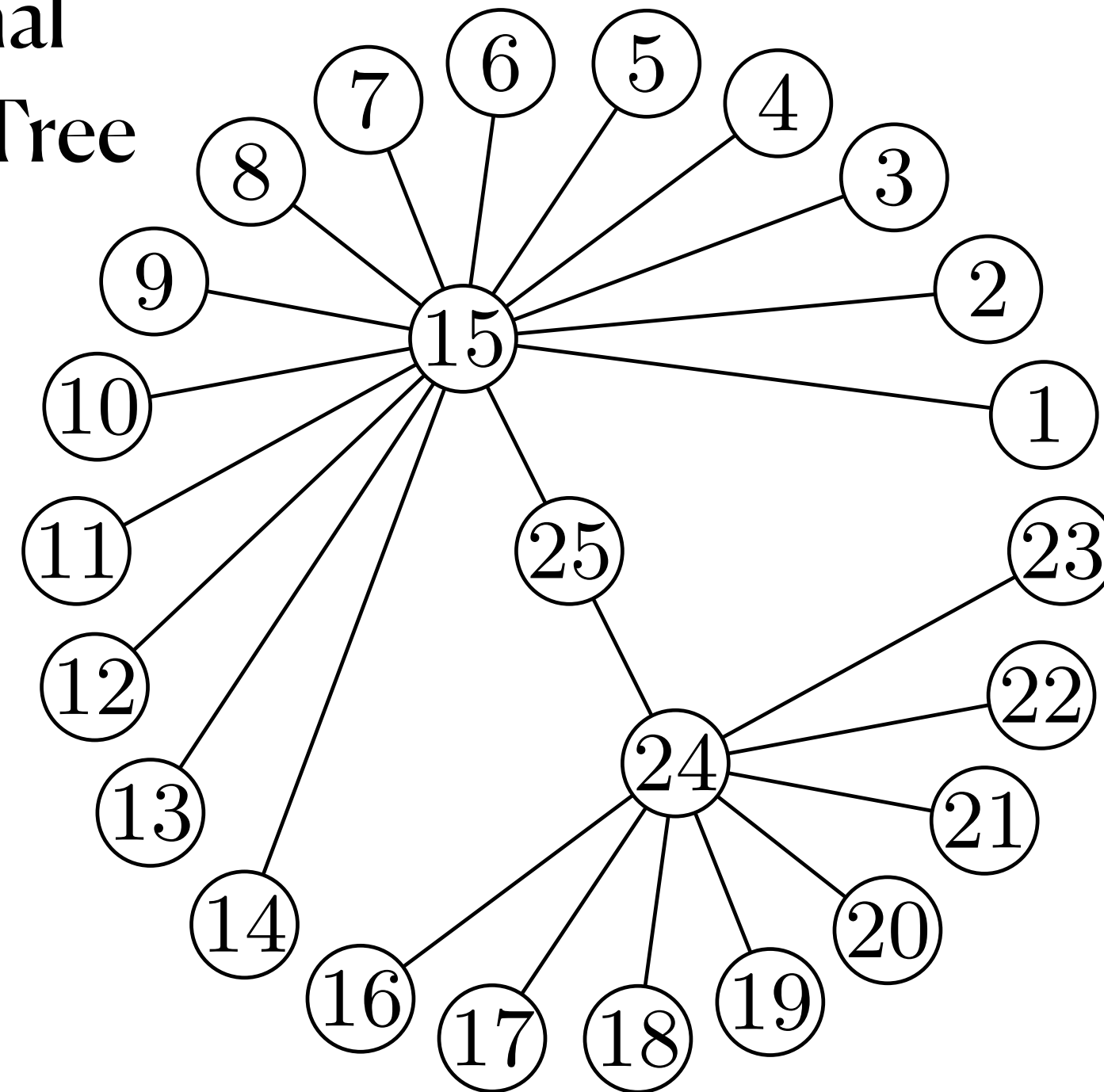
• $\alpha = 3$



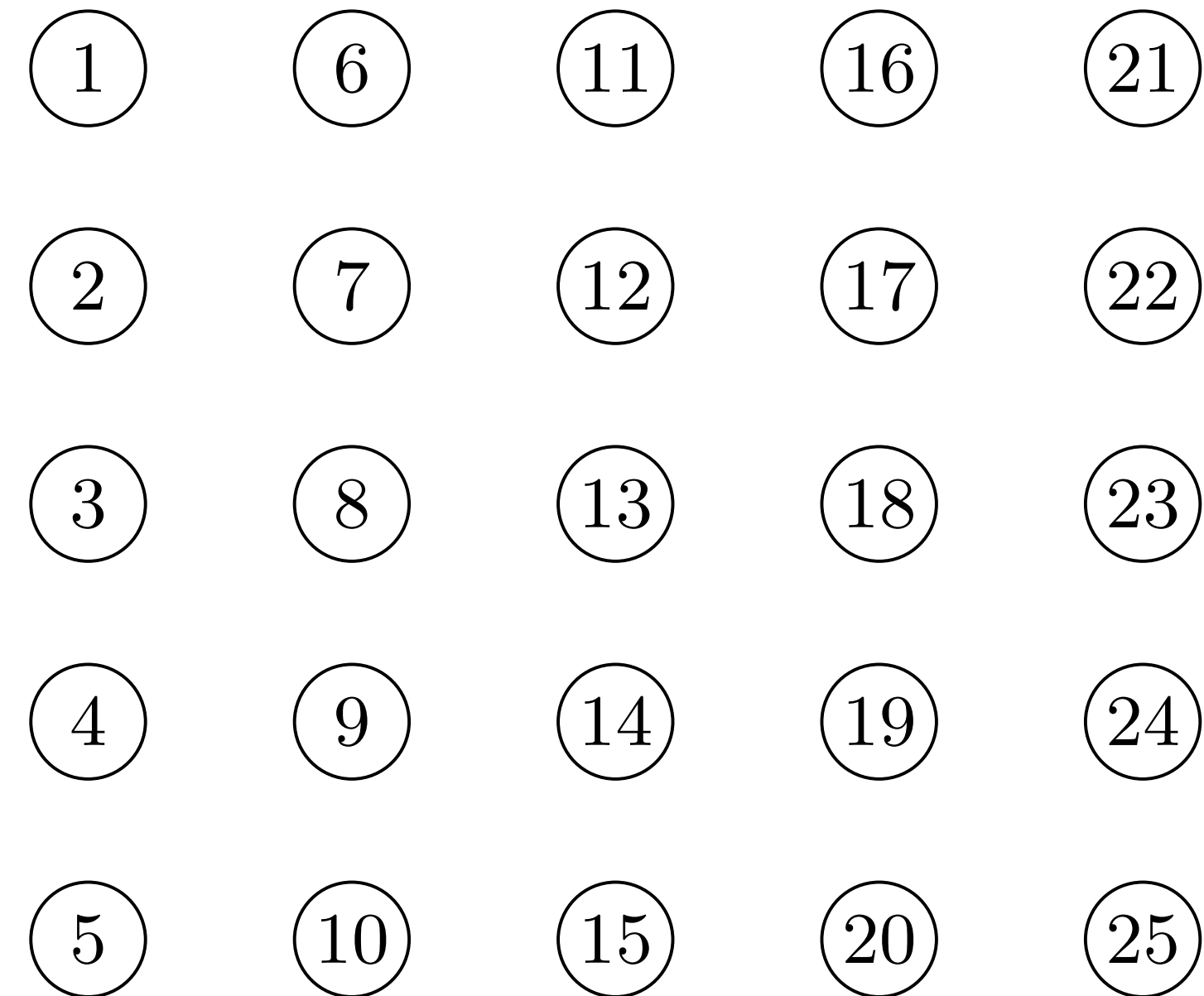
Entangled Merkle Tree

Entangled Merkle Tree

Original
Merkle Tree

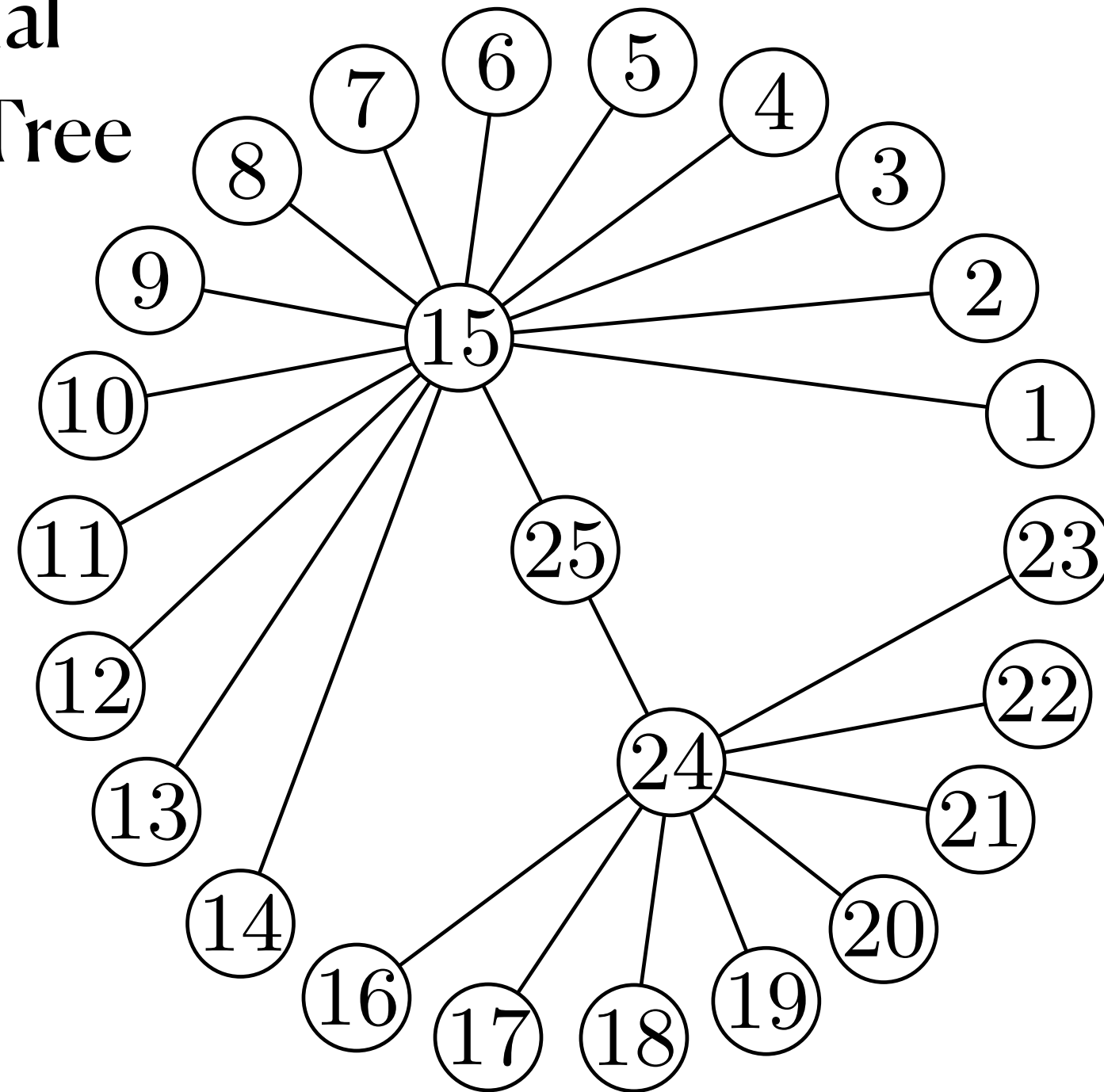


Lattice

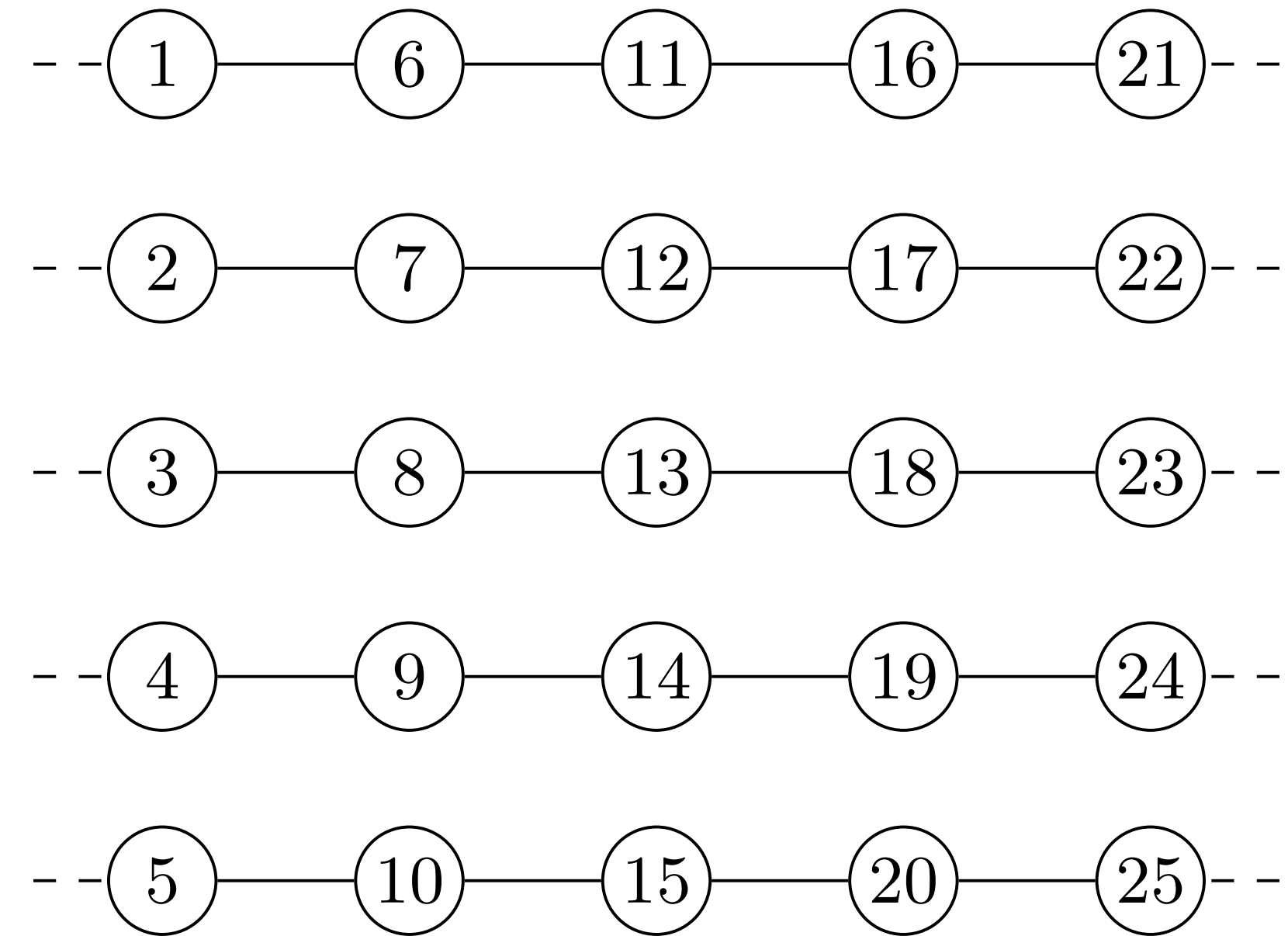


Entangled Merkle Tree

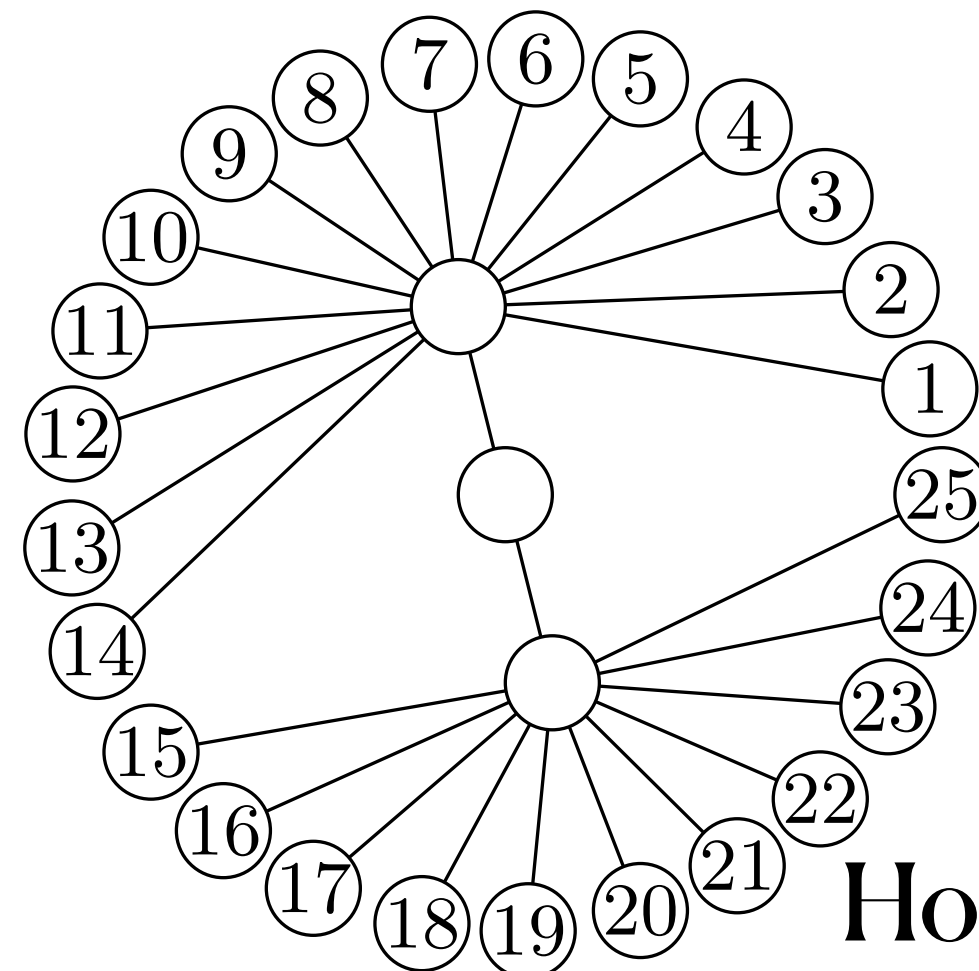
Original
Merkle Tree



Lattice

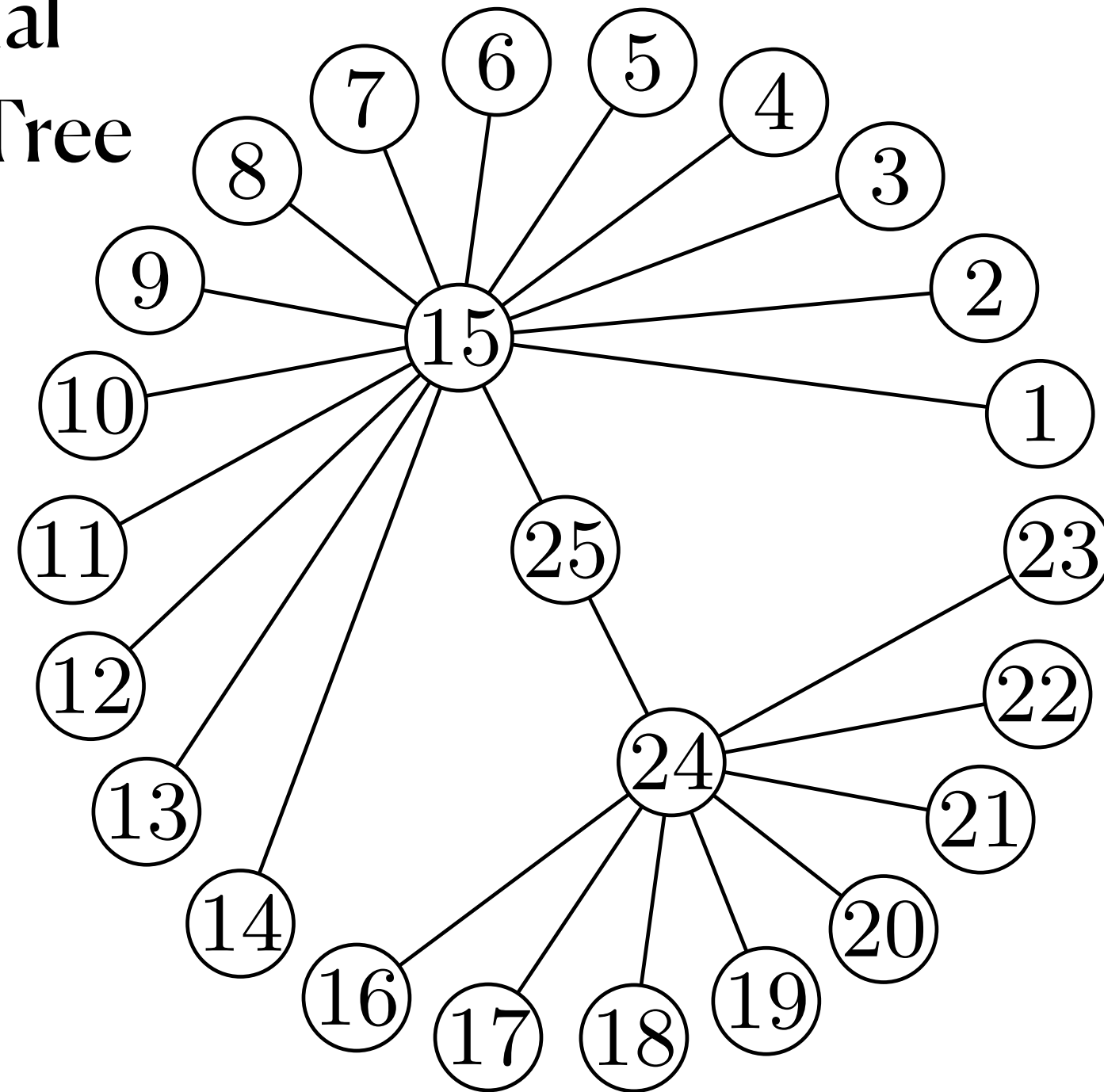


Horizontal
Merkle Tree

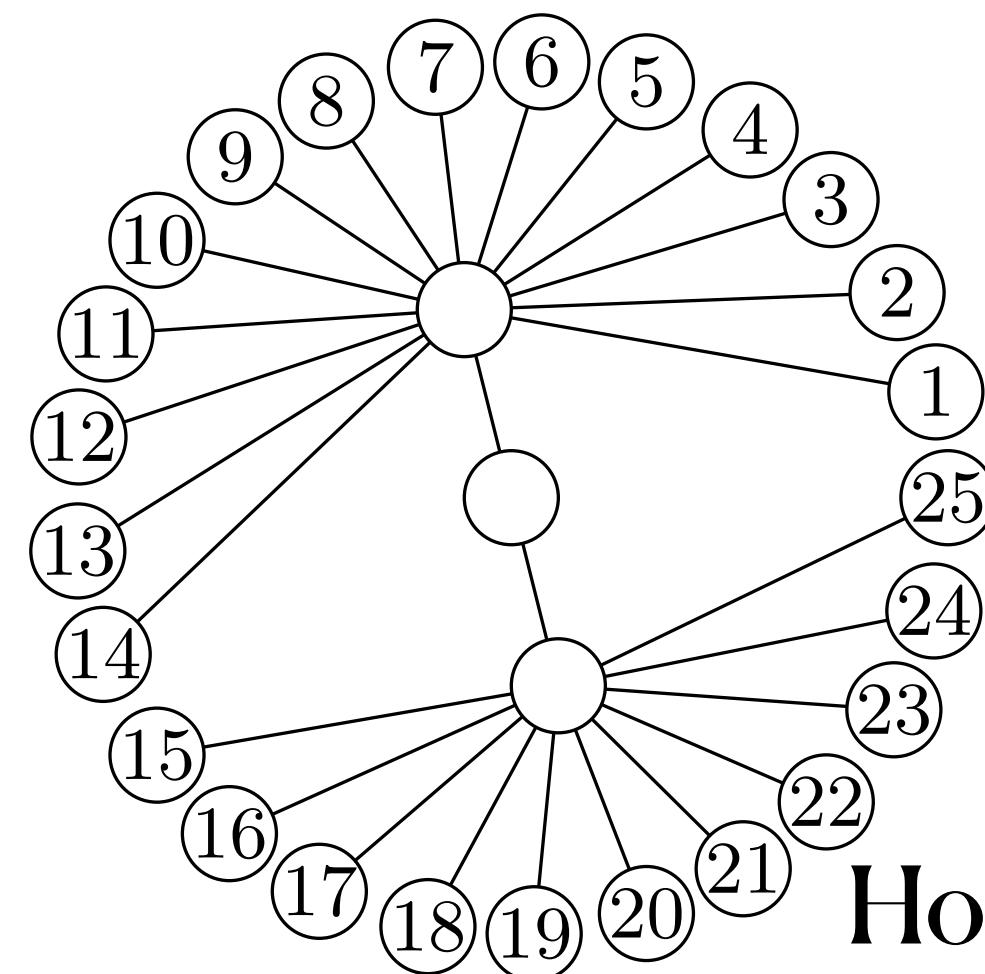
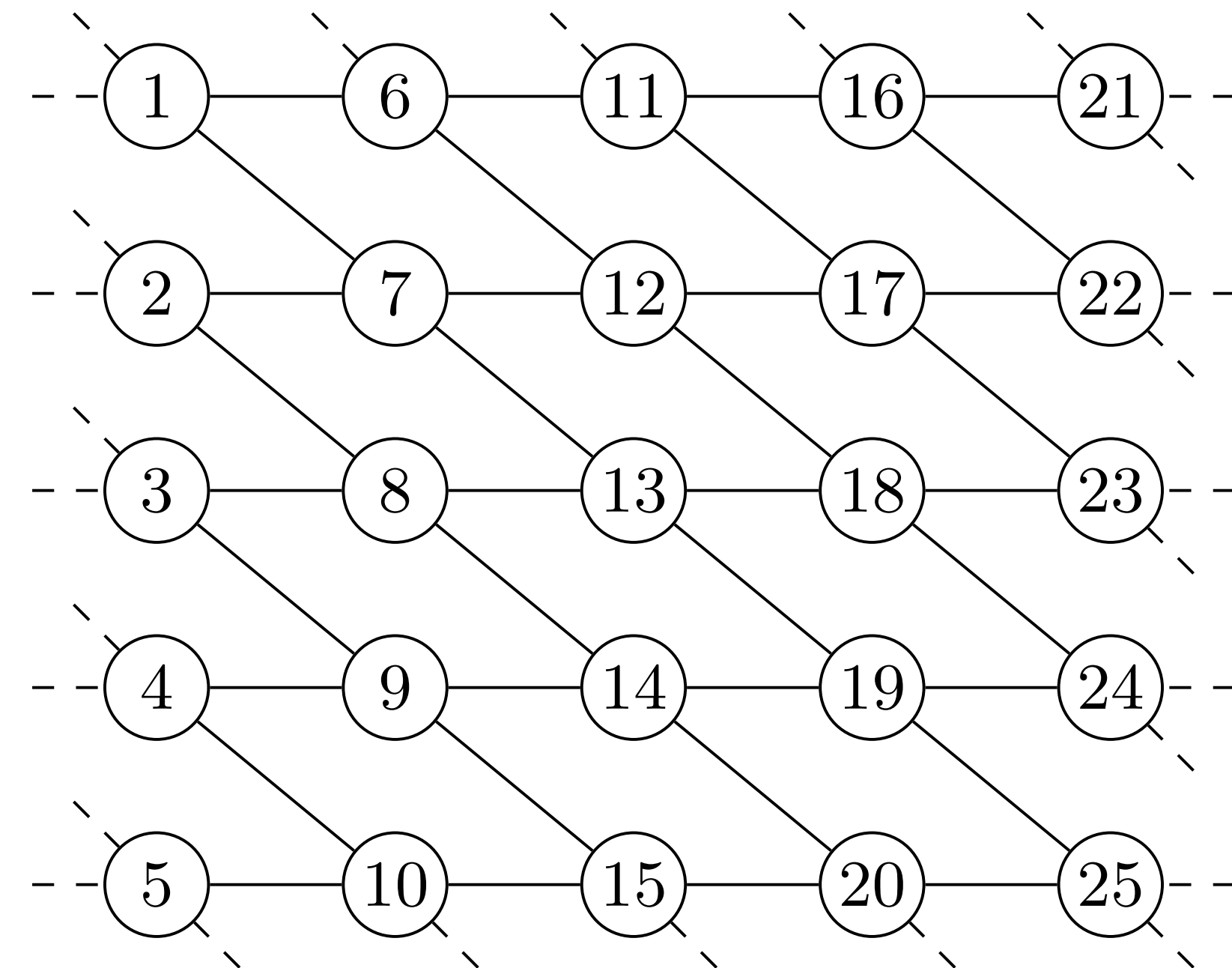


Entangled Merkle Tree

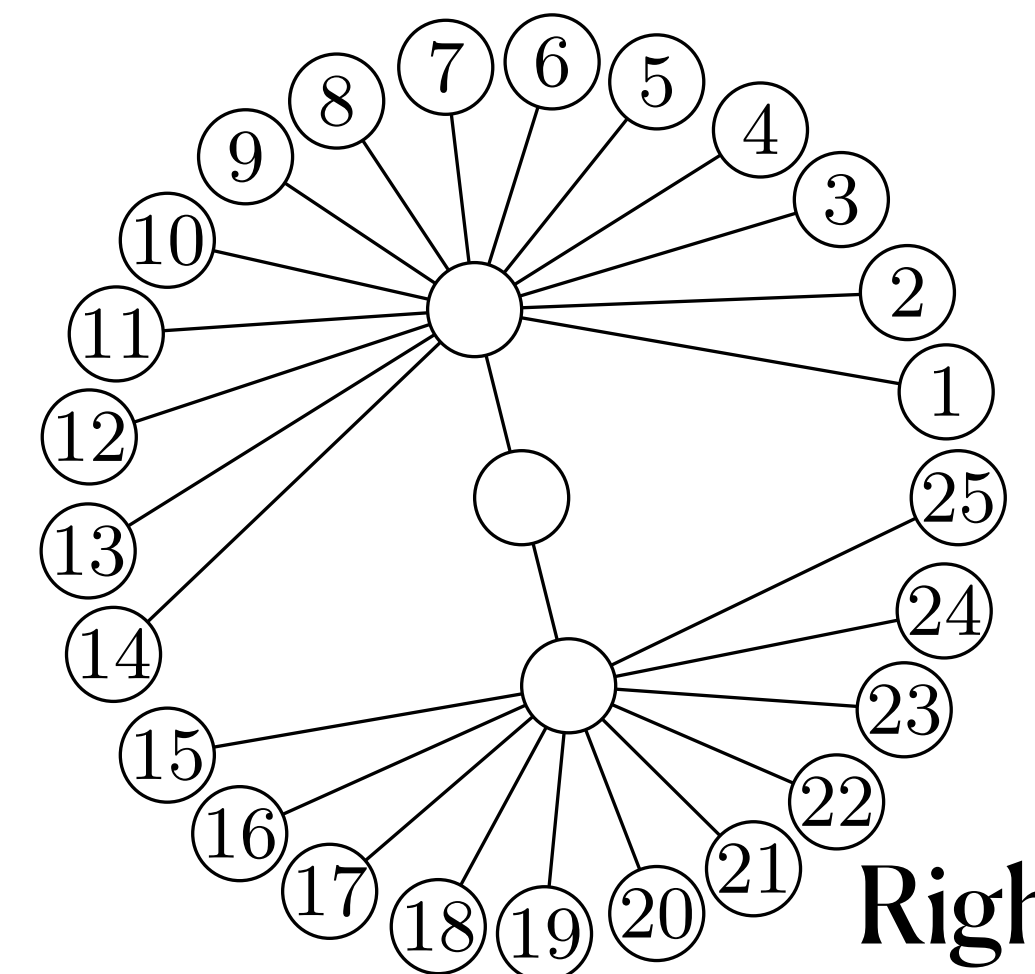
Original
Merkle Tree



Lattice



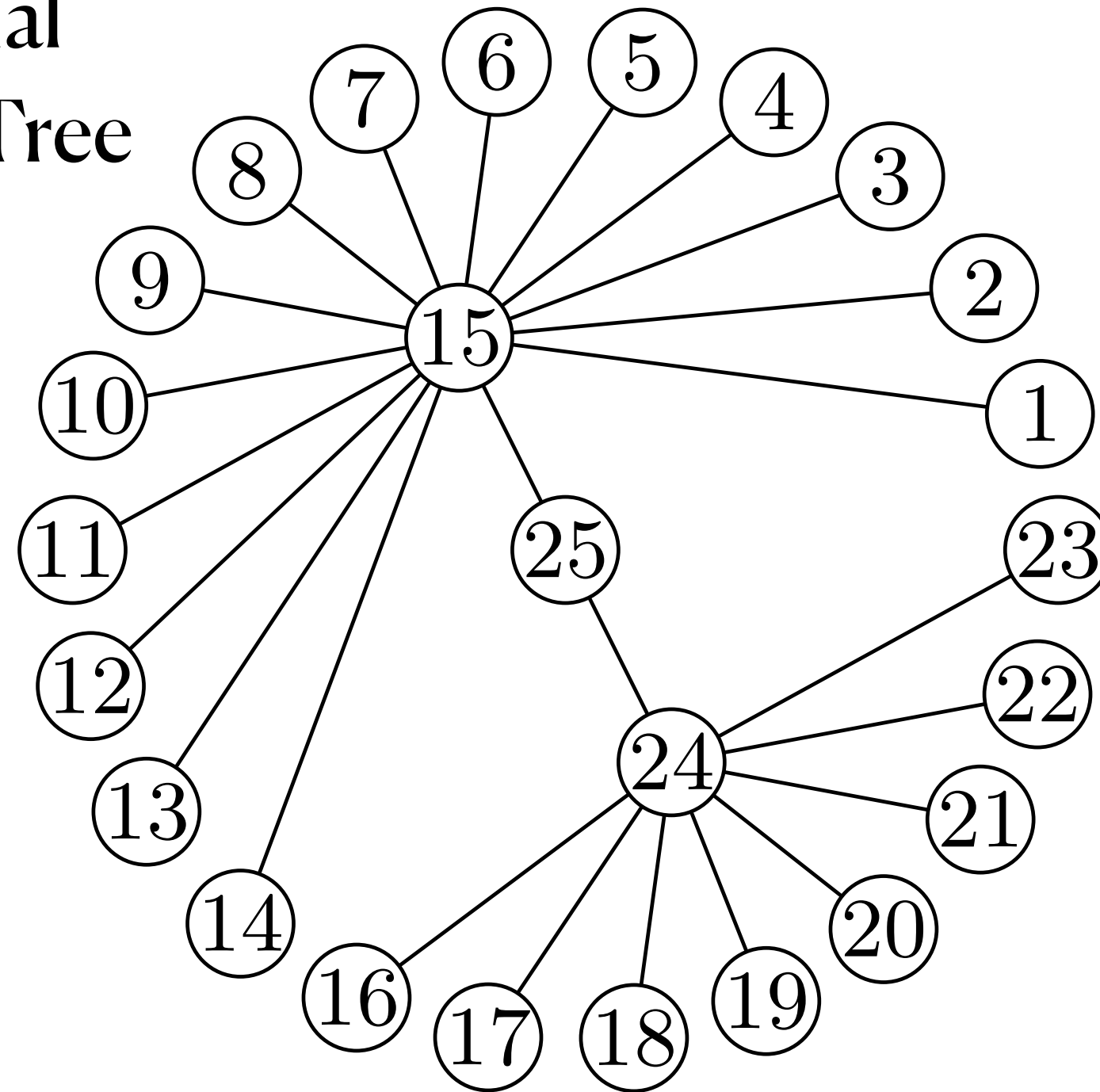
Horizontal
Merkle Tree



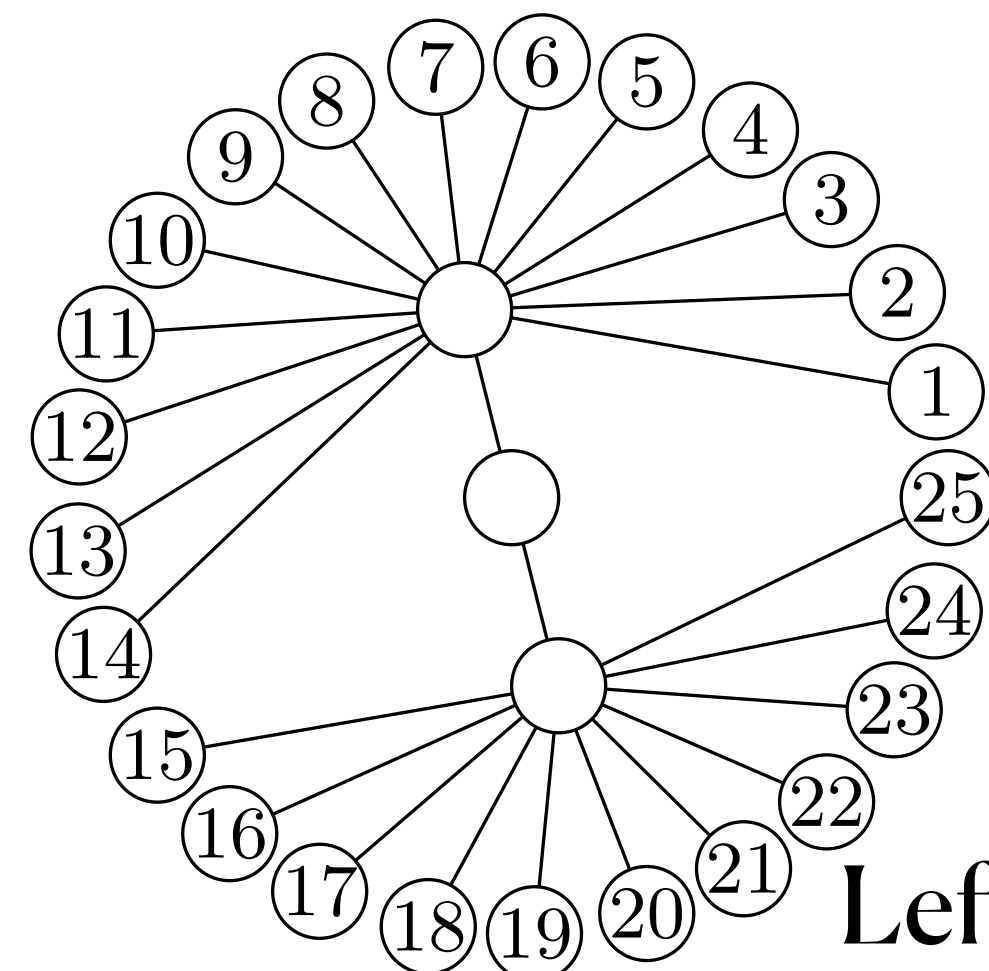
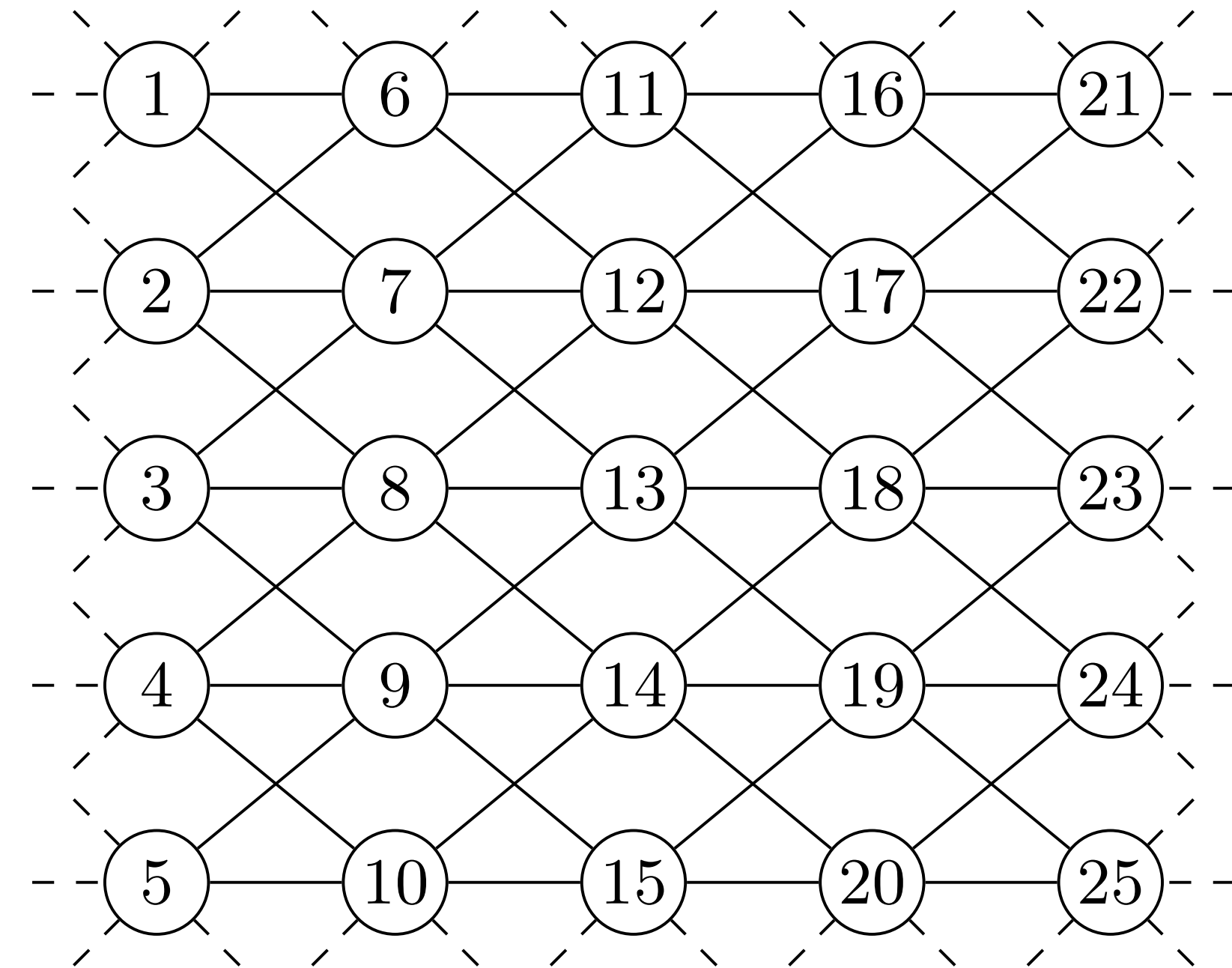
Right
Merkle Tree

Entangled Merkle Tree

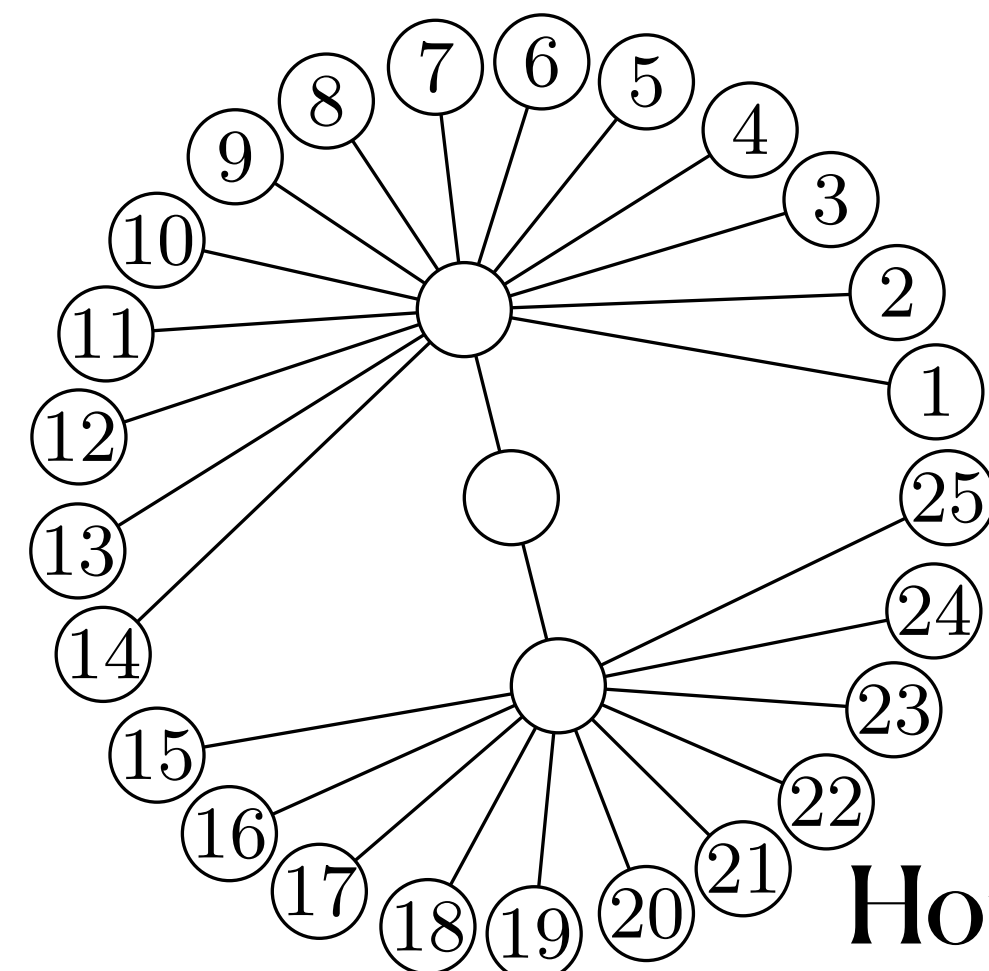
Original
Merkle Tree



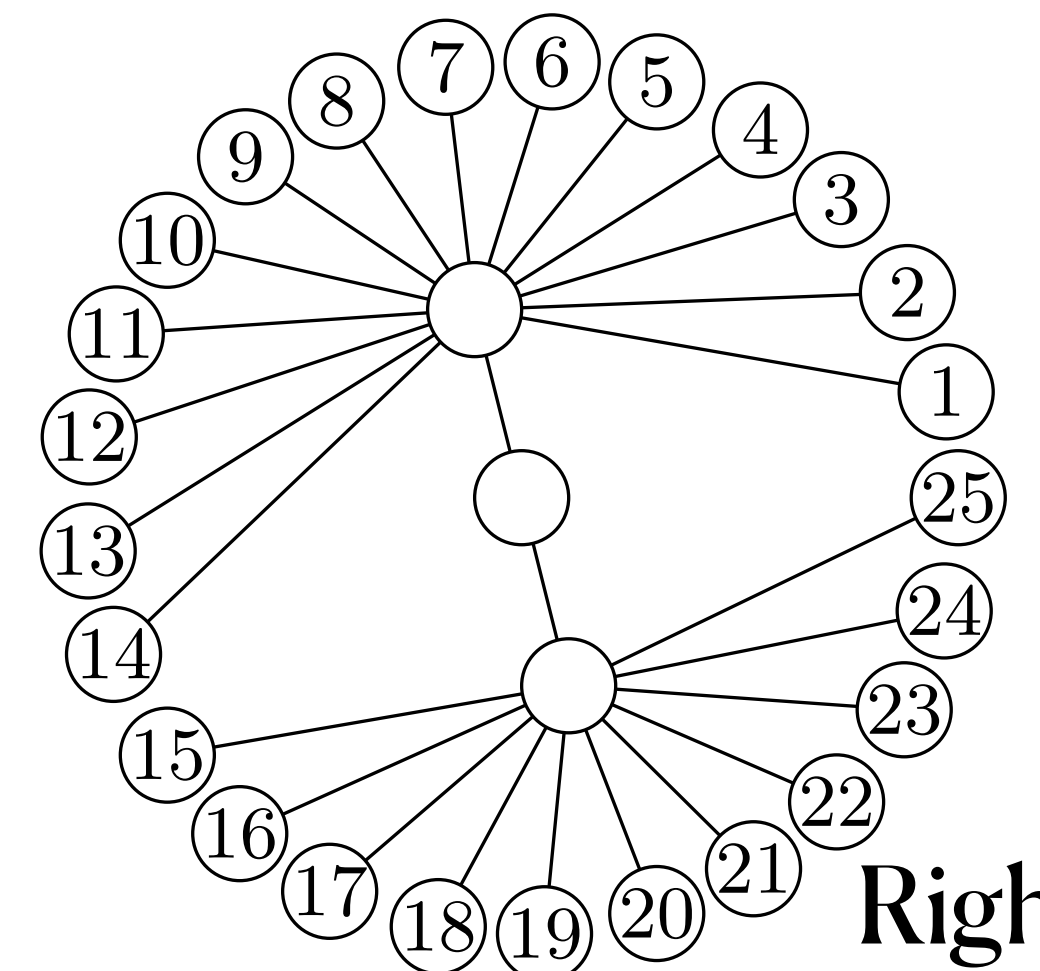
Lattice



Left
Merkle Tree



Horizontal
Merkle Tree

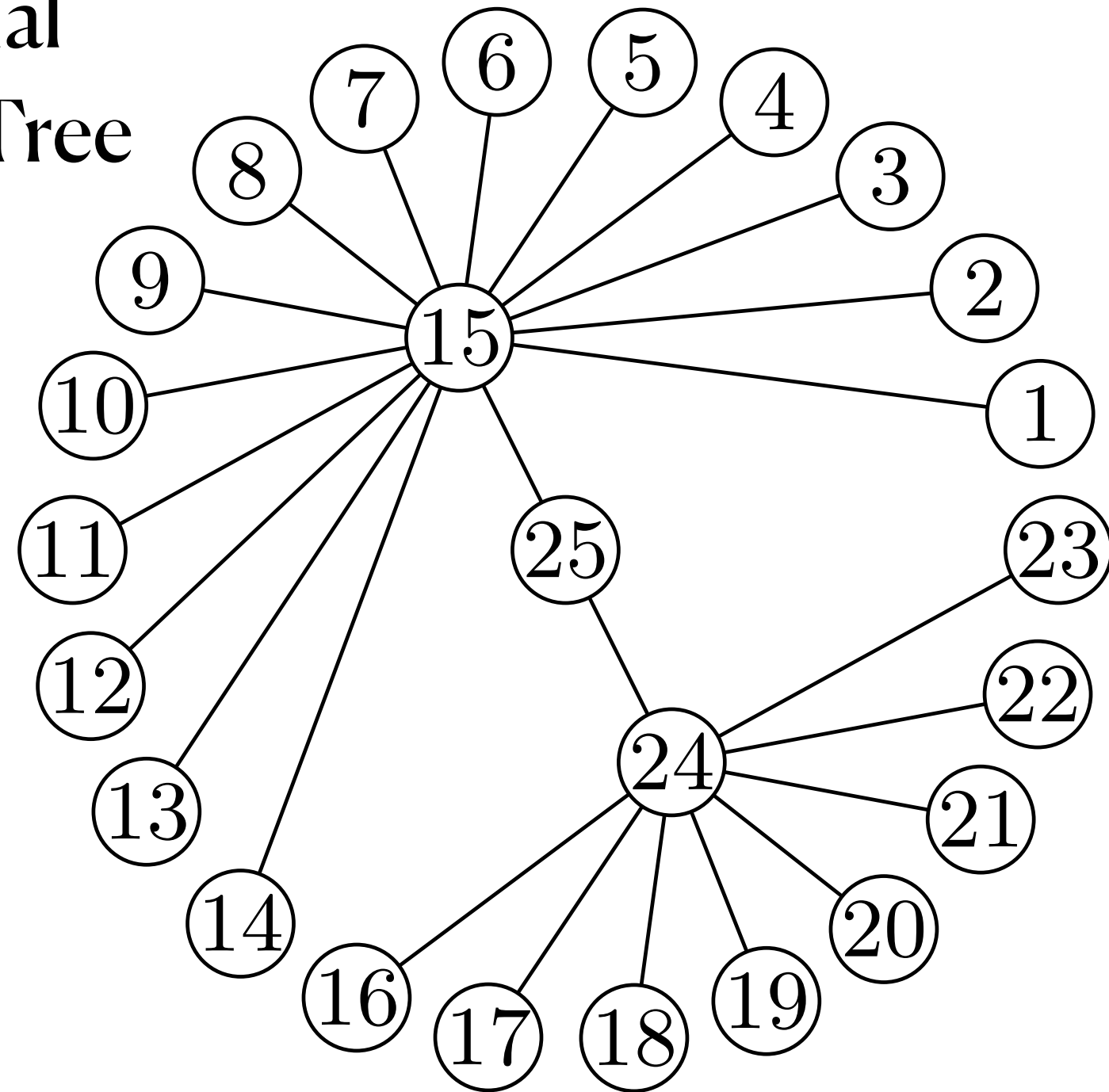


Right
Merkle Tree

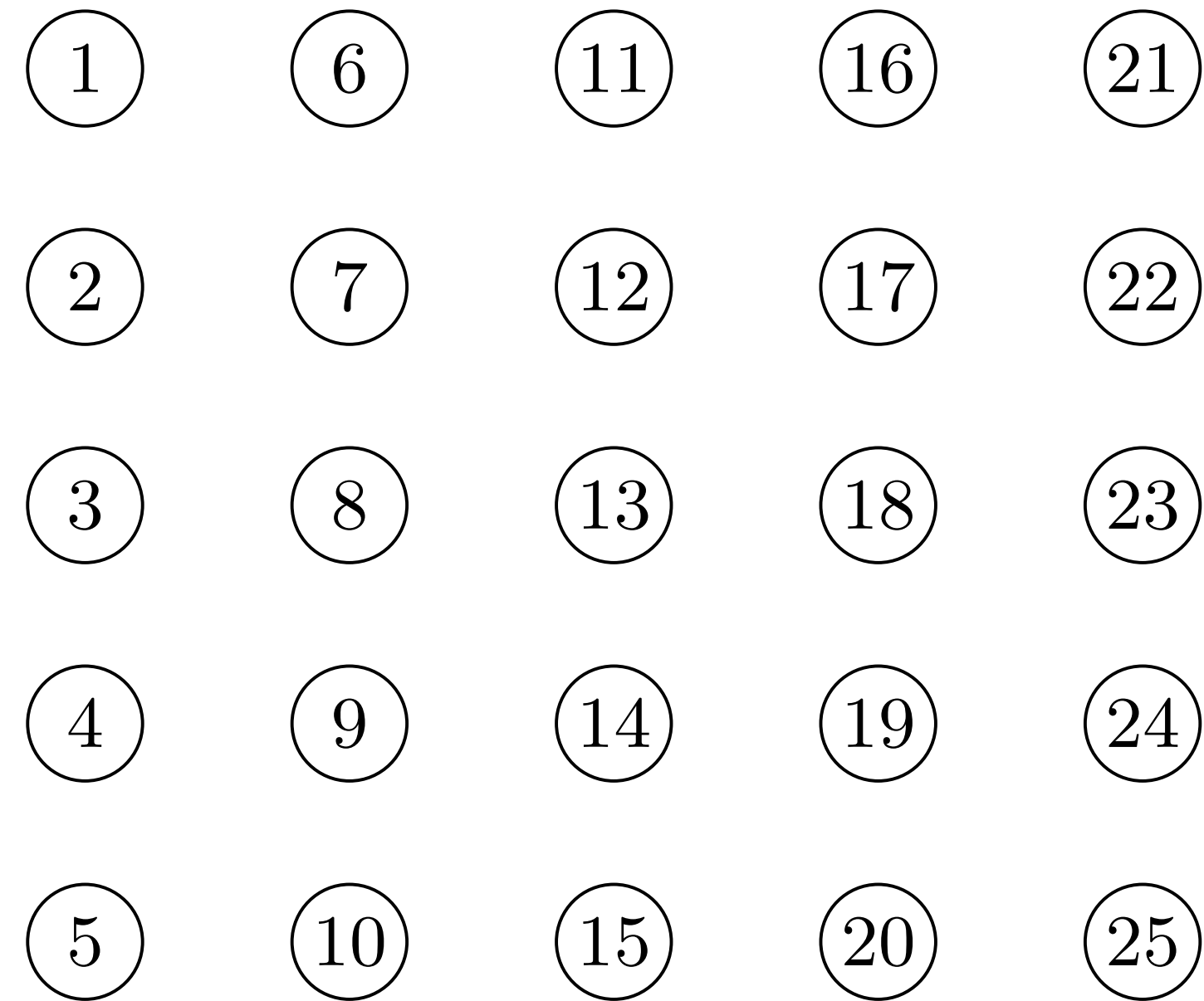
Downloading Chunks

Downloading Chunks

Original
Merkle Tree

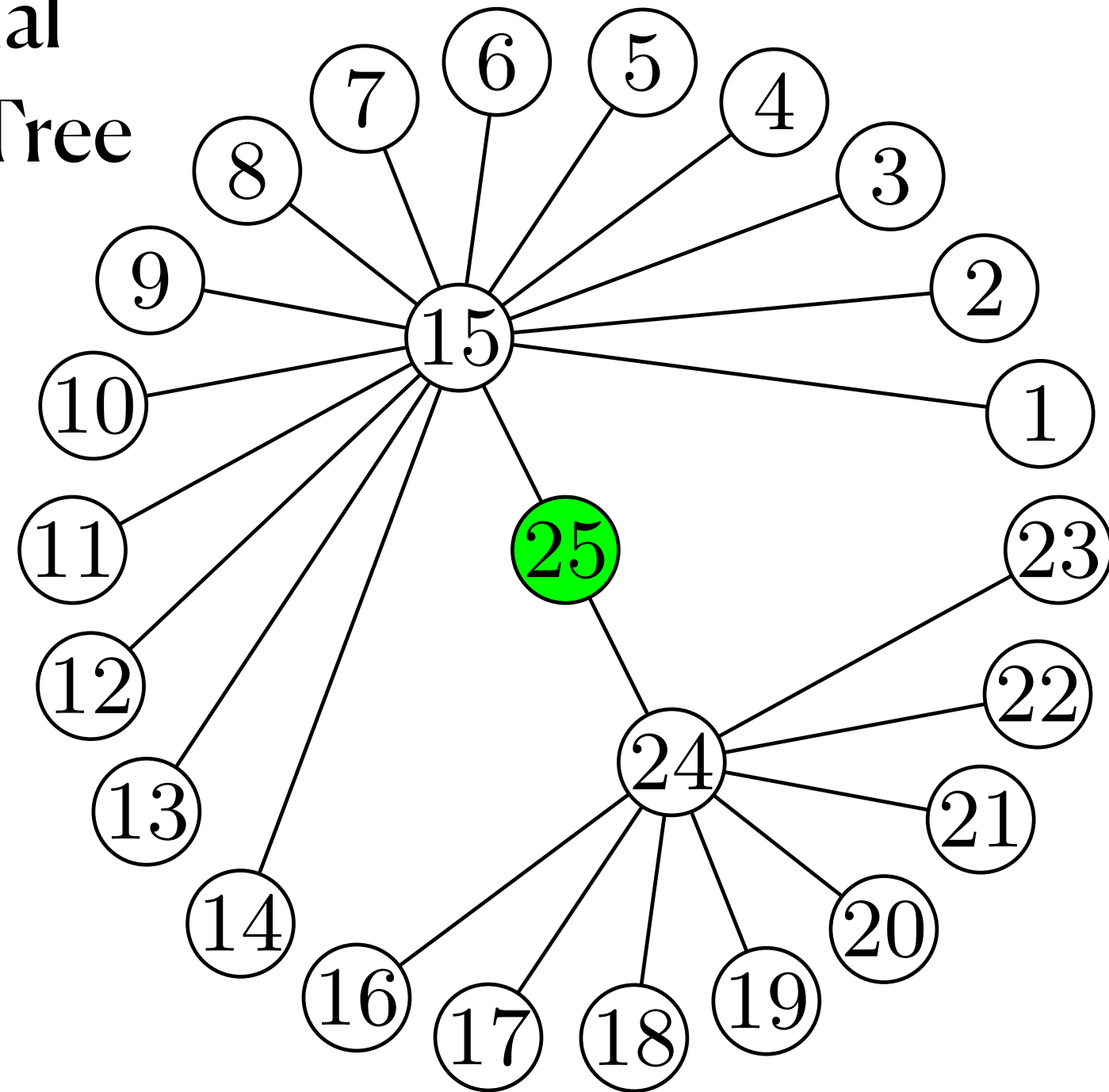


Lattice

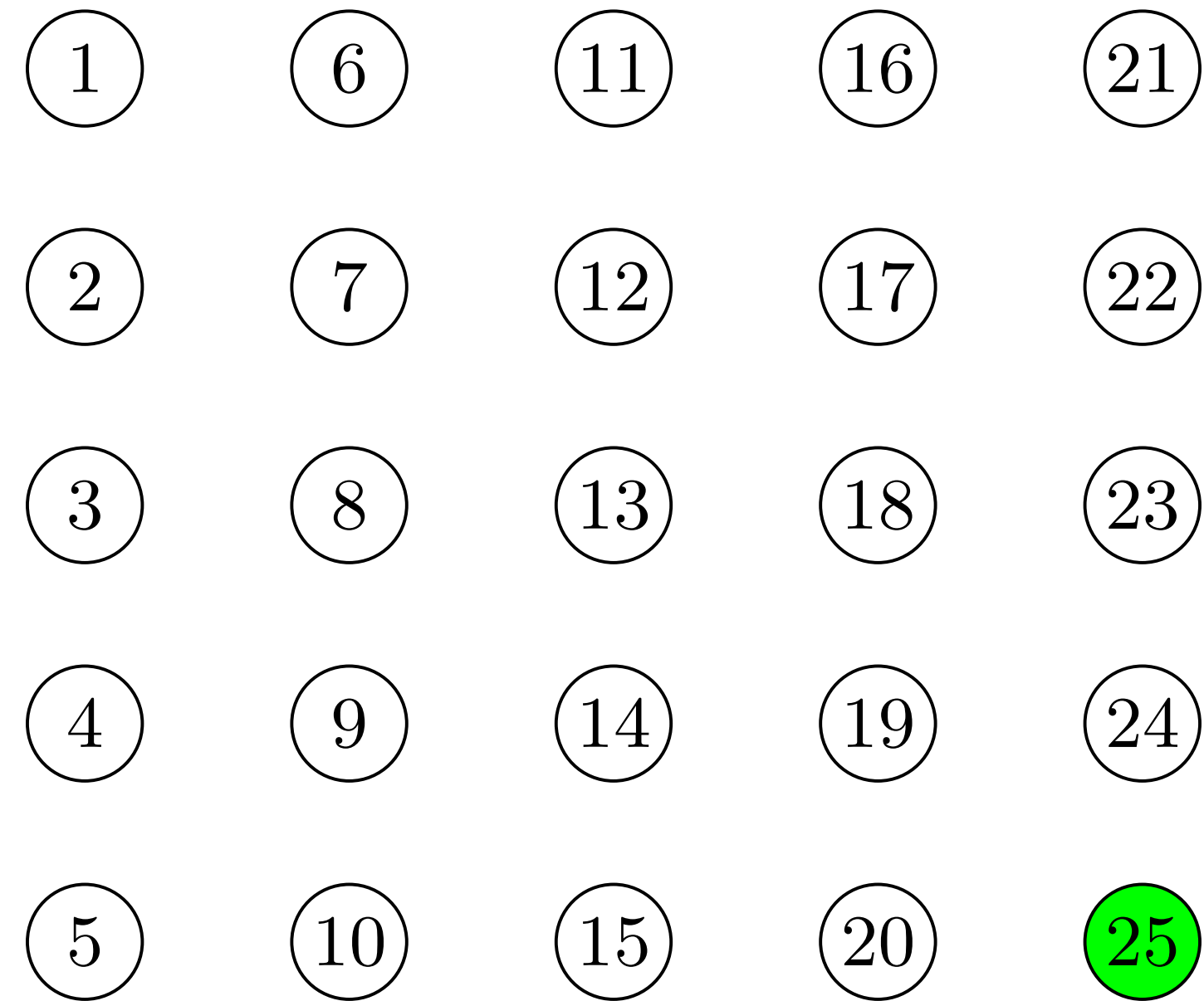


Downloading Chunks

Original
Merkle Tree

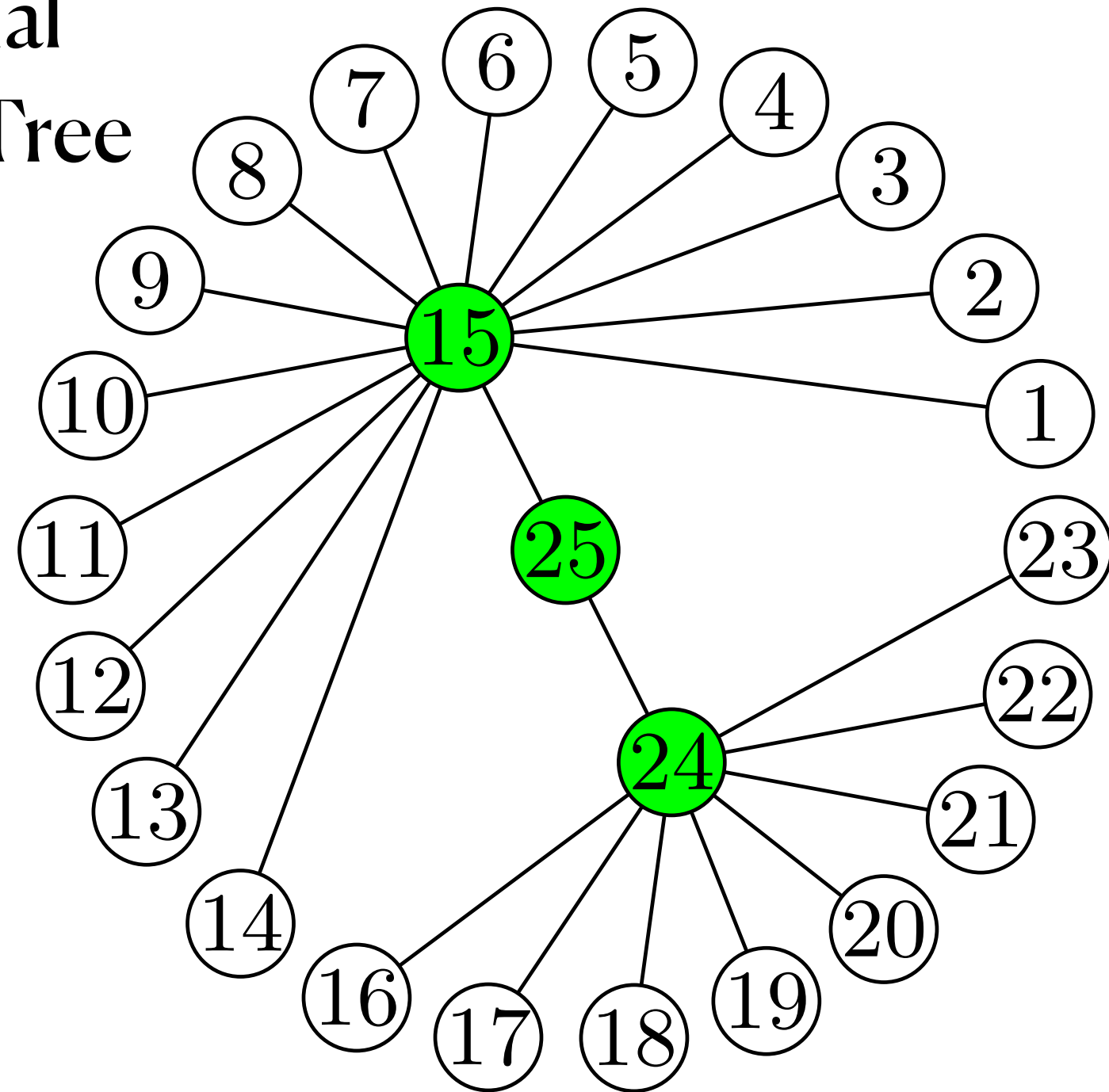


Lattice

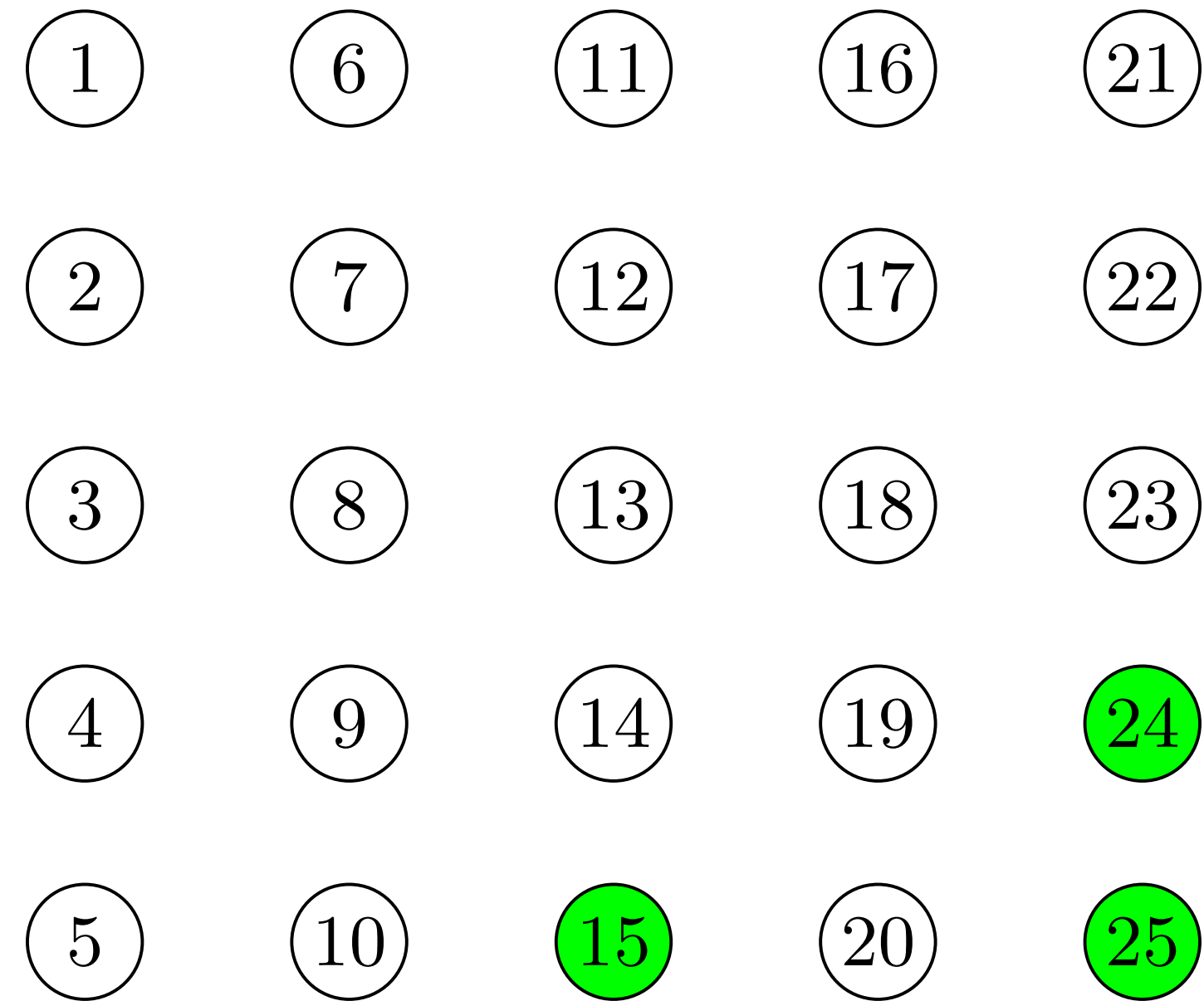


Downloading Chunks

Original
Merkle Tree

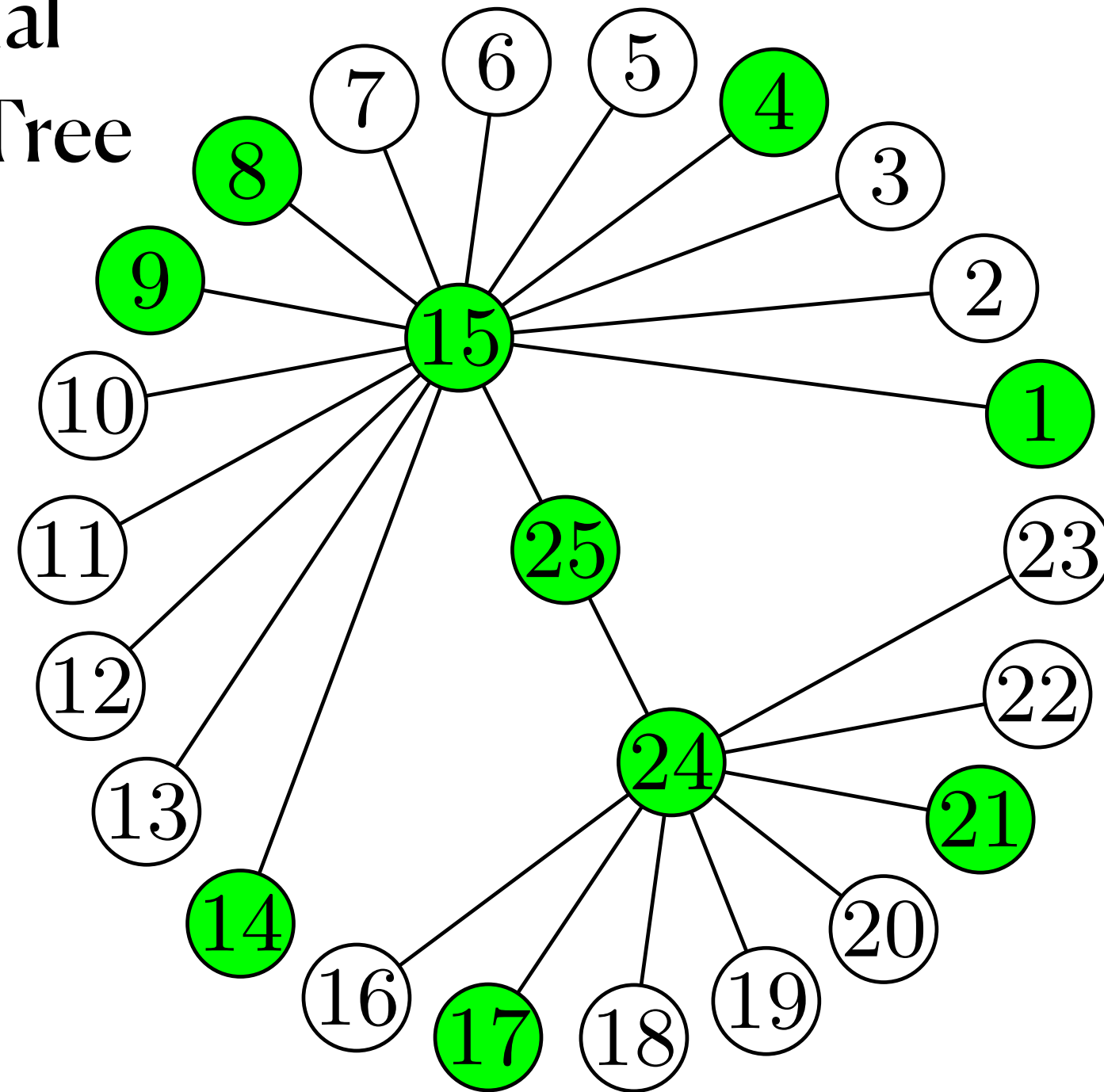


Lattice

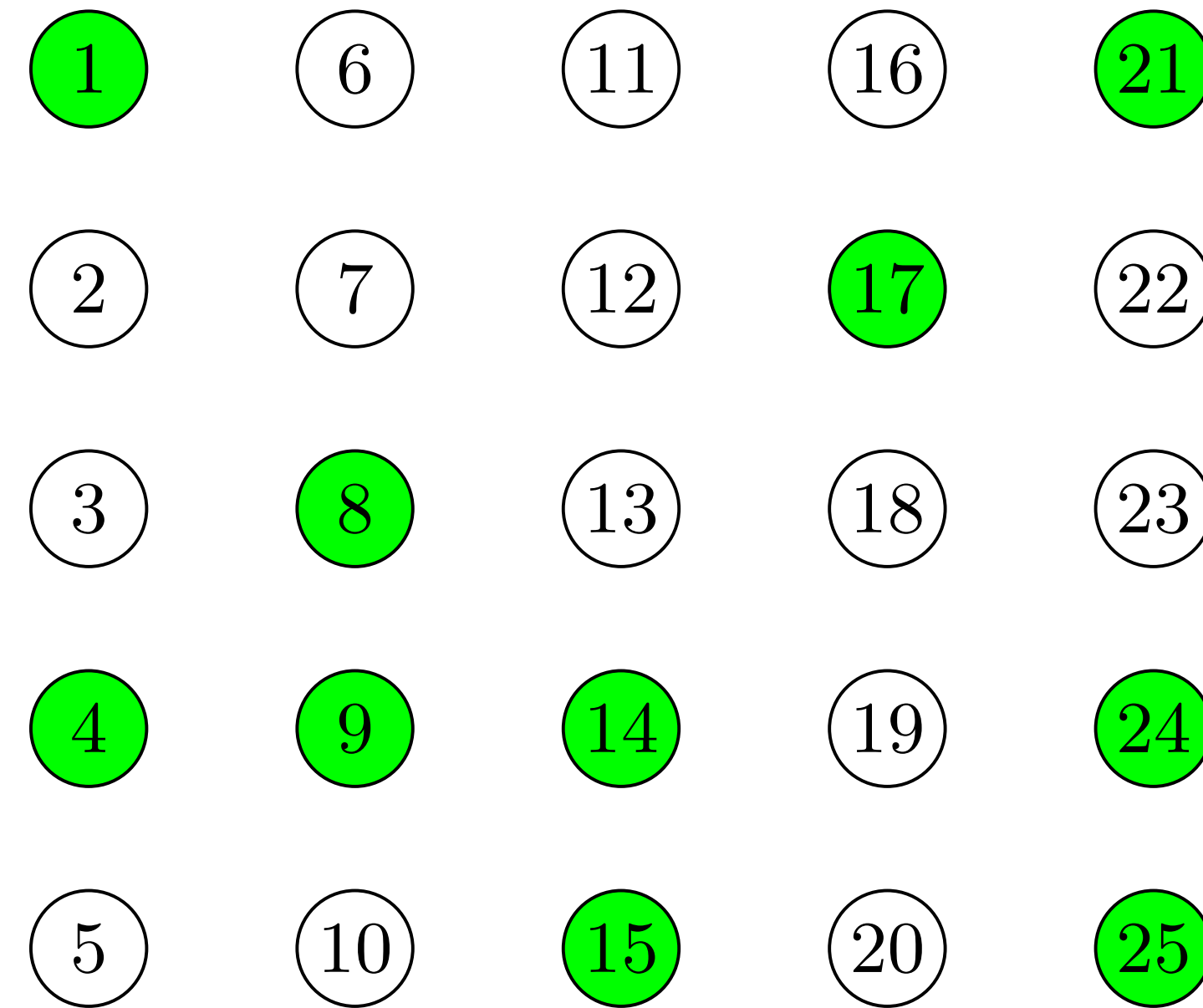


Downloading Chunks

Original
Merkle Tree

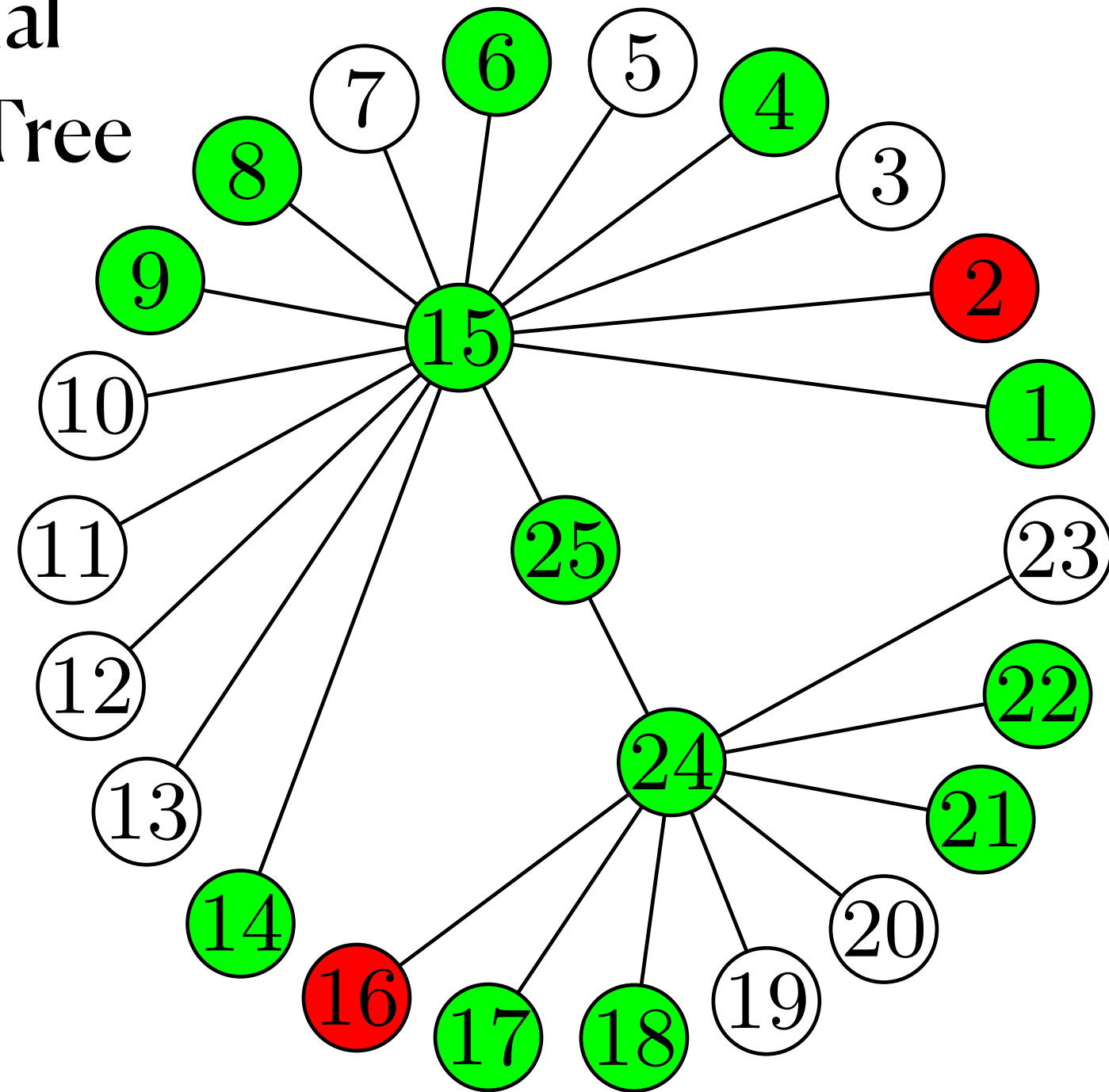


Lattice

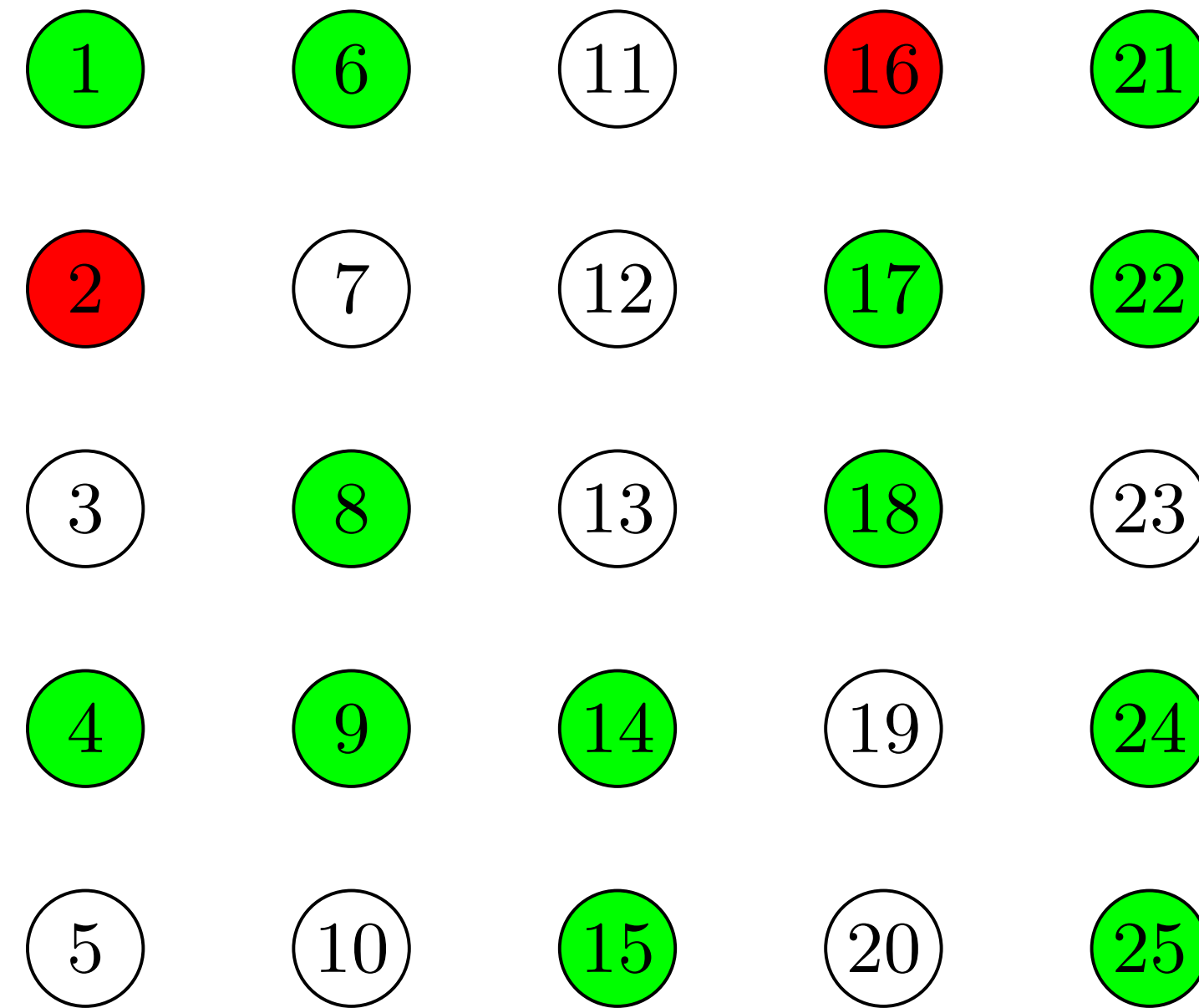


Downloading Chunks

Original
Merkle Tree

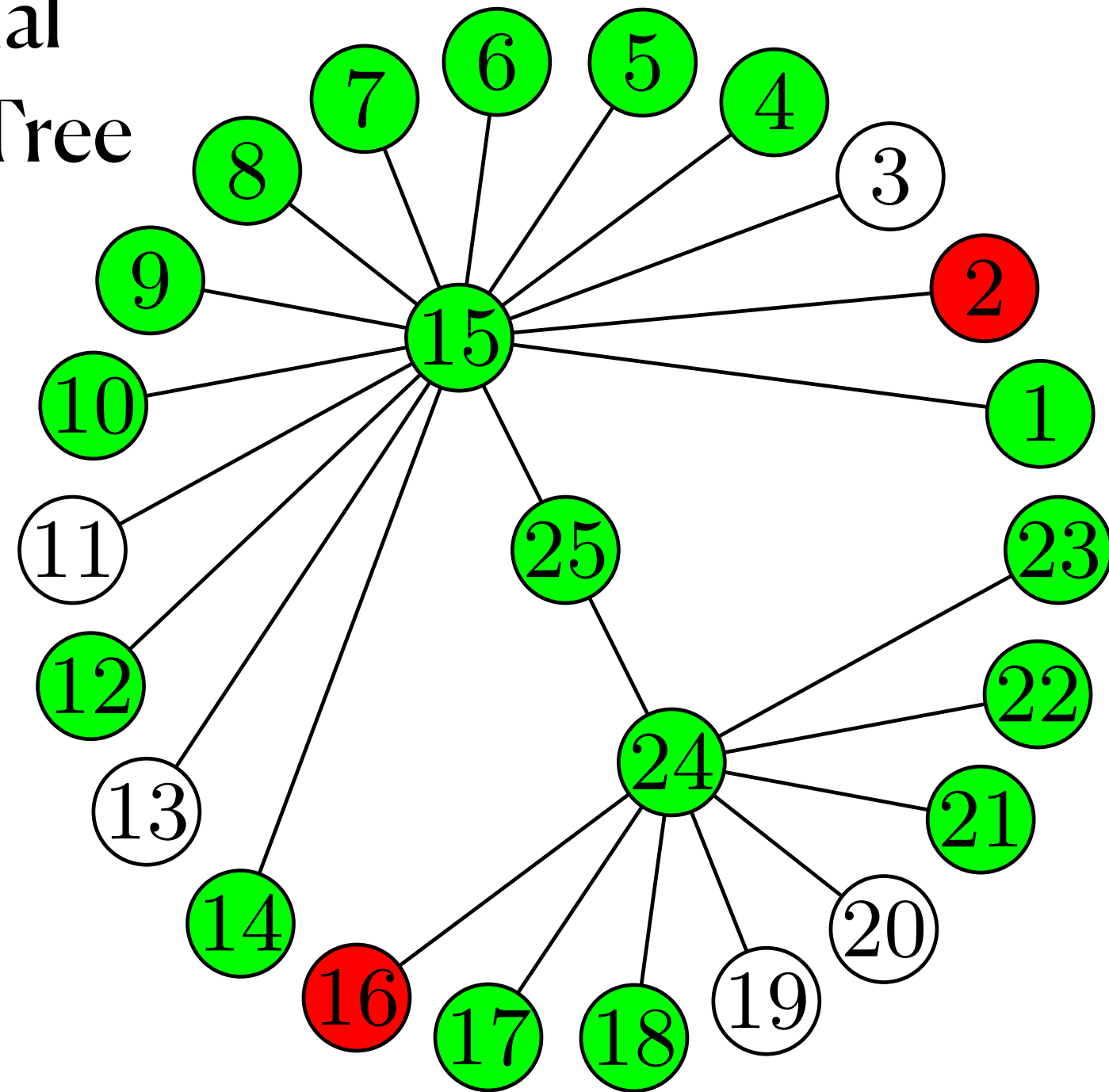


Lattice

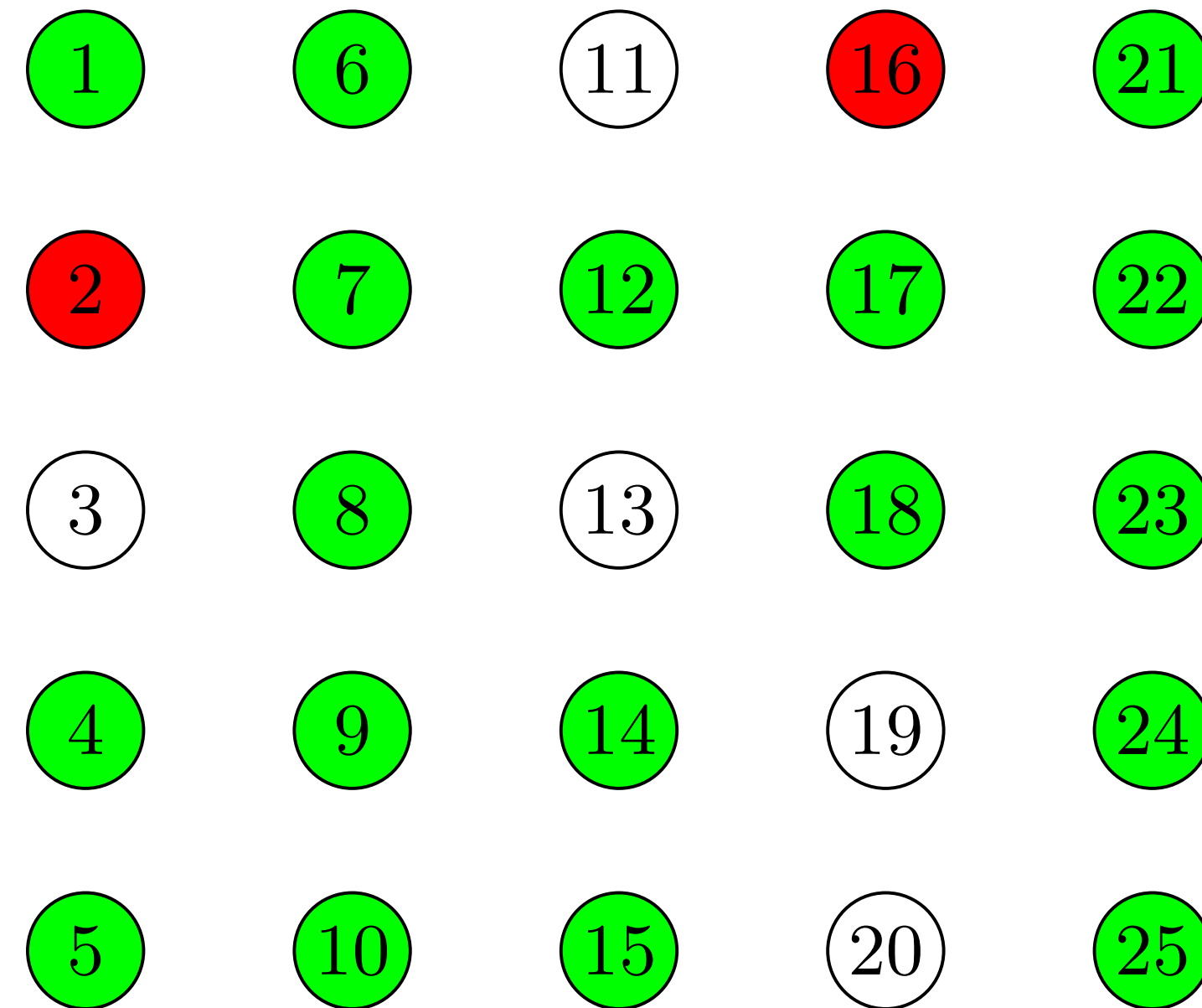


Downloading Chunks

Original
Merkle Tree

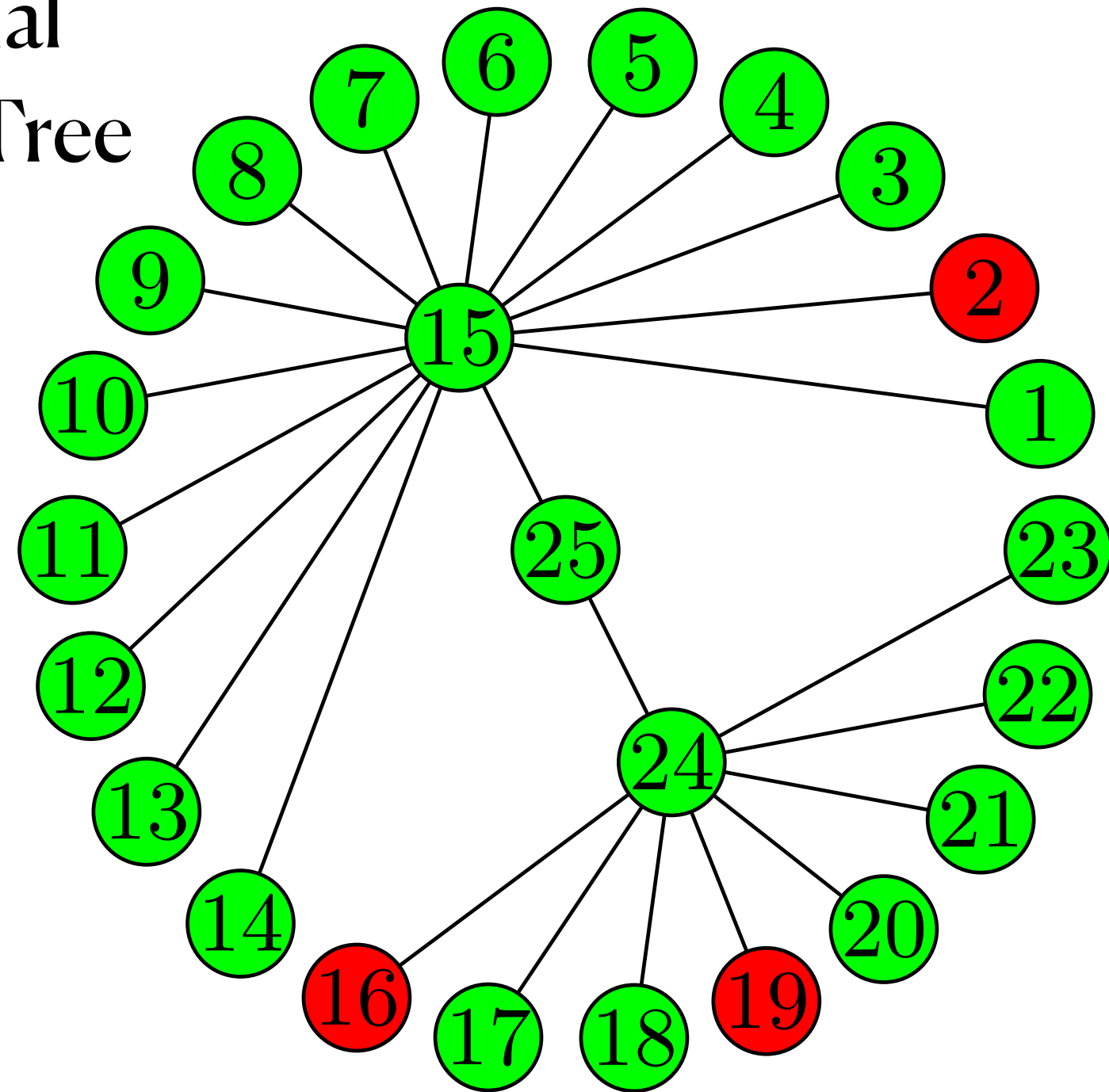


Lattice

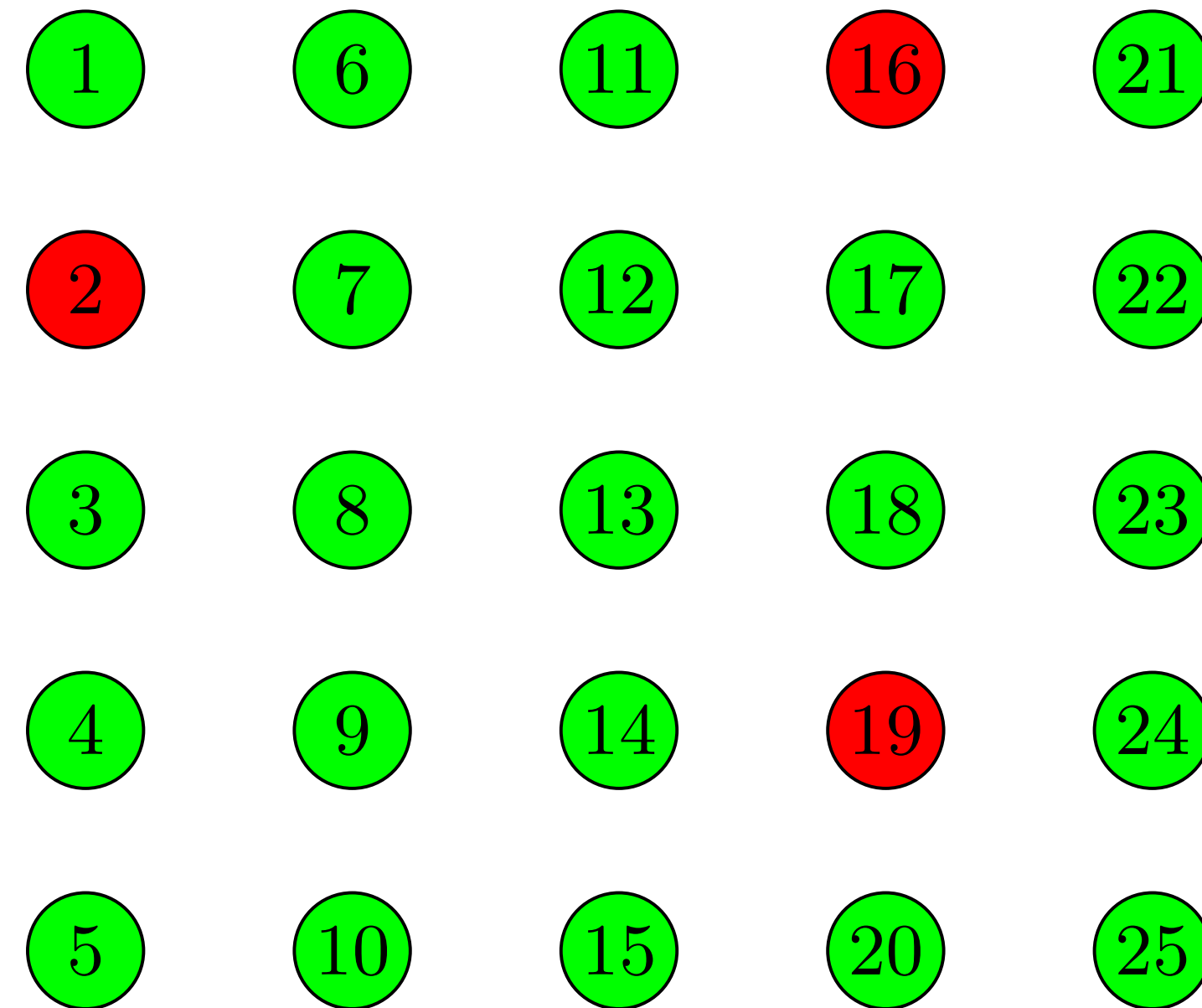


Downloading Chunks

Original
Merkle Tree



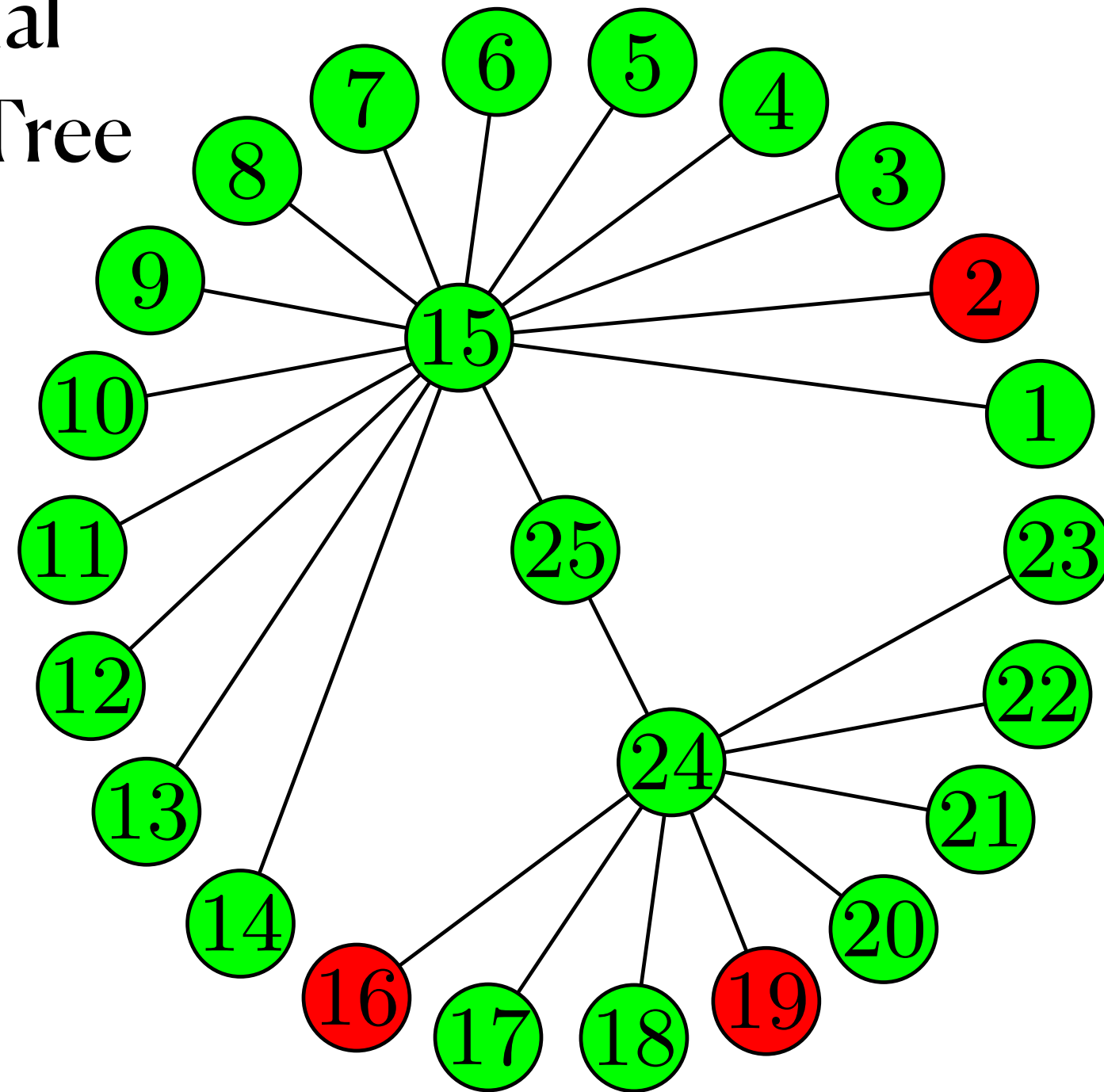
Lattice



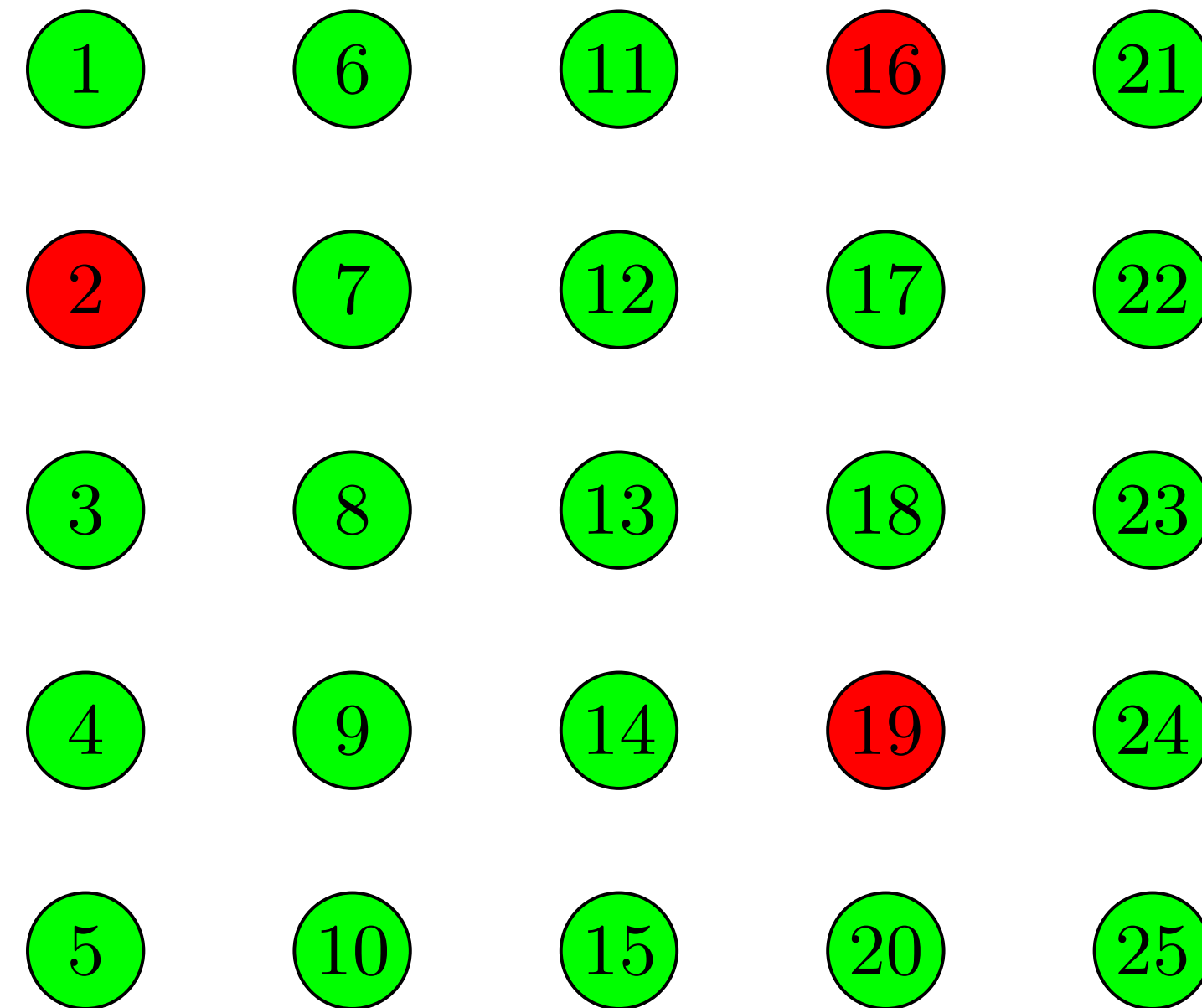
Repair Missing Chunks

Repair Missing Chunks

Original
Merkle Tree

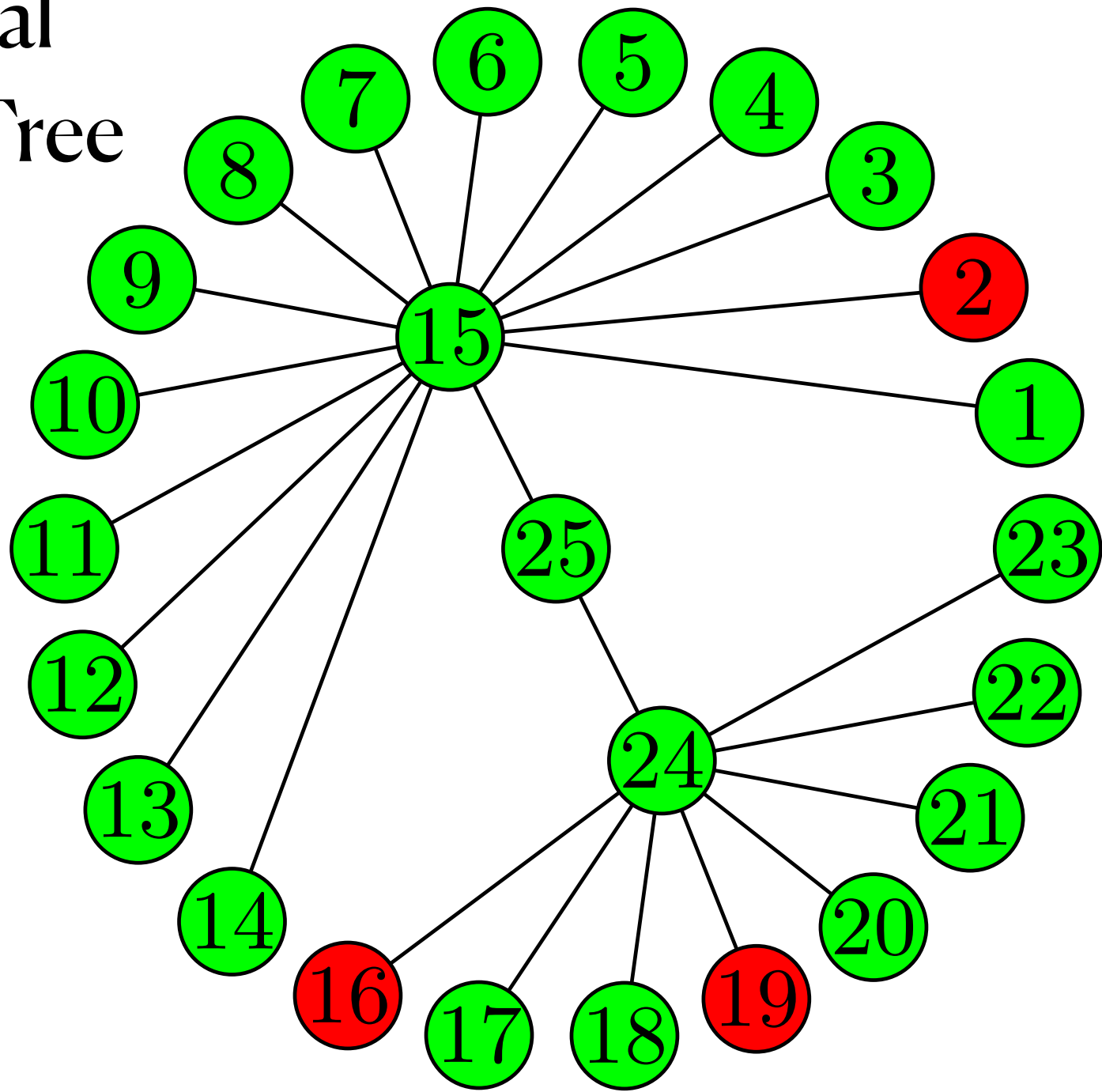


Lattice

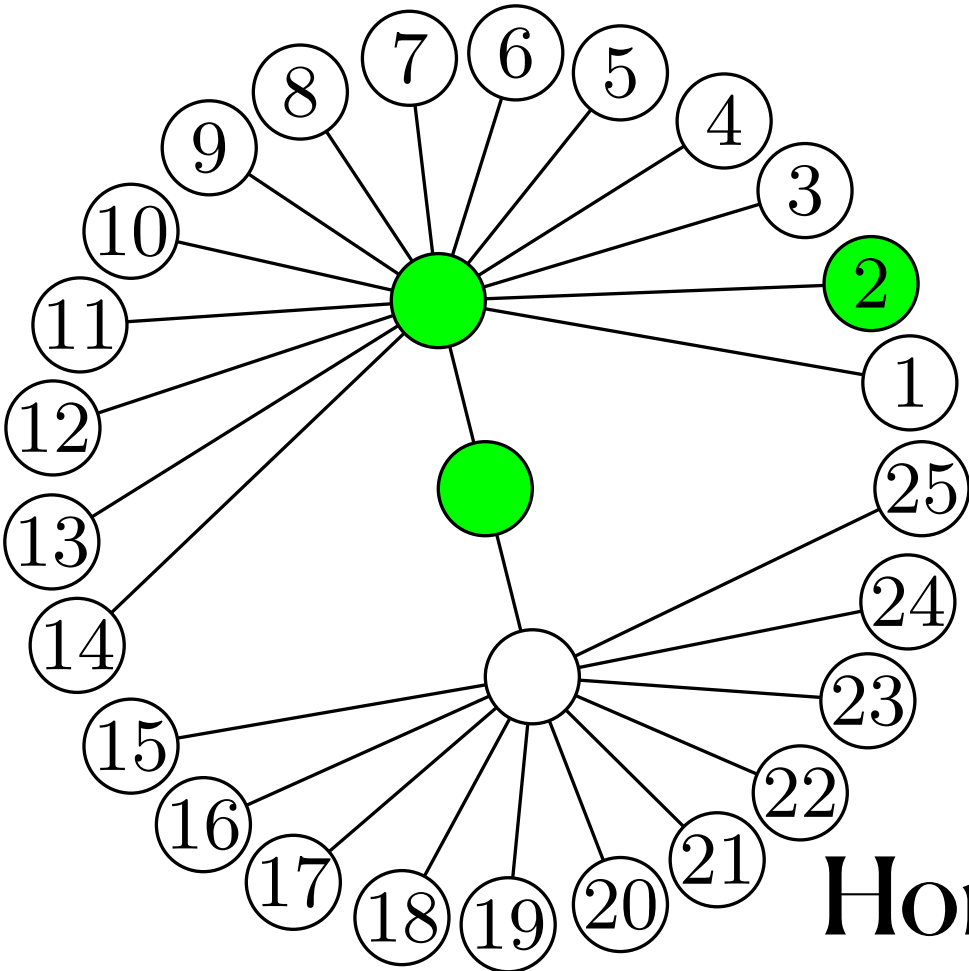
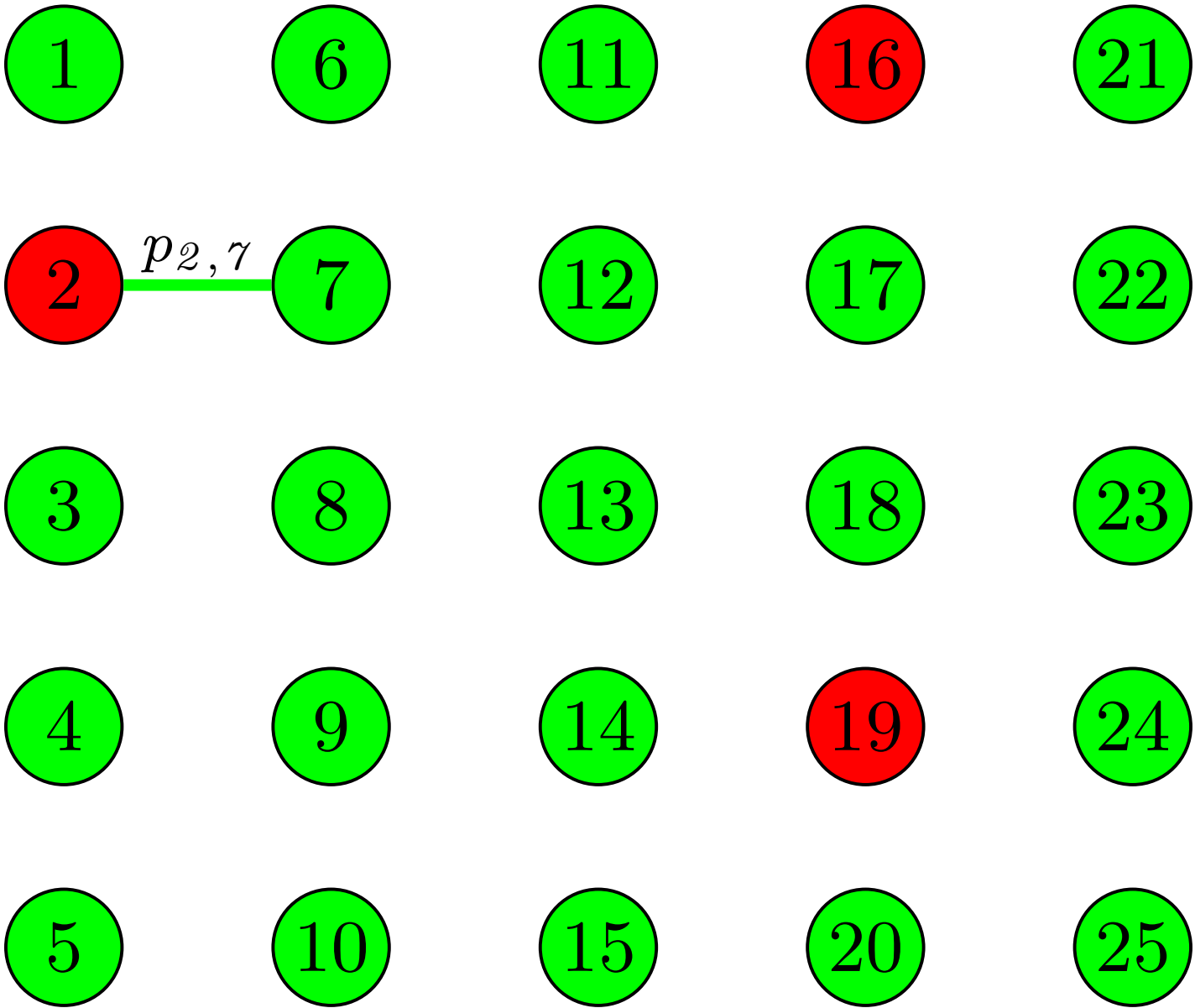


Repairing Data Chunk 2

Original
Merkle Tree



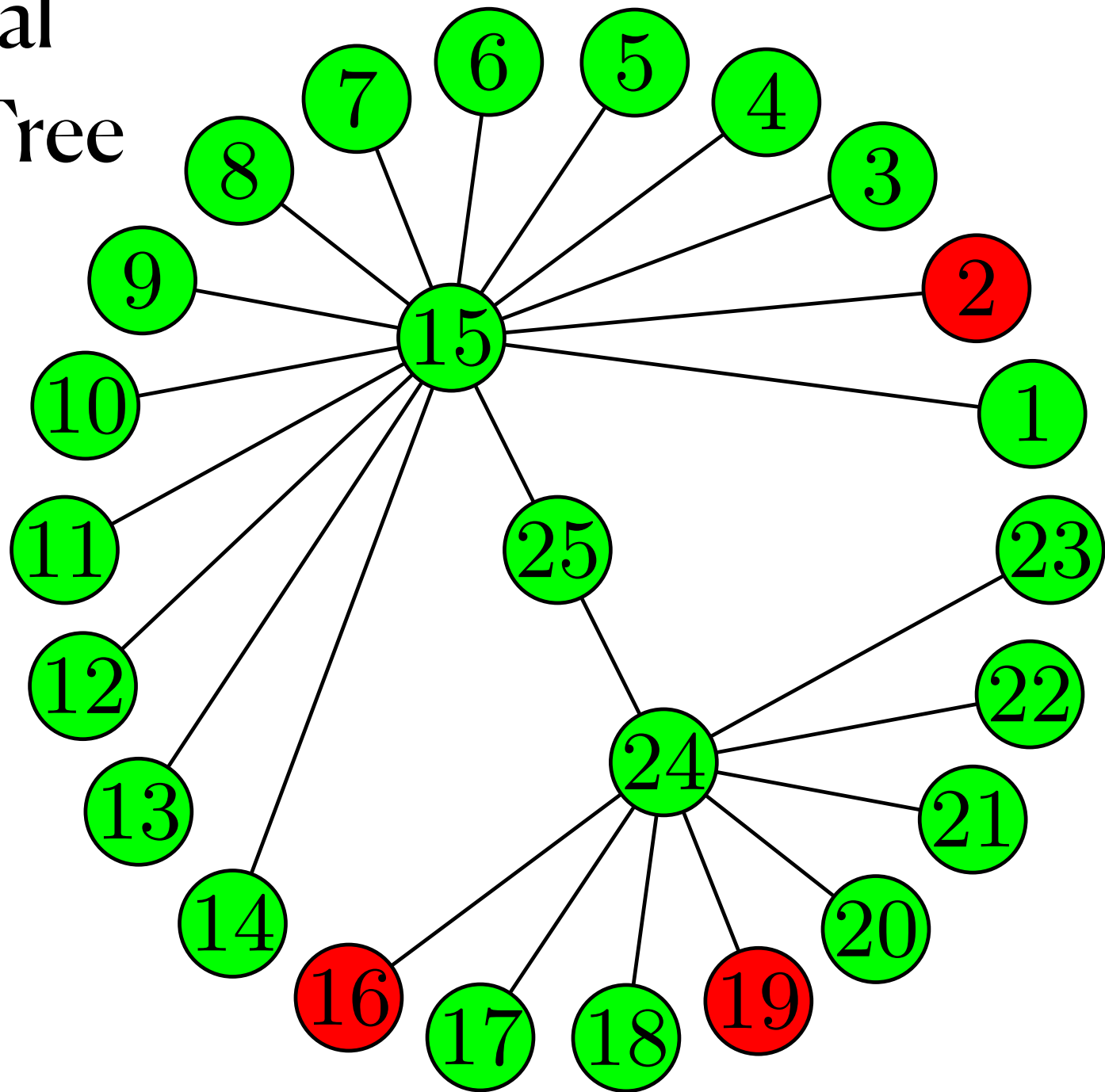
Lattice



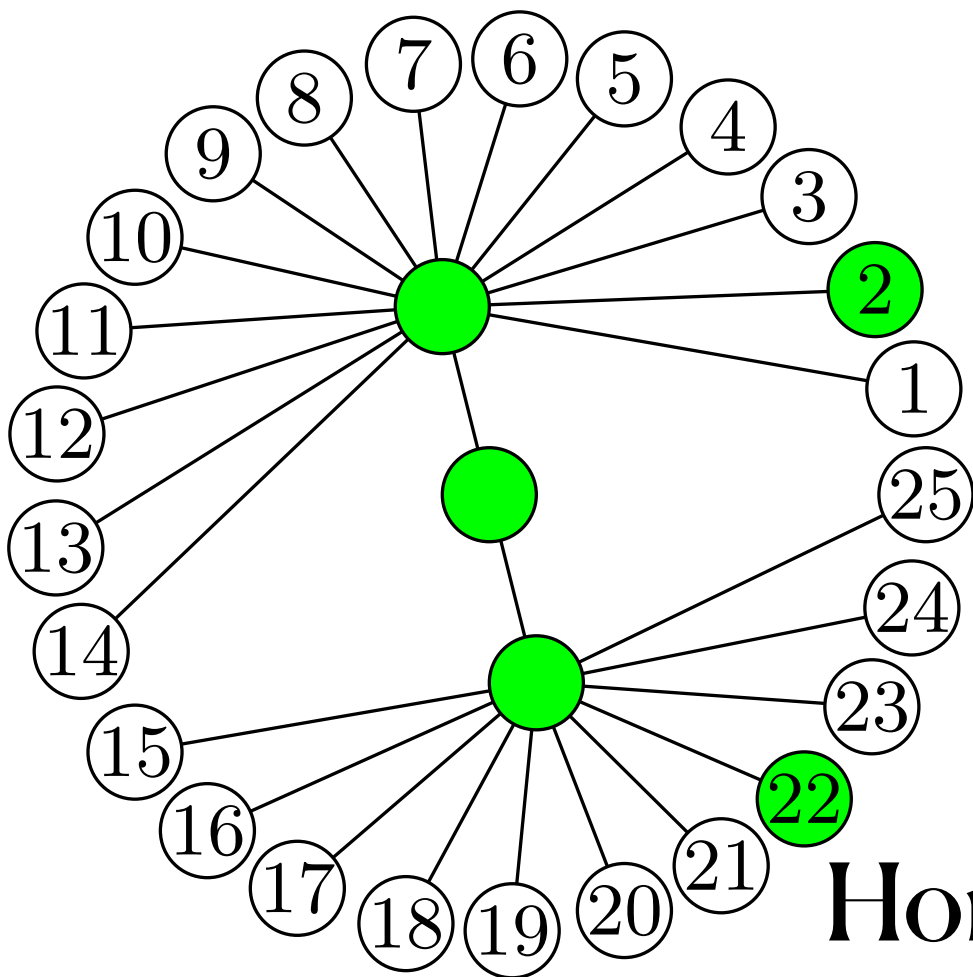
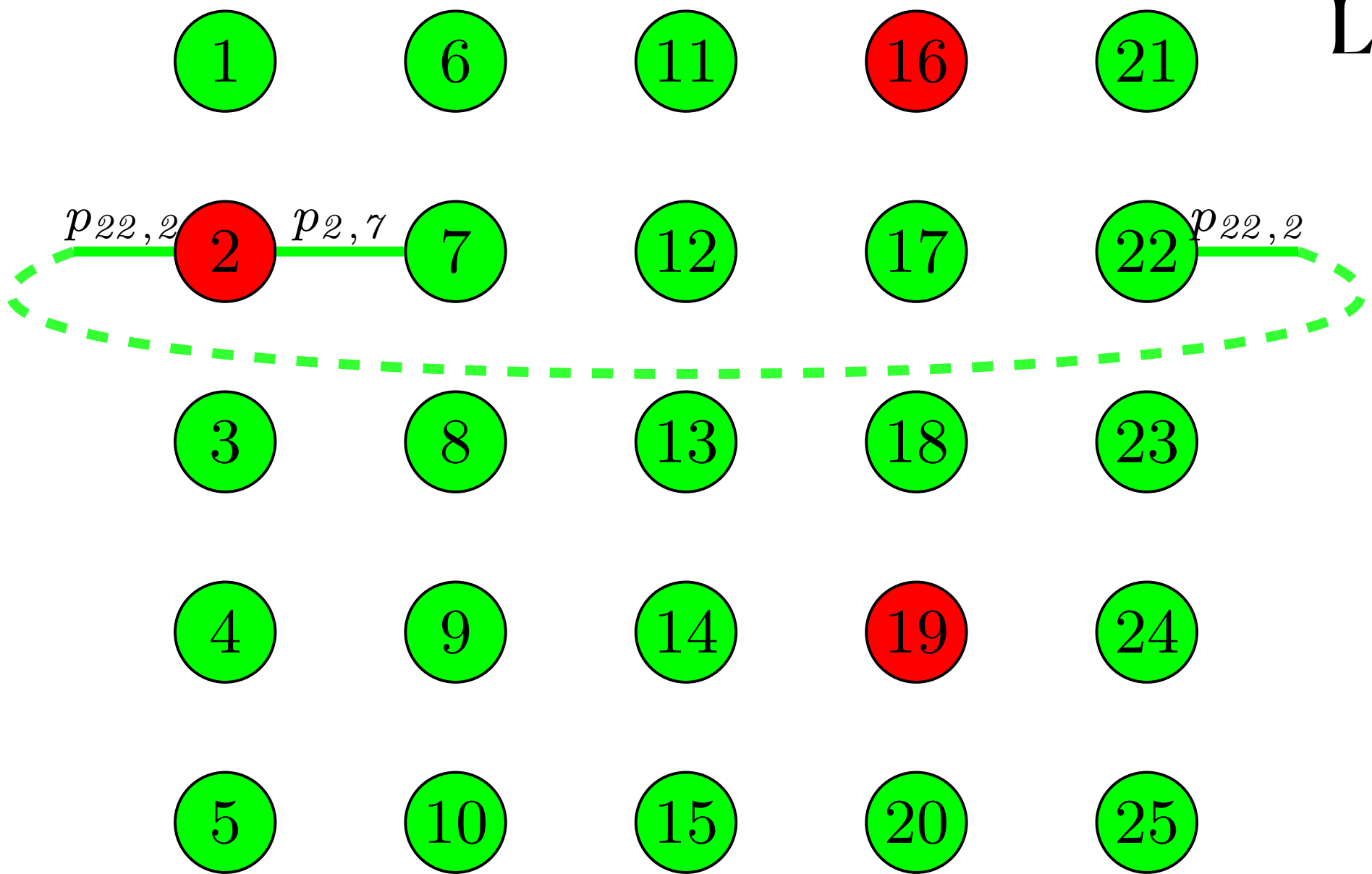
Horizontal
Merkle Tree

Repairing Data Chunk 2

Original
Merkle Tree



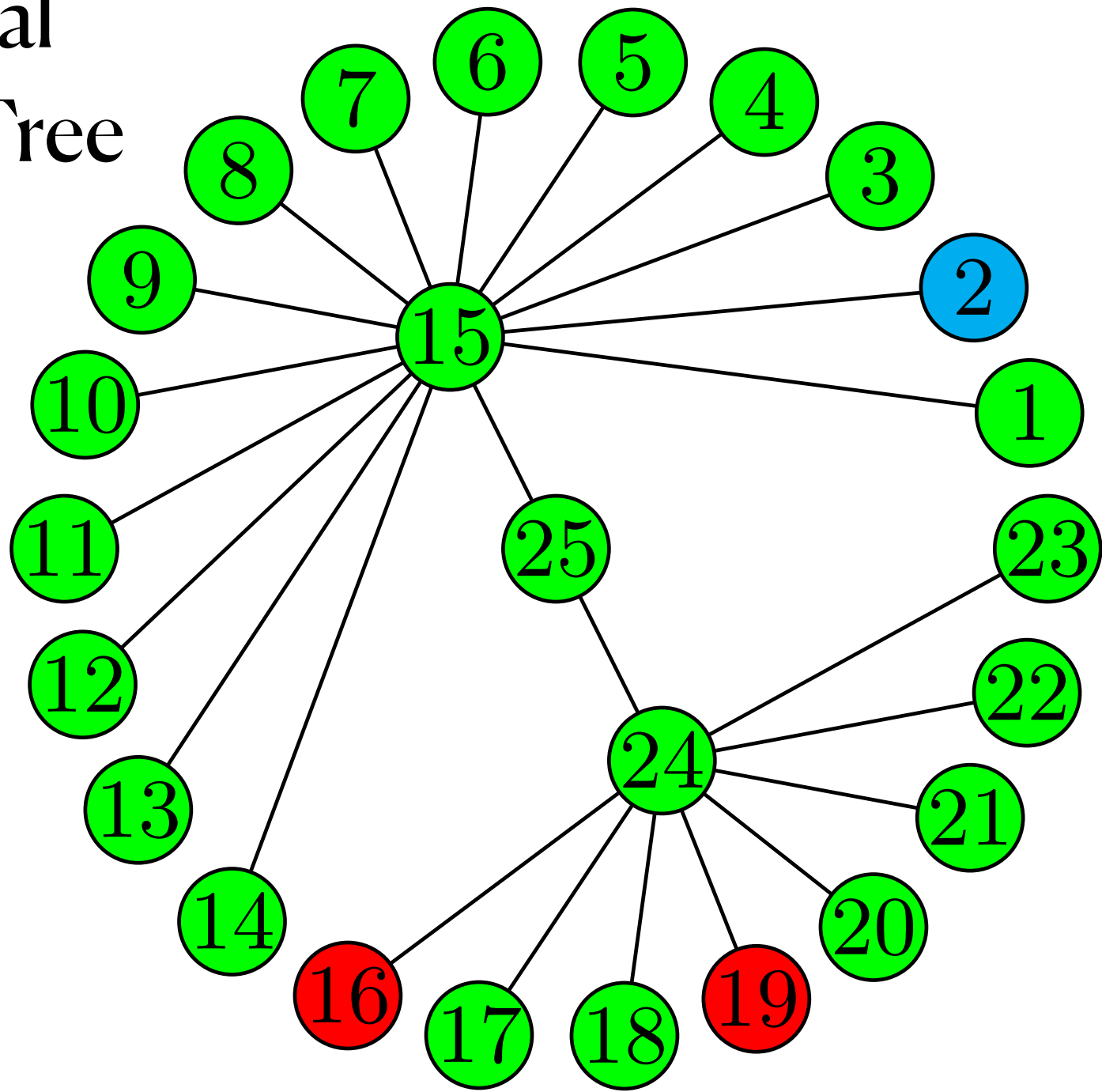
Lattice



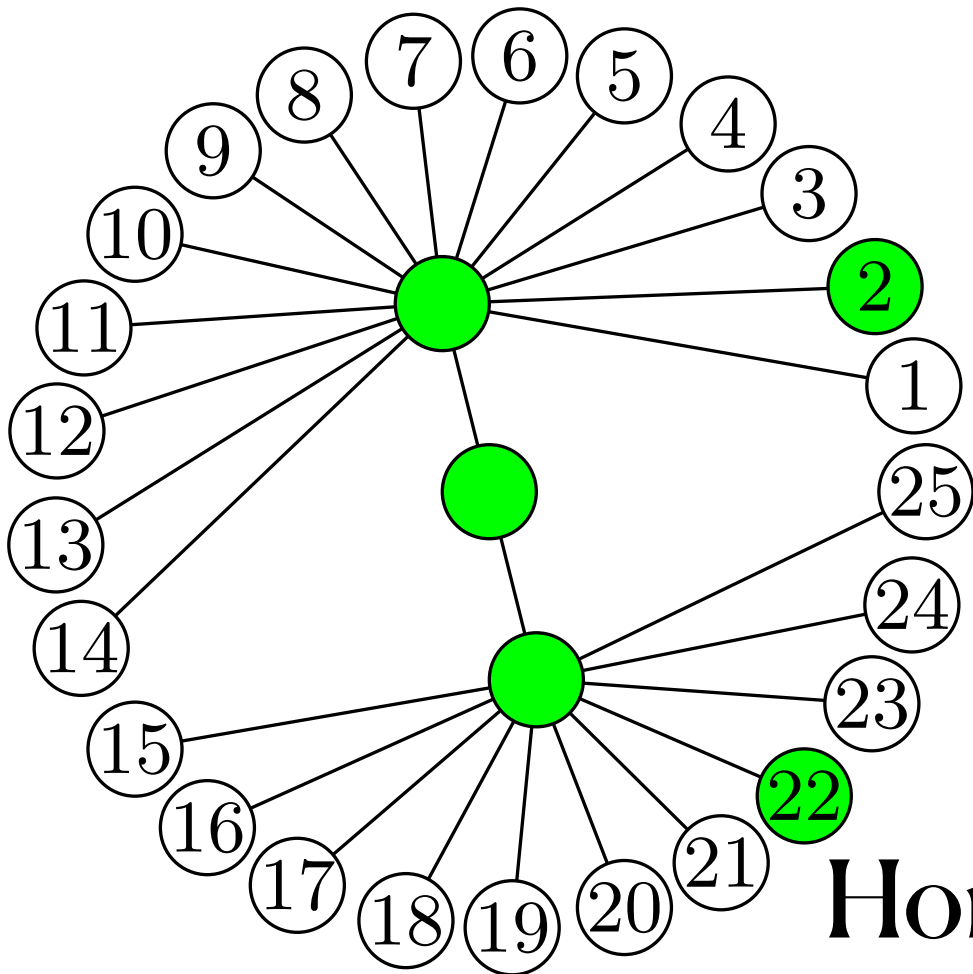
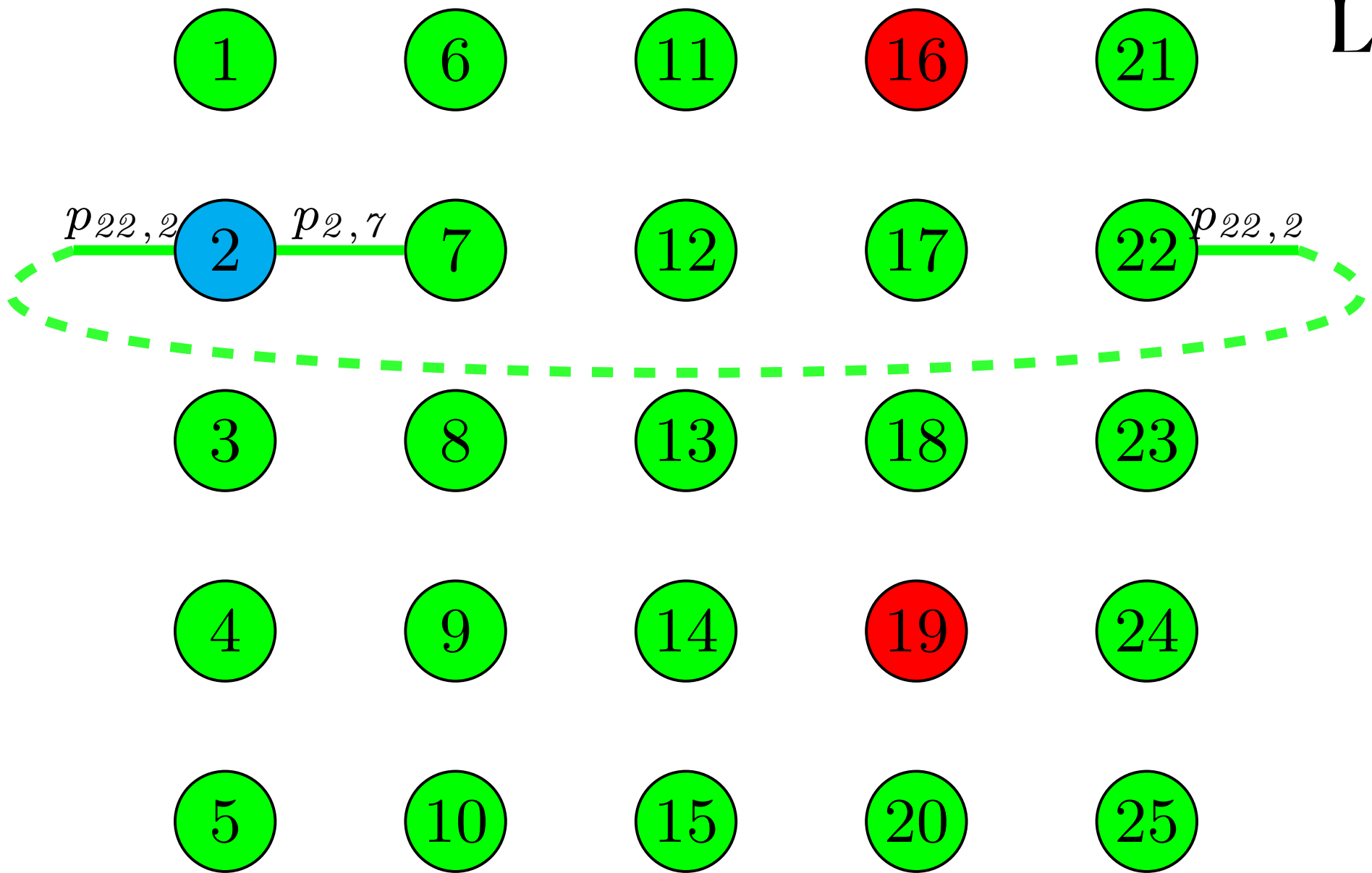
Horizontal
Merkle Tree

Repaired Data Chunk 2

Original
Merkle Tree



Lattice

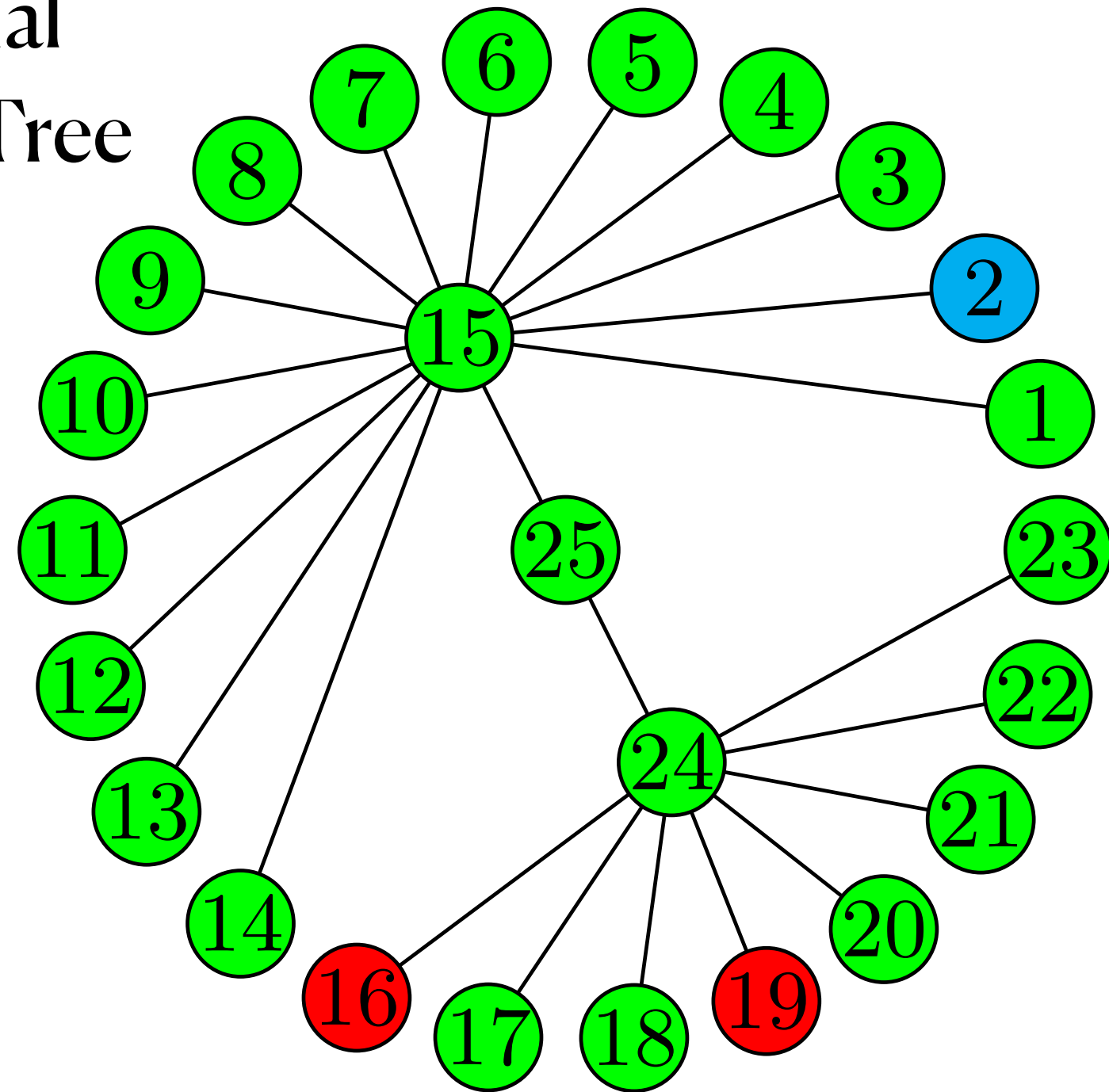


Horizontal
Merkle Tree

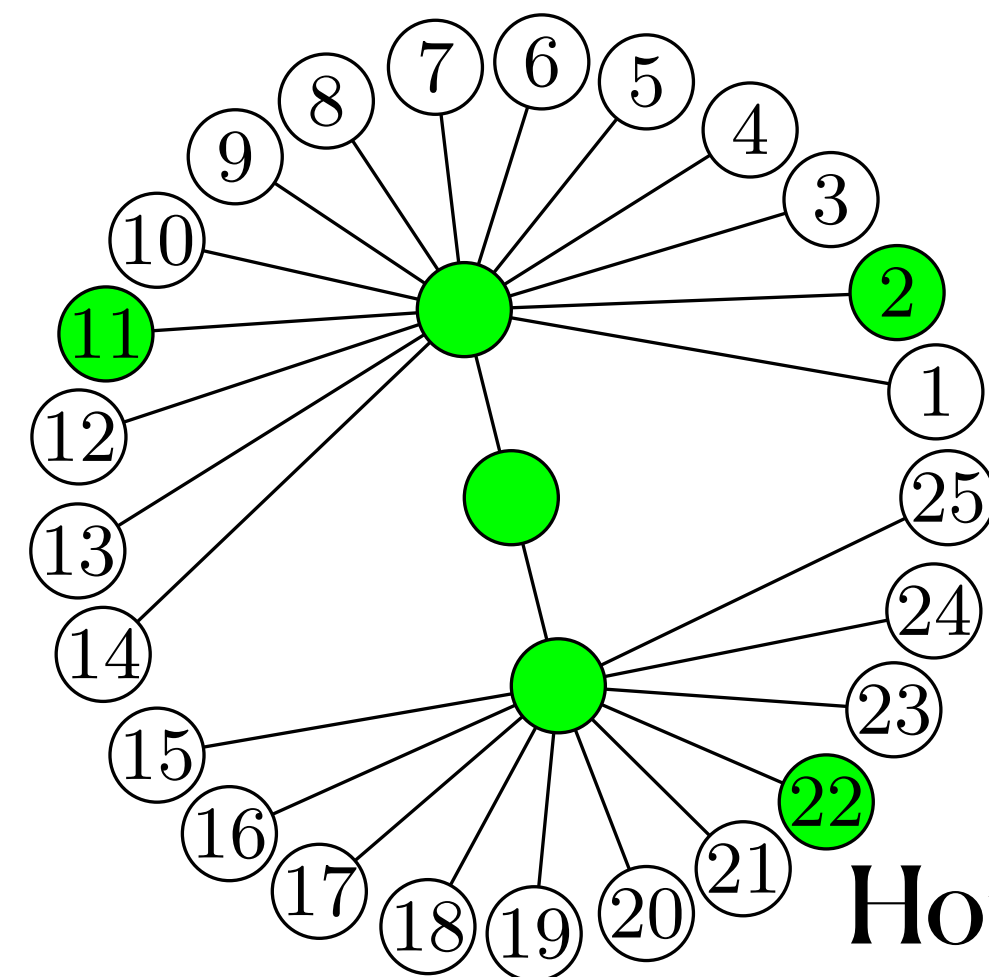
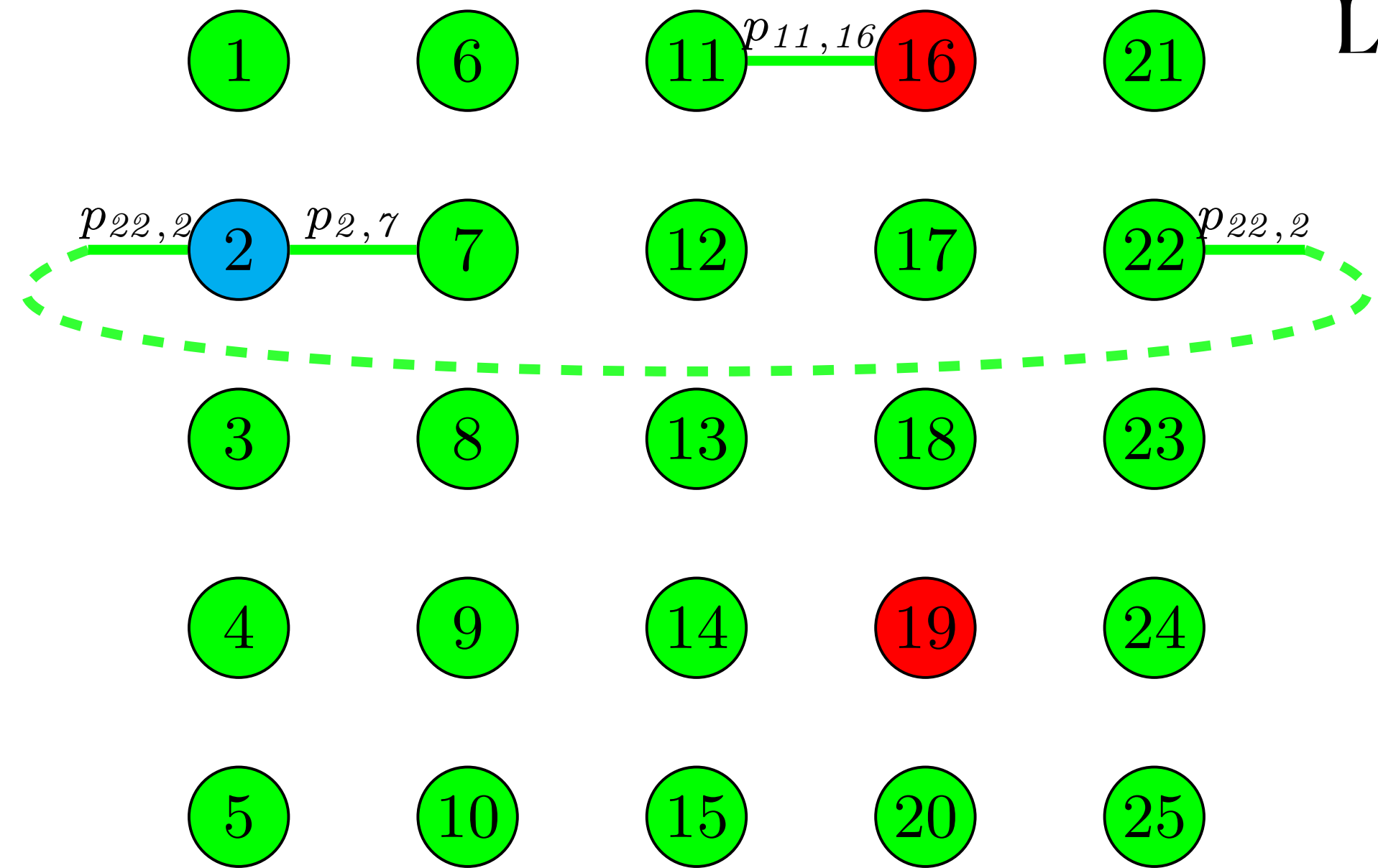
Repairing Data Chunk 16

Repairing Data Chunk 16

Original
Merkle Tree



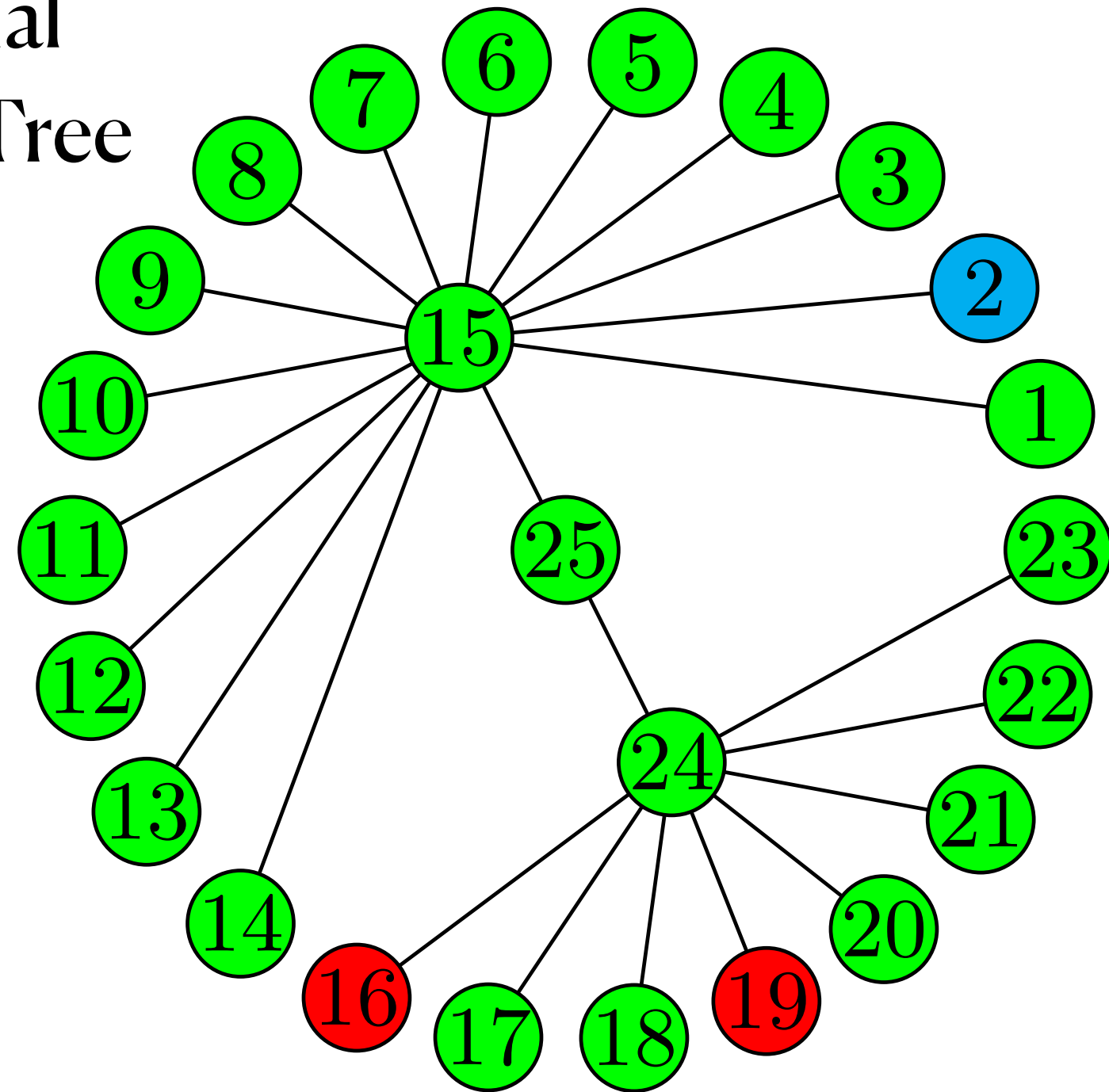
Lattice



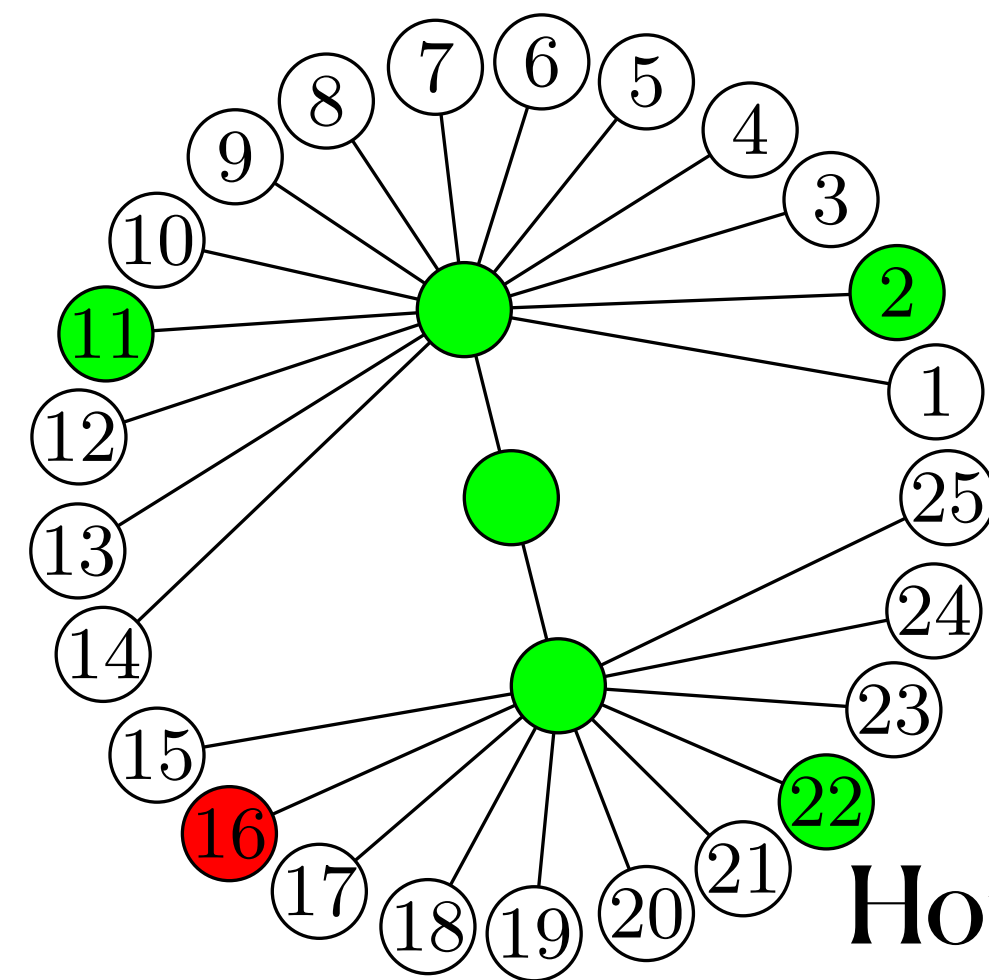
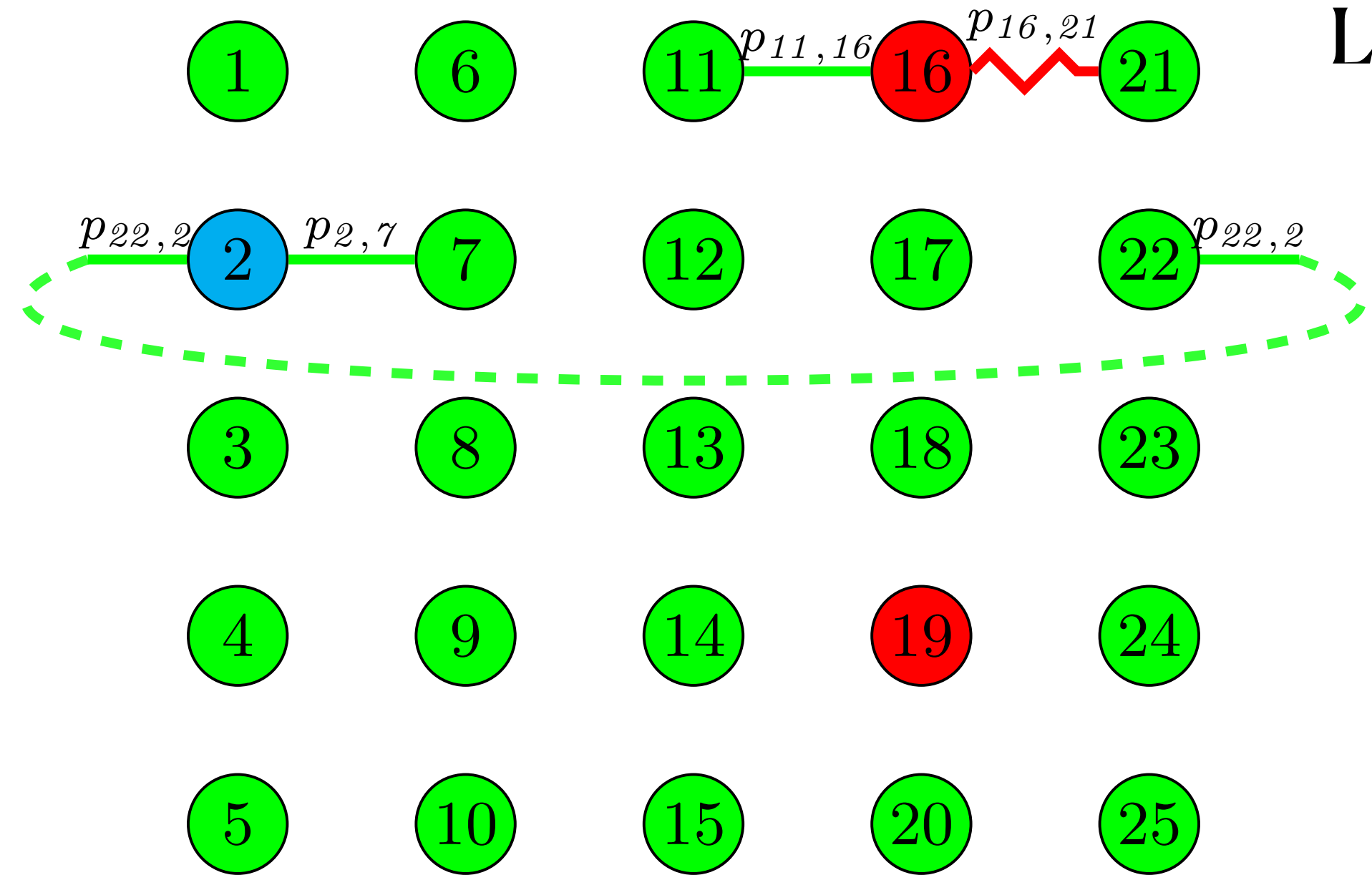
Horizontal
Merkle Tree

Repairing Data Chunk 16

Original
Merkle Tree



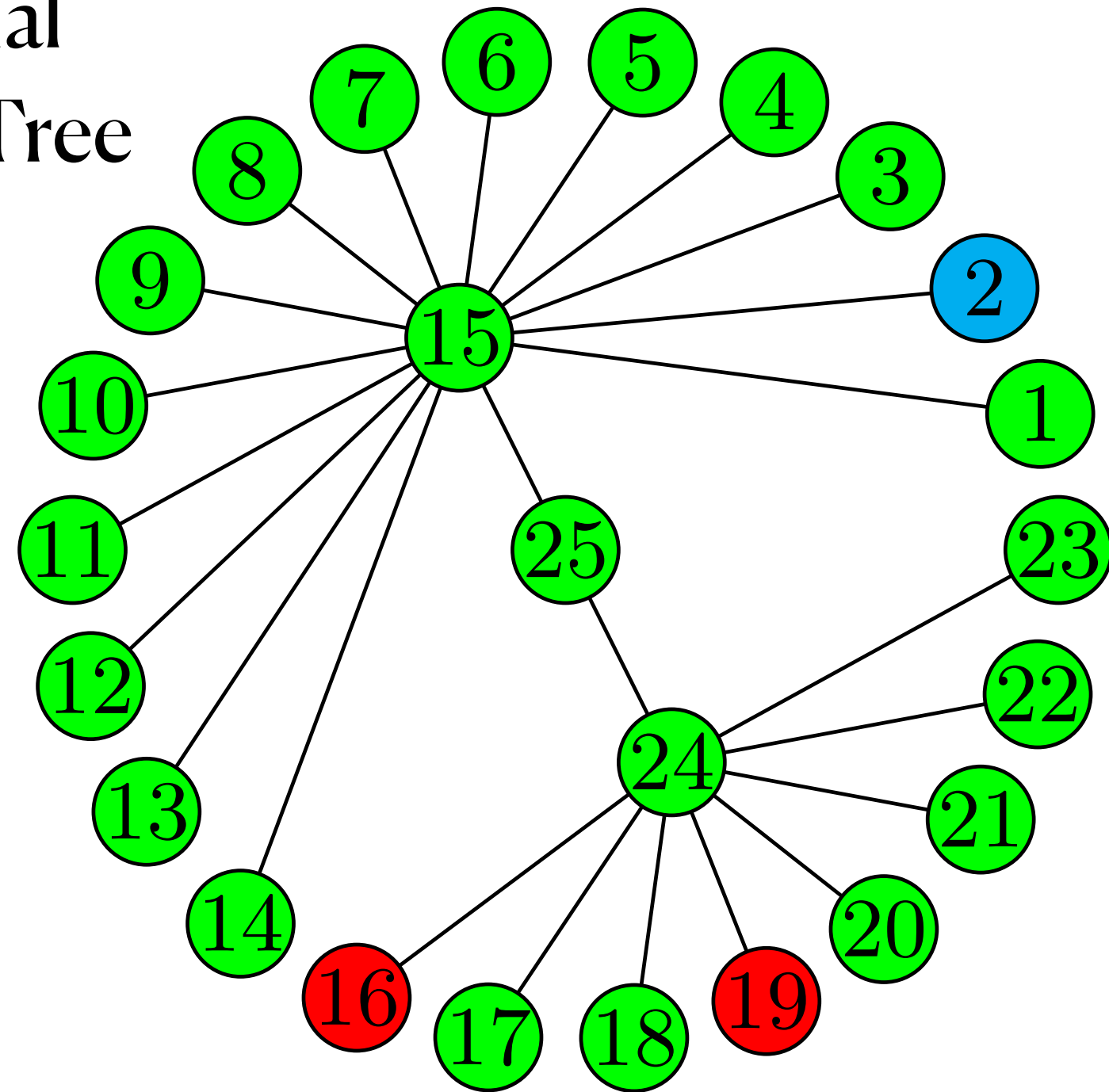
Lattice



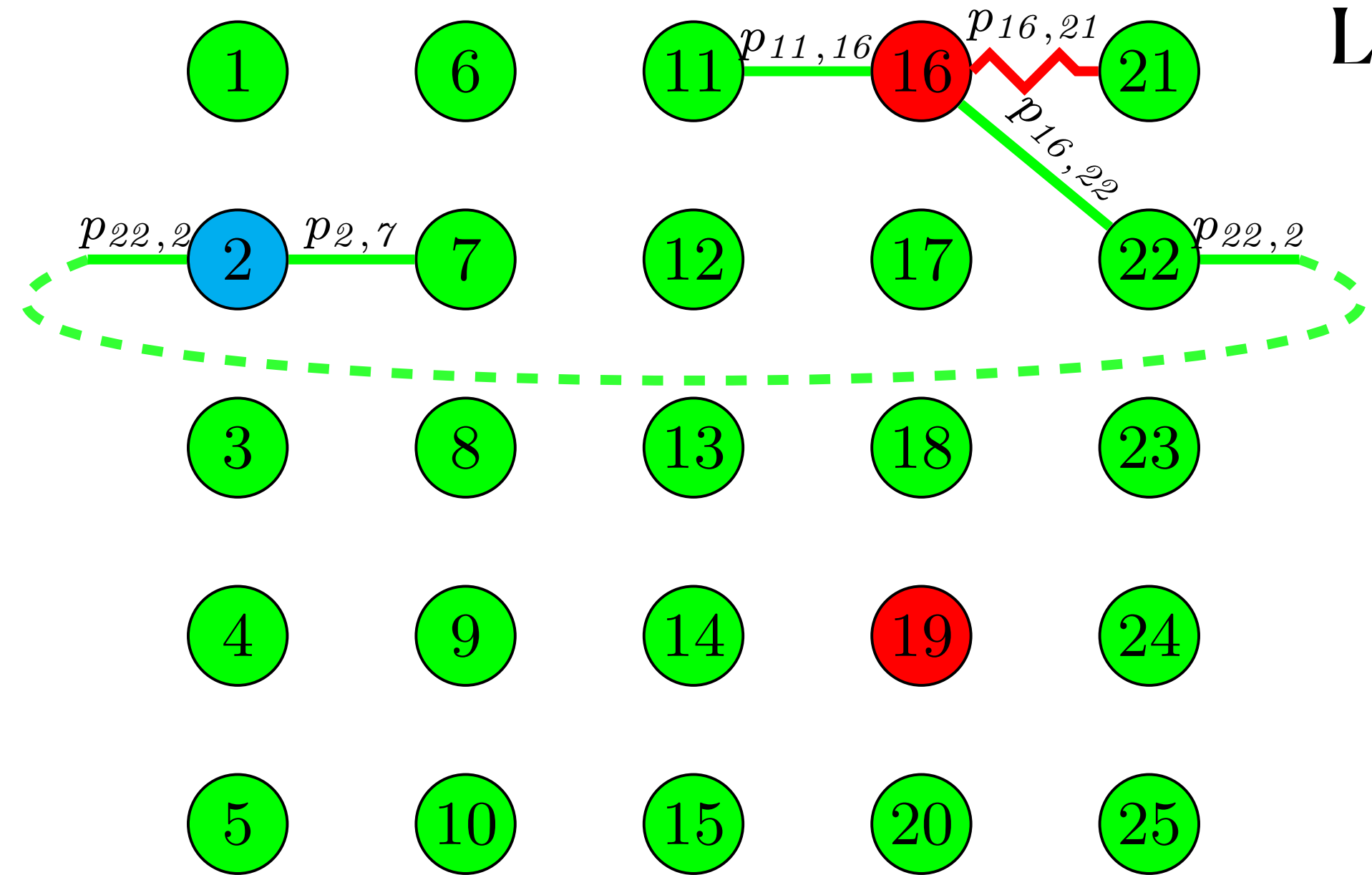
Horizontal
Merkle Tree

Repairing Data Chunk 16

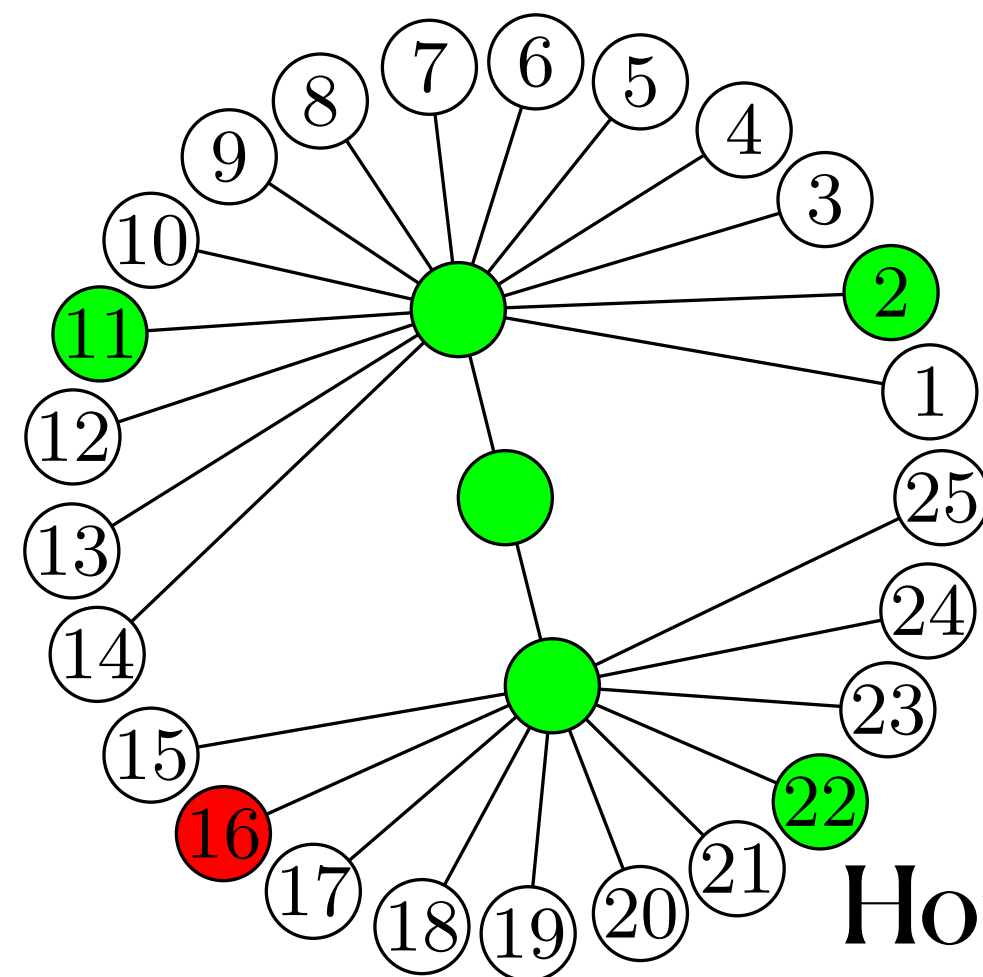
Original
Merkle Tree



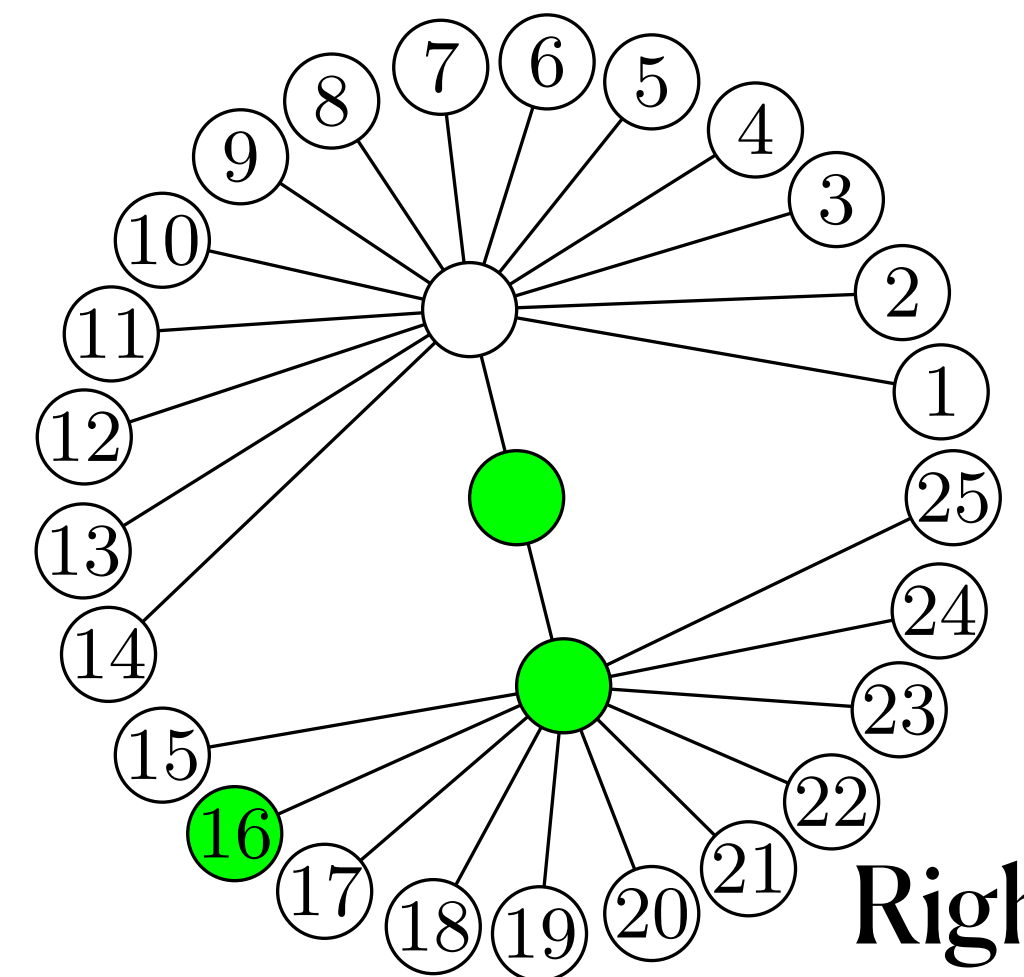
Lattice



Horizontal
Merkle Tree

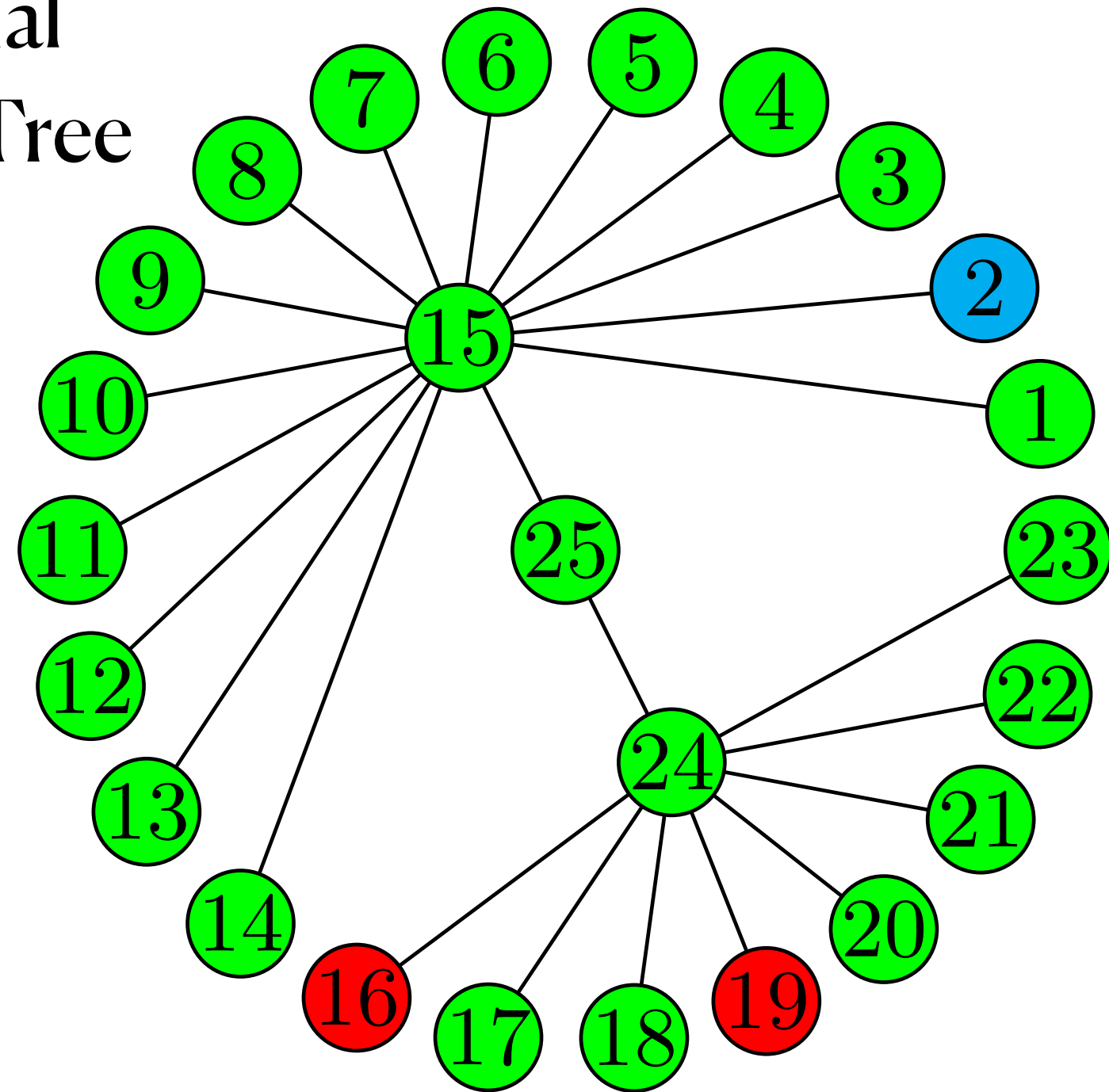


Right
Merkle Tree

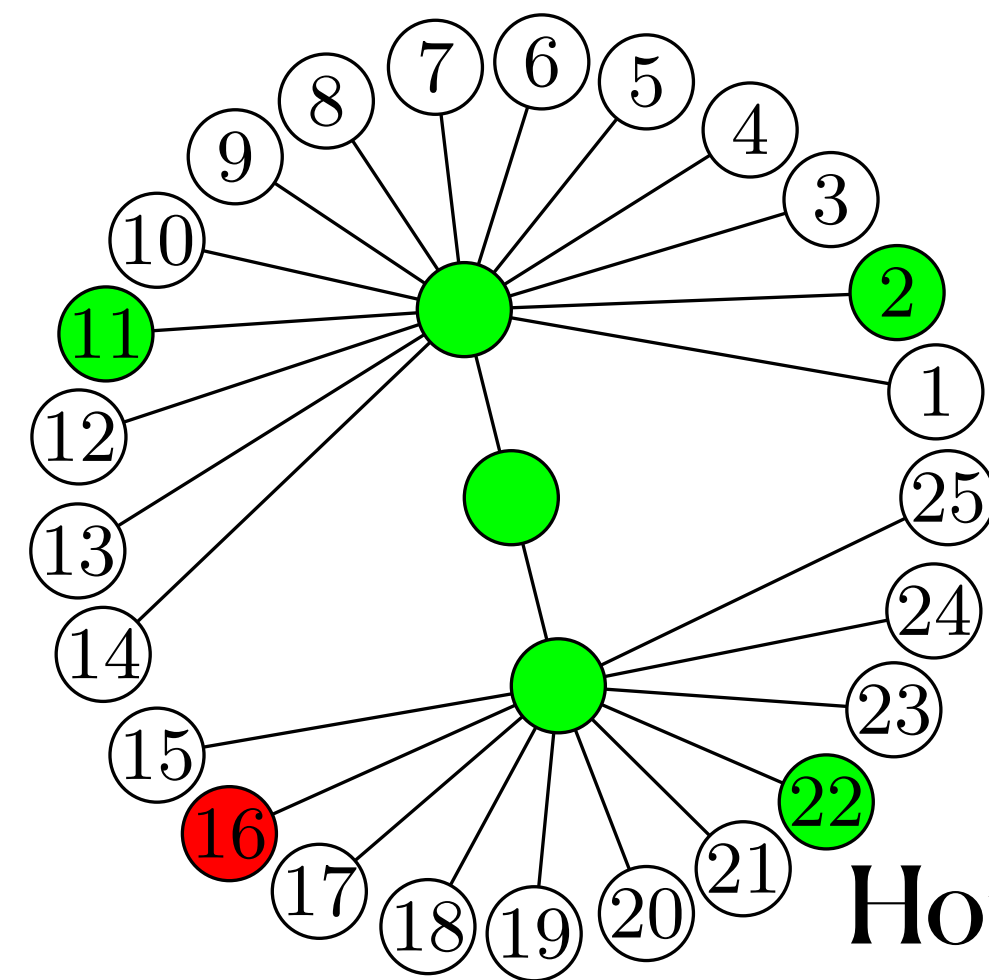
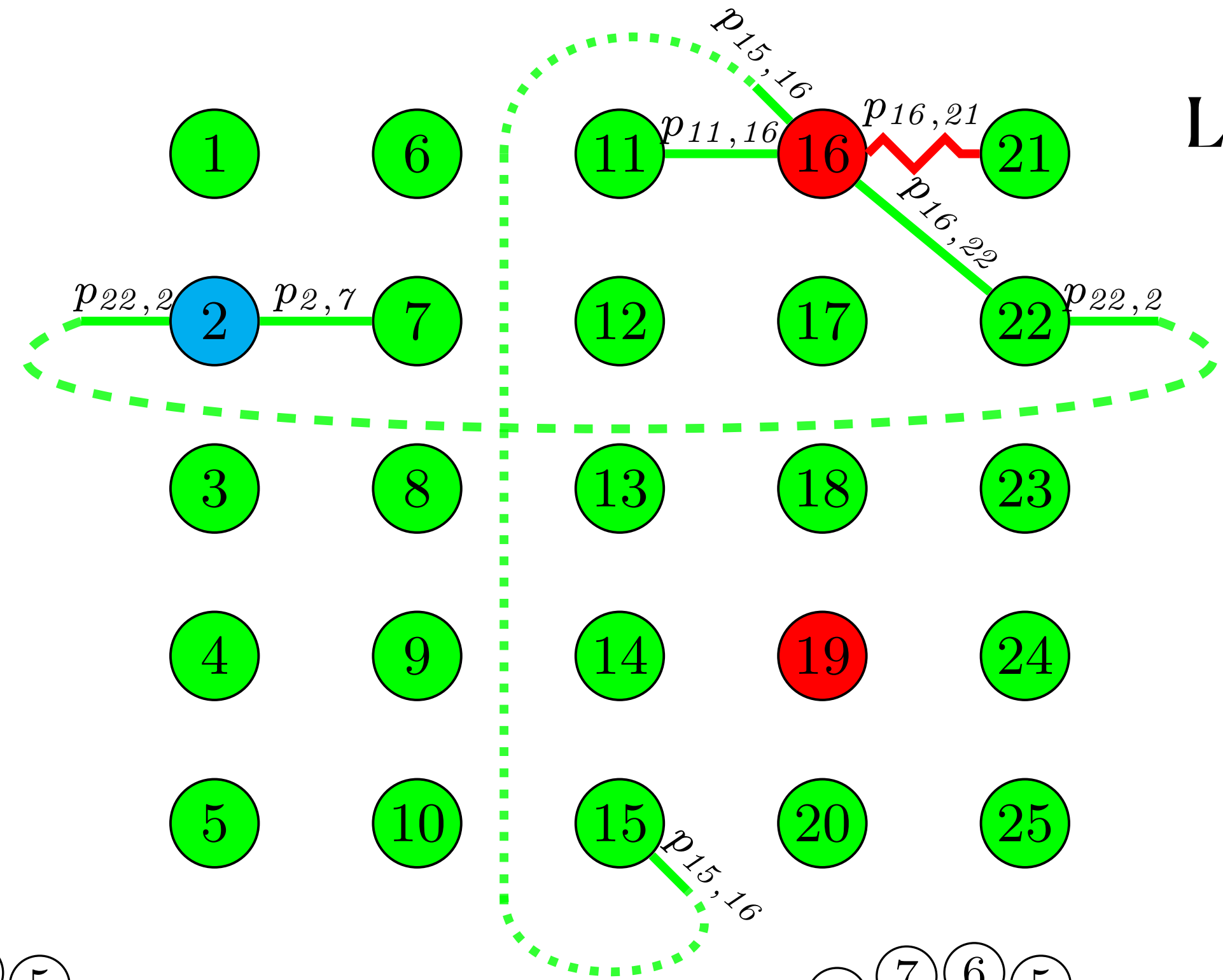


Repairing Data Chunk 16

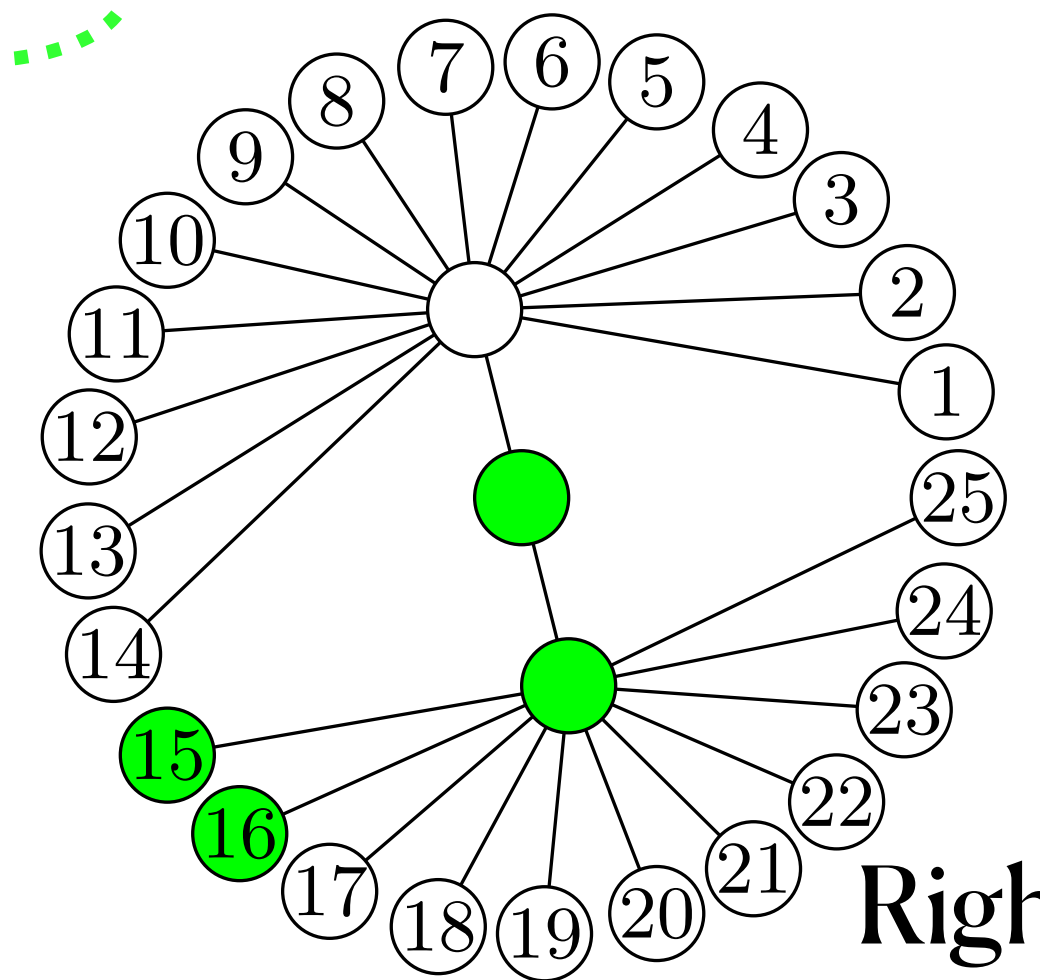
Original
Merkle Tree



Lattice



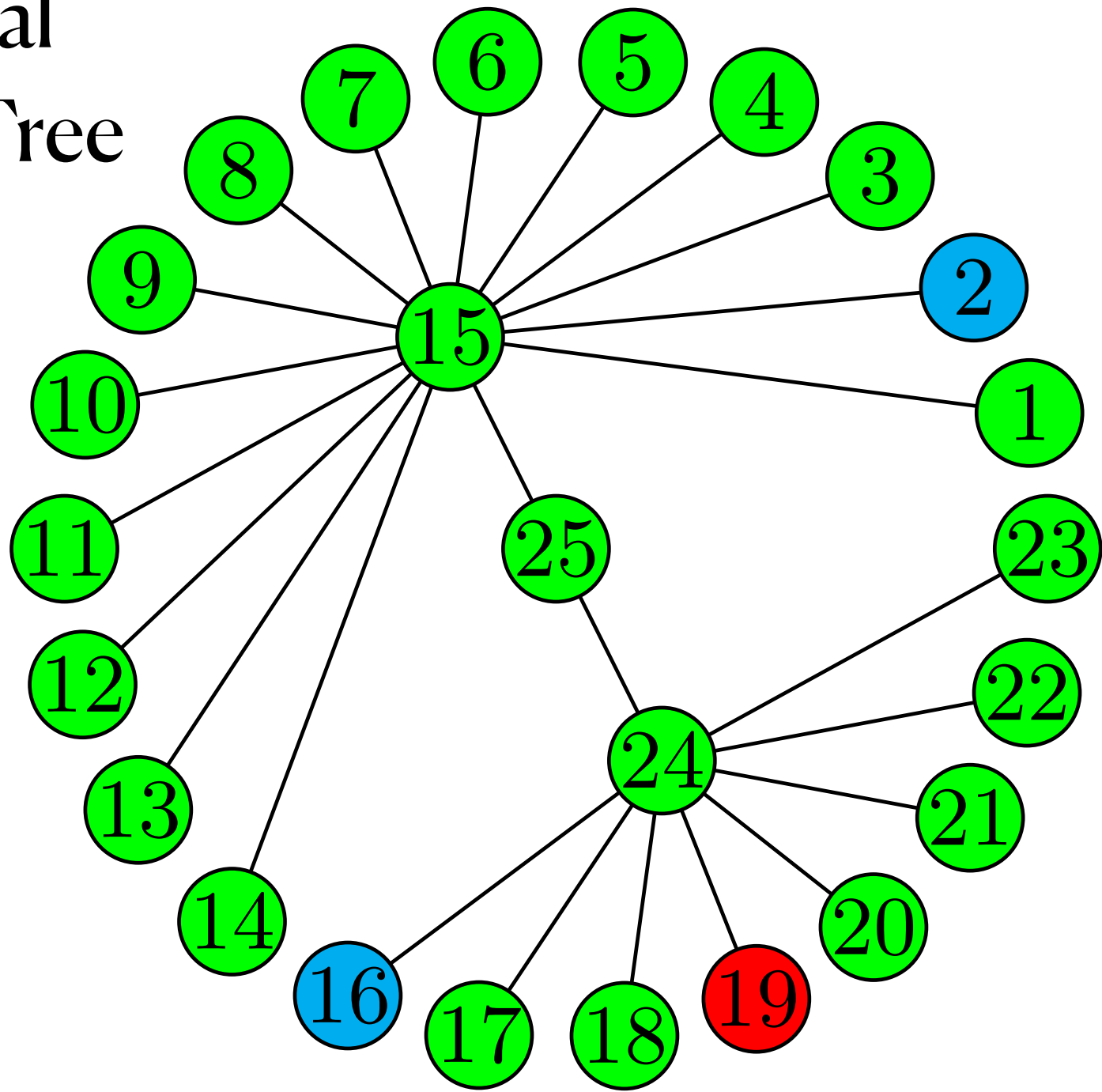
Horizontal
Merkle Tree



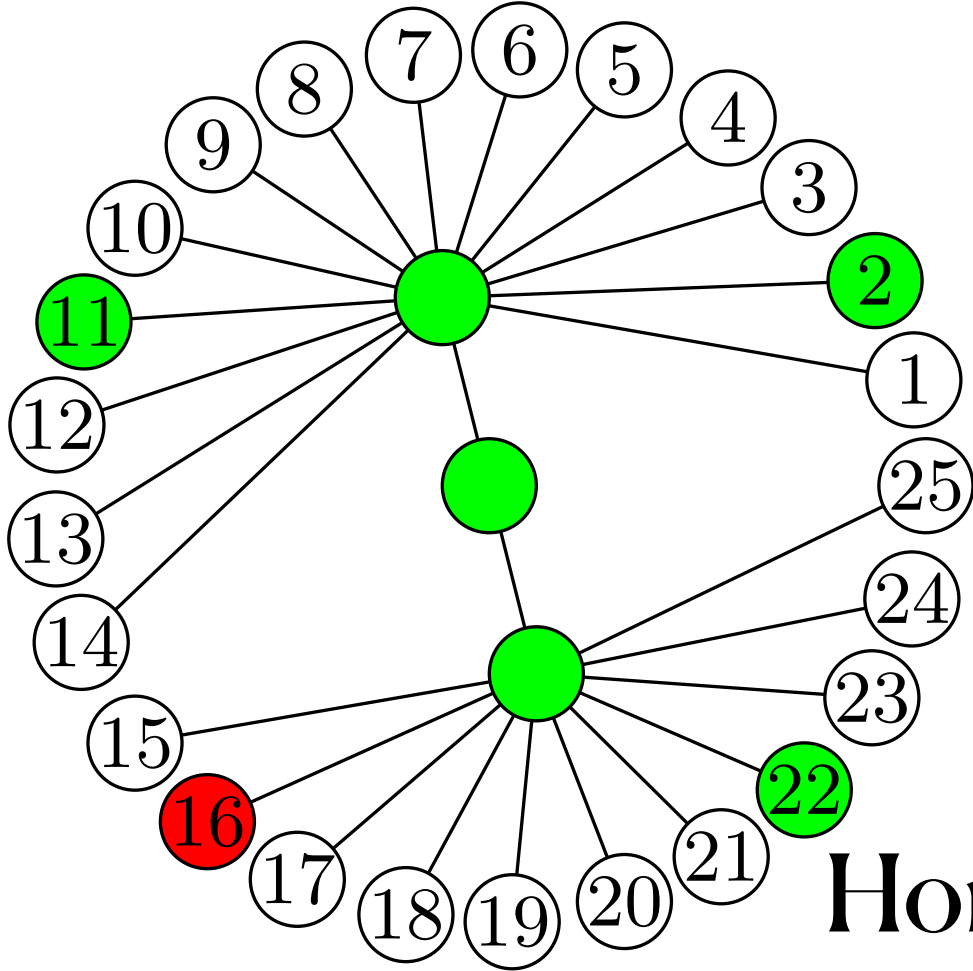
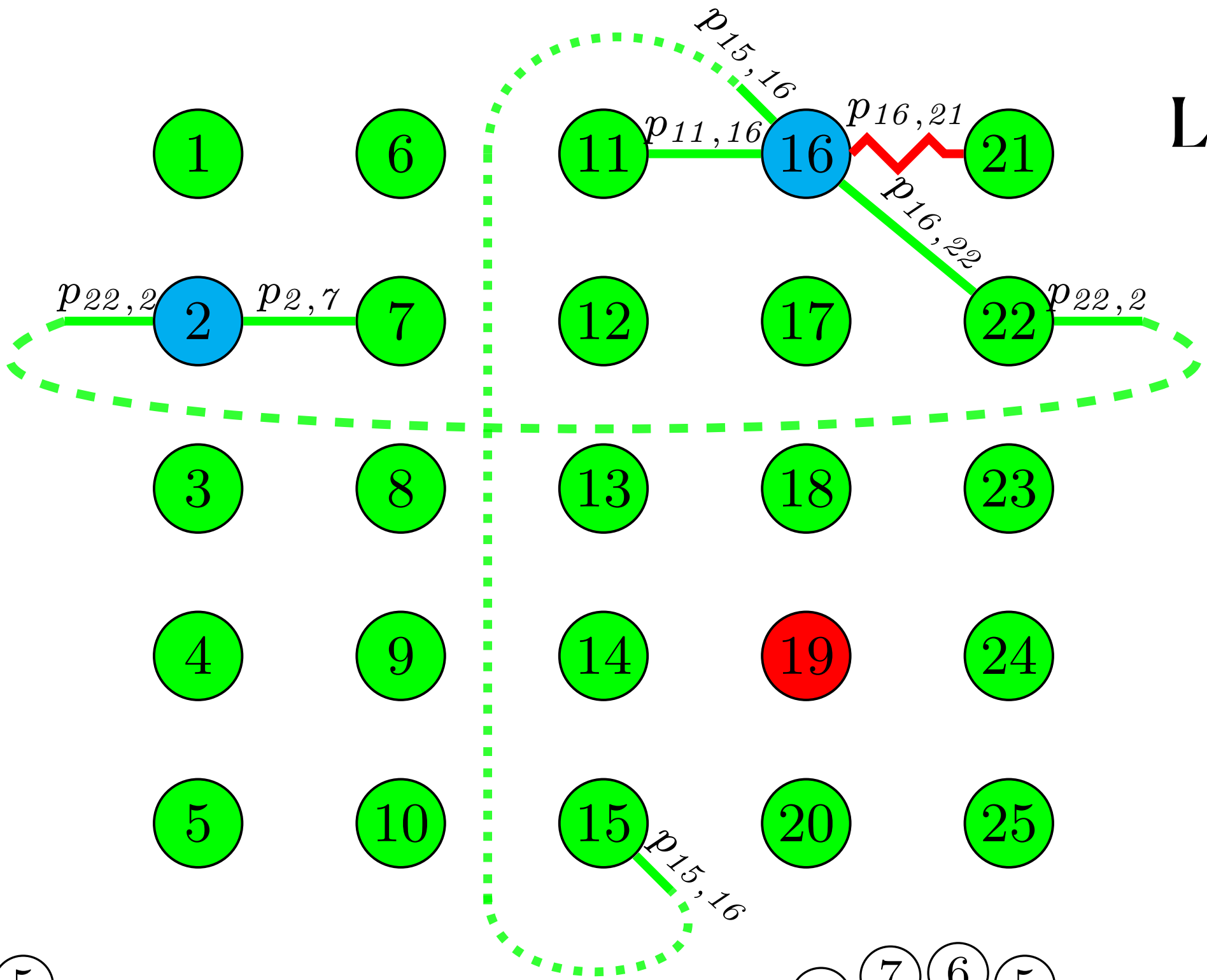
Right
Merkle Tree

Repaired Data Chunk 16

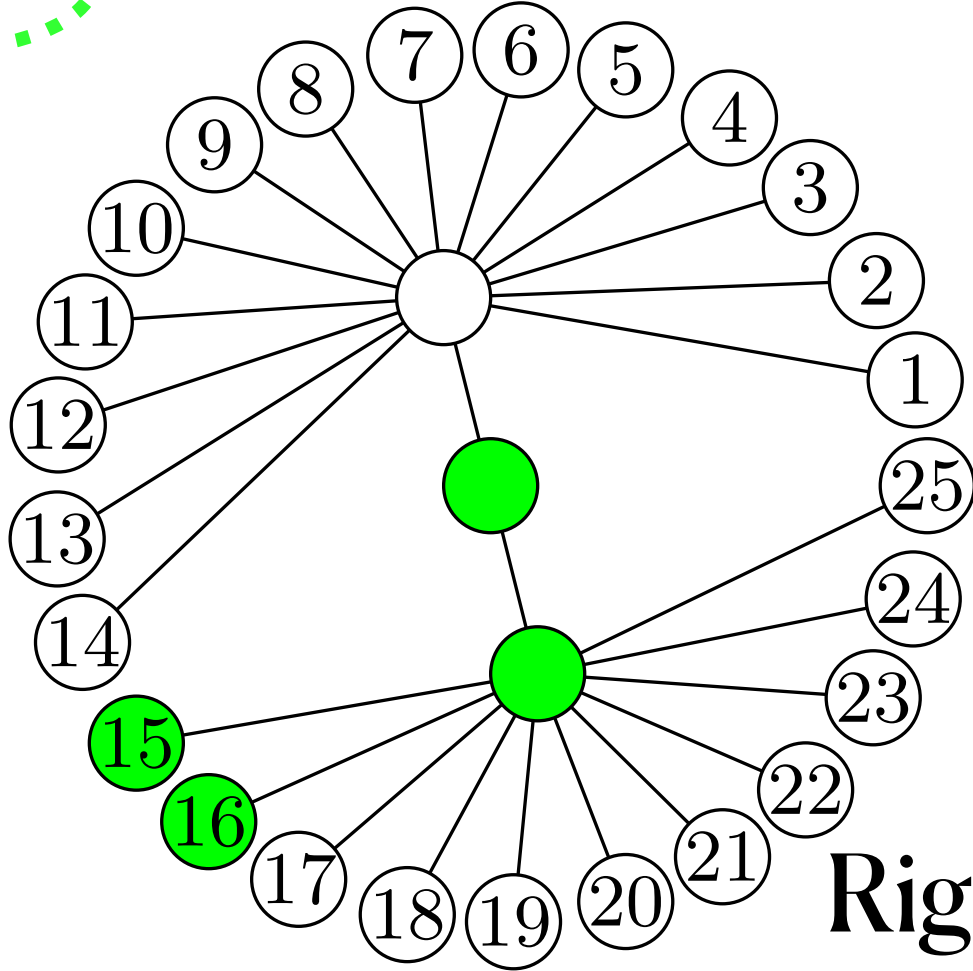
Original
Merkle Tree



Lattice



Horizontal
Merkle Tree

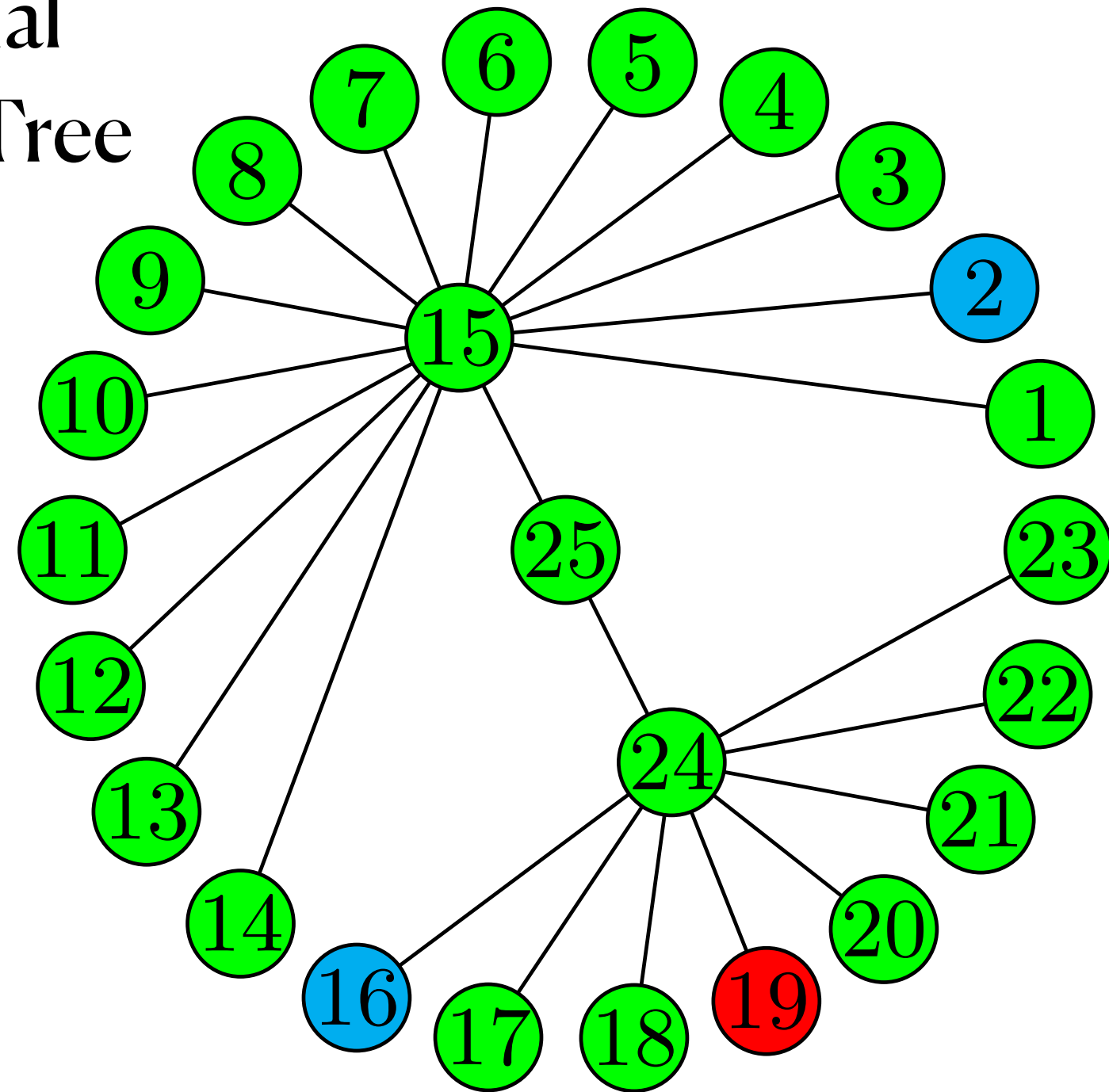


Right
Merkle Tree

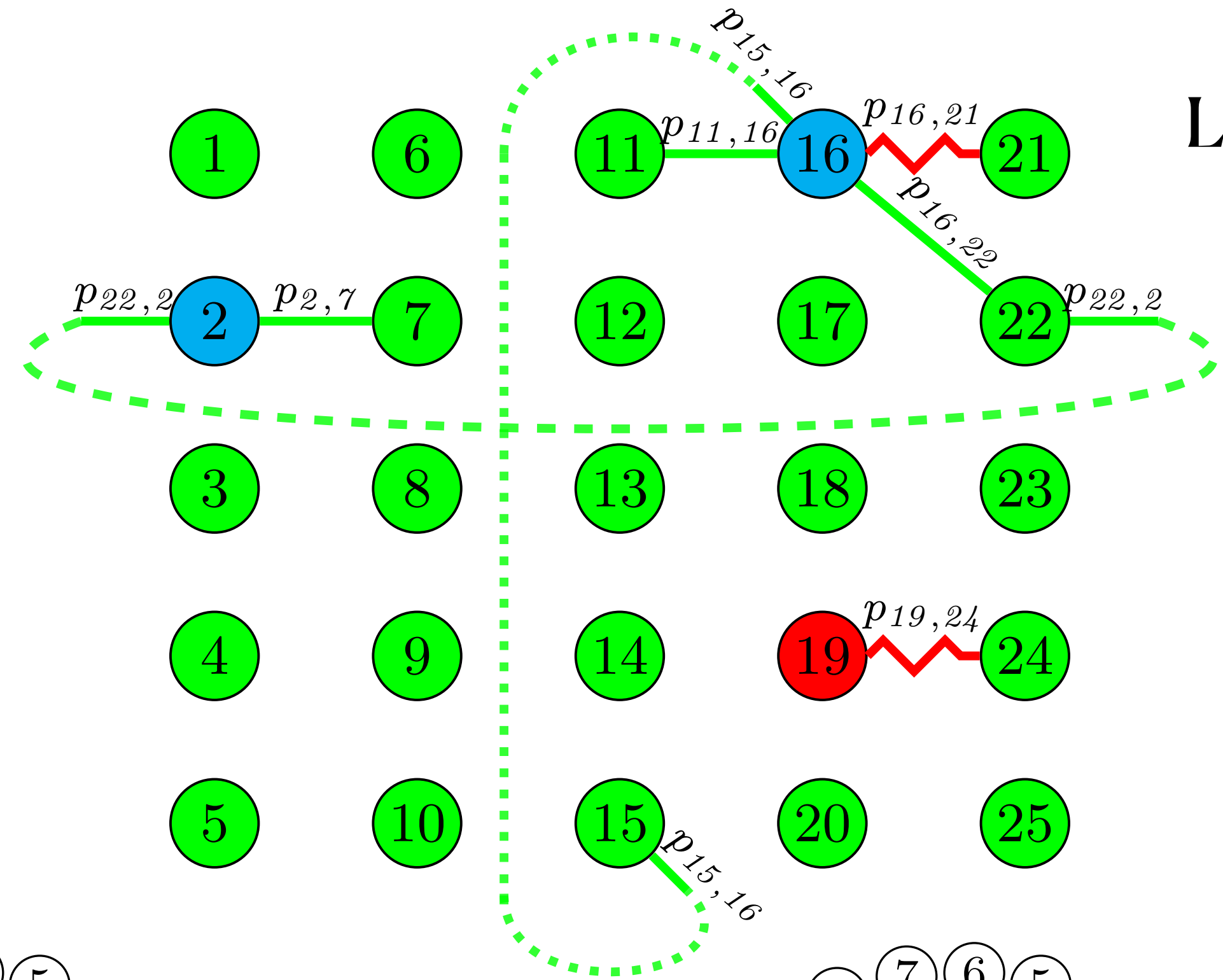
Repairing Data Chunk 19

Repairing Data Chunk 19

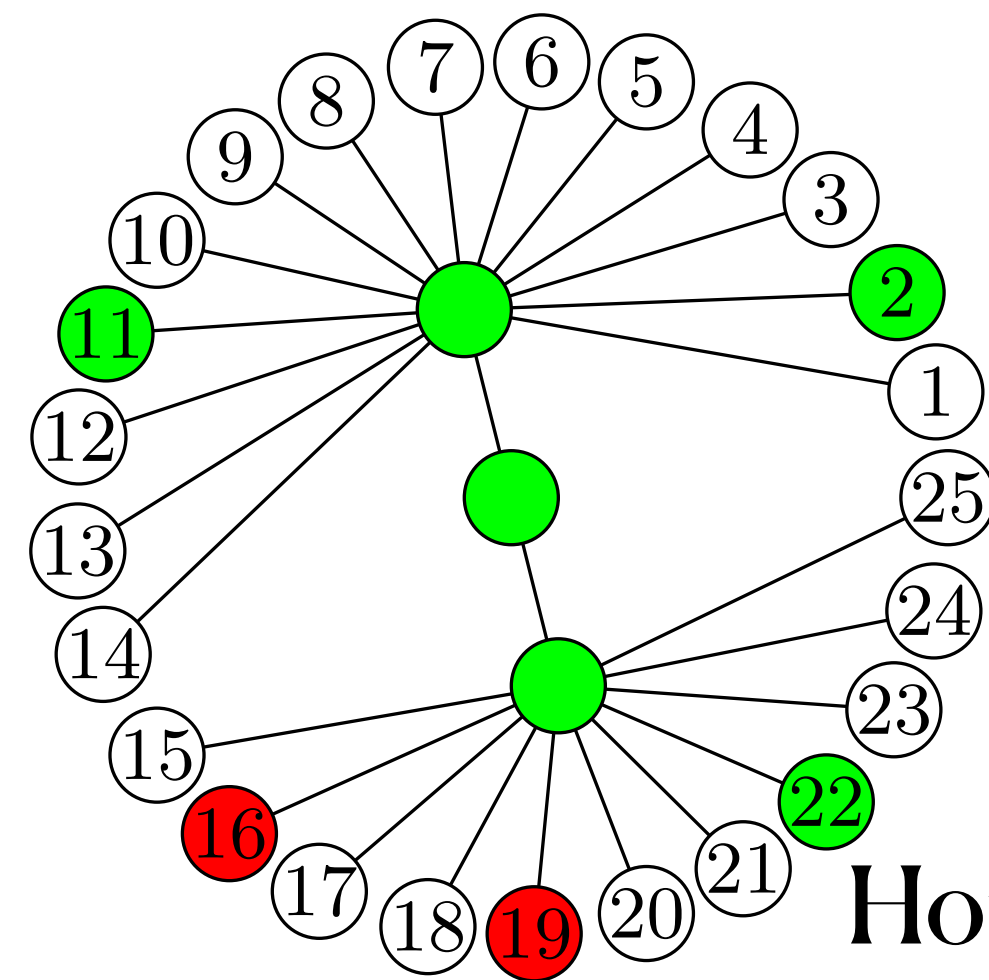
Original
Merkle Tree



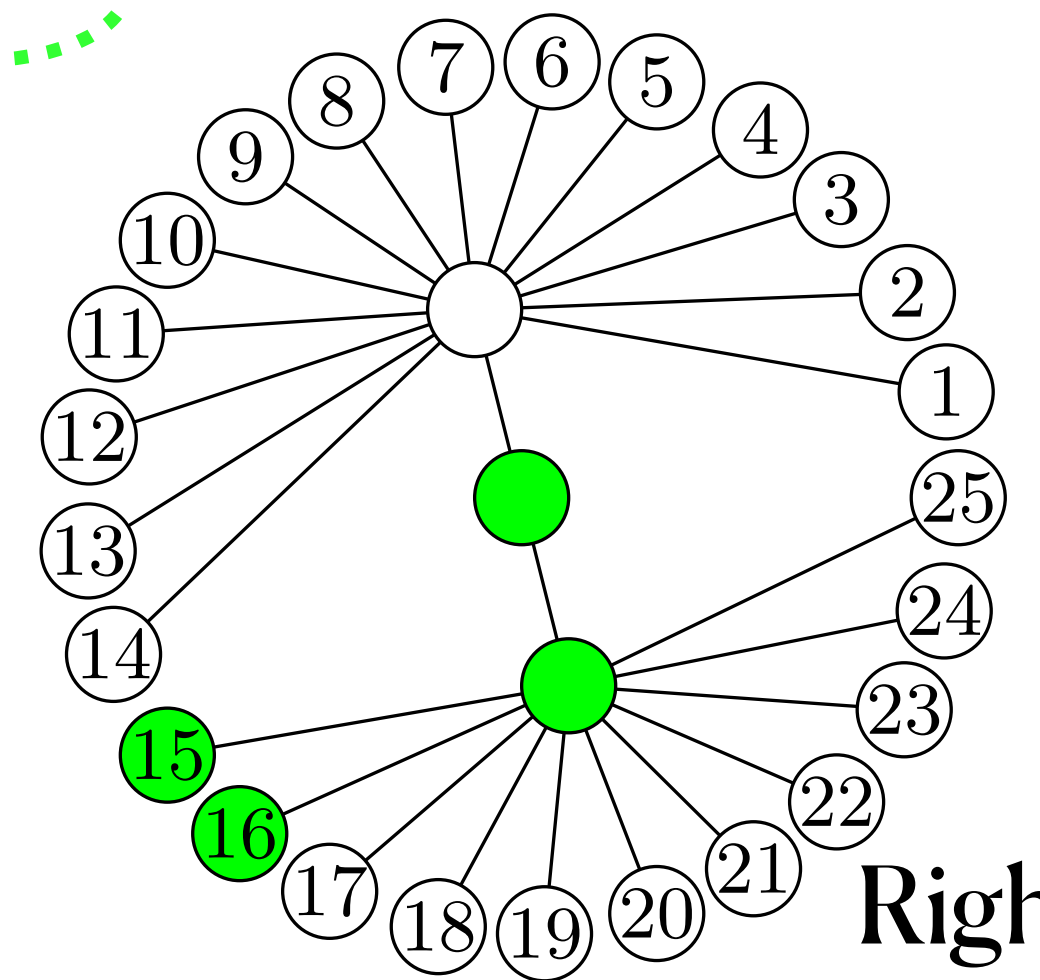
Lattice



Horizontal
Merkle Tree

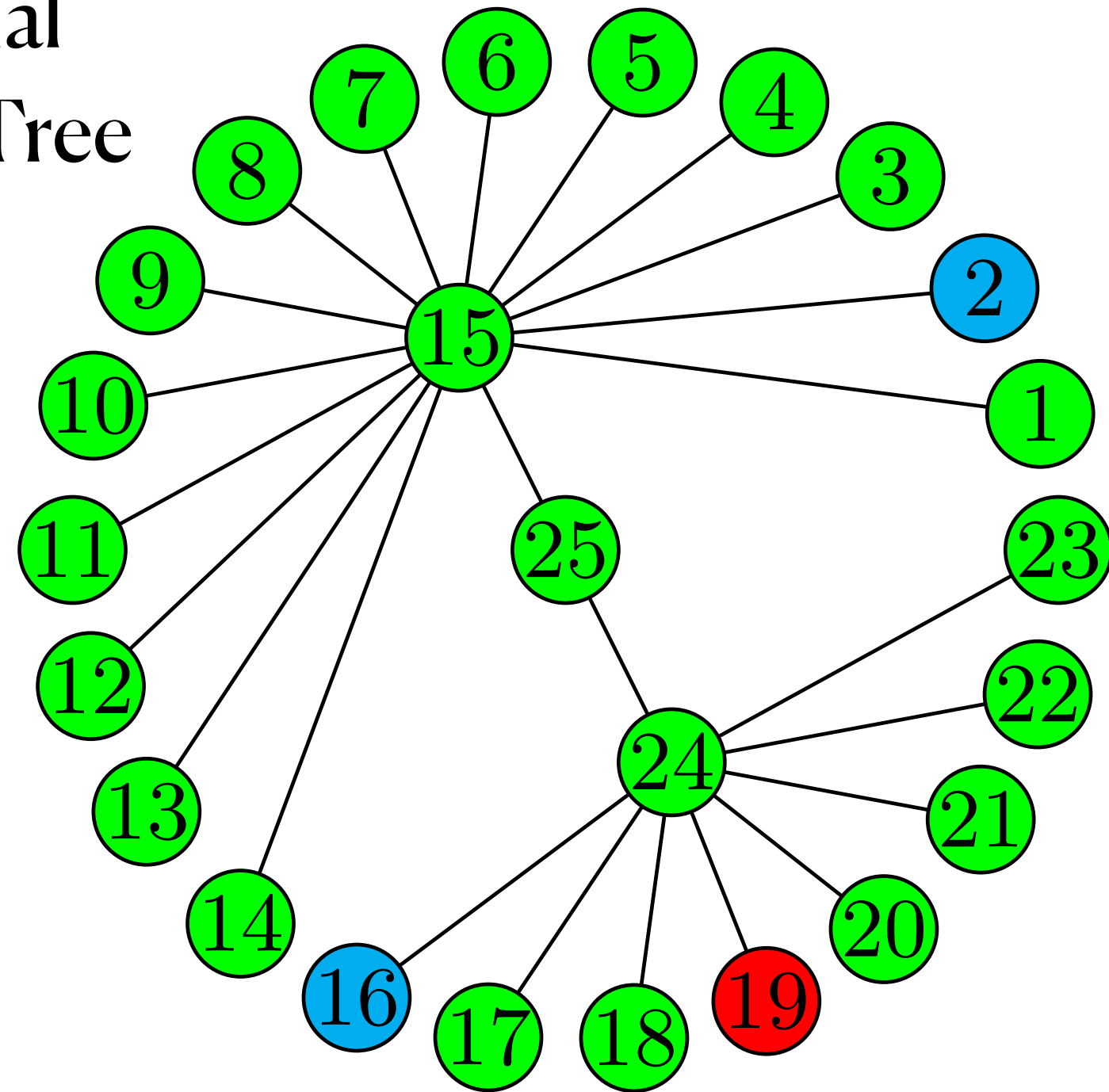


Right
Merkle Tree

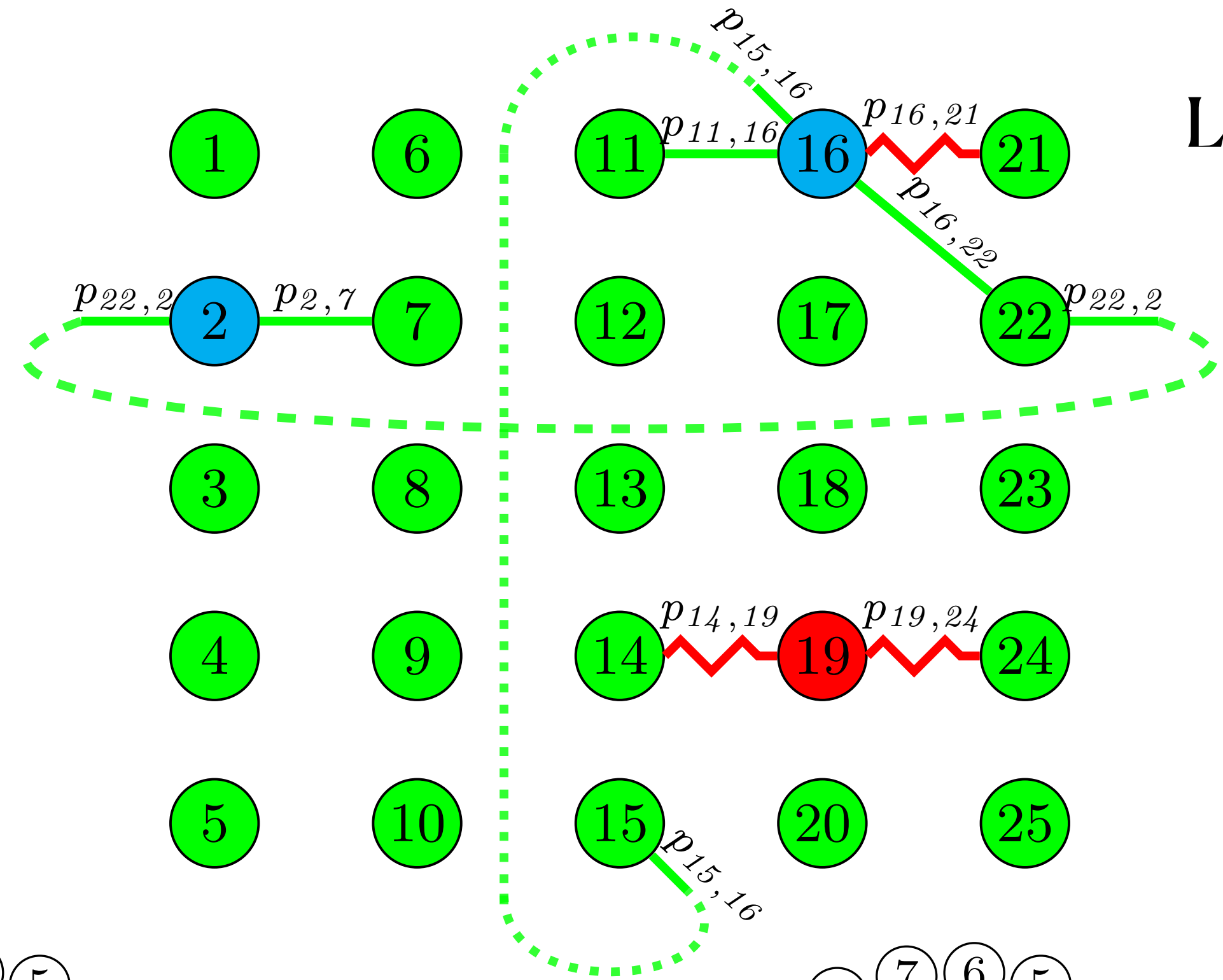


Repairing Data Chunk 19

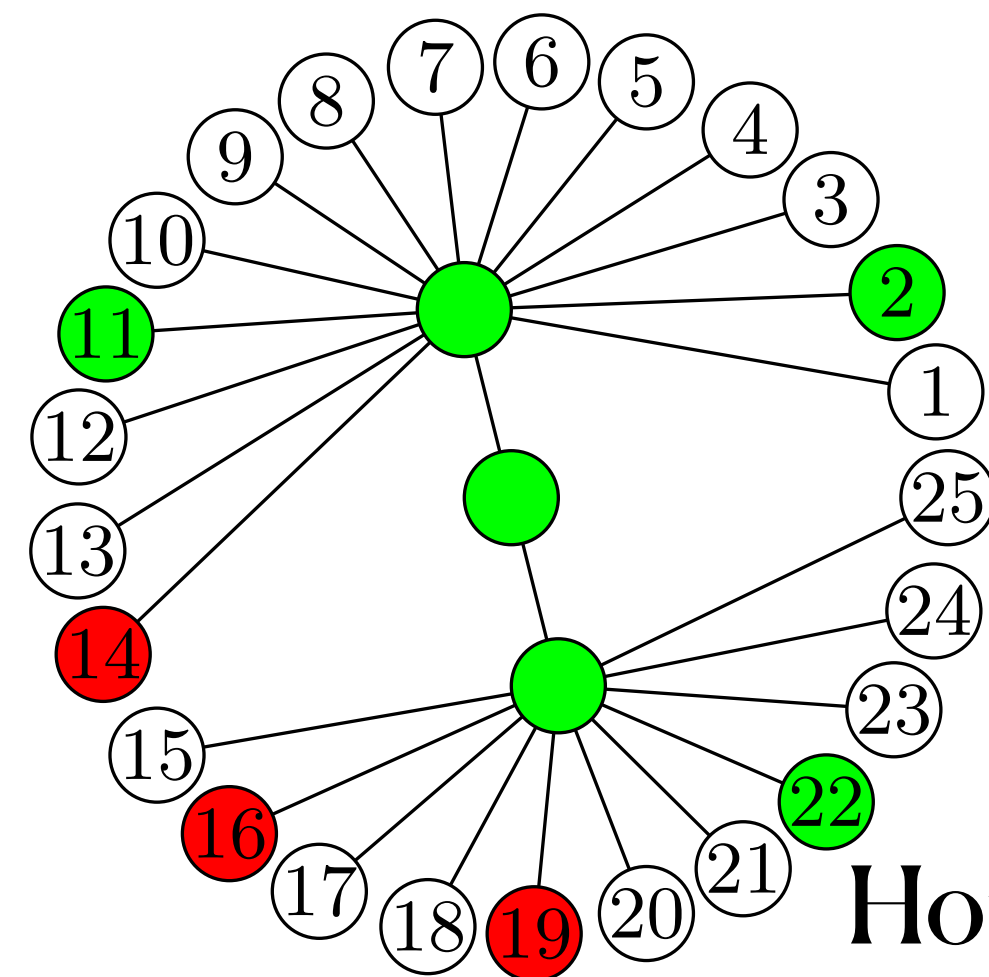
Original
Merkle Tree



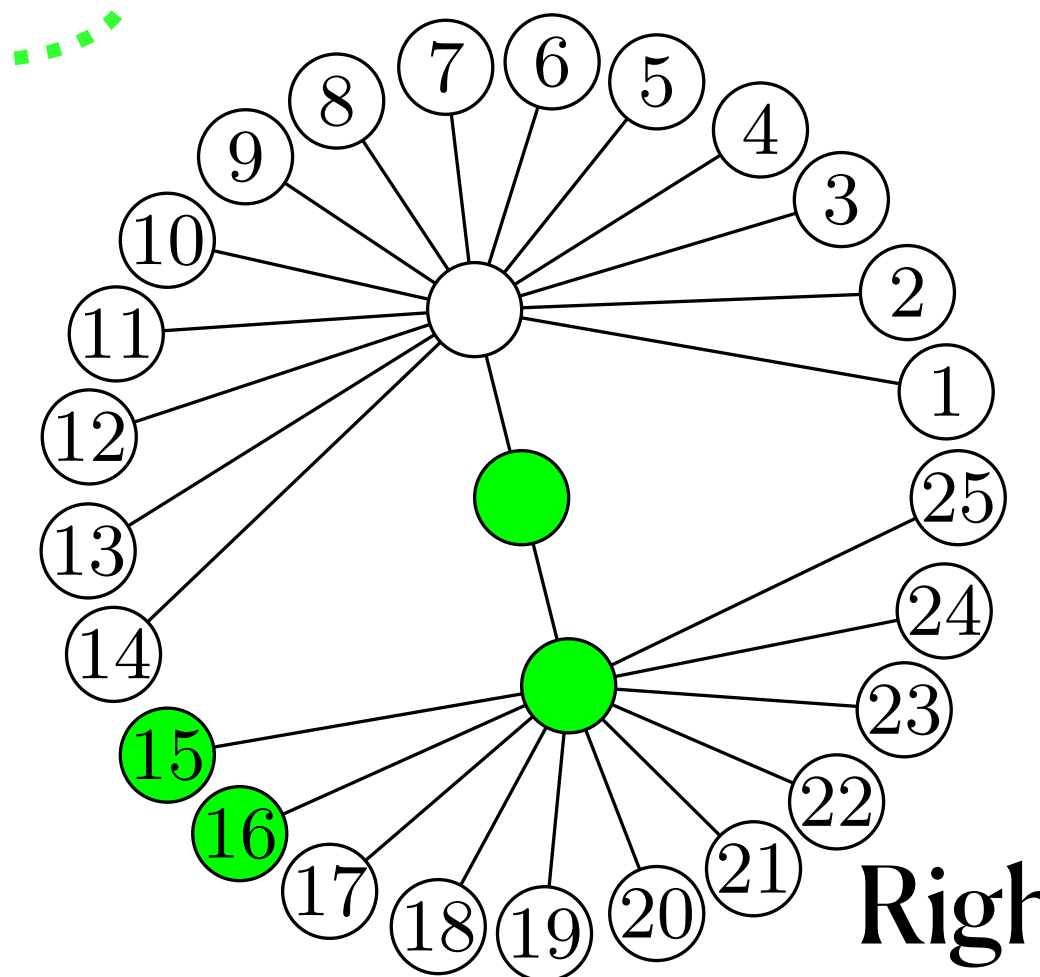
Lattice



Horizontal
Merkle Tree

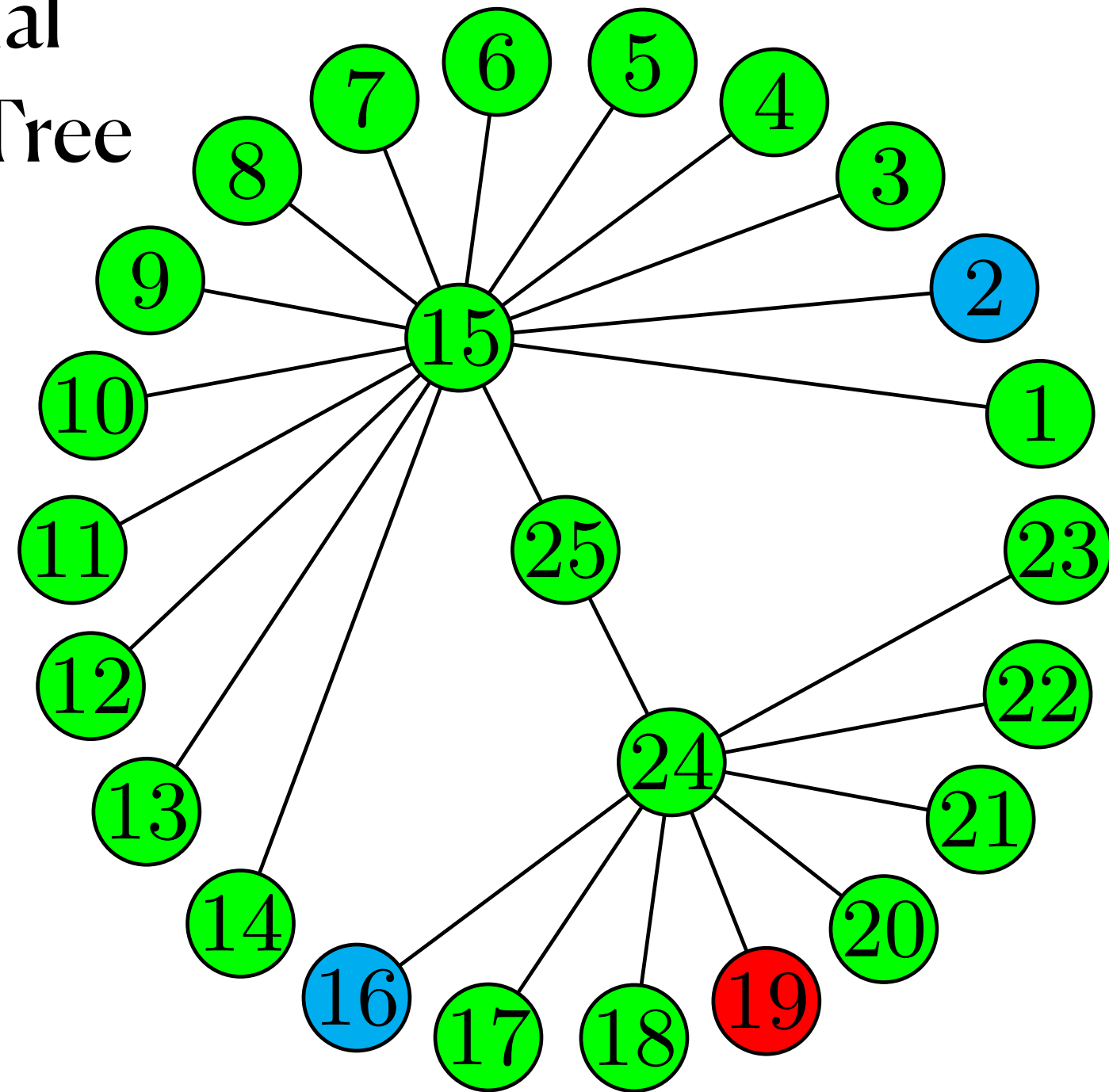


Right
Merkle Tree

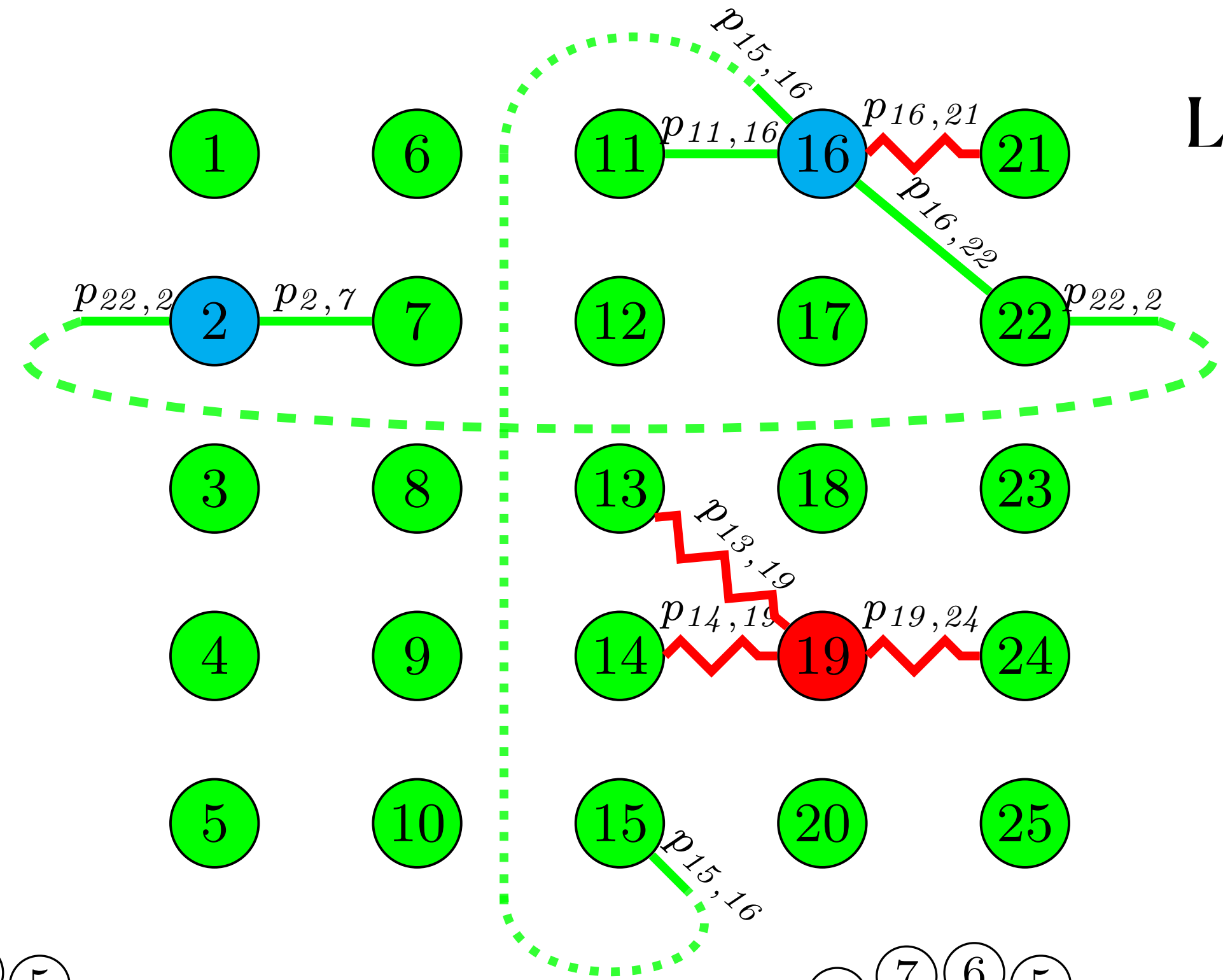


Repairing Data Chunk 19

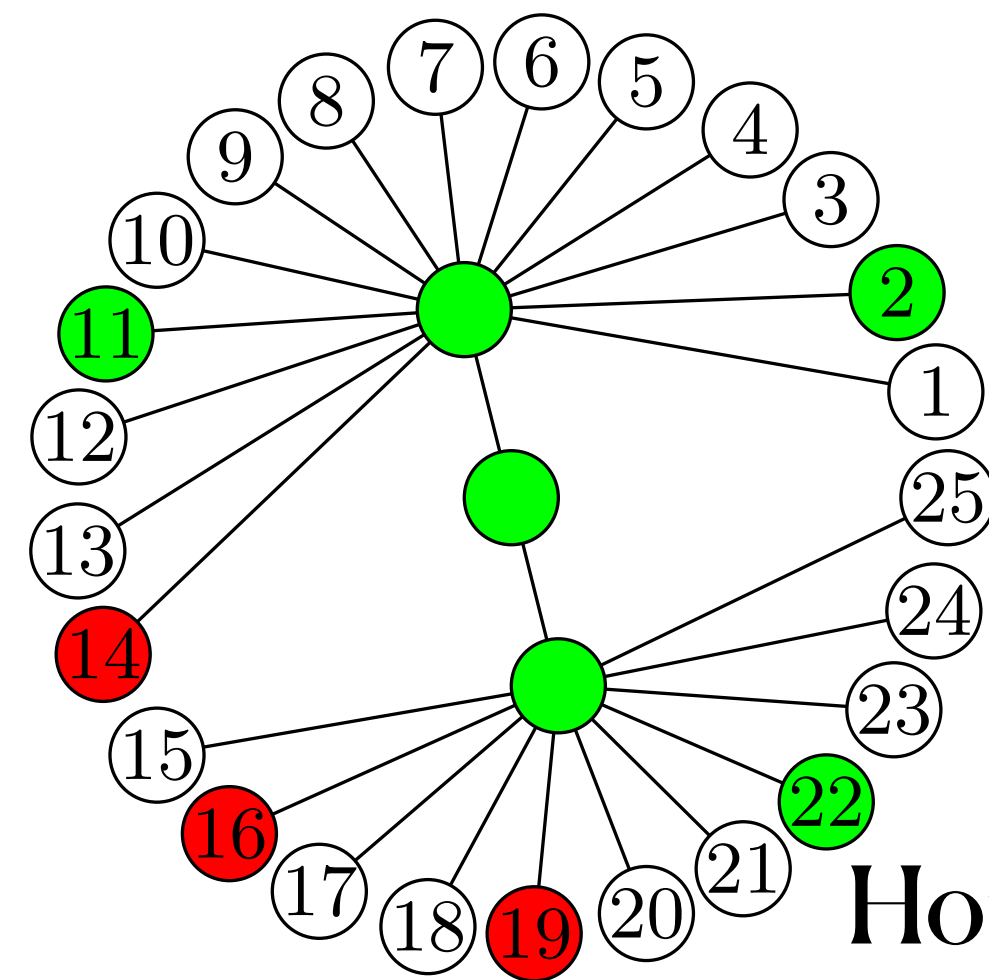
Original
Merkle Tree



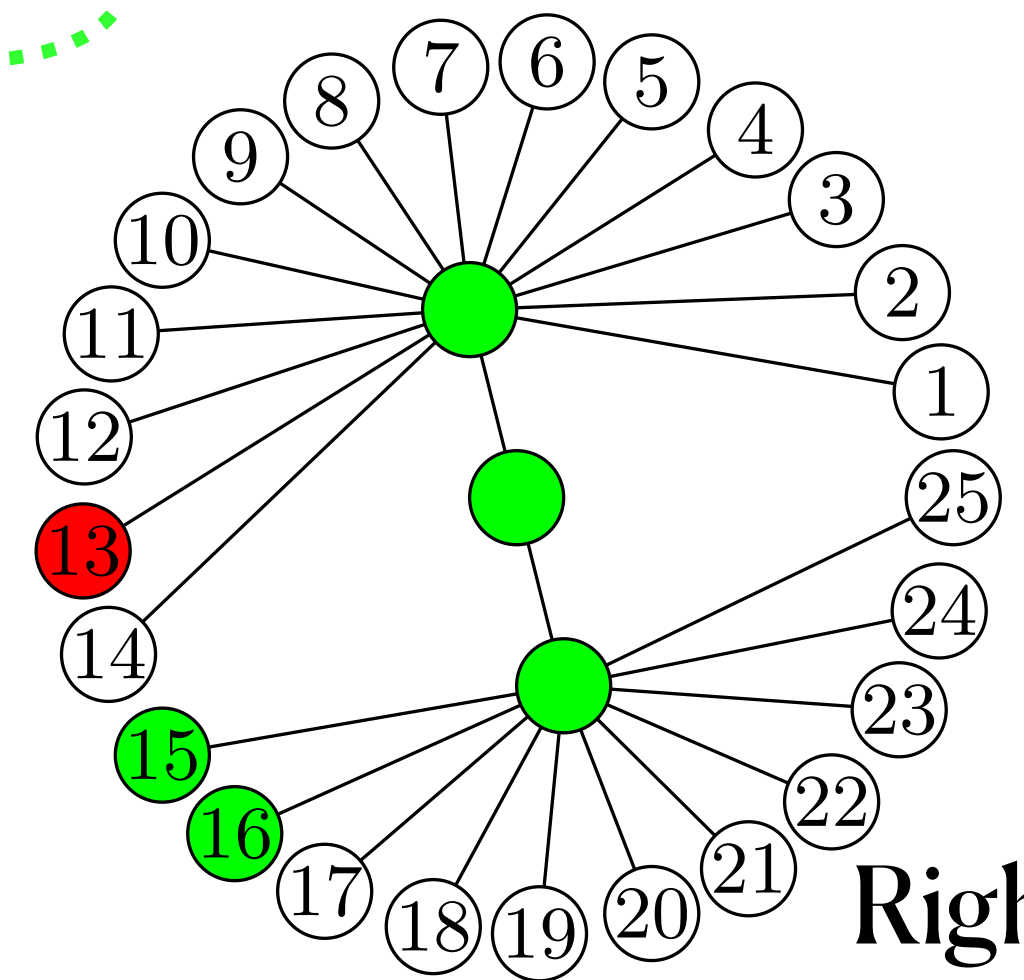
Lattice



Horizontal
Merkle Tree

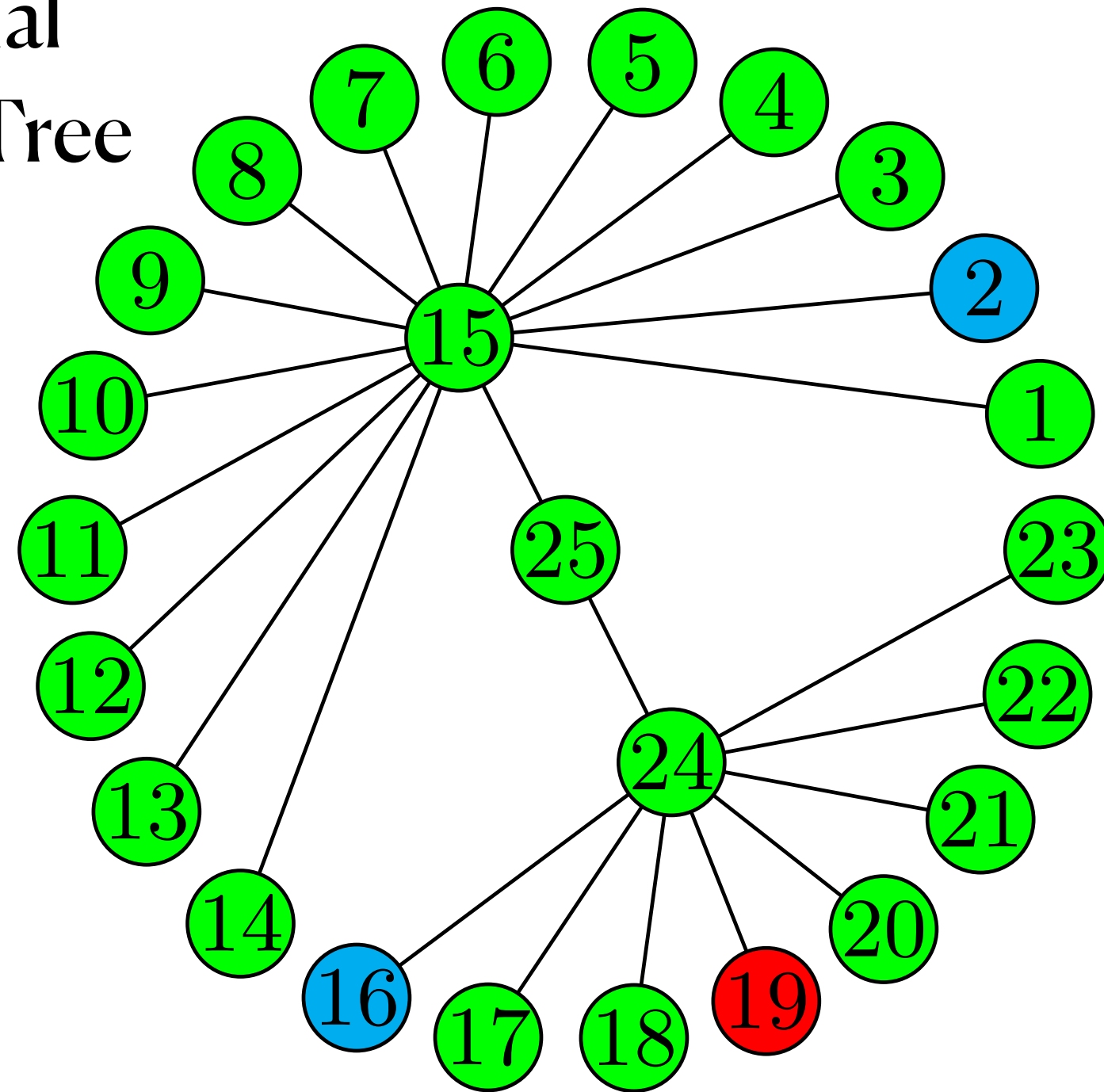


Right
Merkle Tree

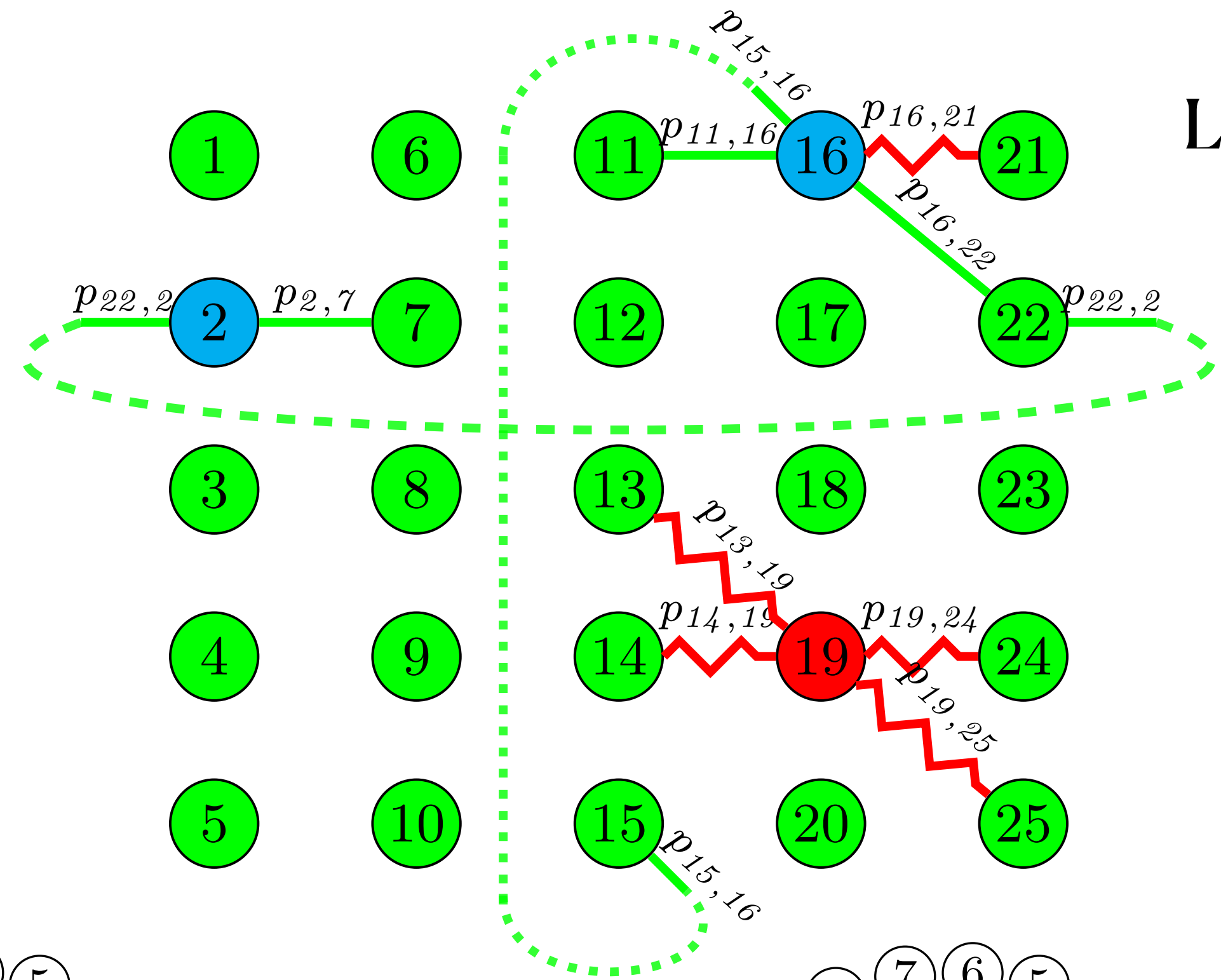


Repairing Data Chunk 19

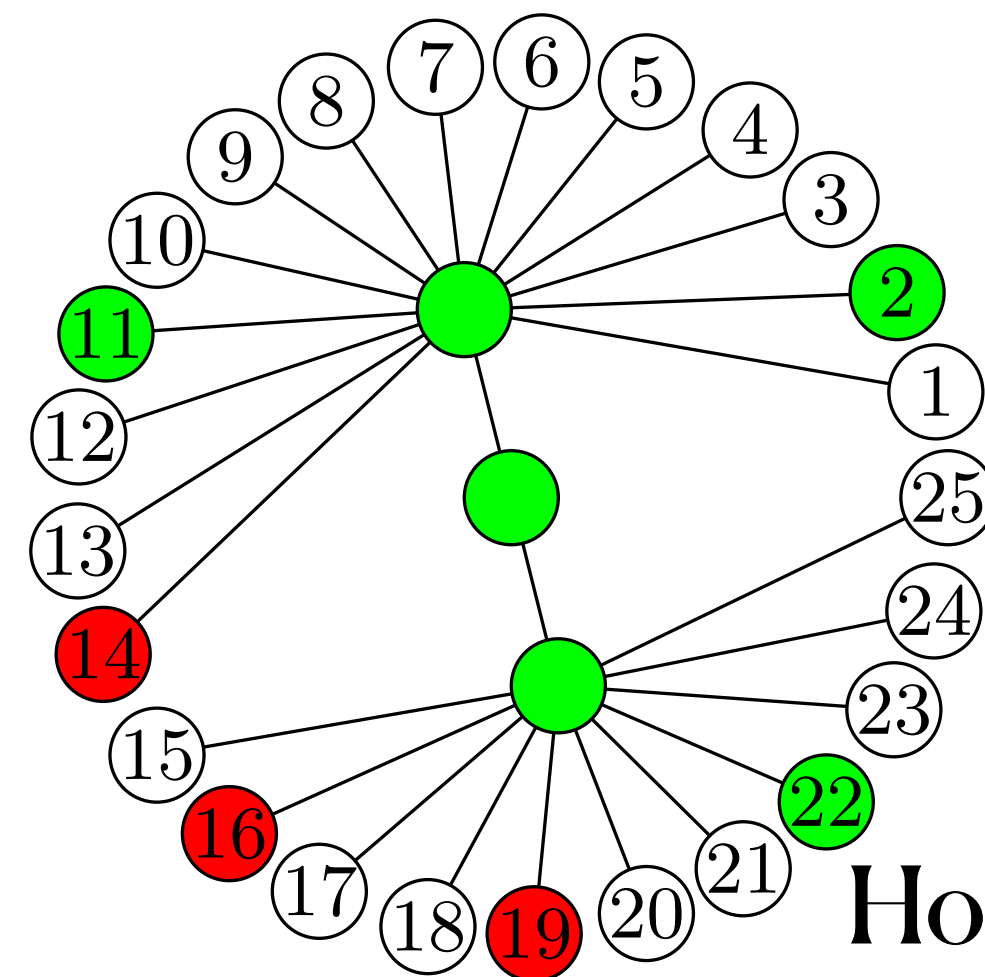
Original
Merkle Tree



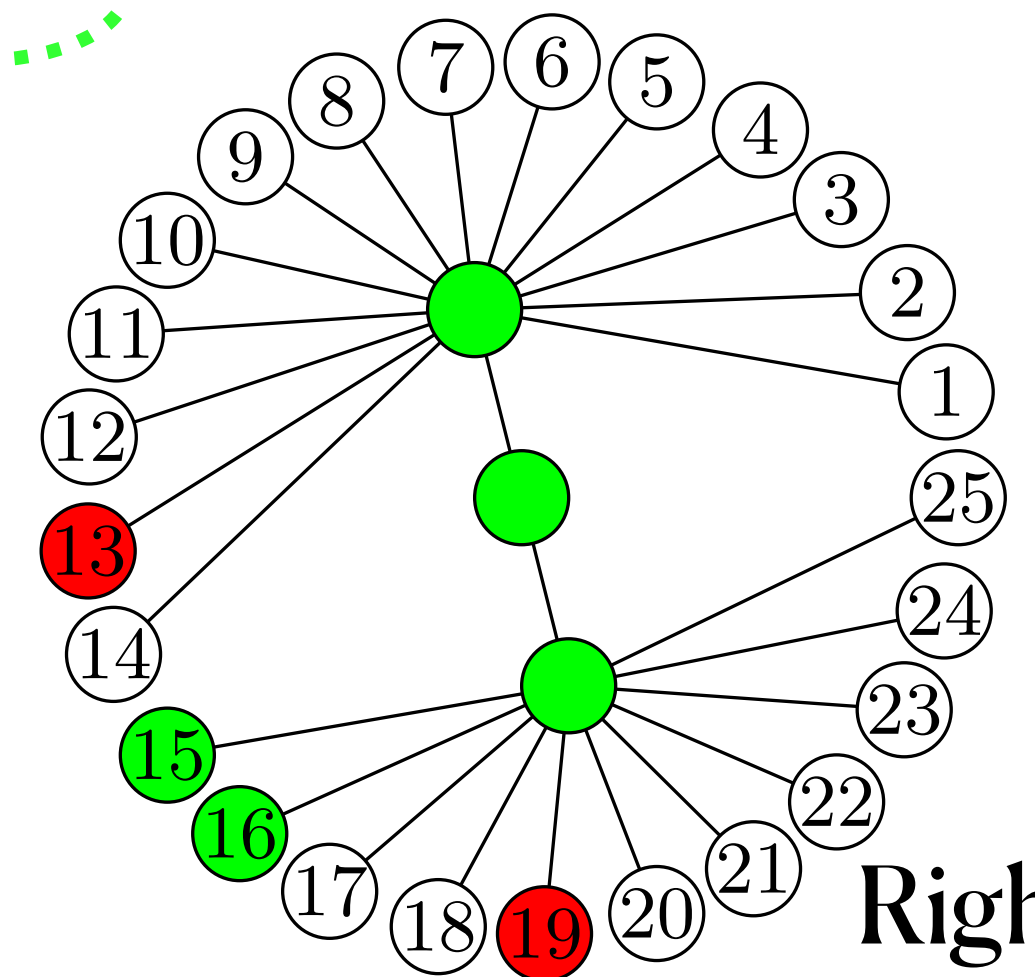
Lattice



Horizontal
Merkle Tree

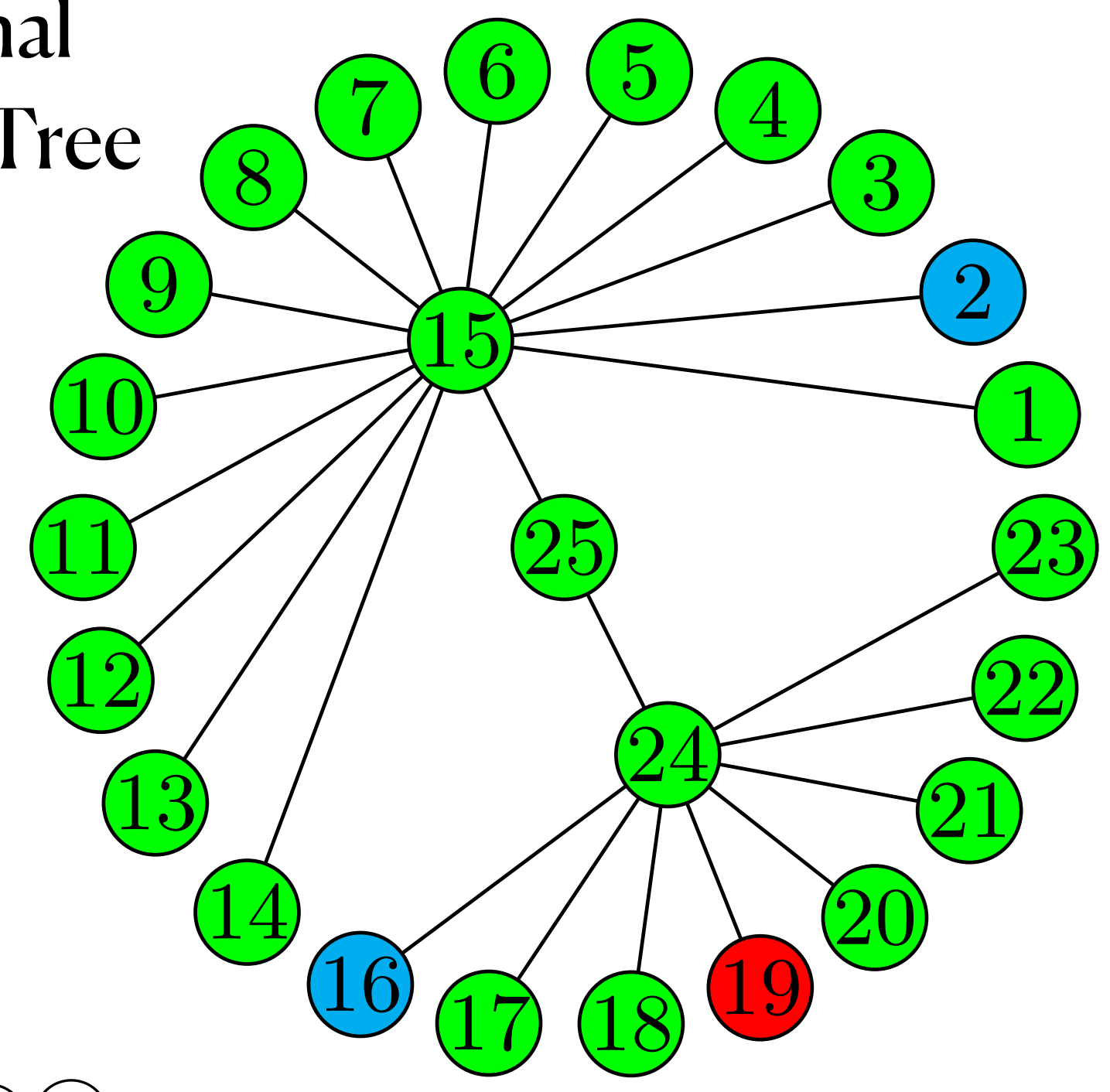


Right
Merkle Tree

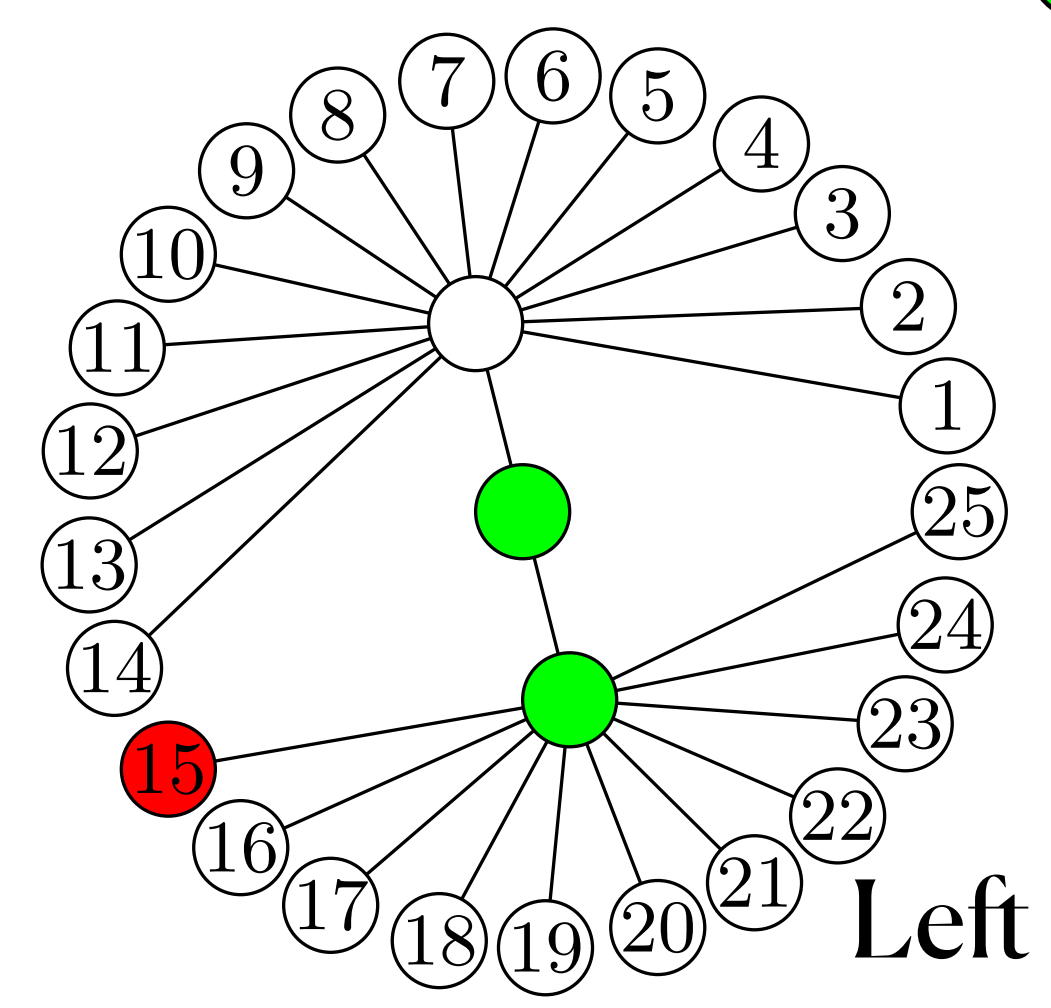
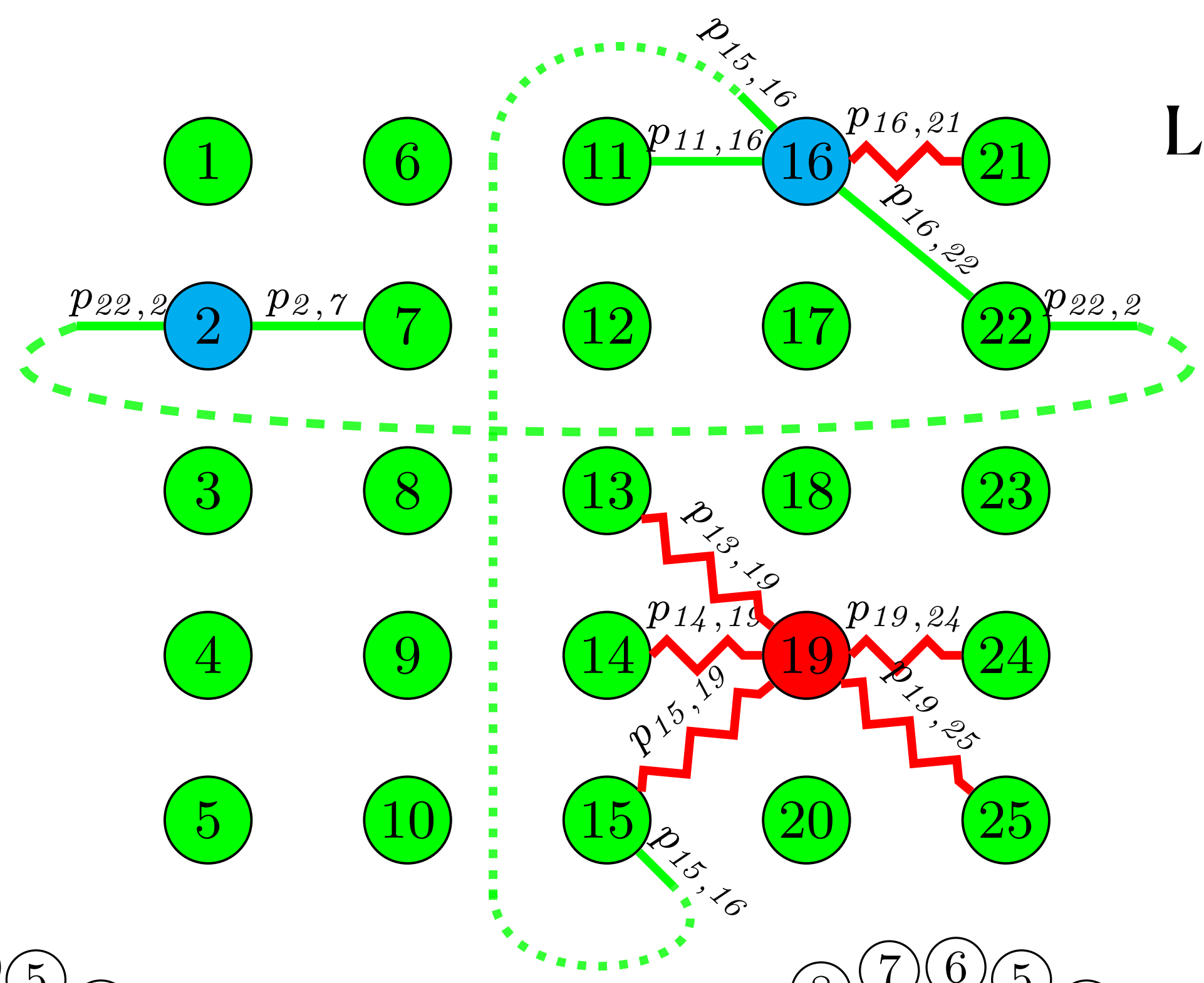


Repairing Data Chunk 19

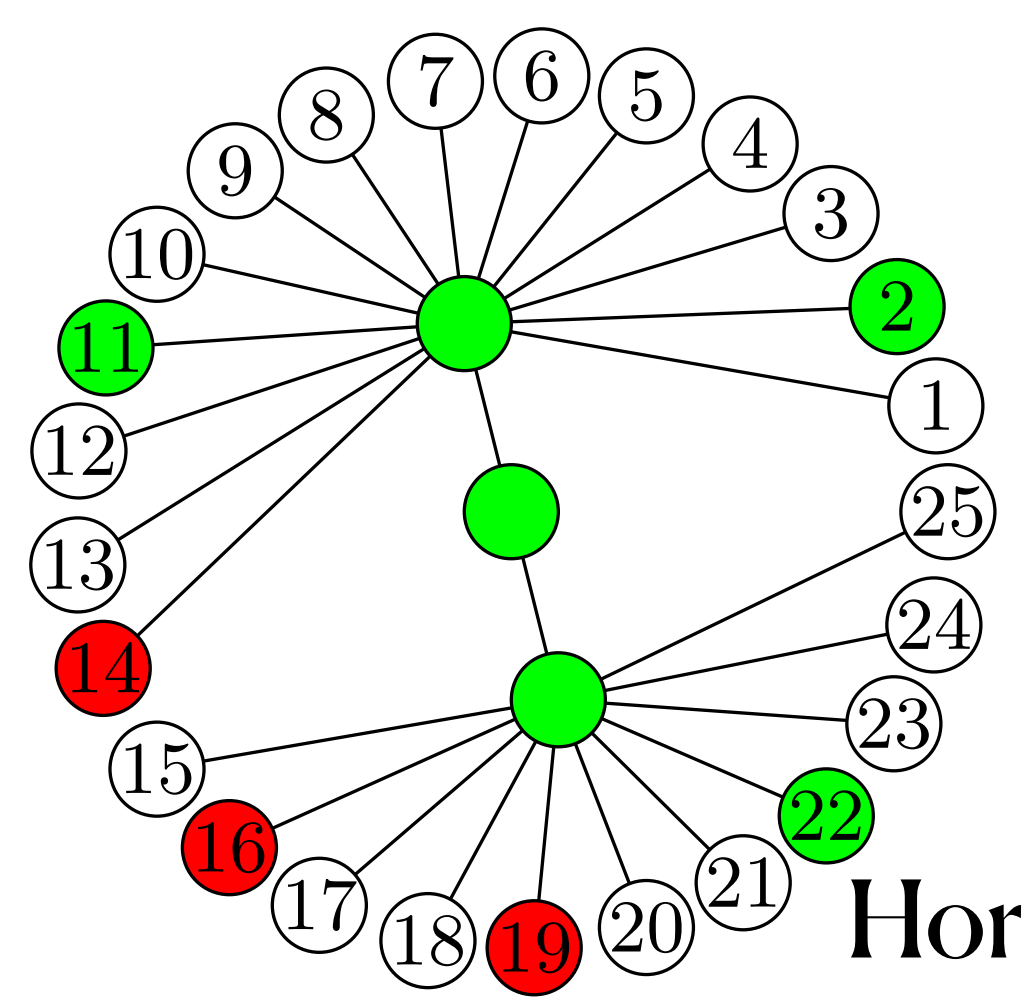
Original
Merkle Tree



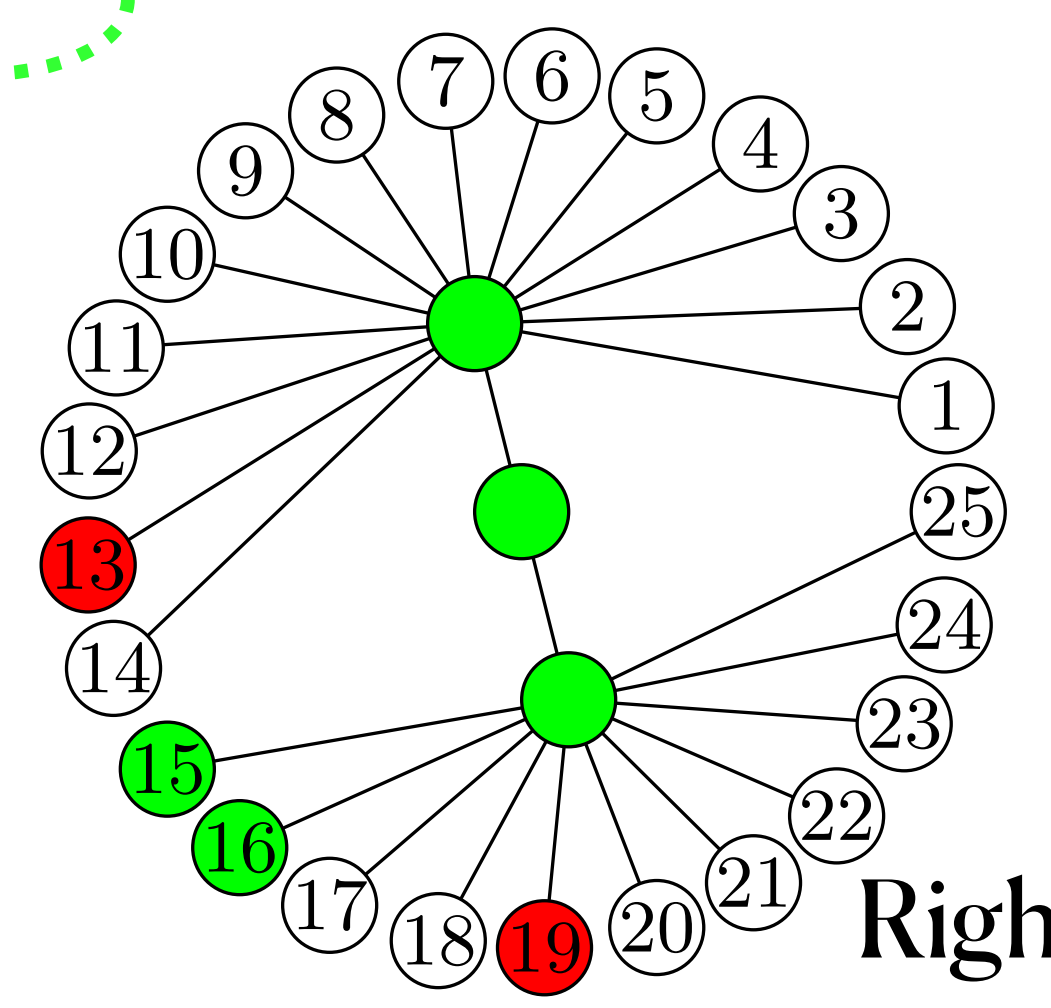
Lattice



Left
Merkle Tree



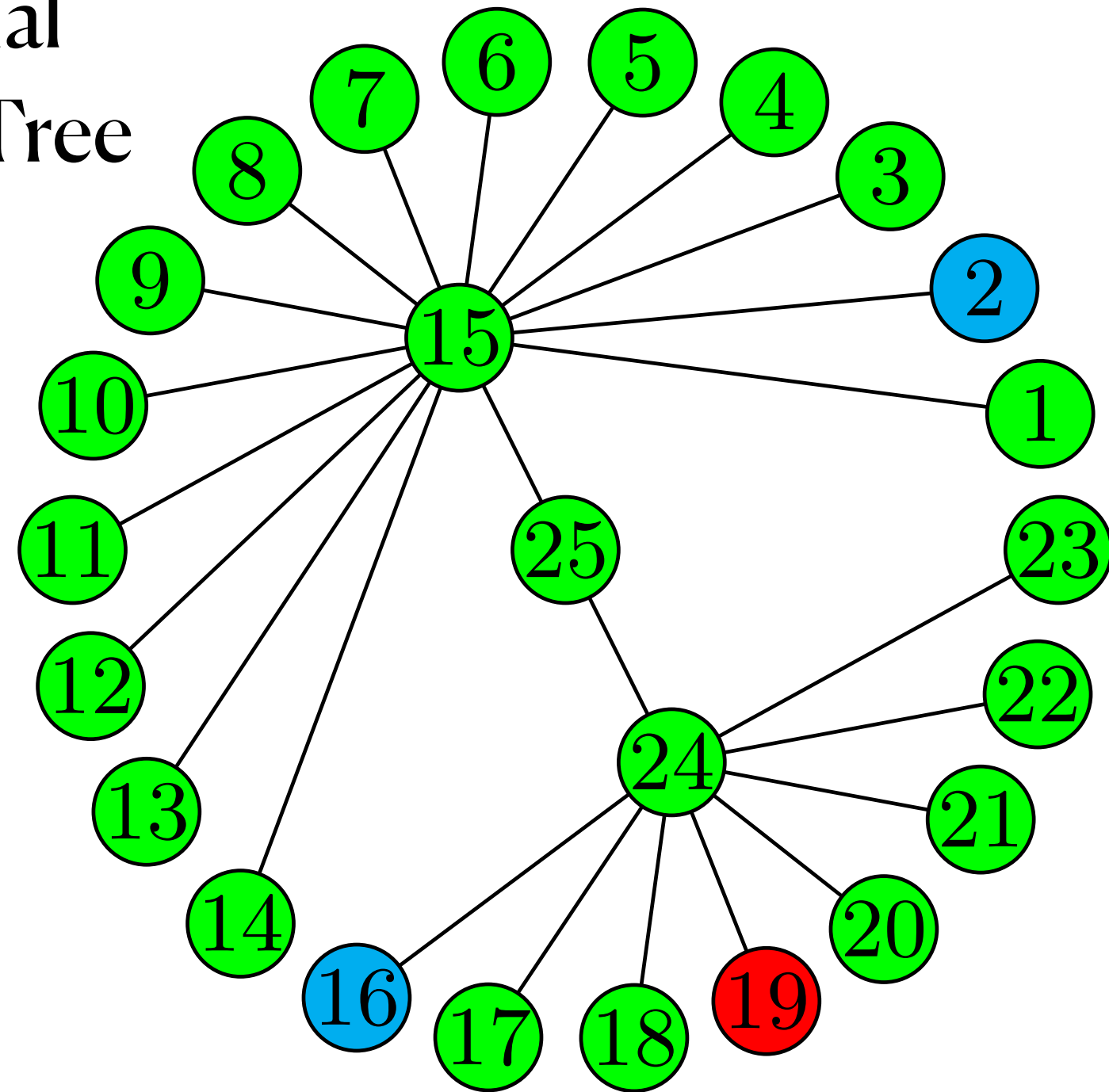
Horizontal
Merkle Tree



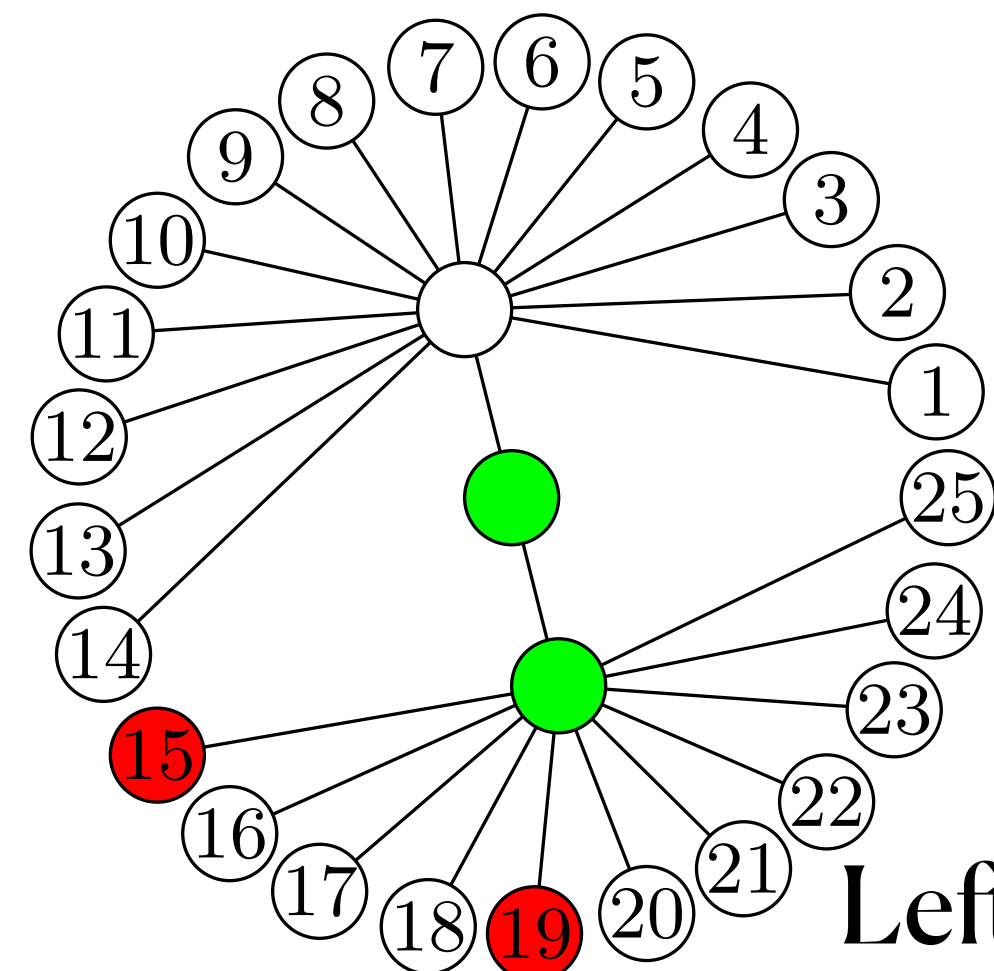
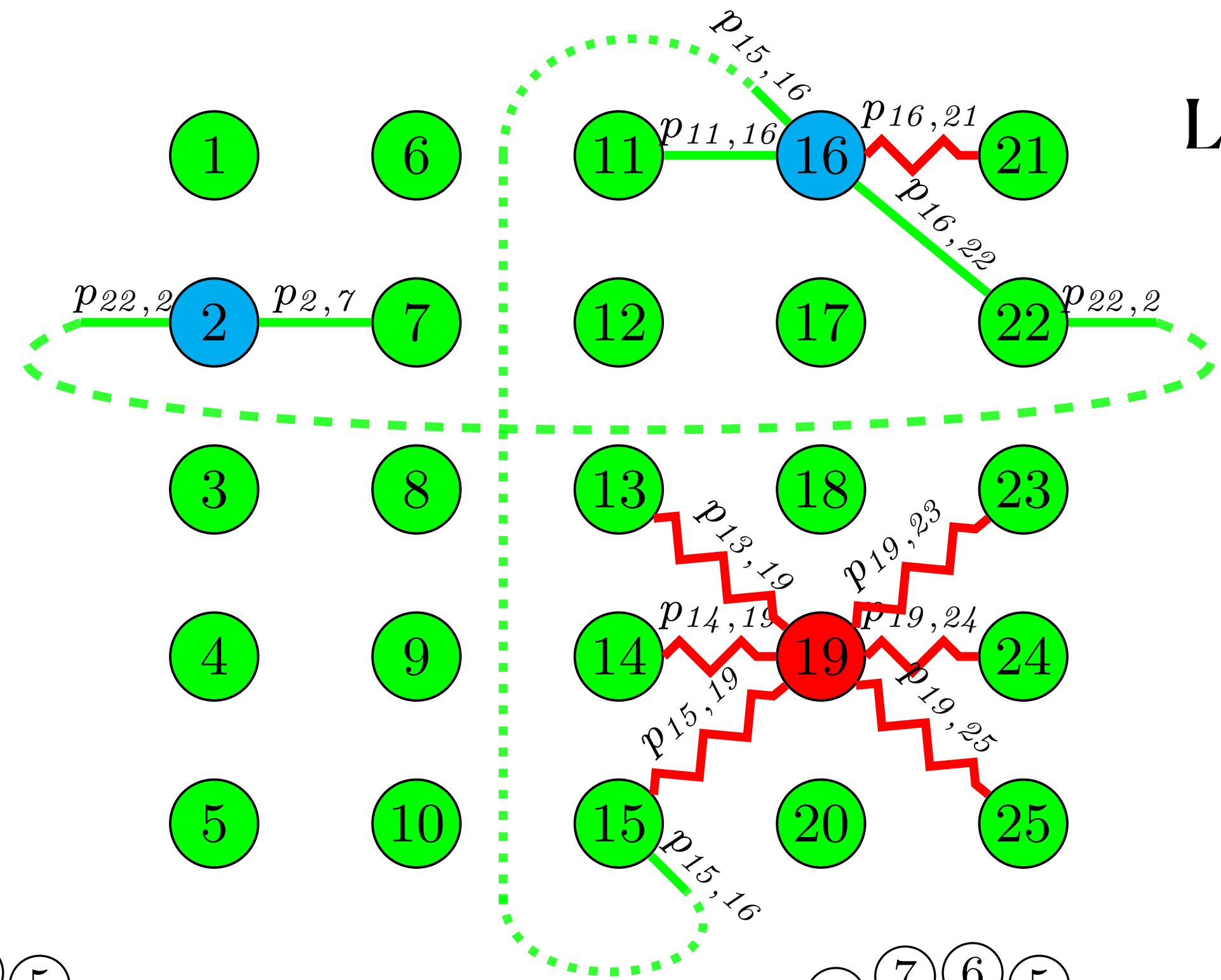
Right
Merkle Tree

Repairing Data Chunk 19

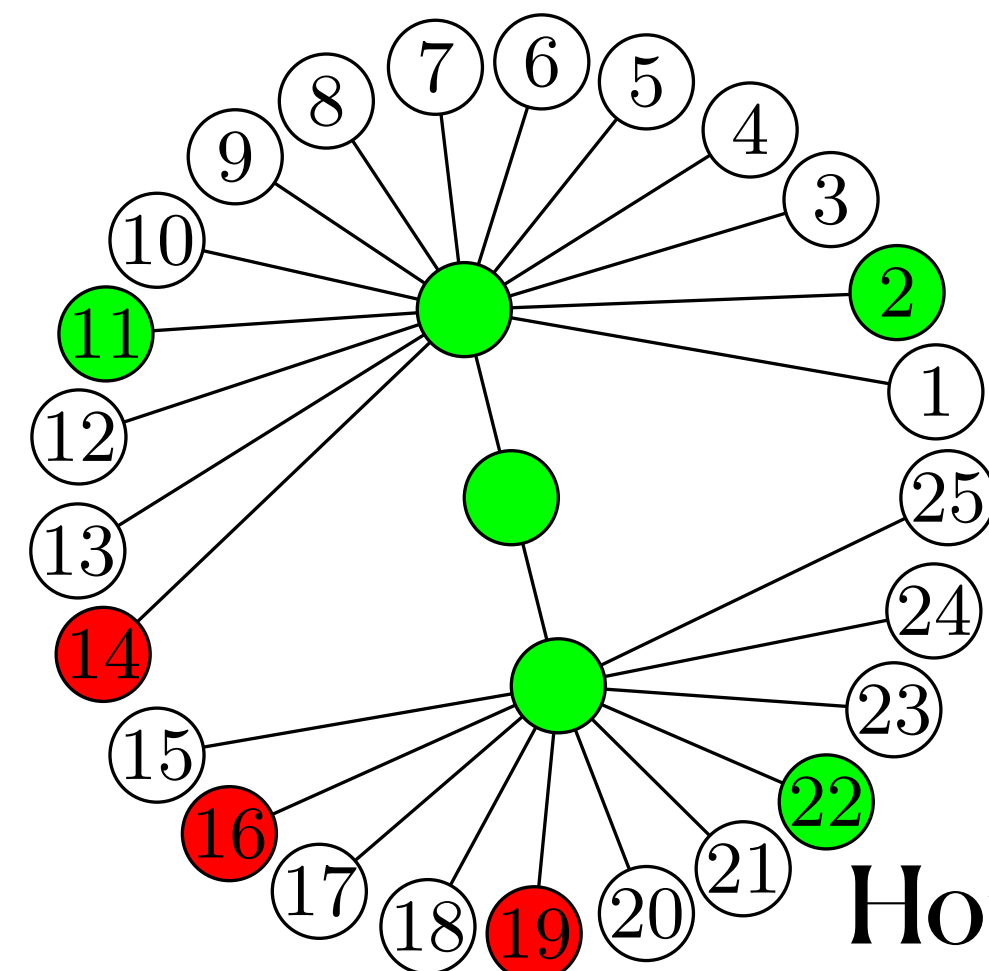
Original
Merkle Tree



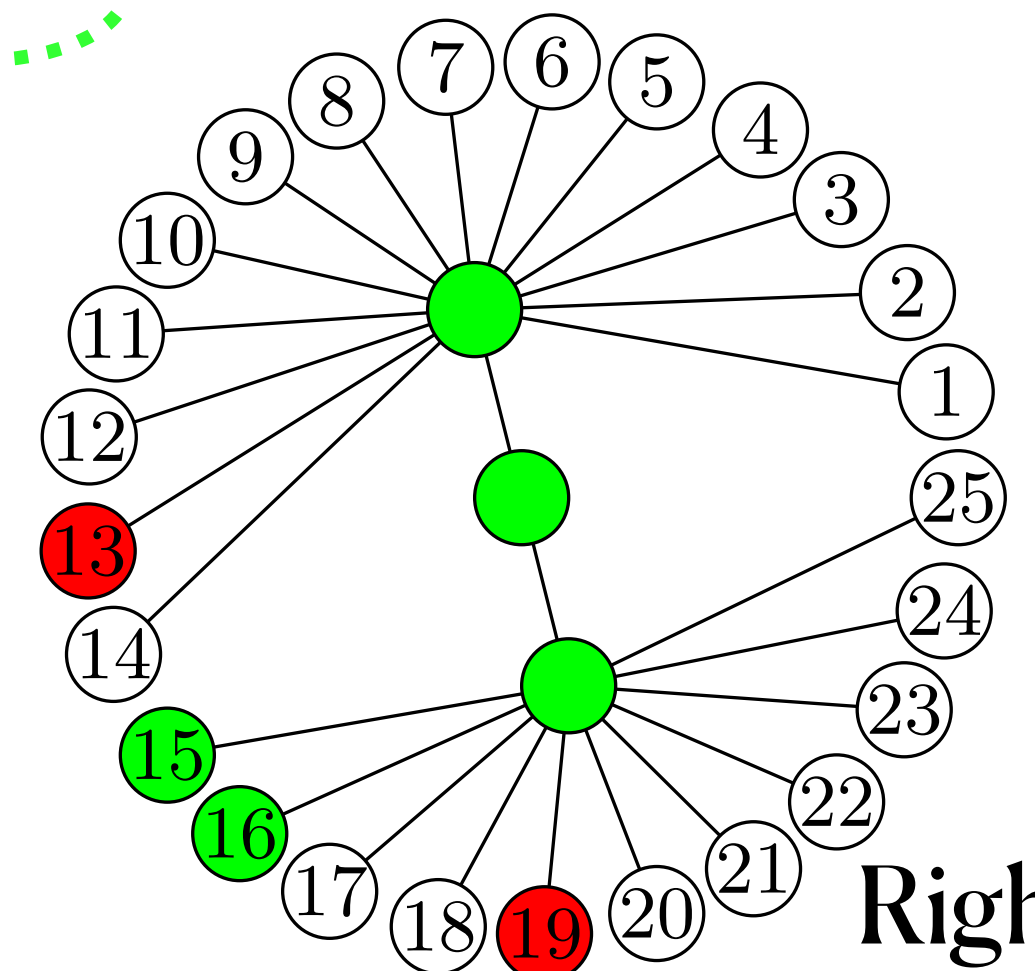
Lattice



Left
Merkle Tree



Horizontal
Merkle Tree

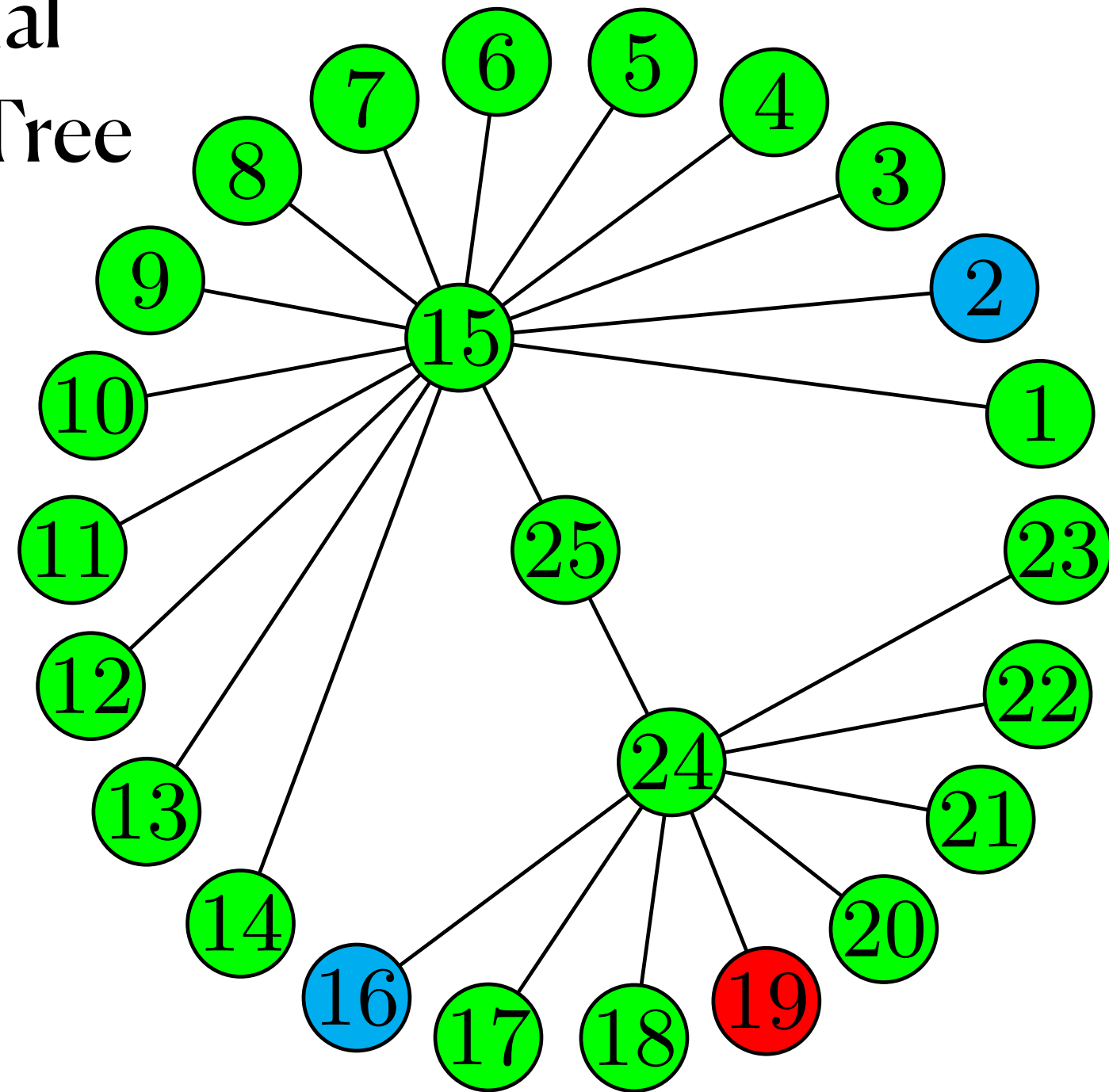


Right
Merkle Tree

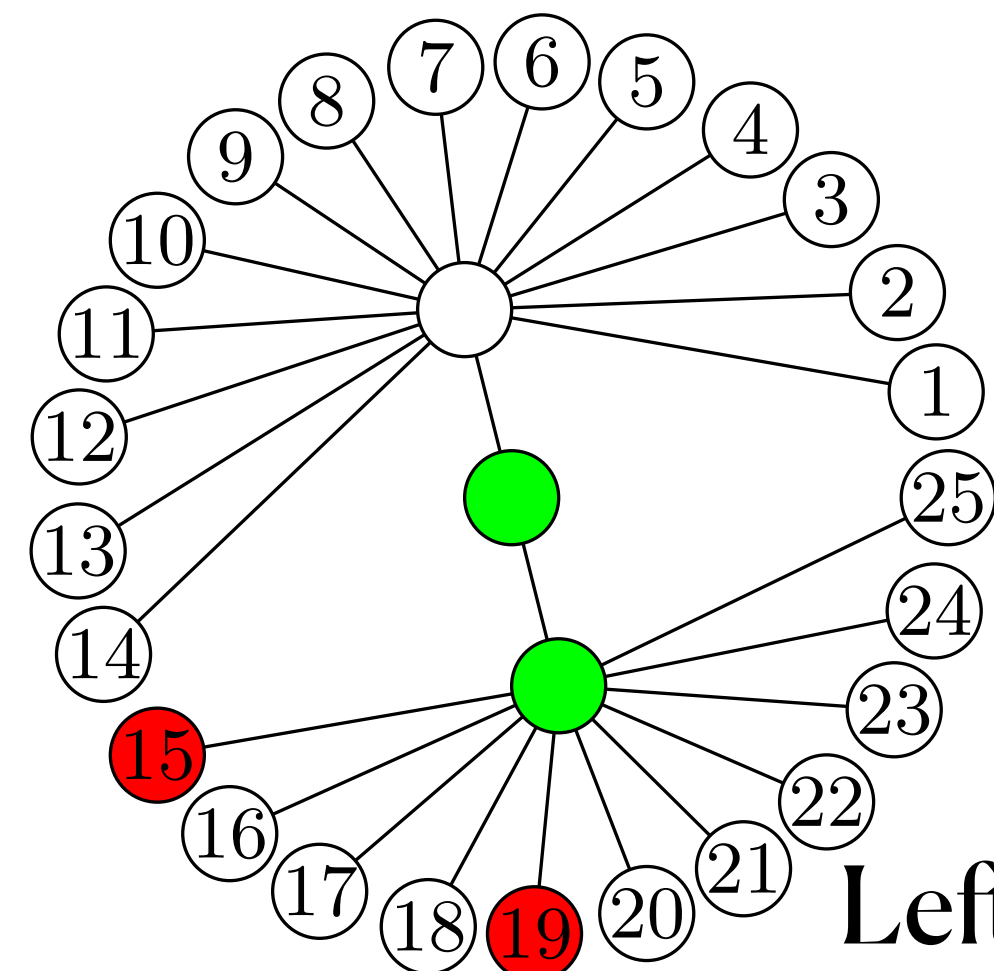
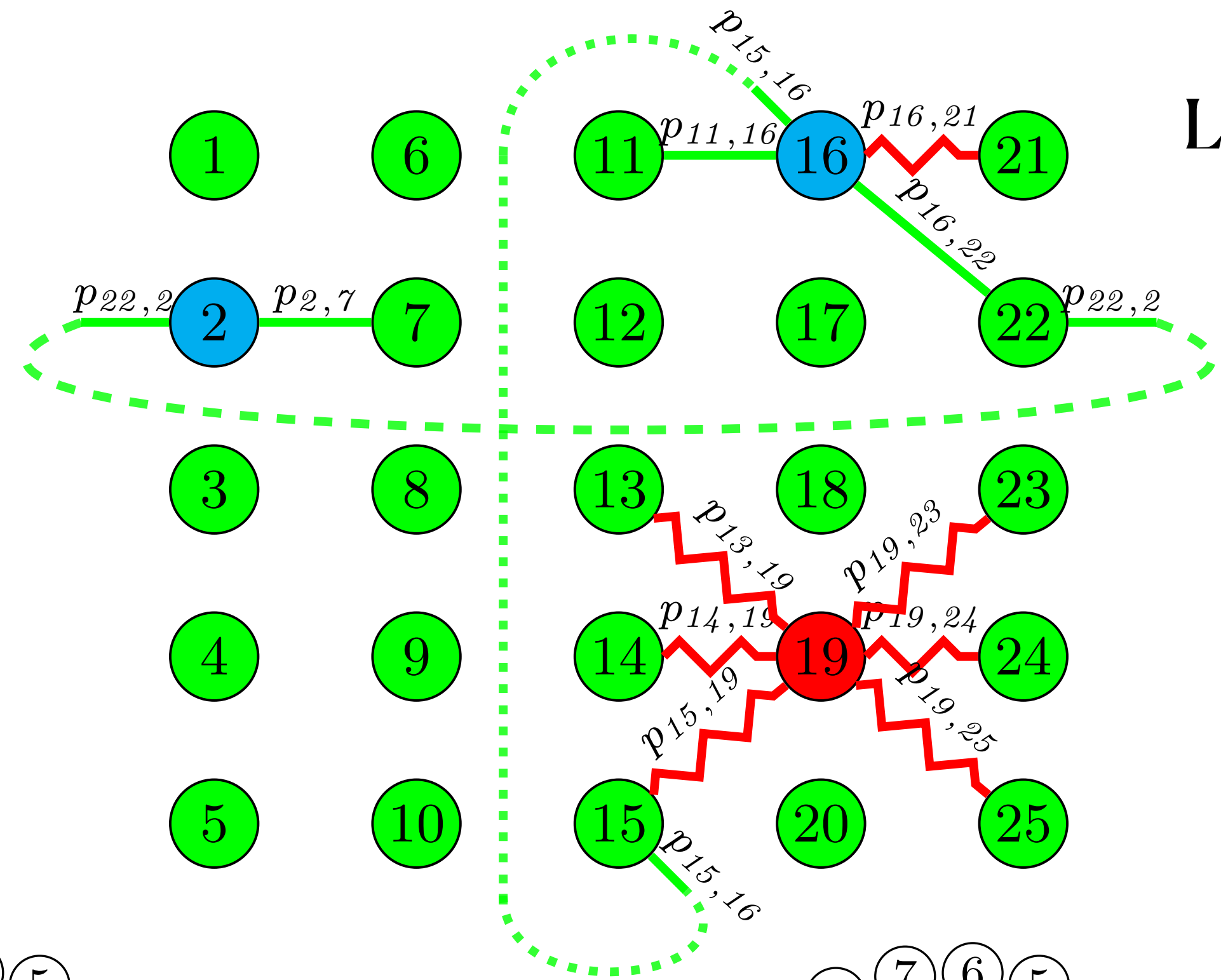
Now What?

Repairing Data Chunk 19

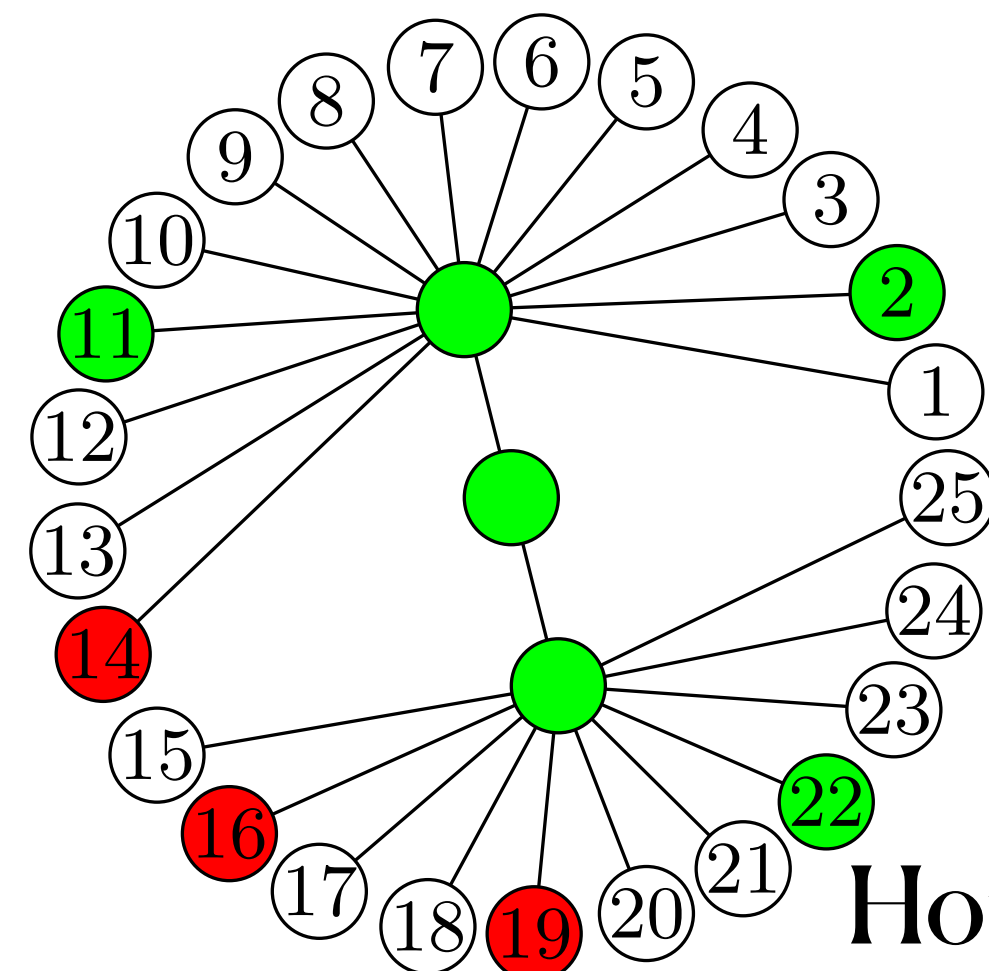
Original
Merkle Tree



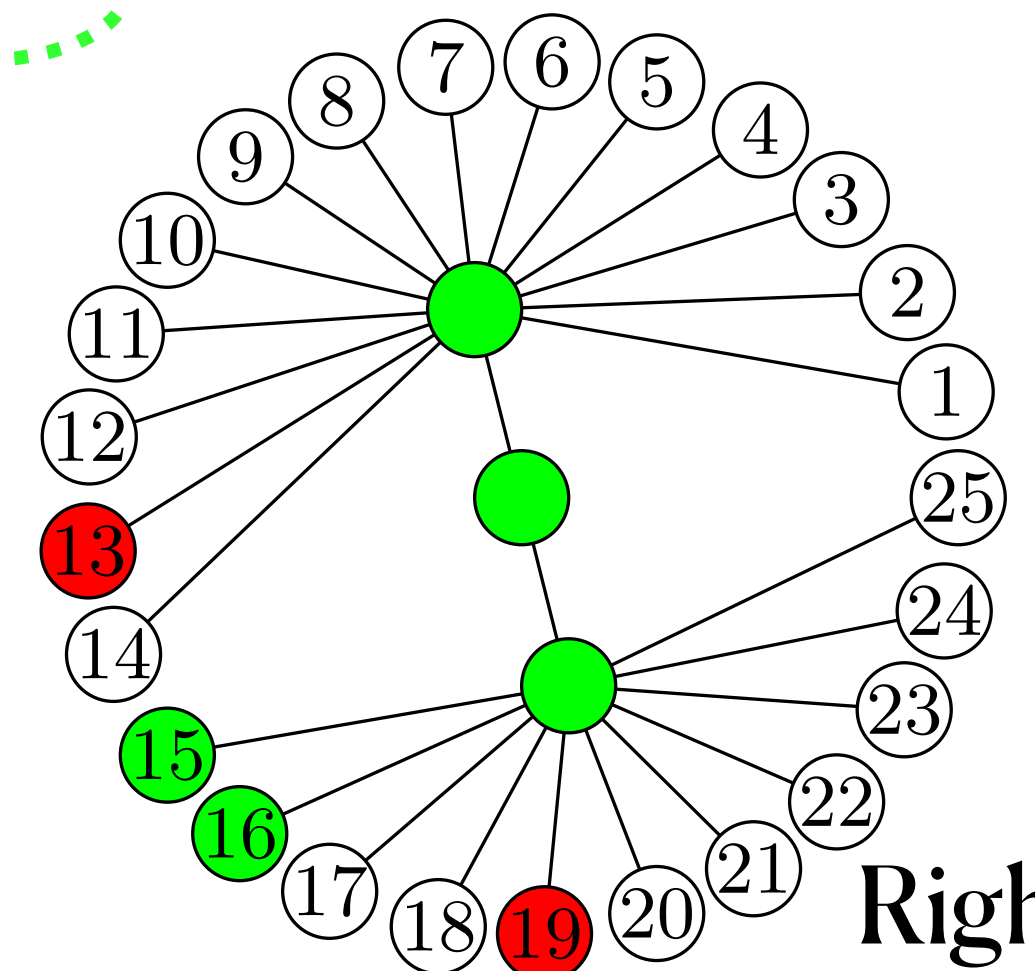
Lattice



Left
Merkle Tree



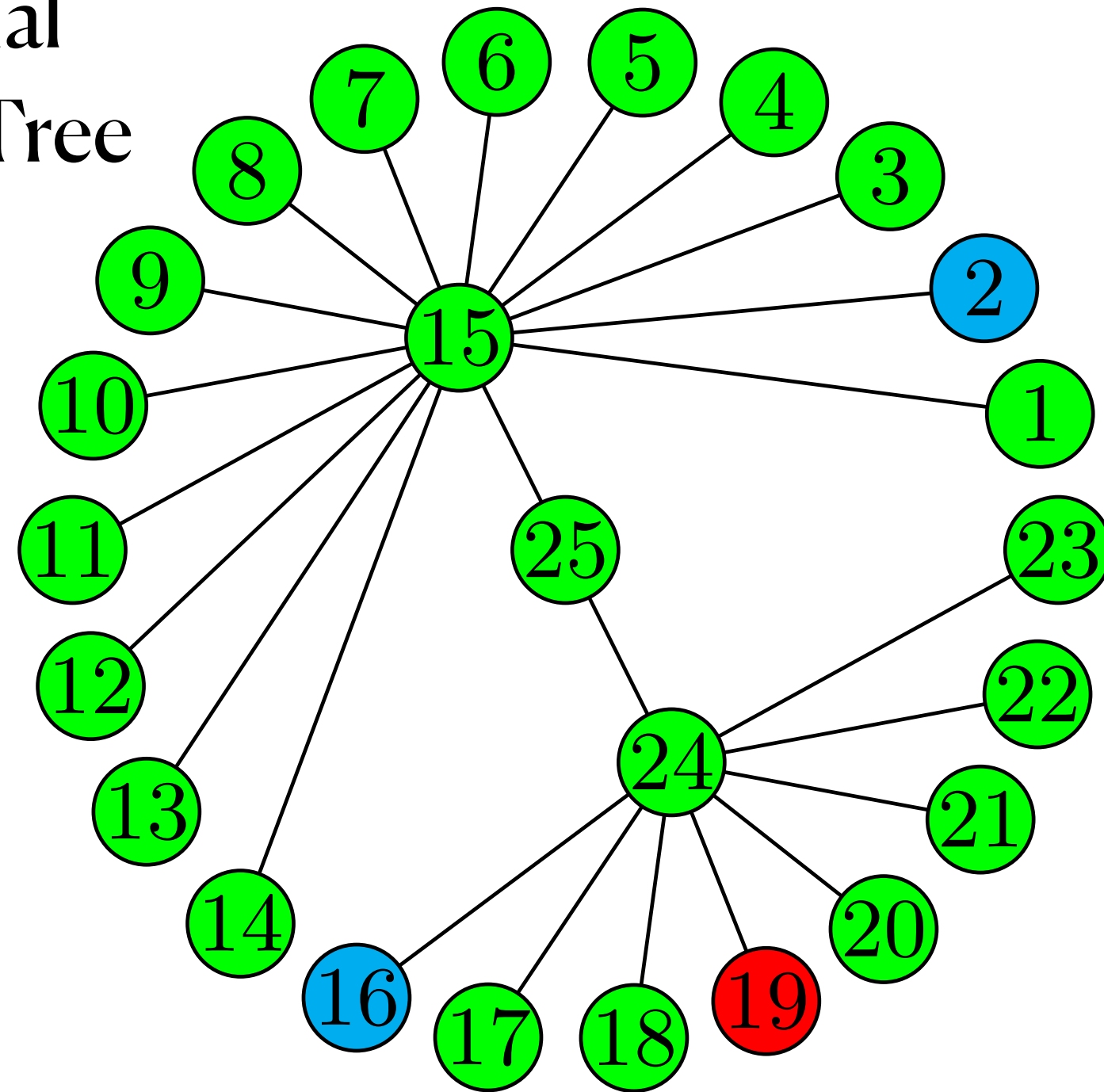
Horizontal
Merkle Tree



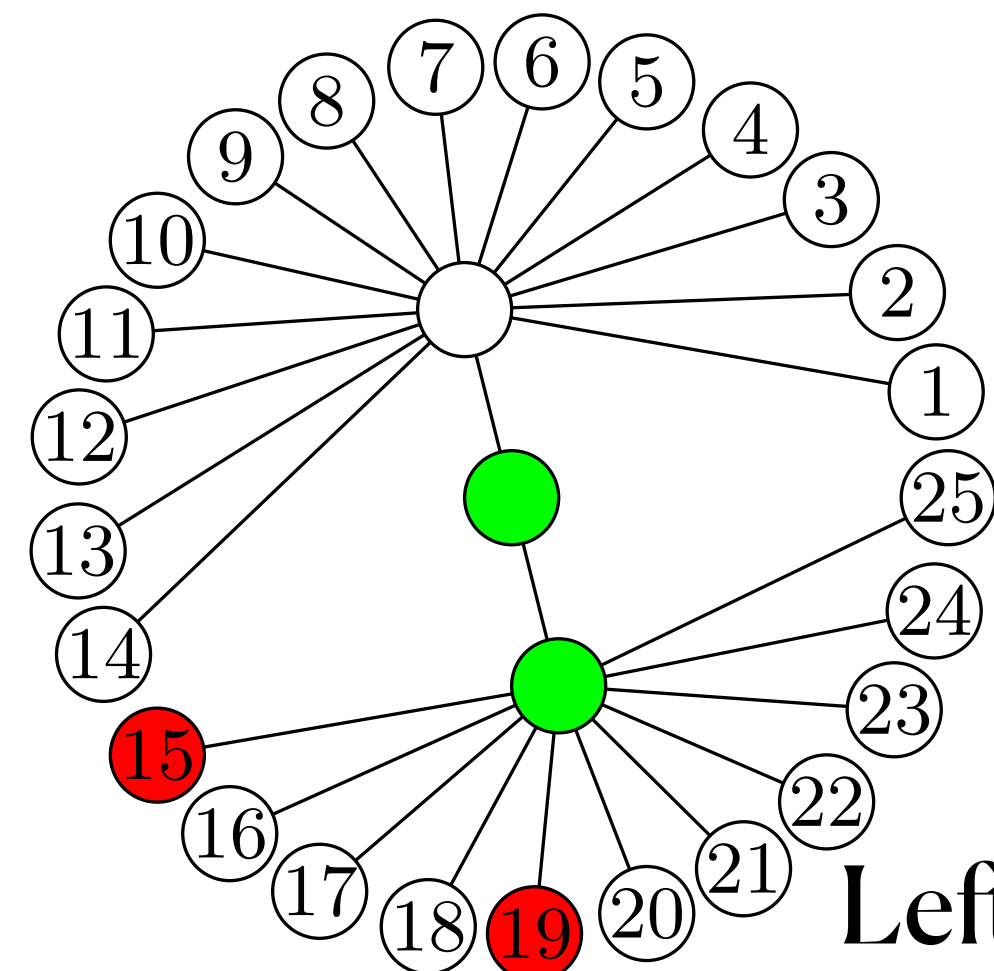
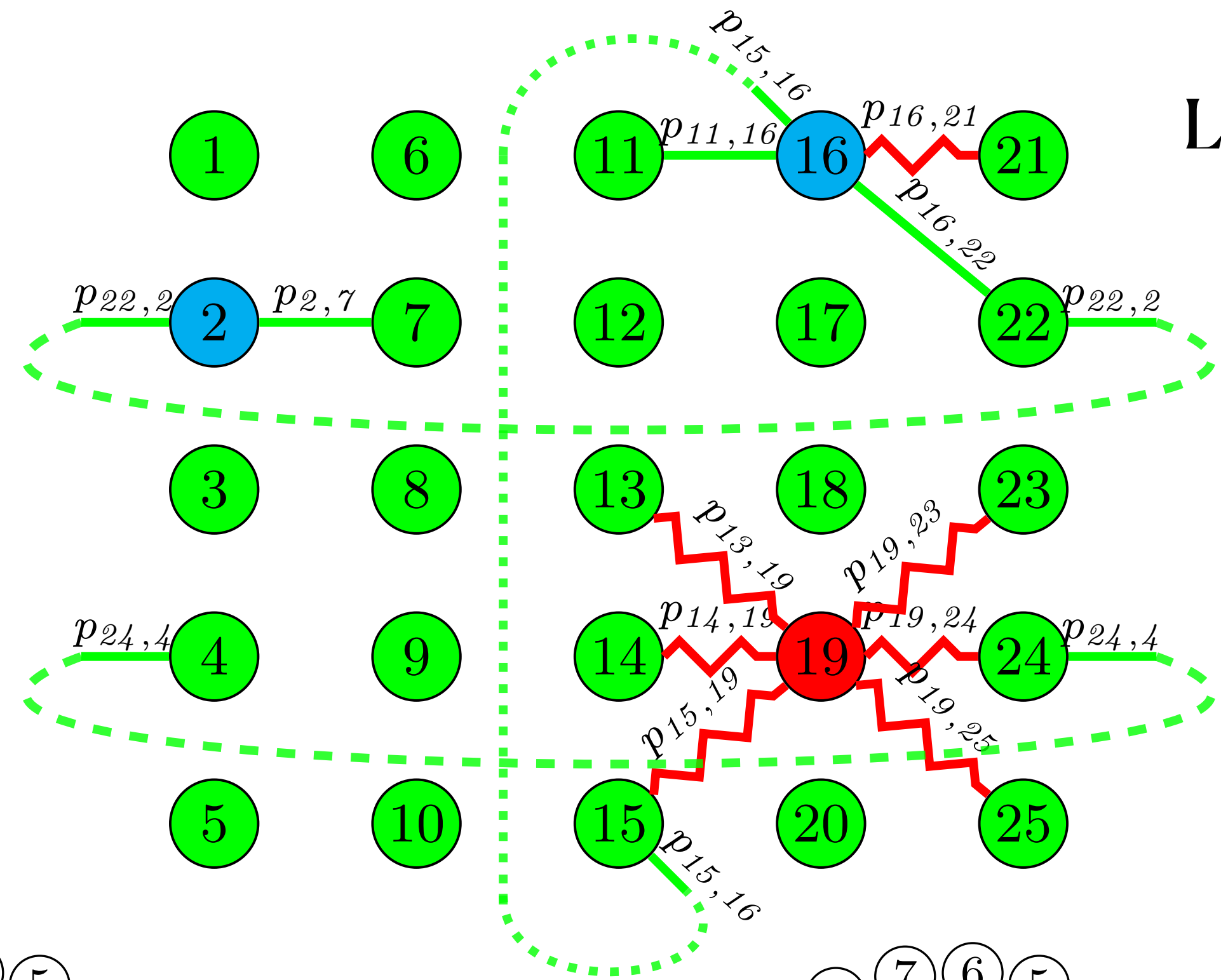
Right
Merkle Tree

Repairing Parity Chunk 19-24

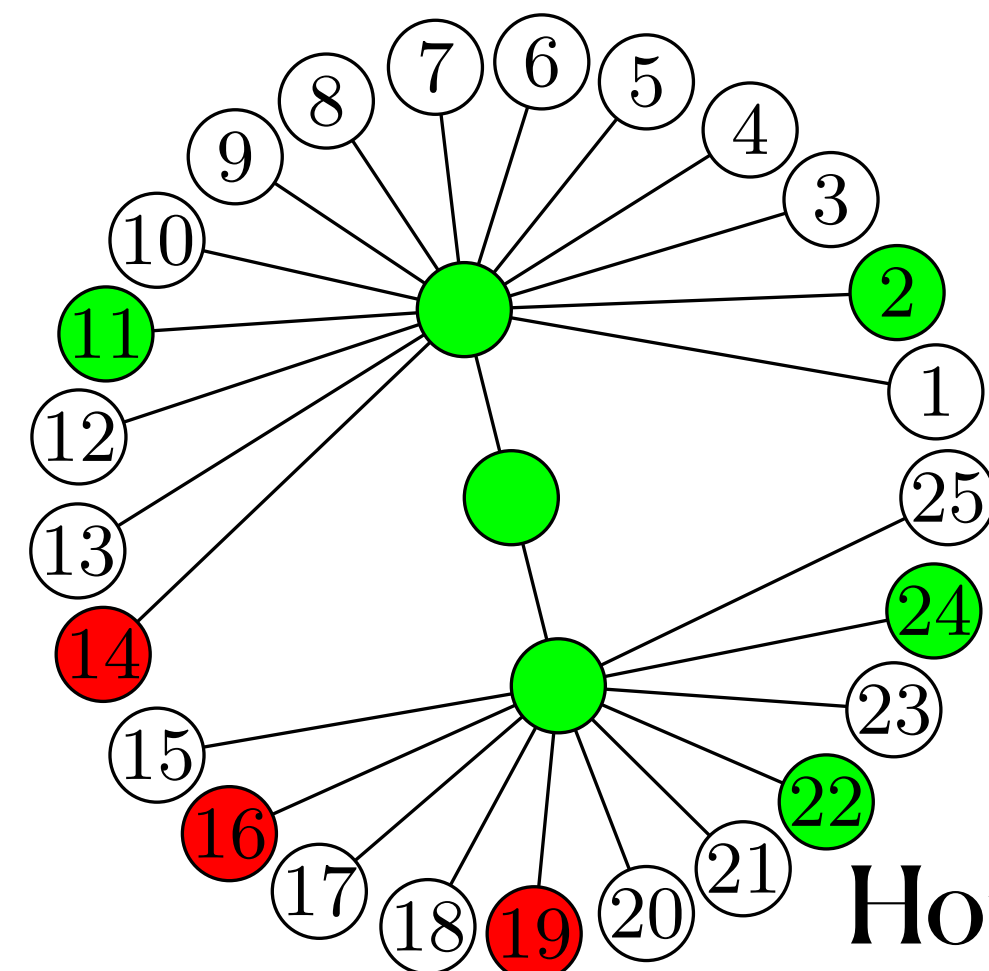
Original
Merkle Tree



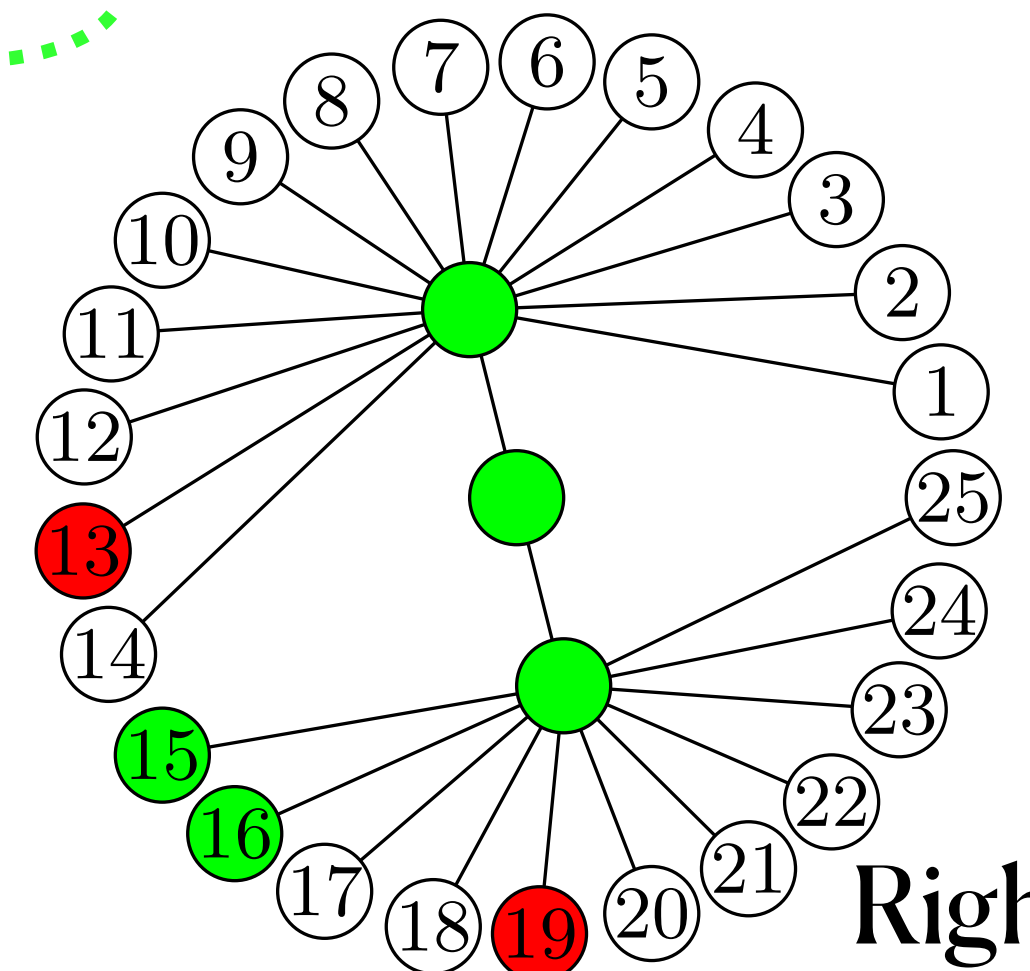
Lattice



Left
Merkle Tree



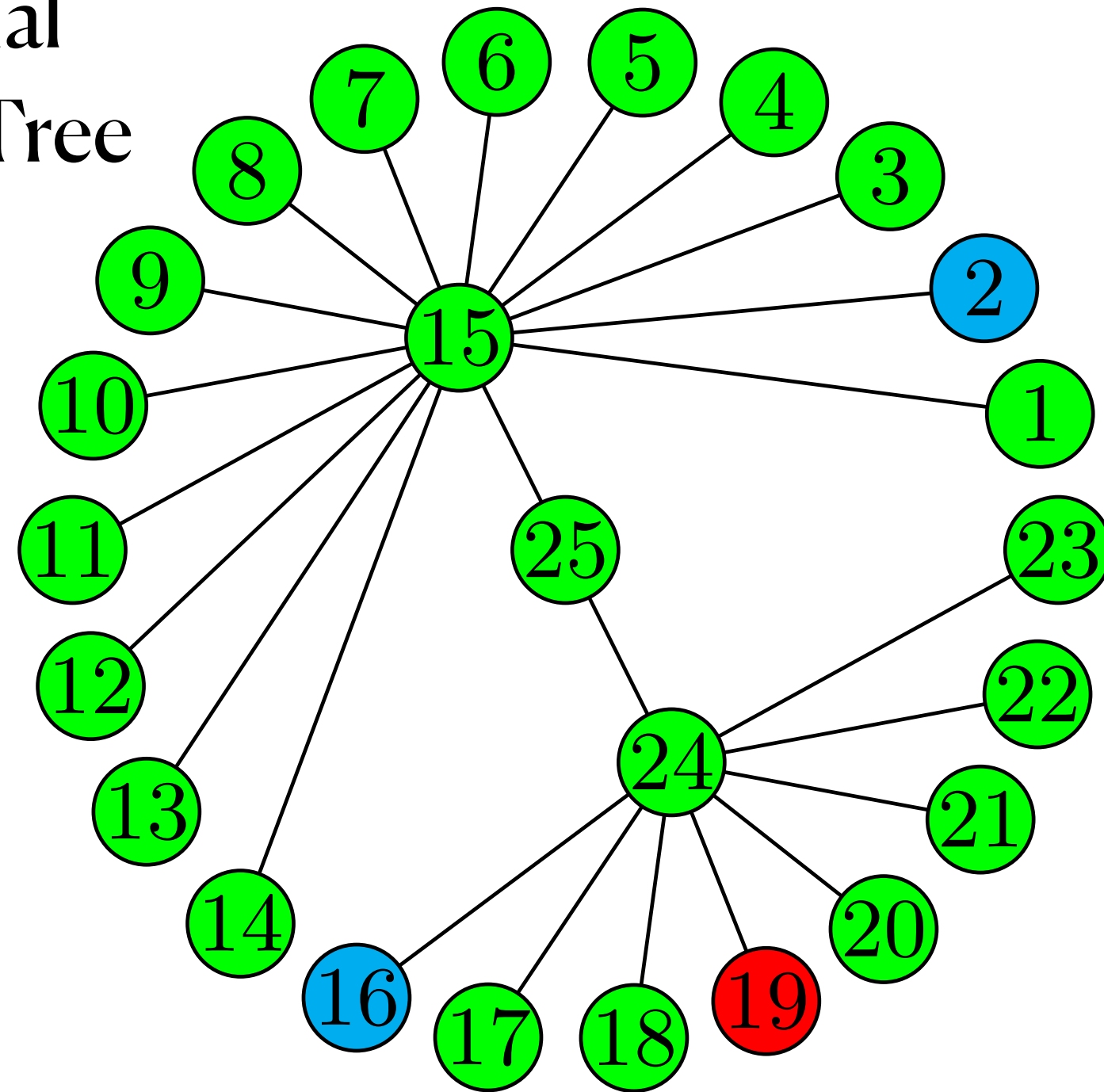
Horizontal
Merkle Tree



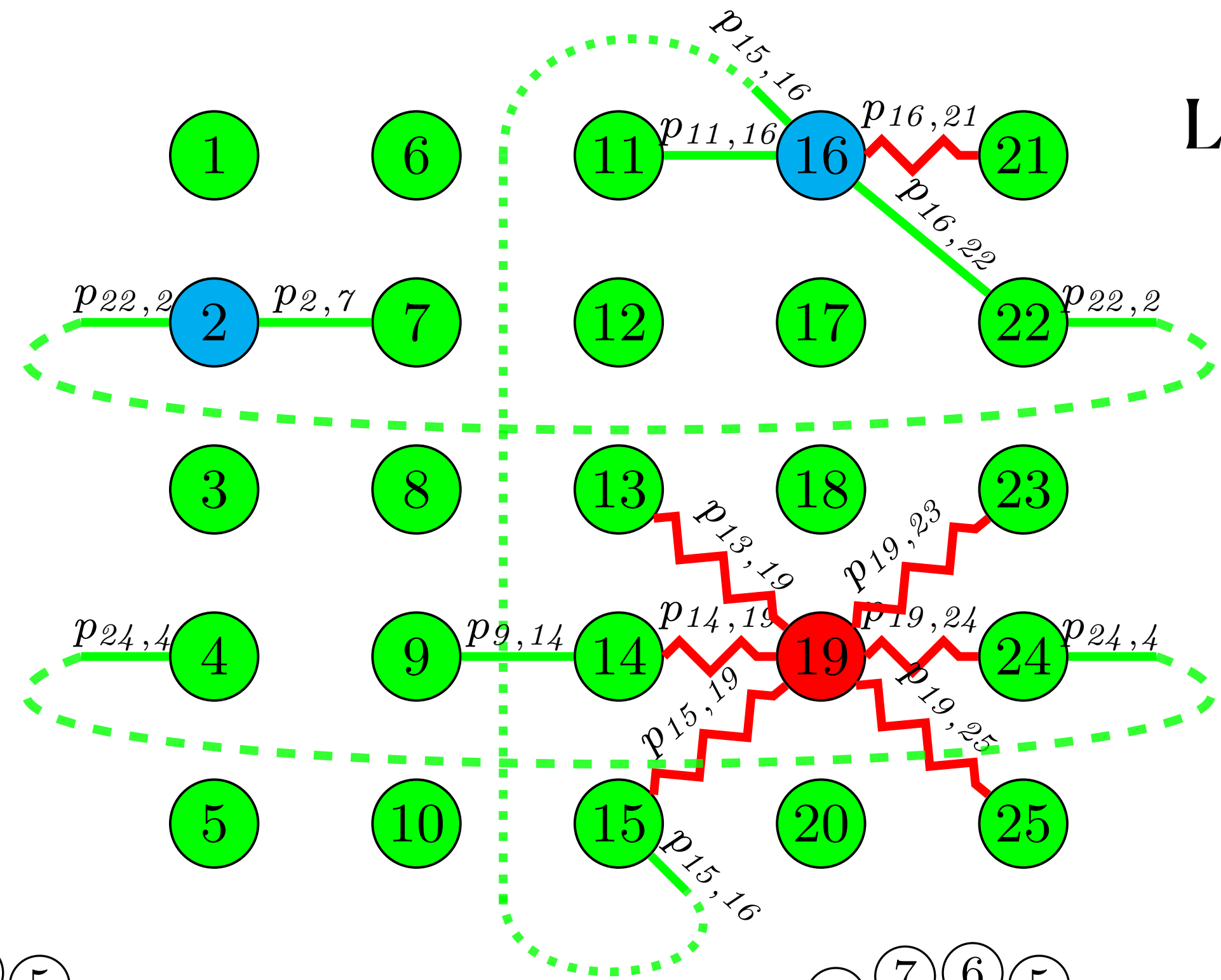
Right
Merkle Tree

Repairing Parity Chunk 14-19

Original Merkle Tree

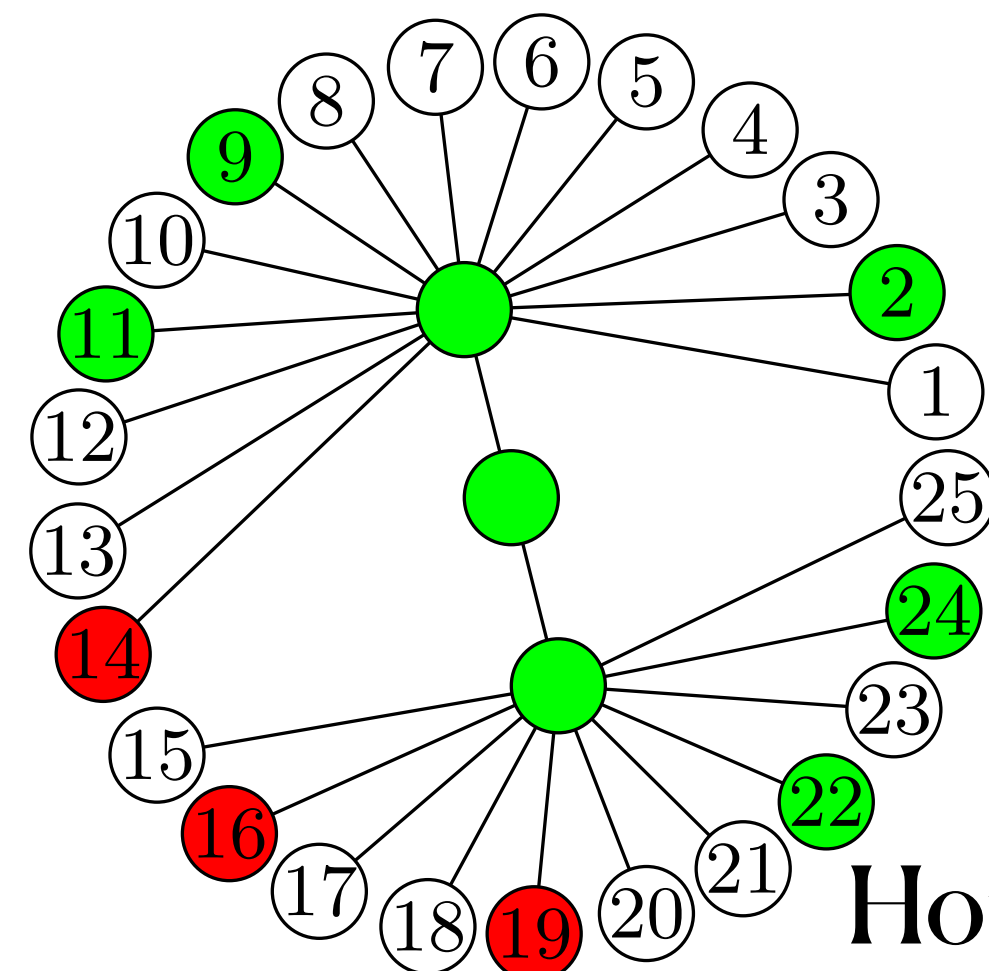


Lattice

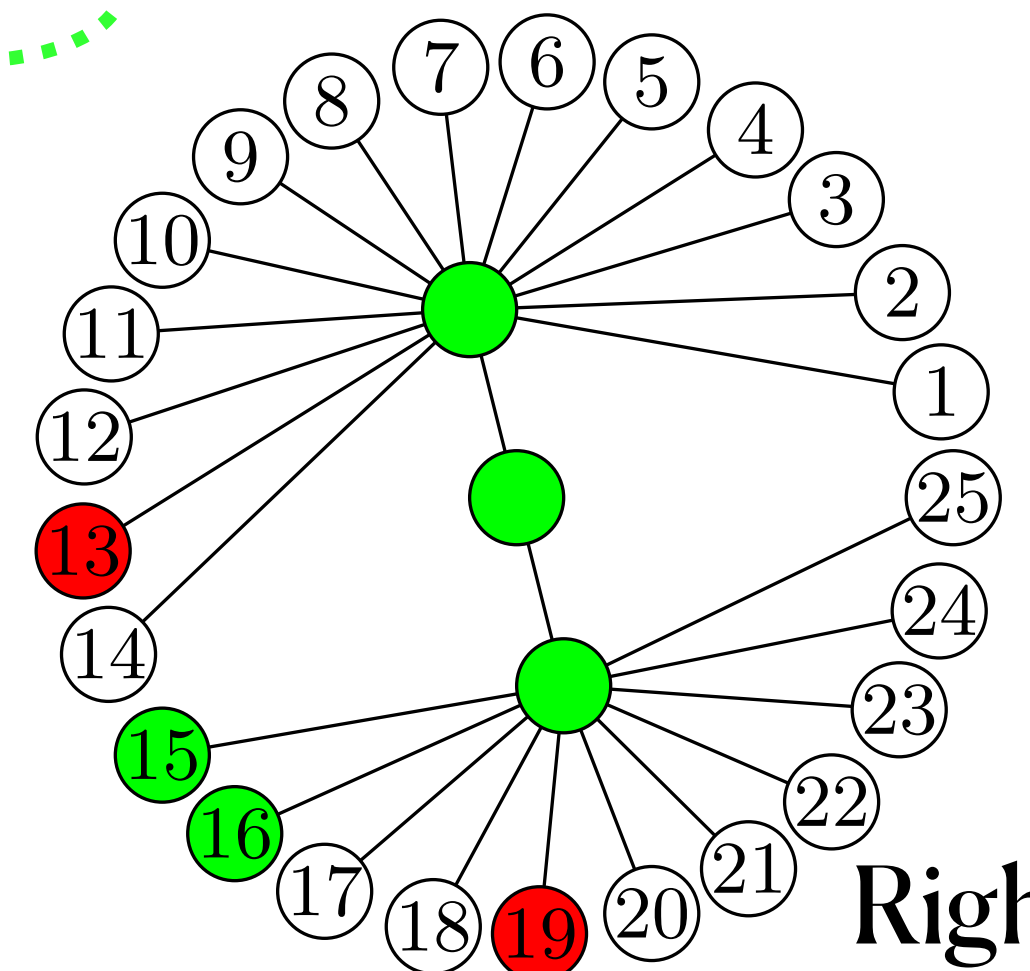


A circular graph with 25 nodes. The top half (nodes 1-14) is connected to a central white node. The bottom half (nodes 15-25) is connected to a central green node. Nodes 15 and 19 are highlighted in red. The word "Left" is written at the bottom right.

Merkle Tree



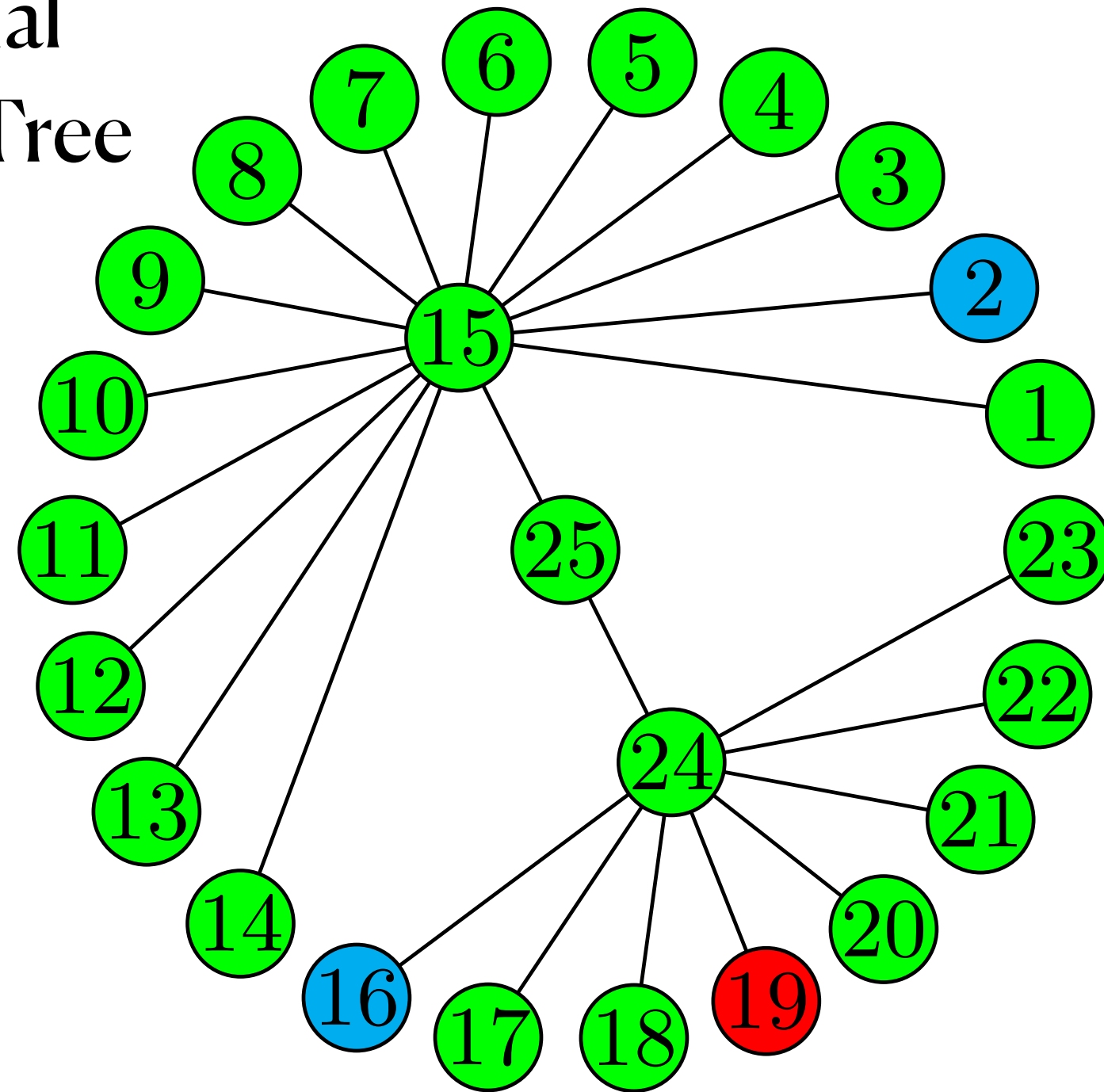
Horizontal Merkle Tree



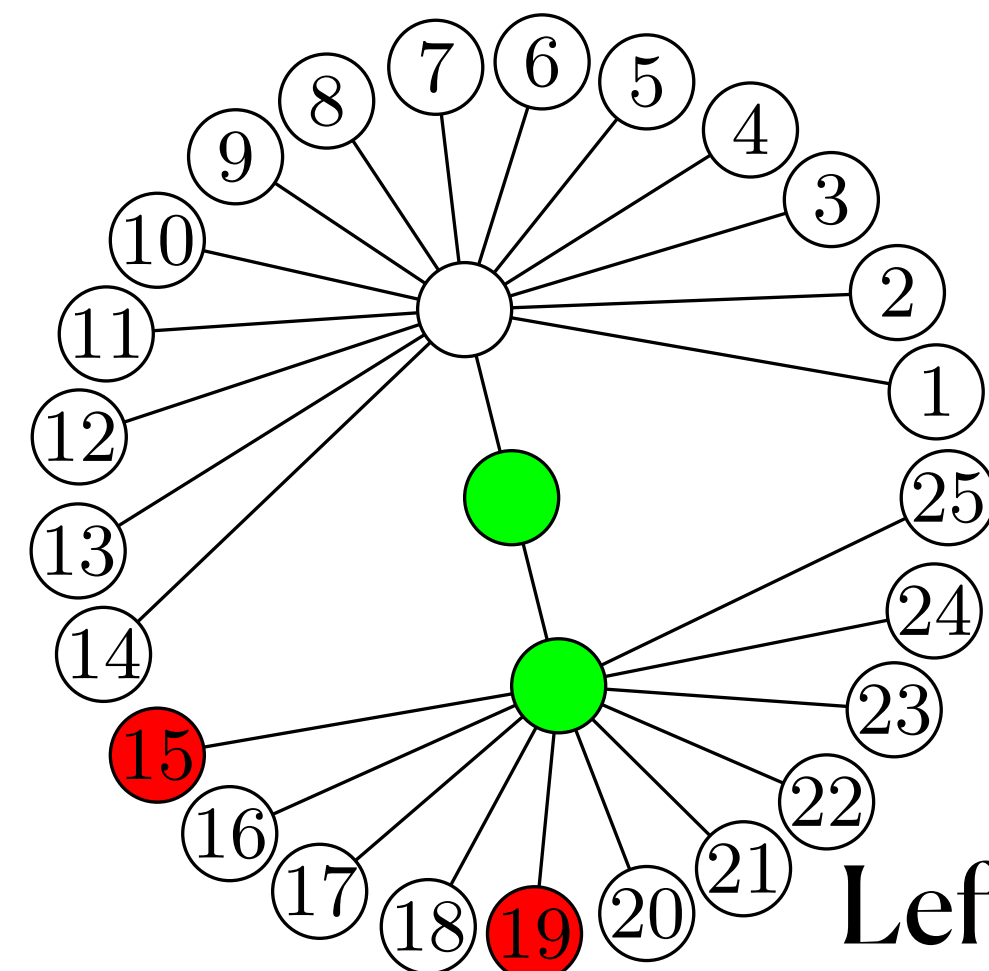
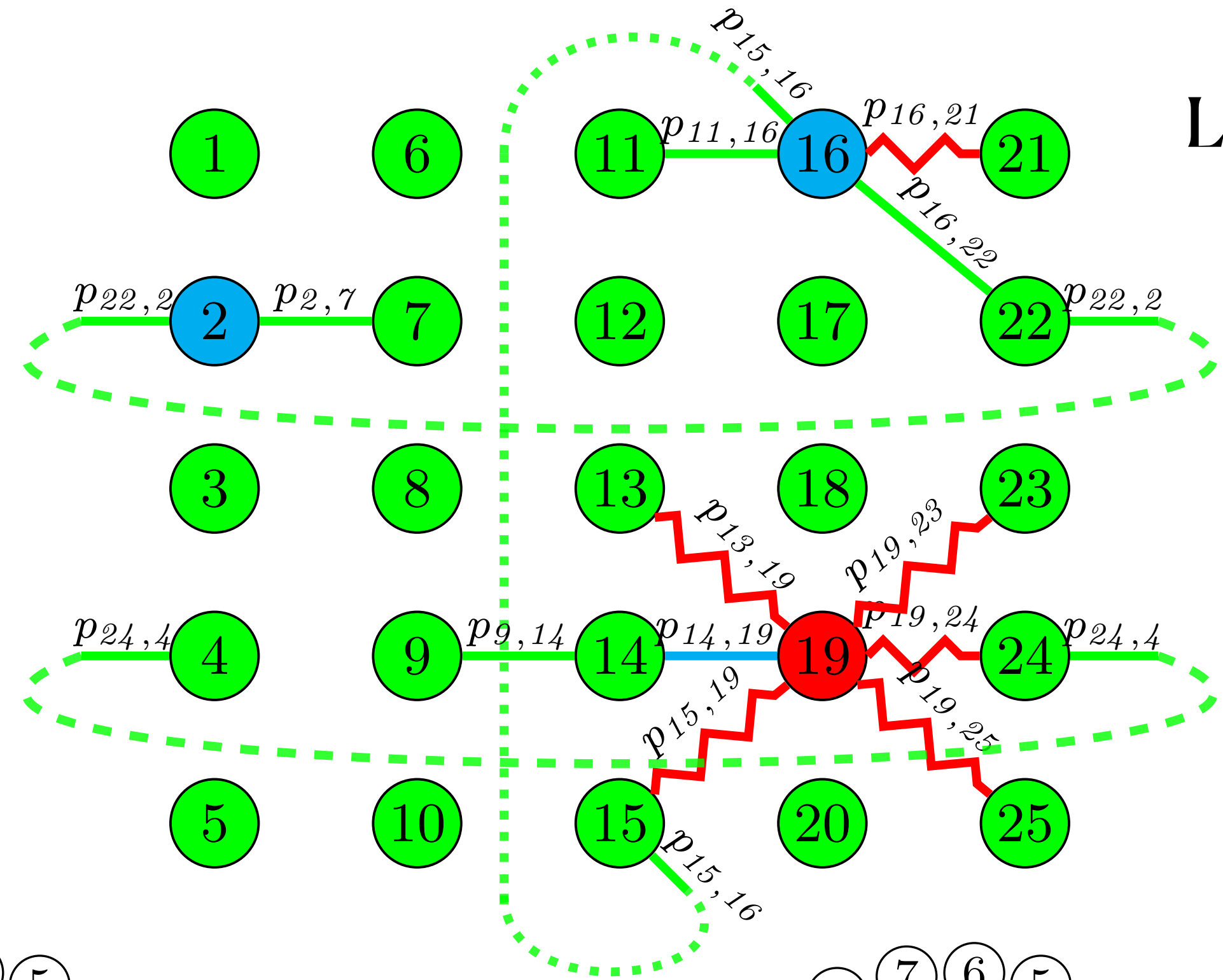
Right Merkle Tree

Repaired Parity Chunk 14-19

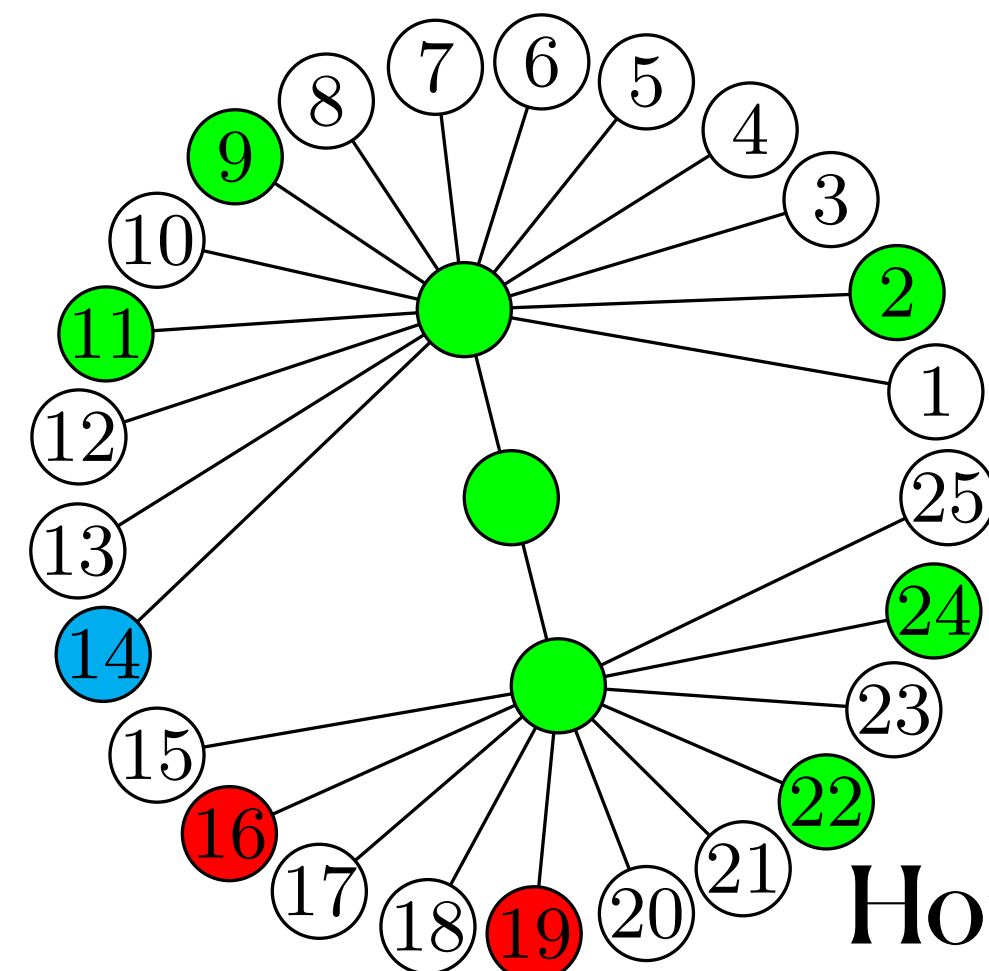
Original
Merkle Tree



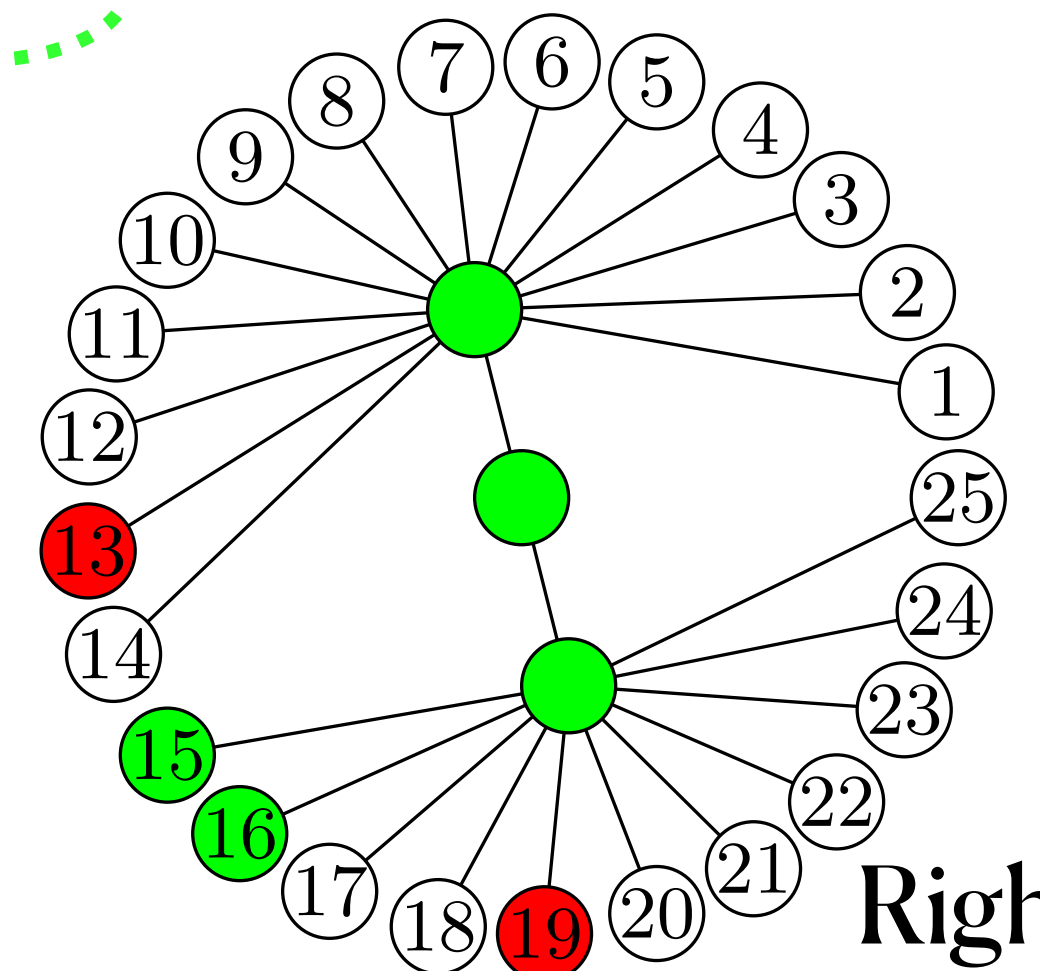
Lattice



Left
Merkle Tree



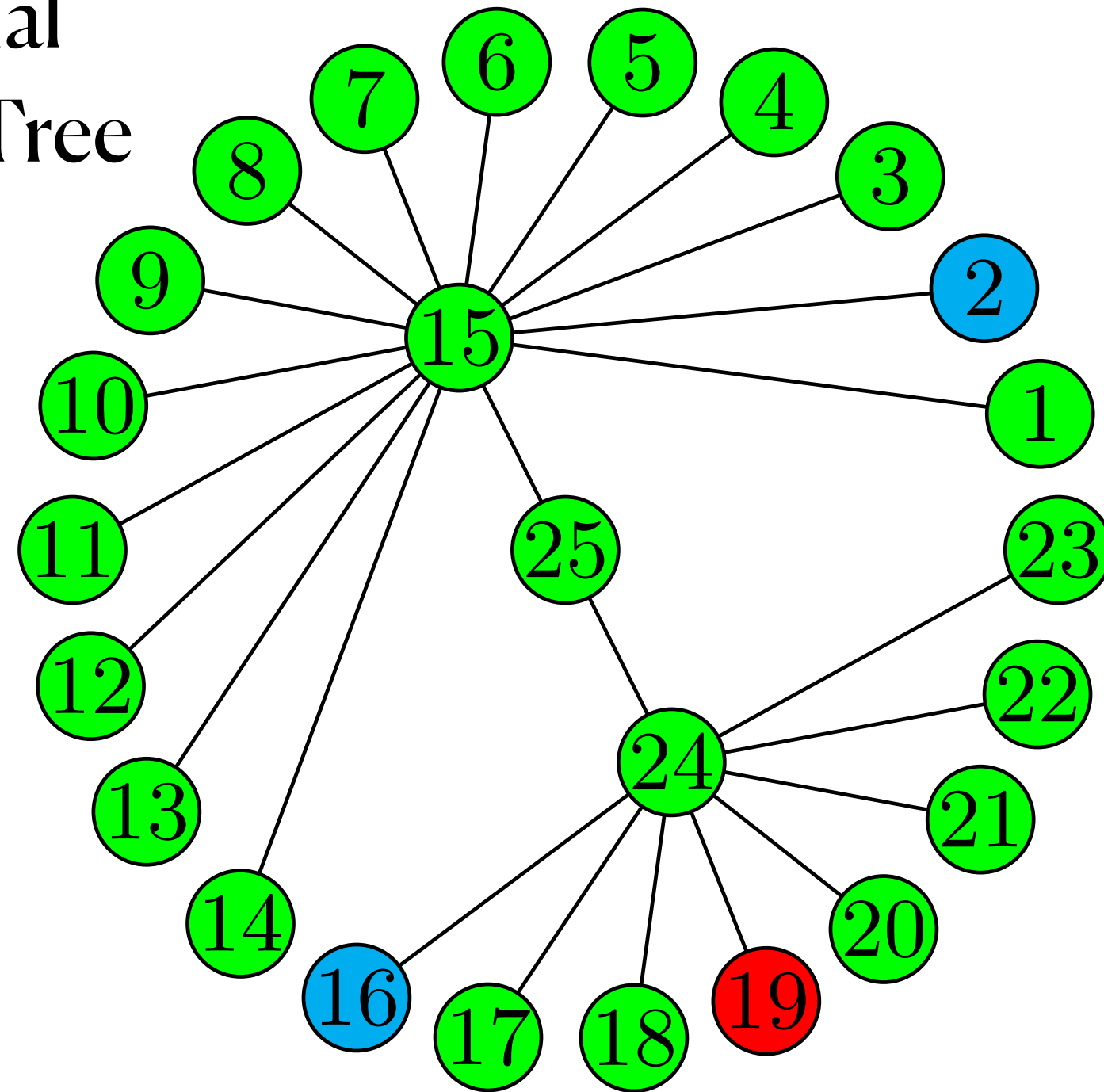
Horizontal
Merkle Tree



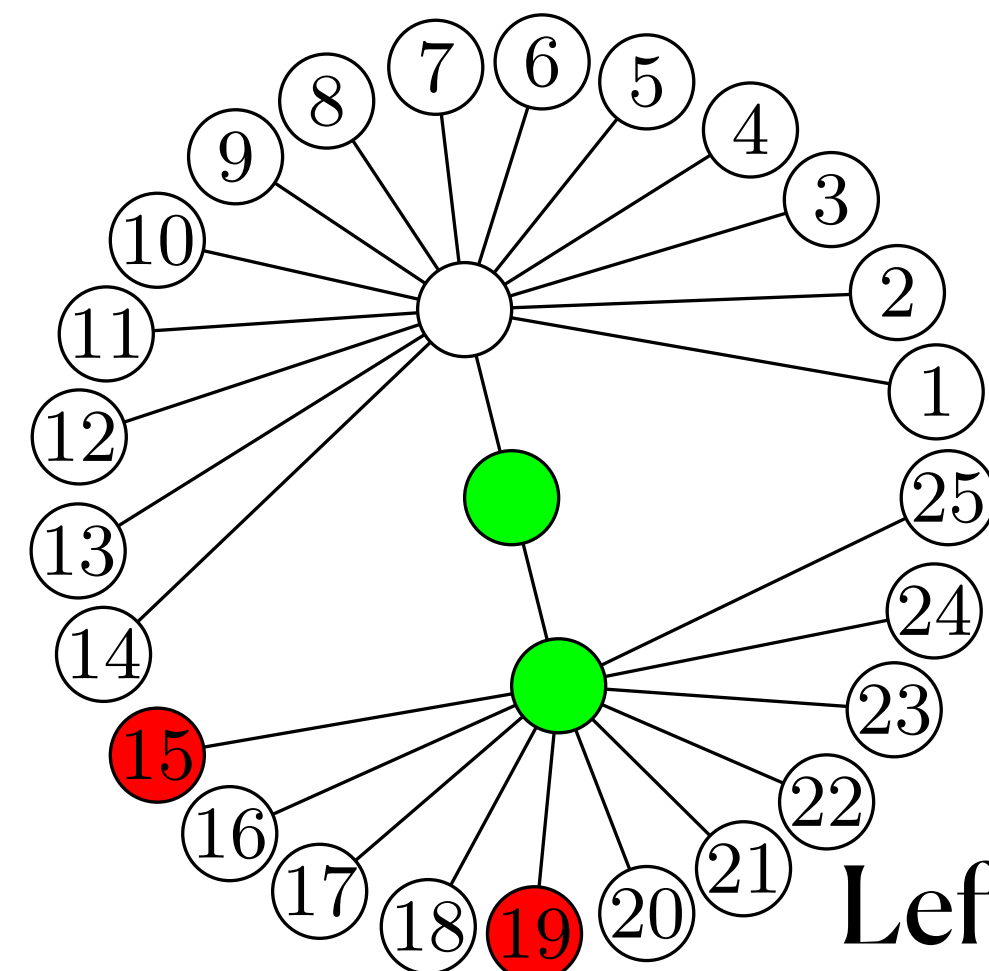
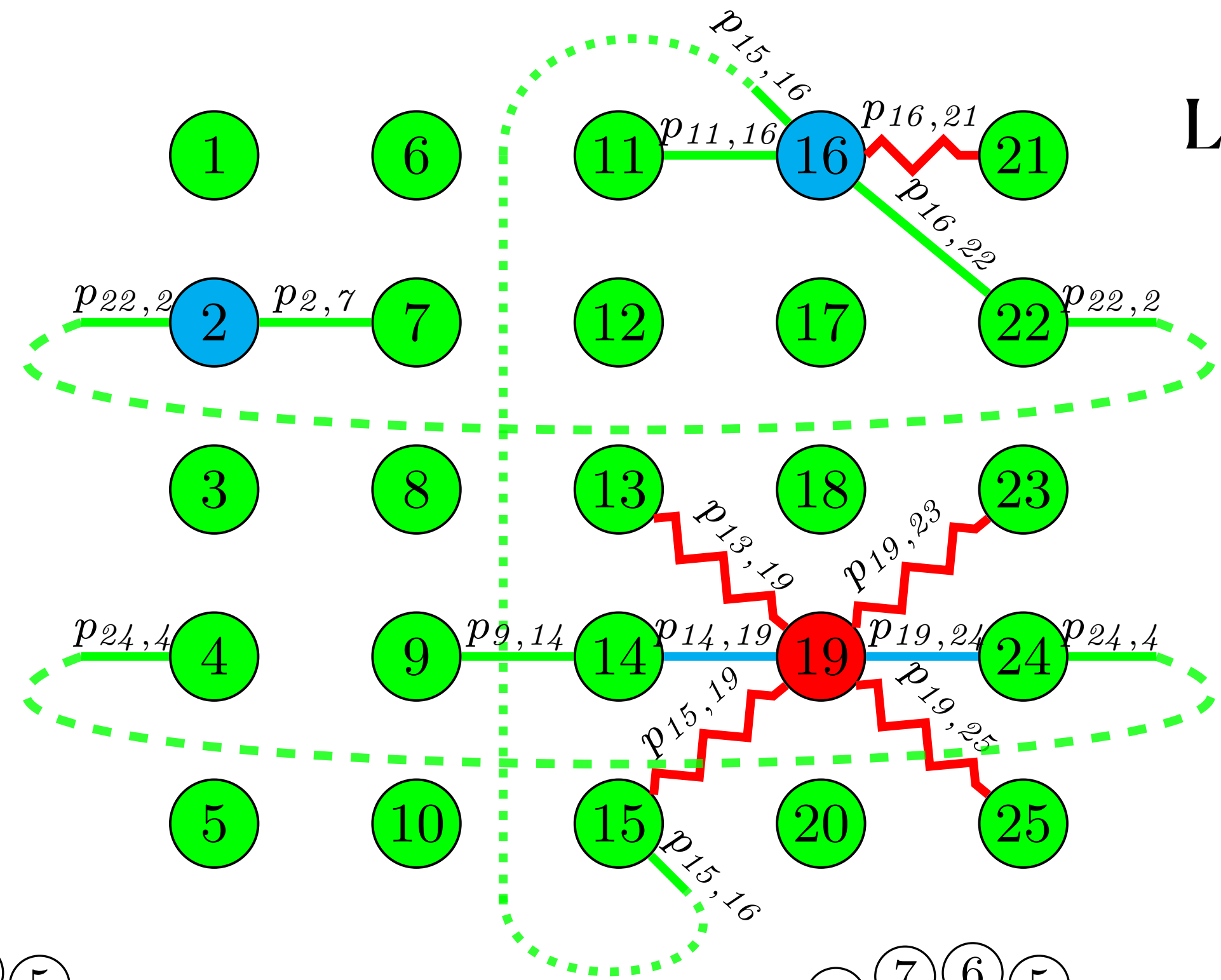
Right
Merkle Tree

Repaired Parity Chunk 19-24

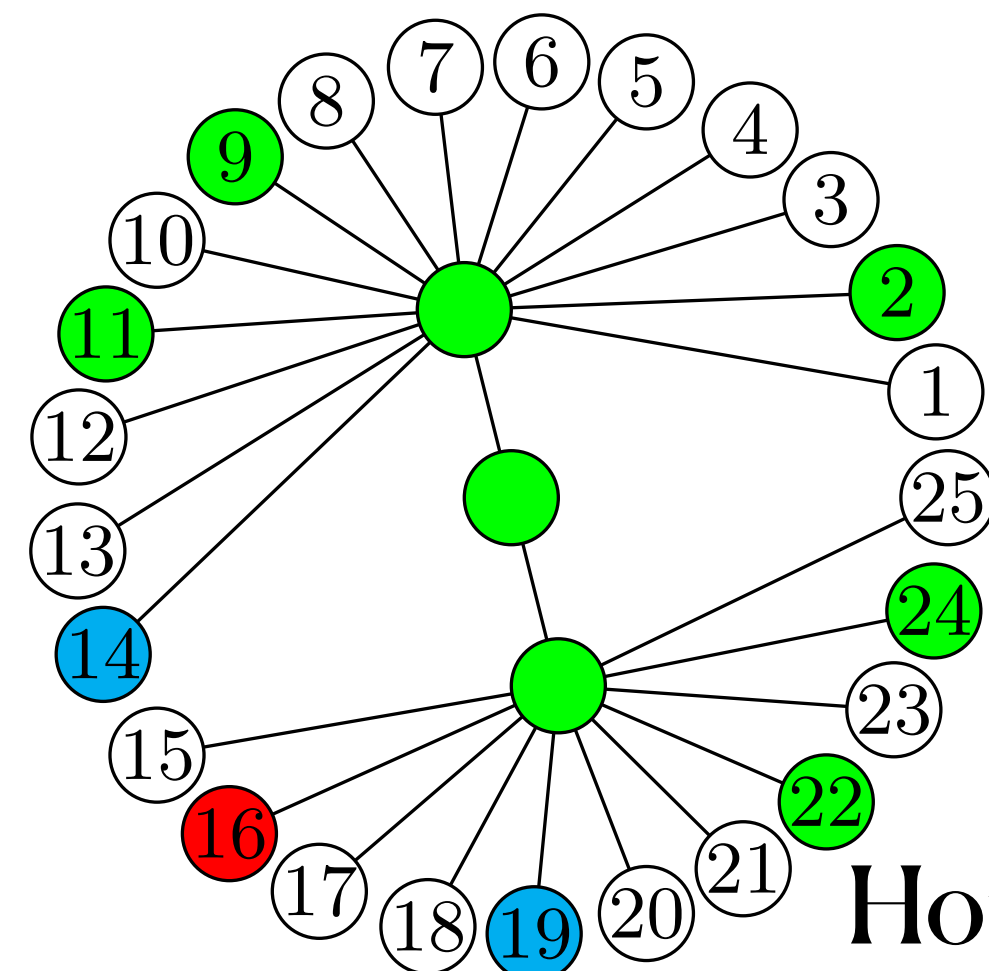
Original
Merkle Tree



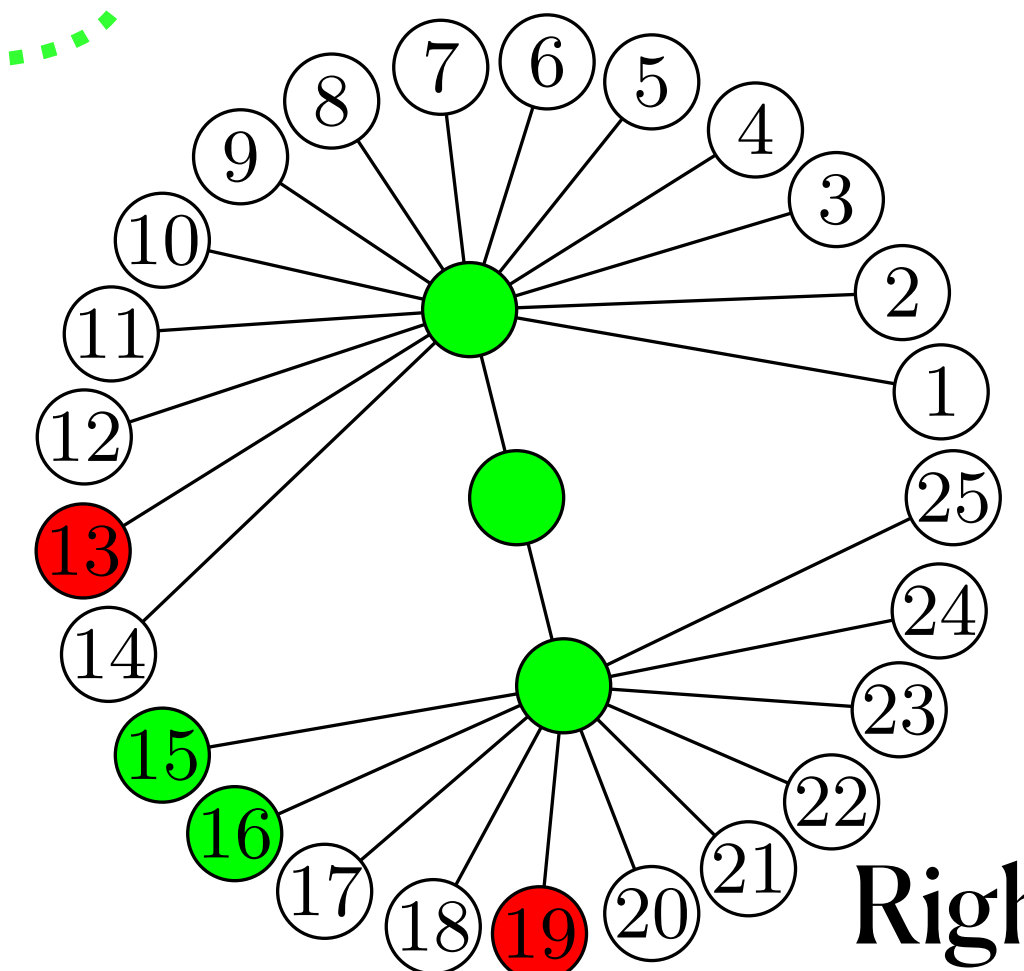
Lattice



Left
Merkle Tree



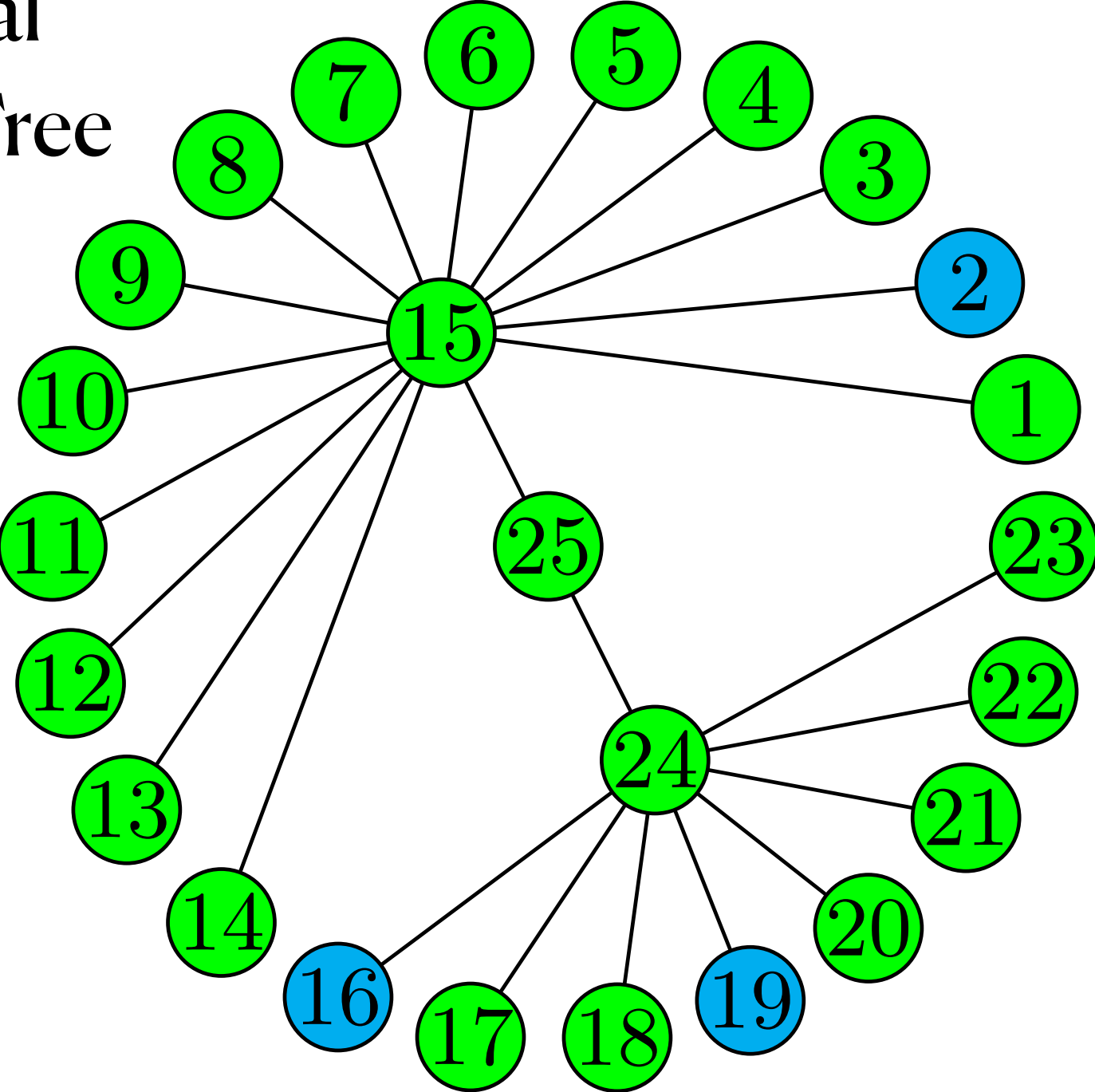
Horizontal
Merkle Tree



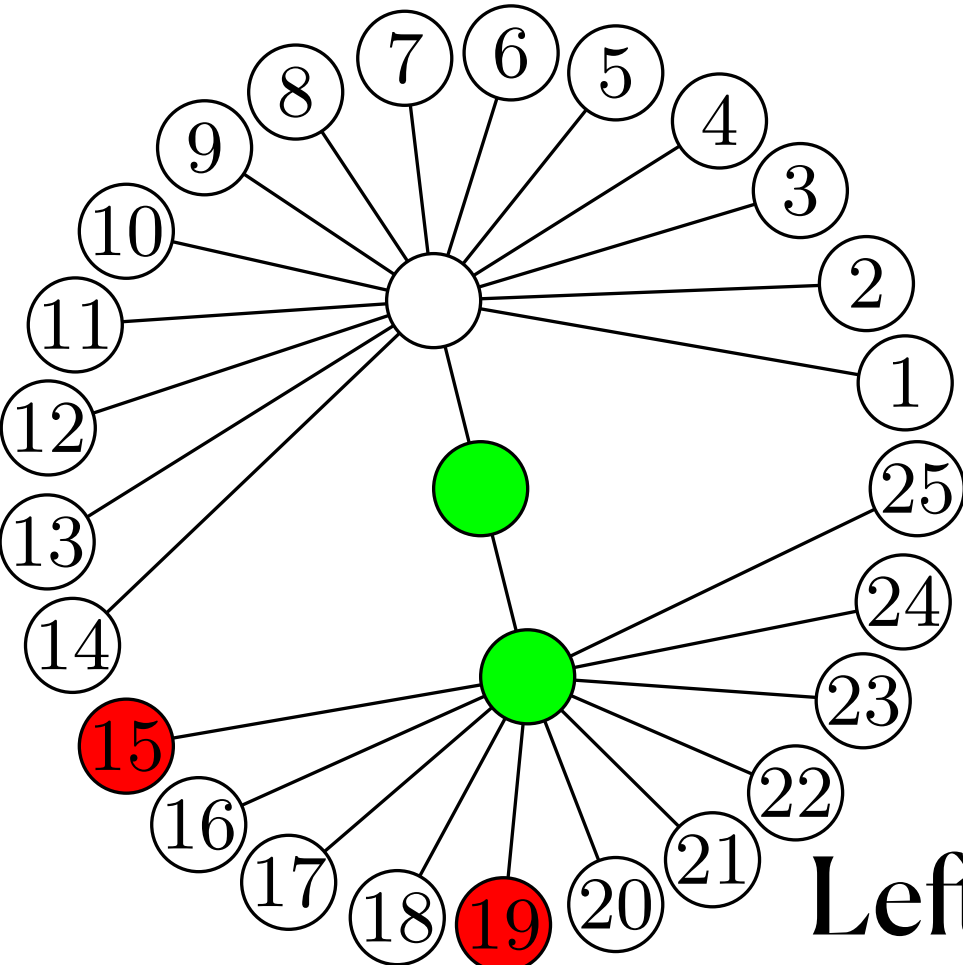
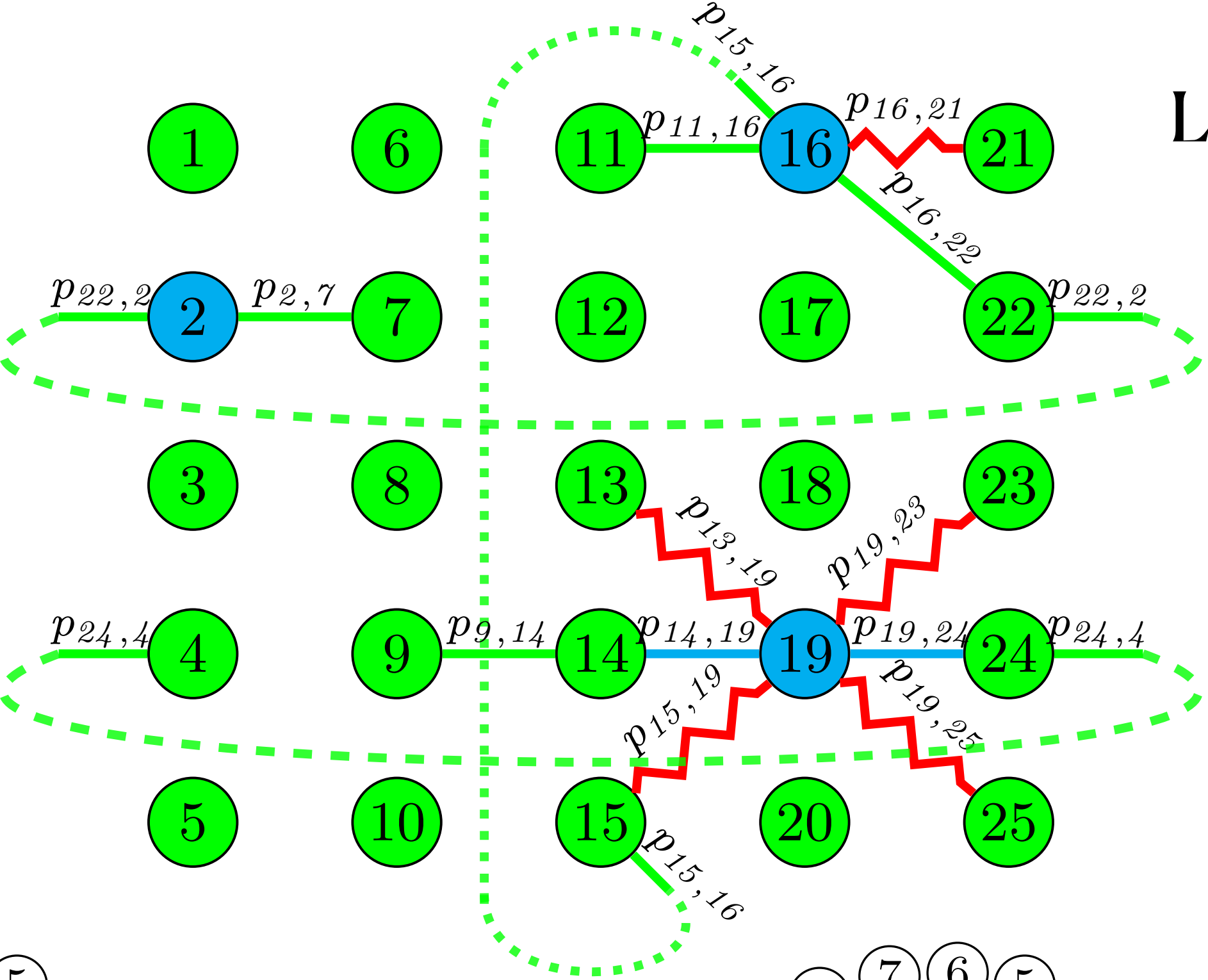
Right
Merkle Tree

Repaired Data Chunk 19

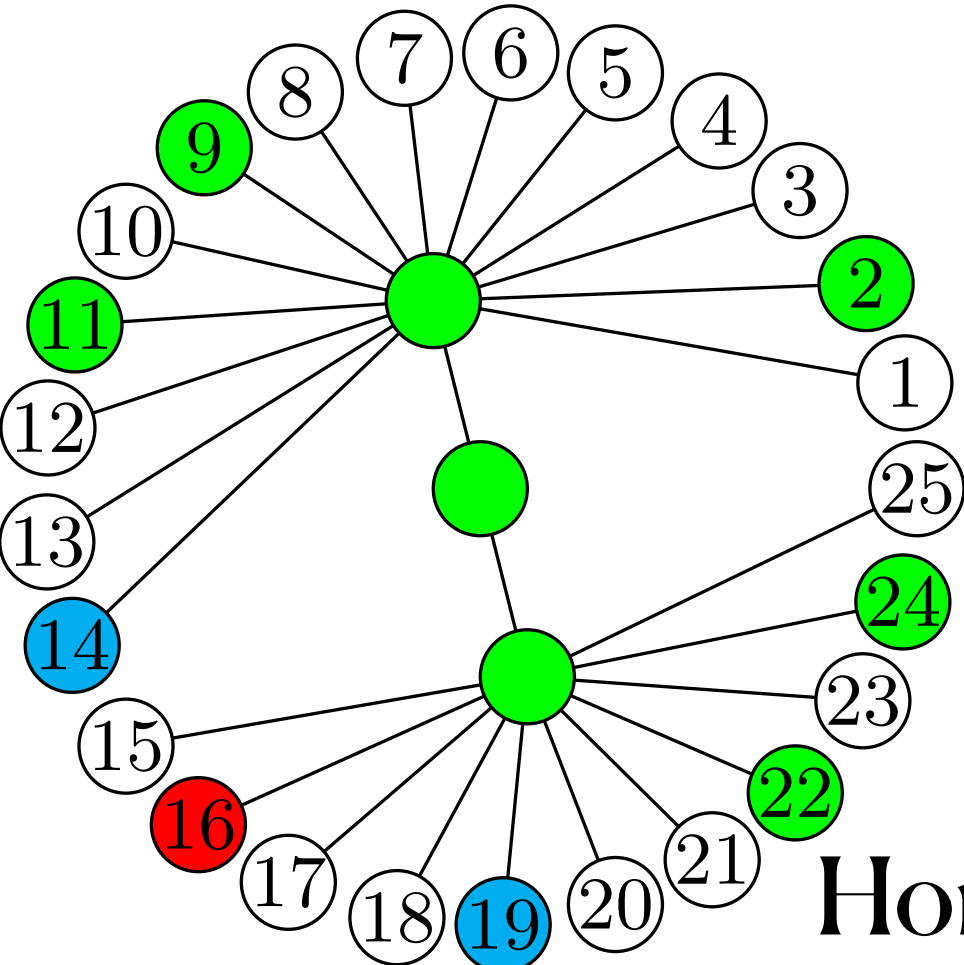
Original
Merkle Tree



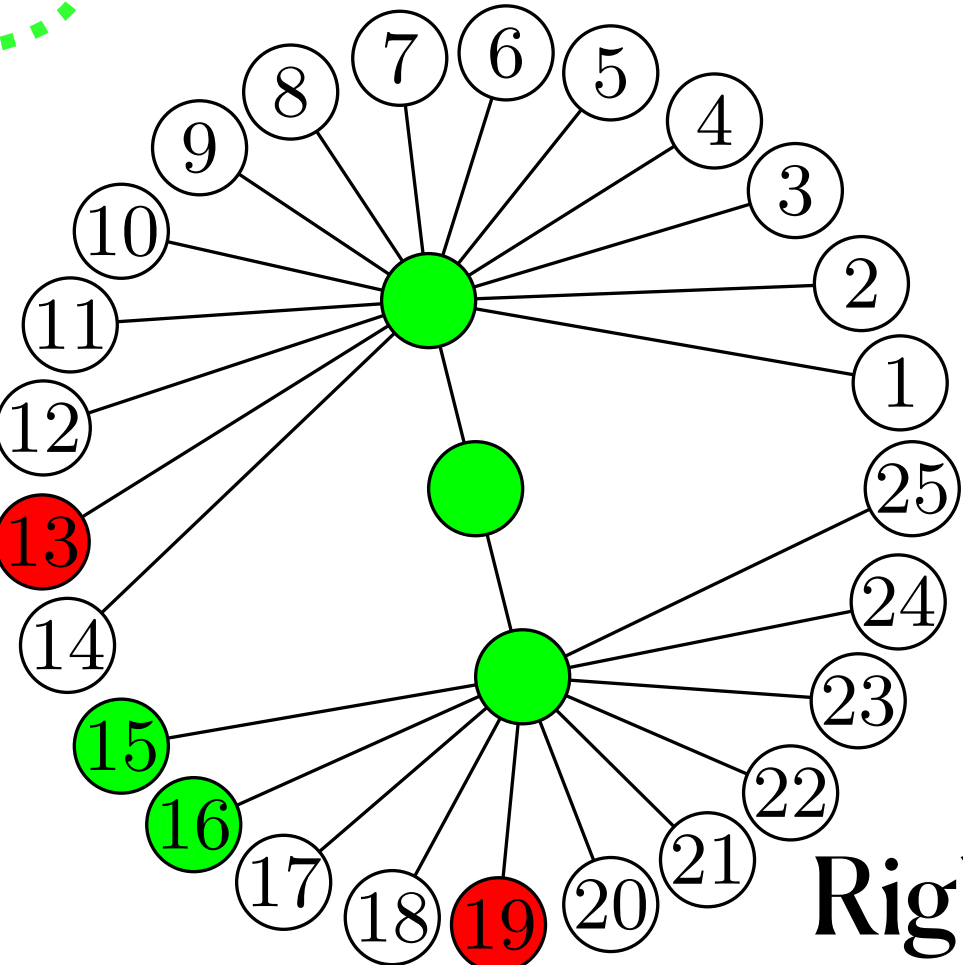
Lattice



Left
Merkle Tree



Horizontal
Merkle Tree

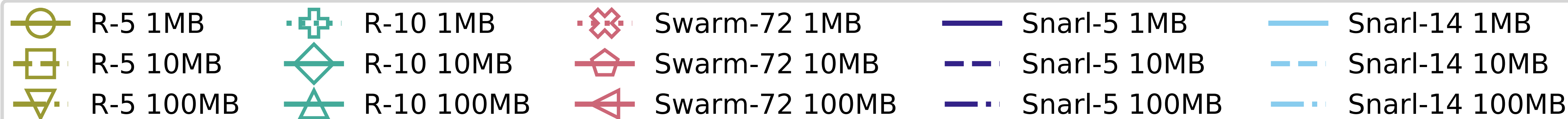
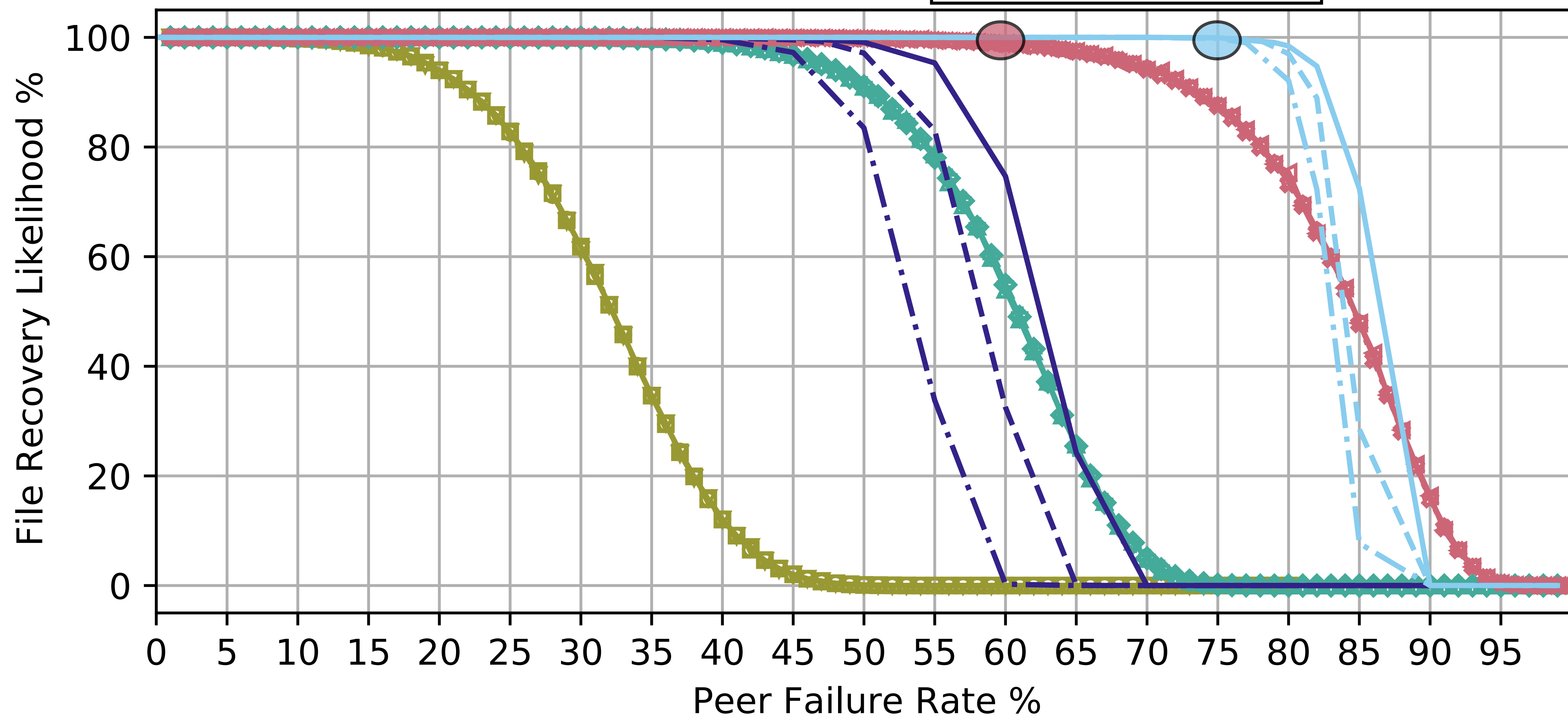


Right
Merkle Tree

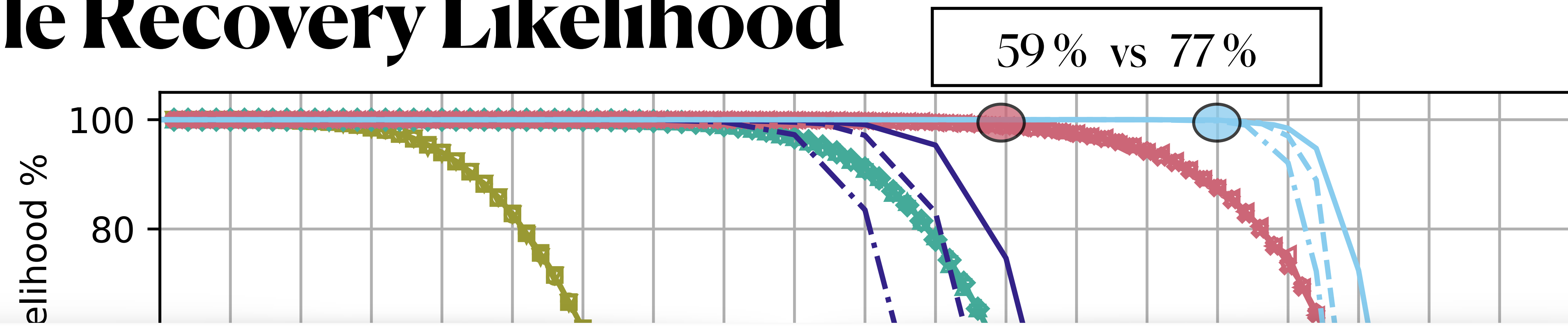
Some Results

File Recovery Likelihood

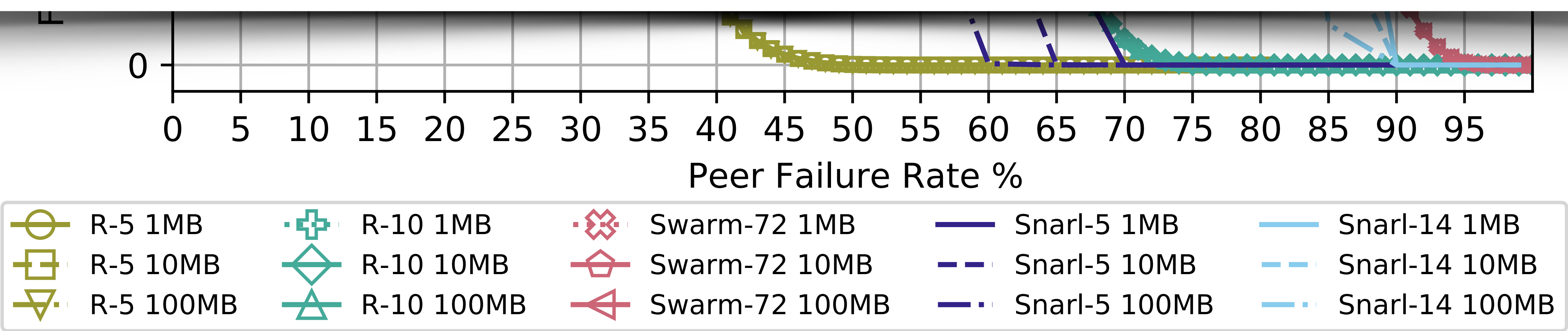
59 % vs 77 %



File Recovery Likelihood

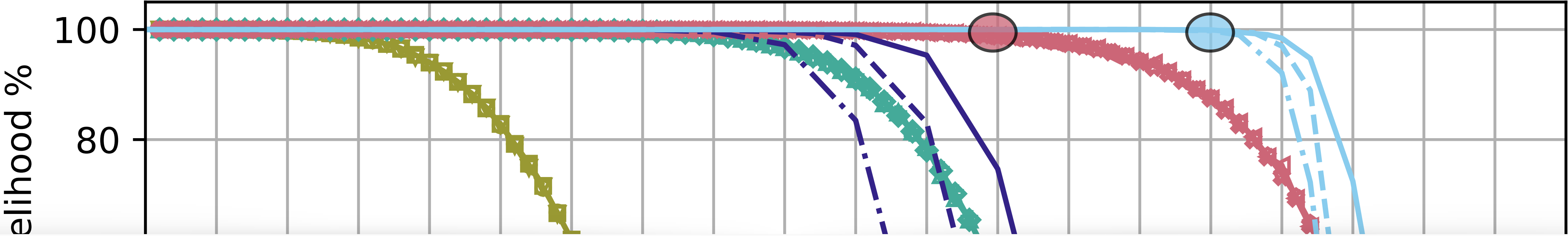


Snarl-14 Can Recover a 100 MB File
Even if 77 % of Peers are Unavailable



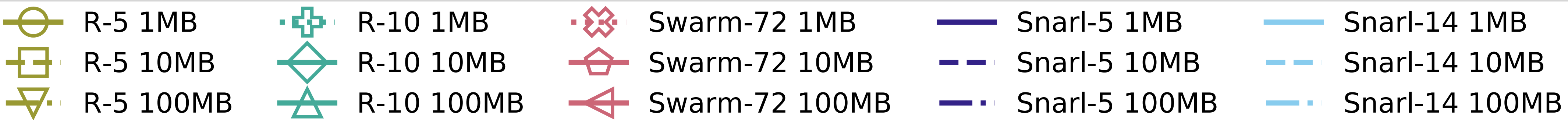
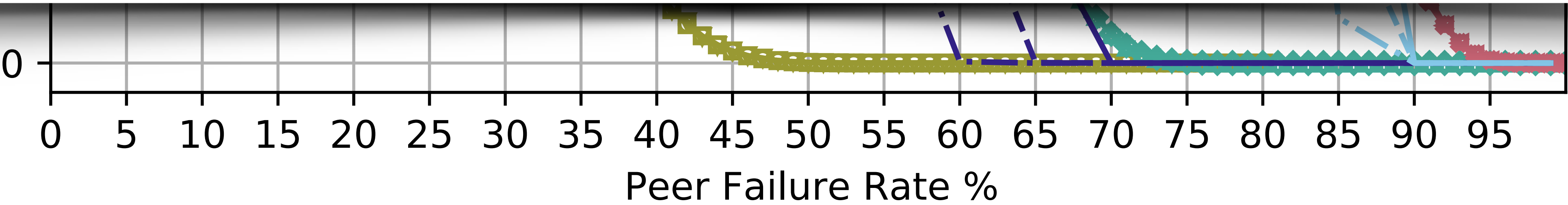
File Recovery Likelihood

59 % vs 77 %



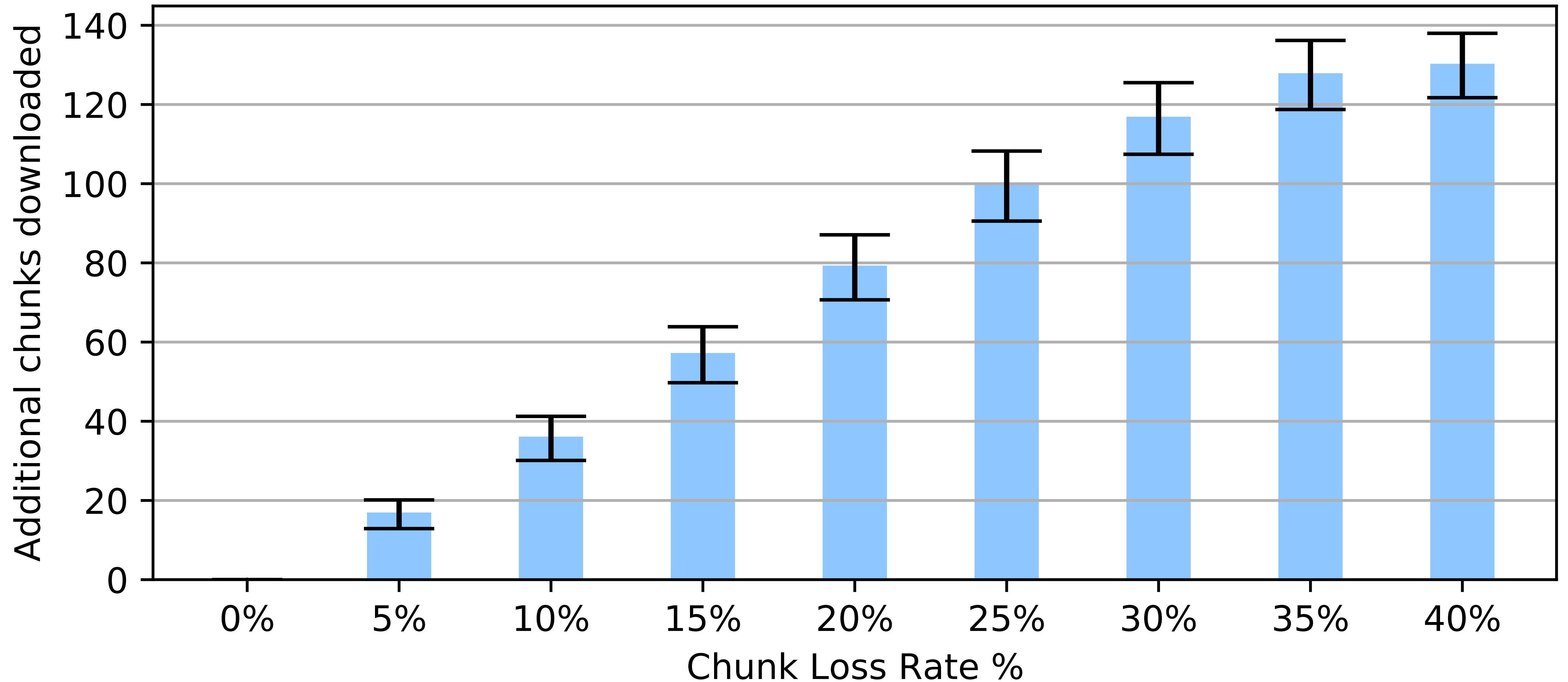
Snarl-14 vs Swarm-72

Improve Storage Utilization by 5×



Parity Chunks Downloaded for Repair

1 MB File — 259 Data Chunks

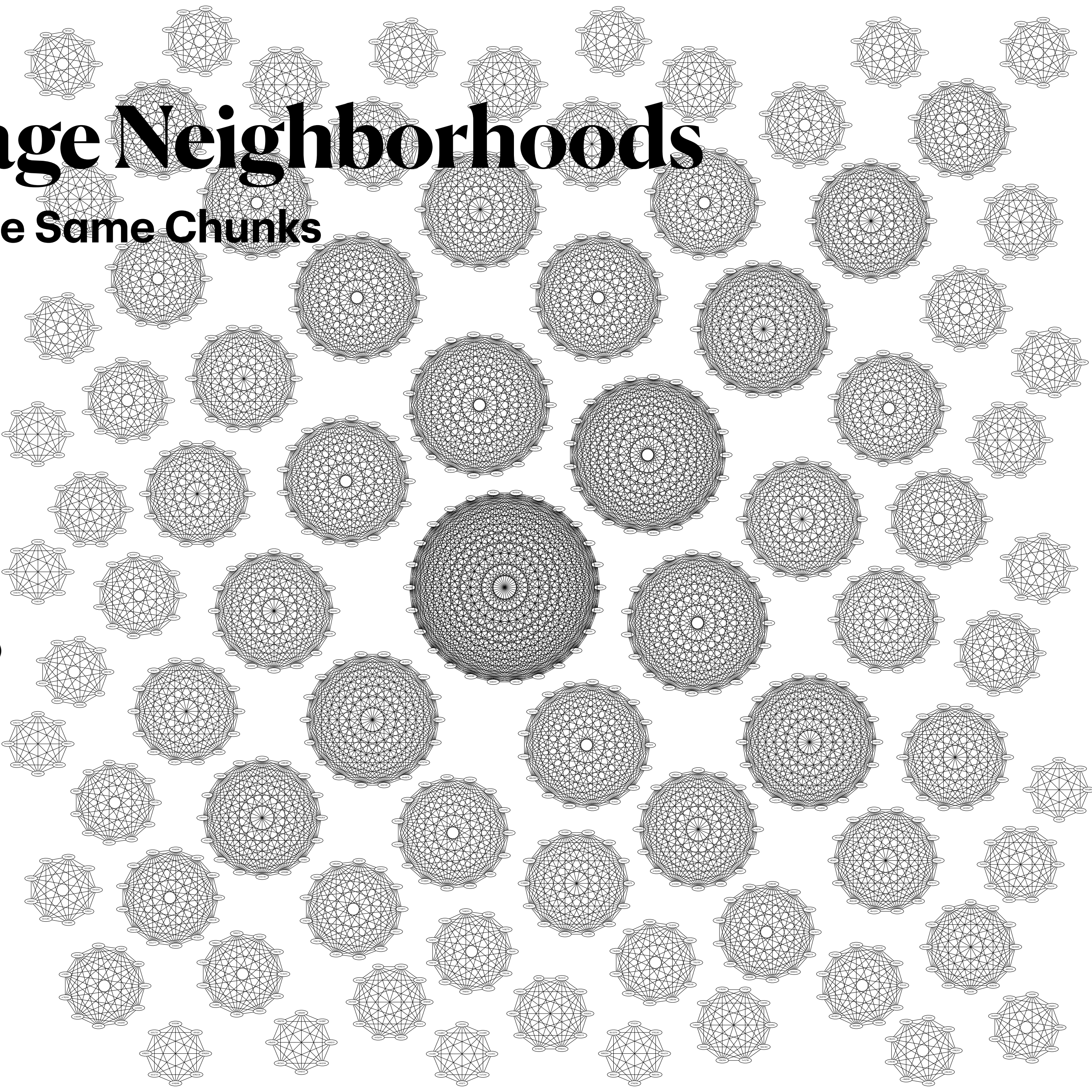
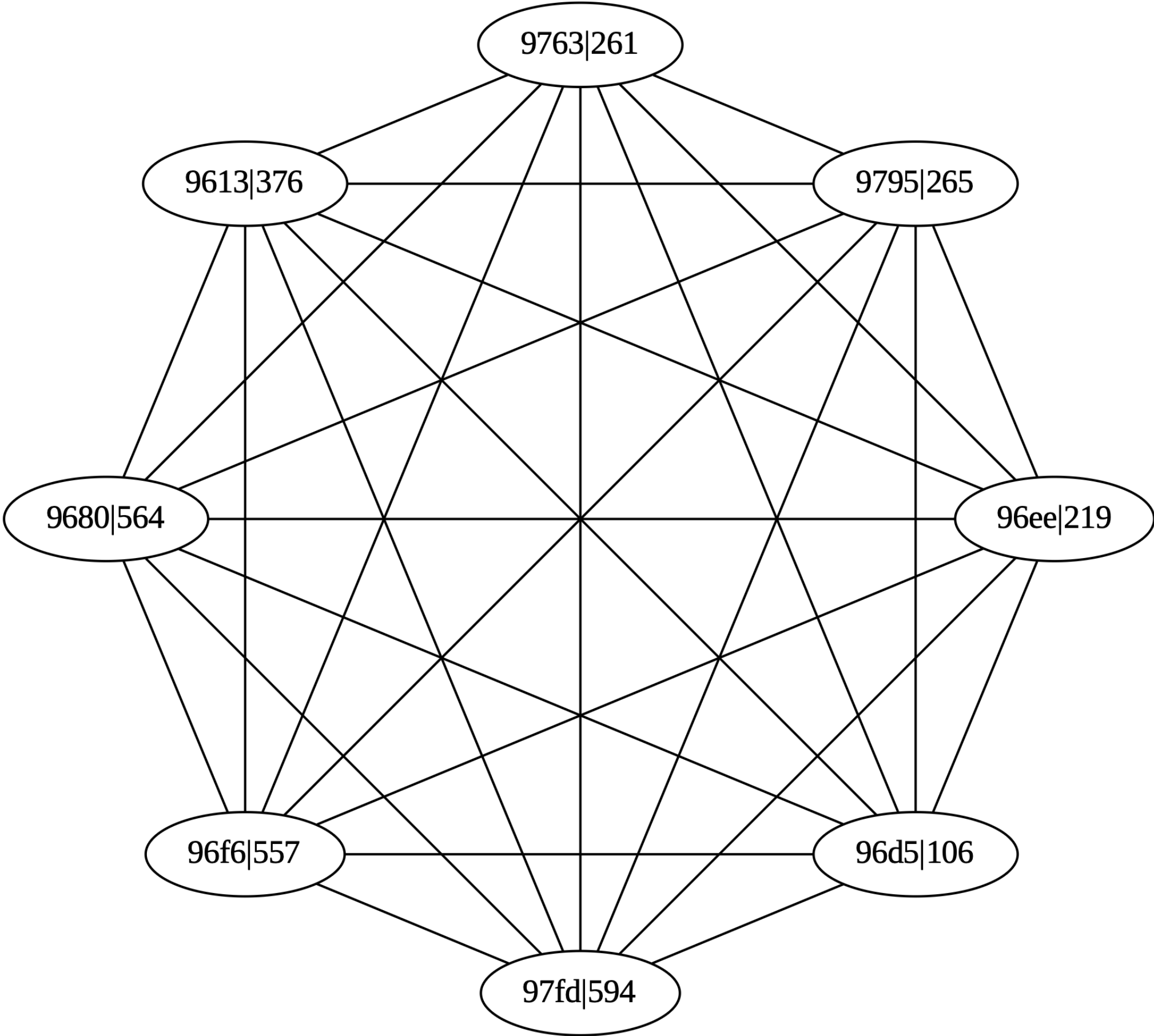


SNIPS

**Synchronizing Decentralized
Storage Peers**

Synchronizing Storage Neighborhoods

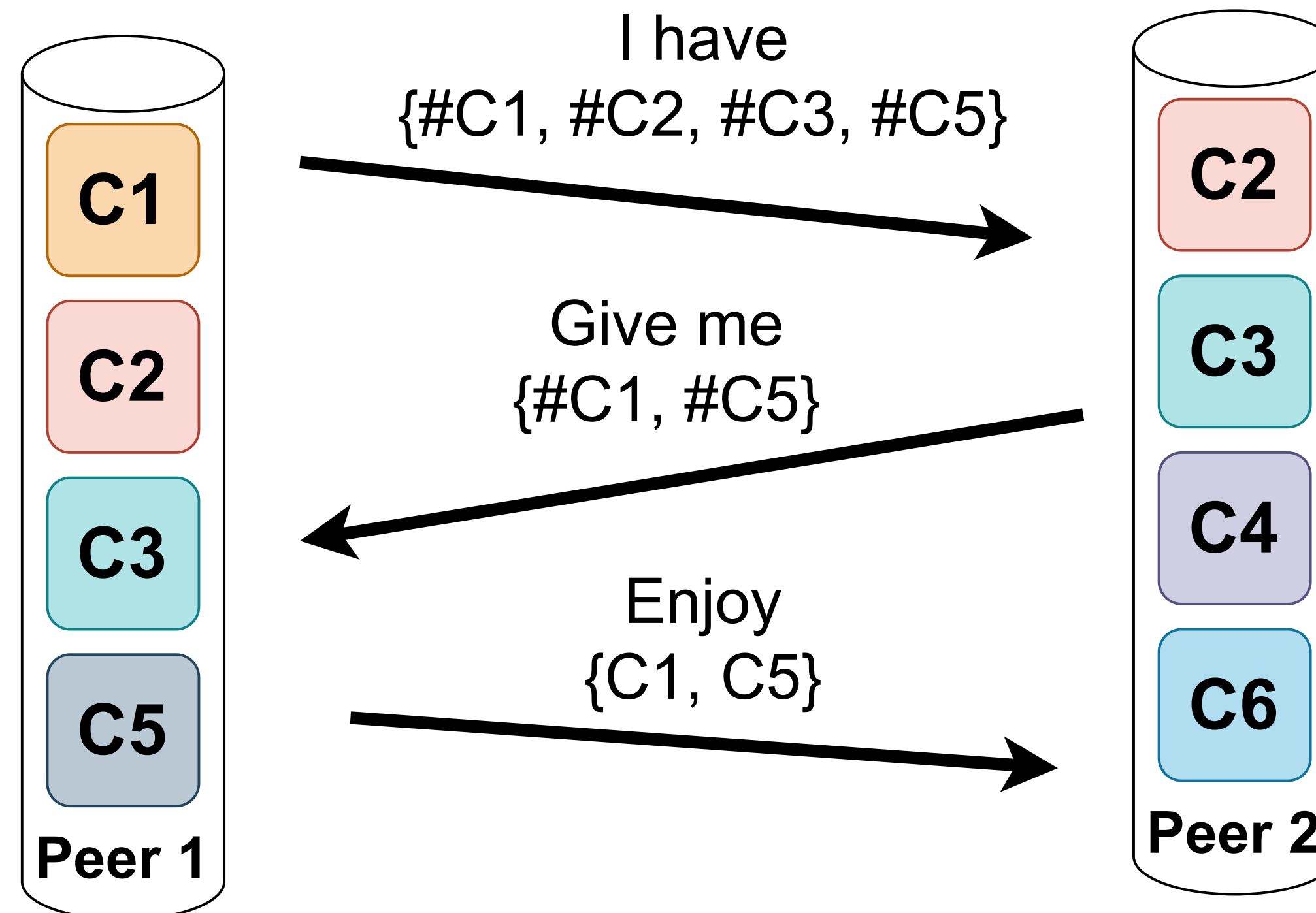
Each Neighborhood Should Store the Same Chunks



Existing Sync Protocols

Swarm and IPFS/Filecoin

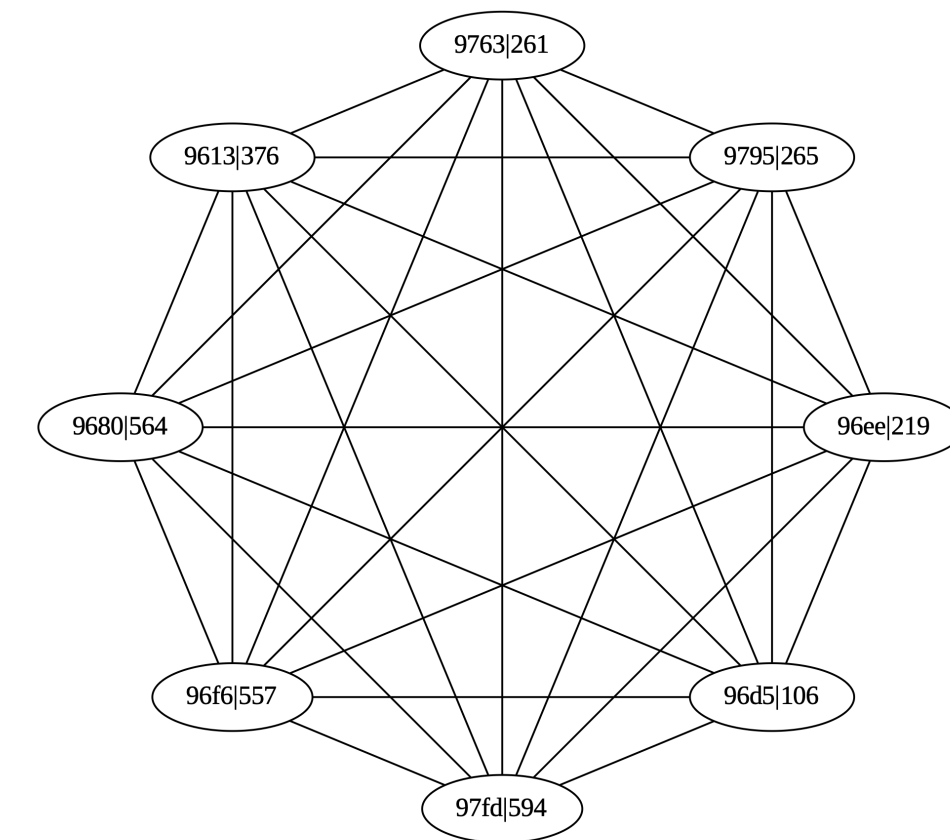
- Exchange Lists of Chunk Identifiers



Limitations With Existing Sync Protocols

Swarm and IPFS/Filecoin

- Exchange **Long** Lists of Chunk Identifiers
 - 32 B/Chunk (4 kB)
 - 1 GB $\rightarrow$ **8 MB sync metadata**
 - Must be Done for Each Pair of Replicas
 - Inefficient at Scale



Key Idea: Minimize Chunk Lists

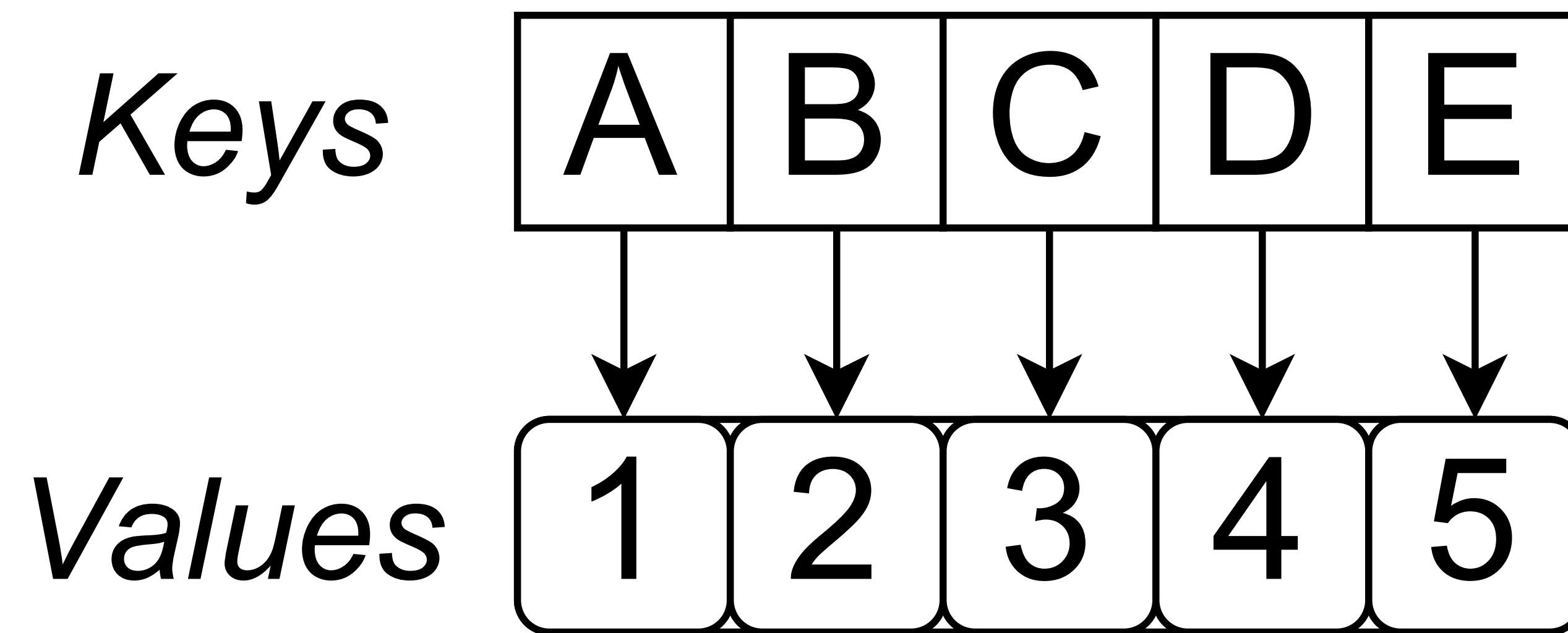
- Succinct Data Structure
 - Minimal Perfect Hash Function ~ 3 bits per chunk
 - Query the Hash Function to Find Missing Chunks
- Sync Protocol

Key Performance Result

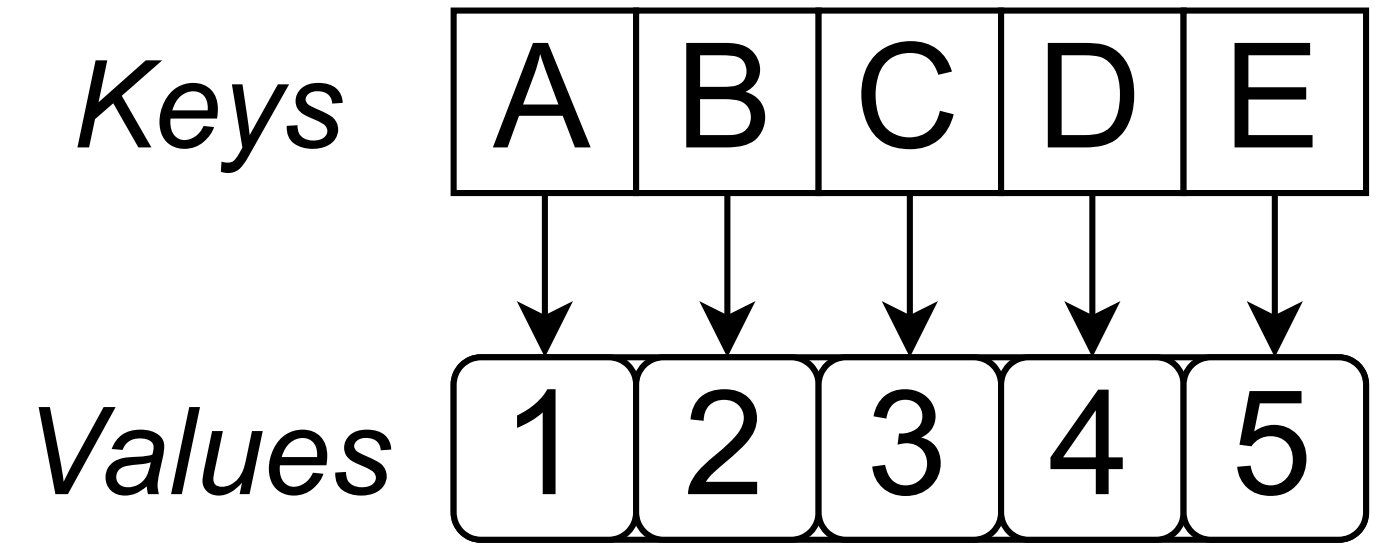
- Reduce Sync Metadata by up to 1-3 Orders of Magnitude
 - 10 GB → **8 MB Sync Metadata**
- Without Modifying the Underlying Storage Structure

Minimal Perfect Hash Function

Minimal Perfect Hash Function

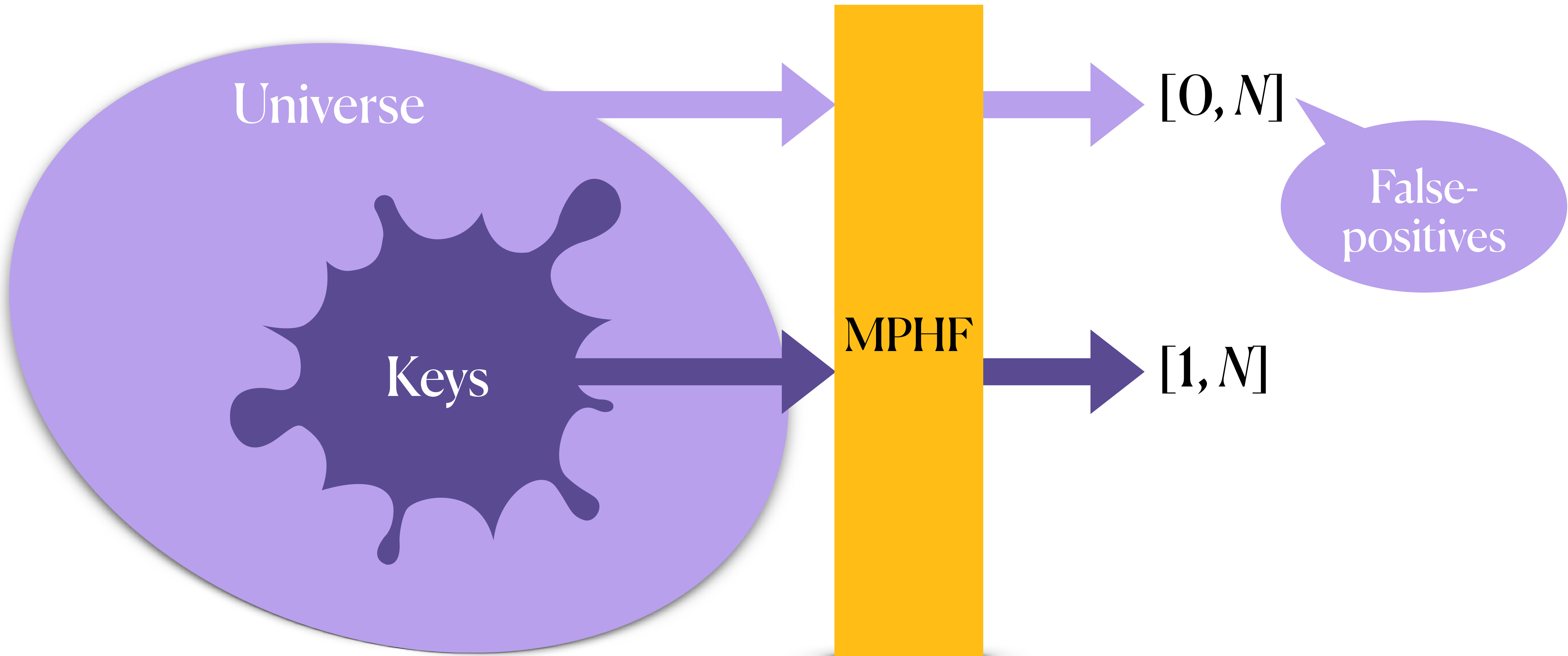


Minimal Perfect Hash Function



- $\text{Create}(\text{keys}) \rightarrow \text{mphf}$
 - Mapping from a large key space to index values
 - ~ 3 bits per key
- $\text{Index}(\text{key}) \rightarrow \text{index value} \in [0, N]$, where 0 = Not Found
 - *key* **might** be in the original set
 - False-positives are possible

Minimal Perfect Hash Function

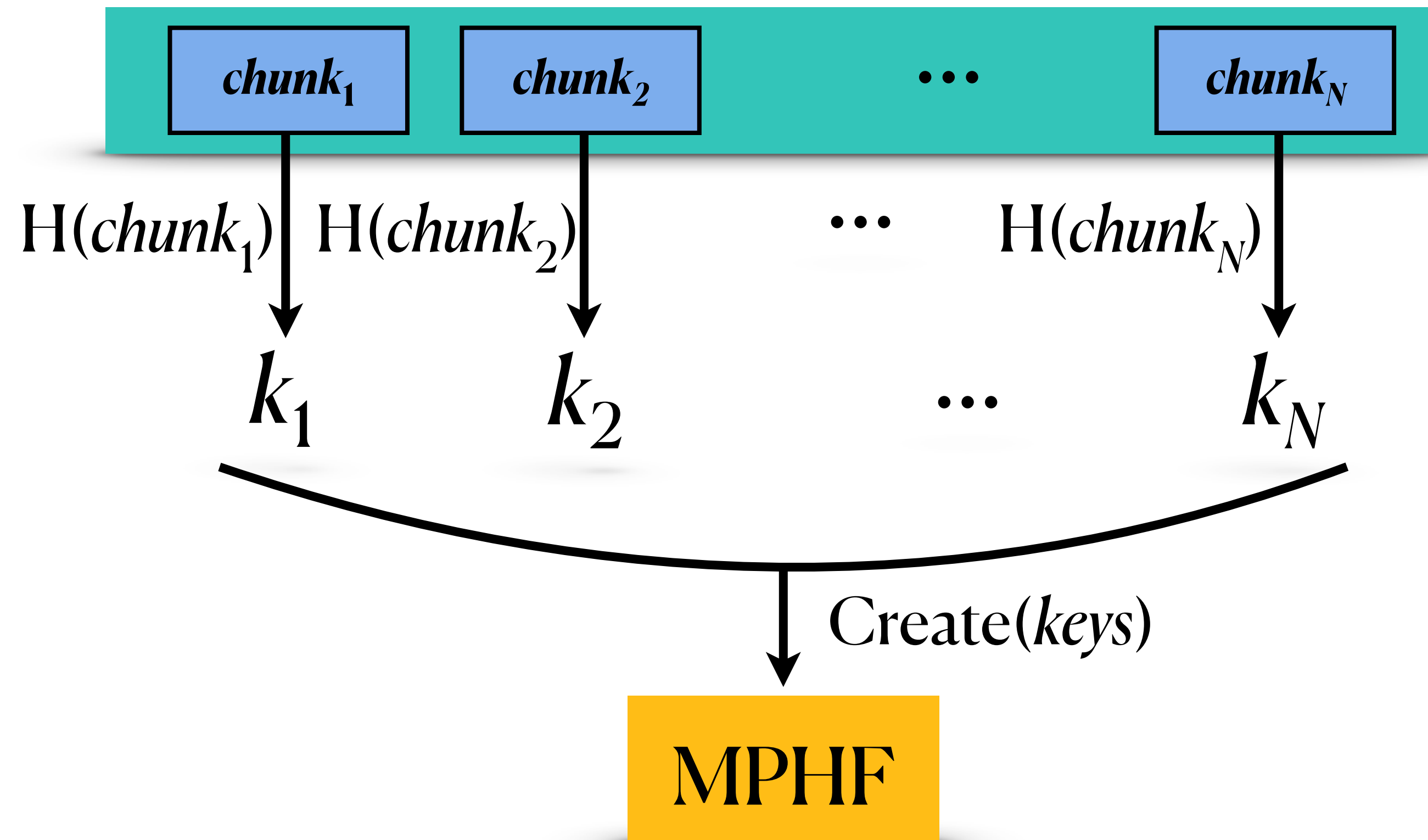


Mapping Storage

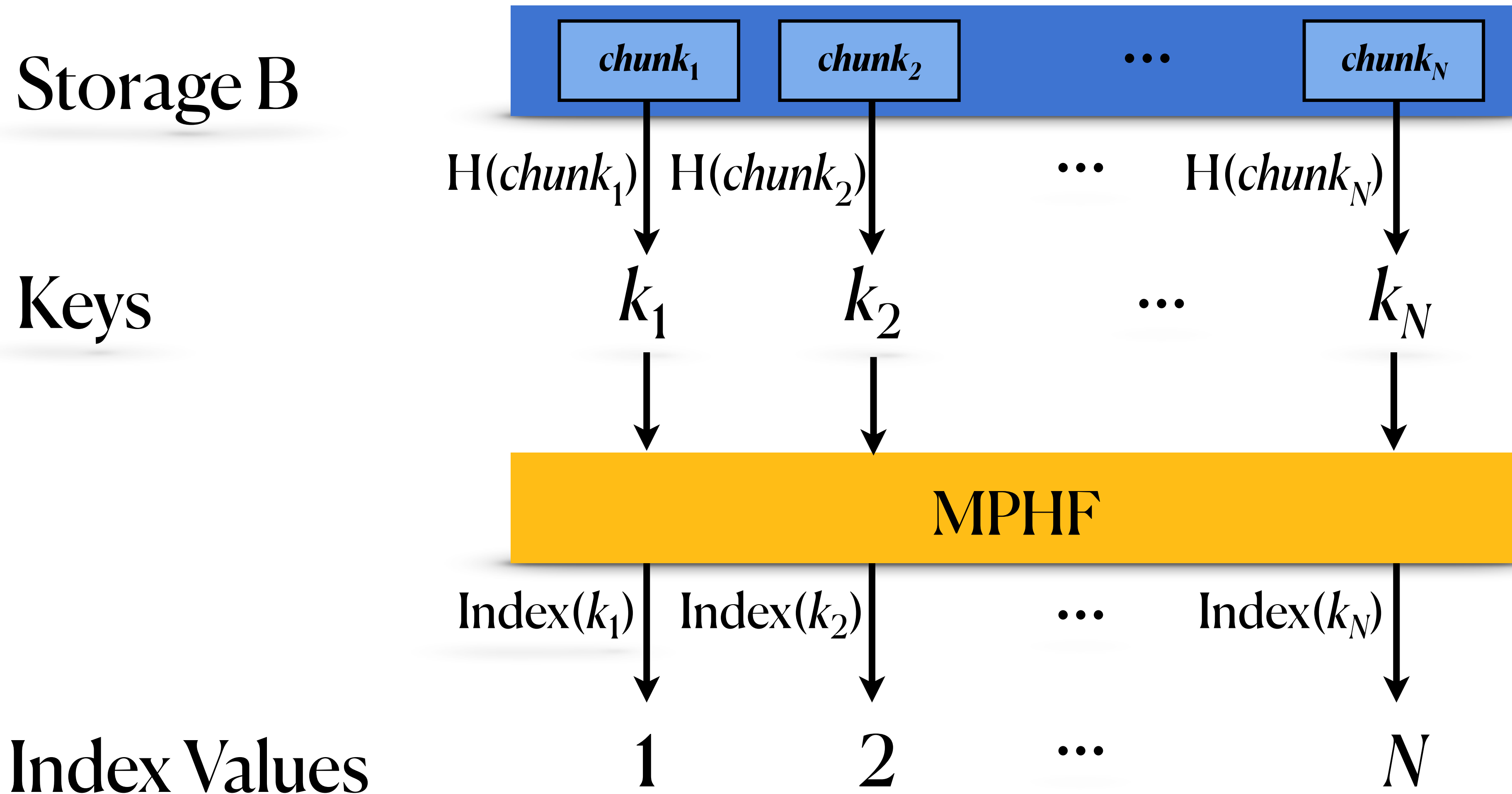
Mapping Storage Structure

Storage A

Keys

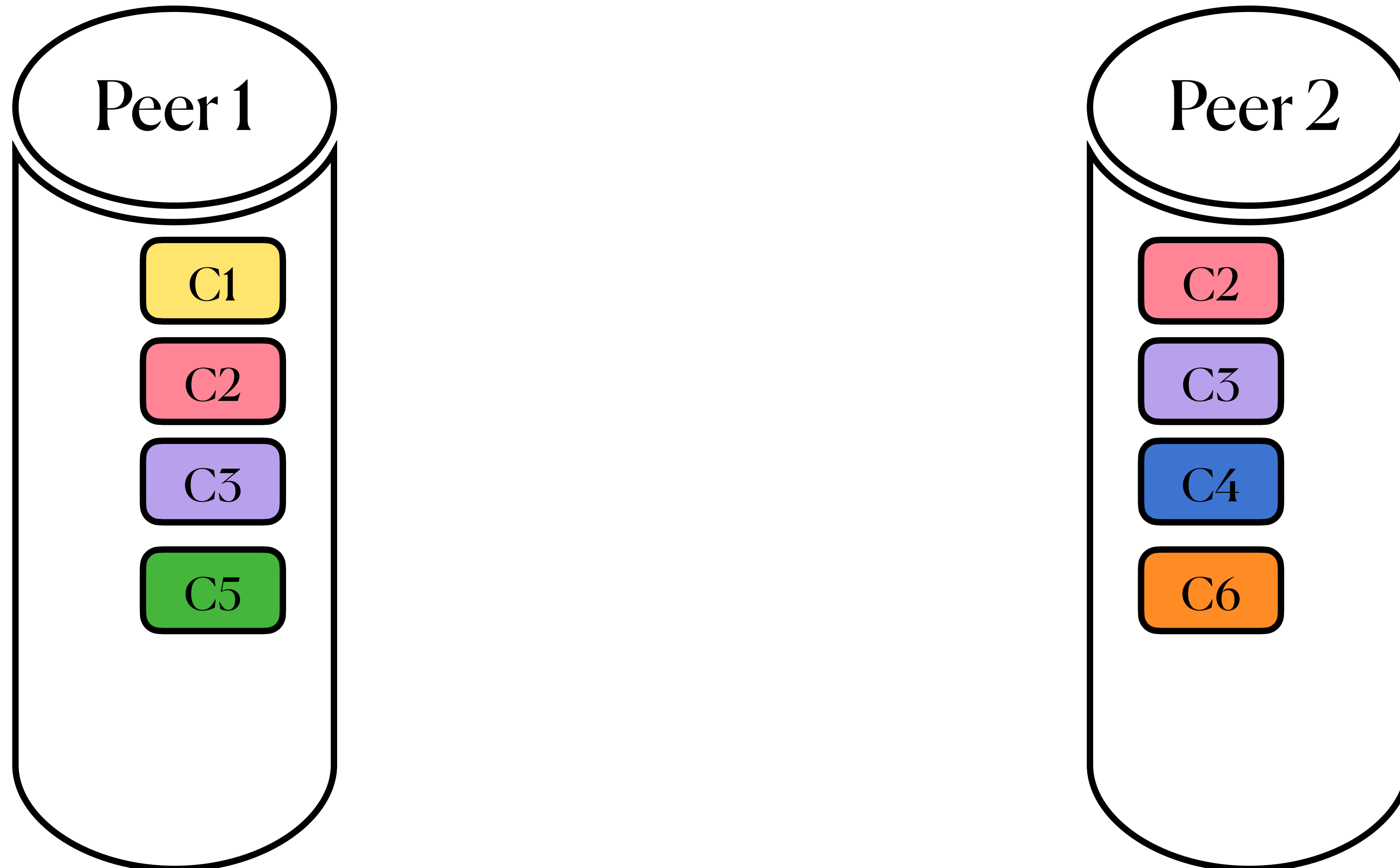


Mapping Storage Structure

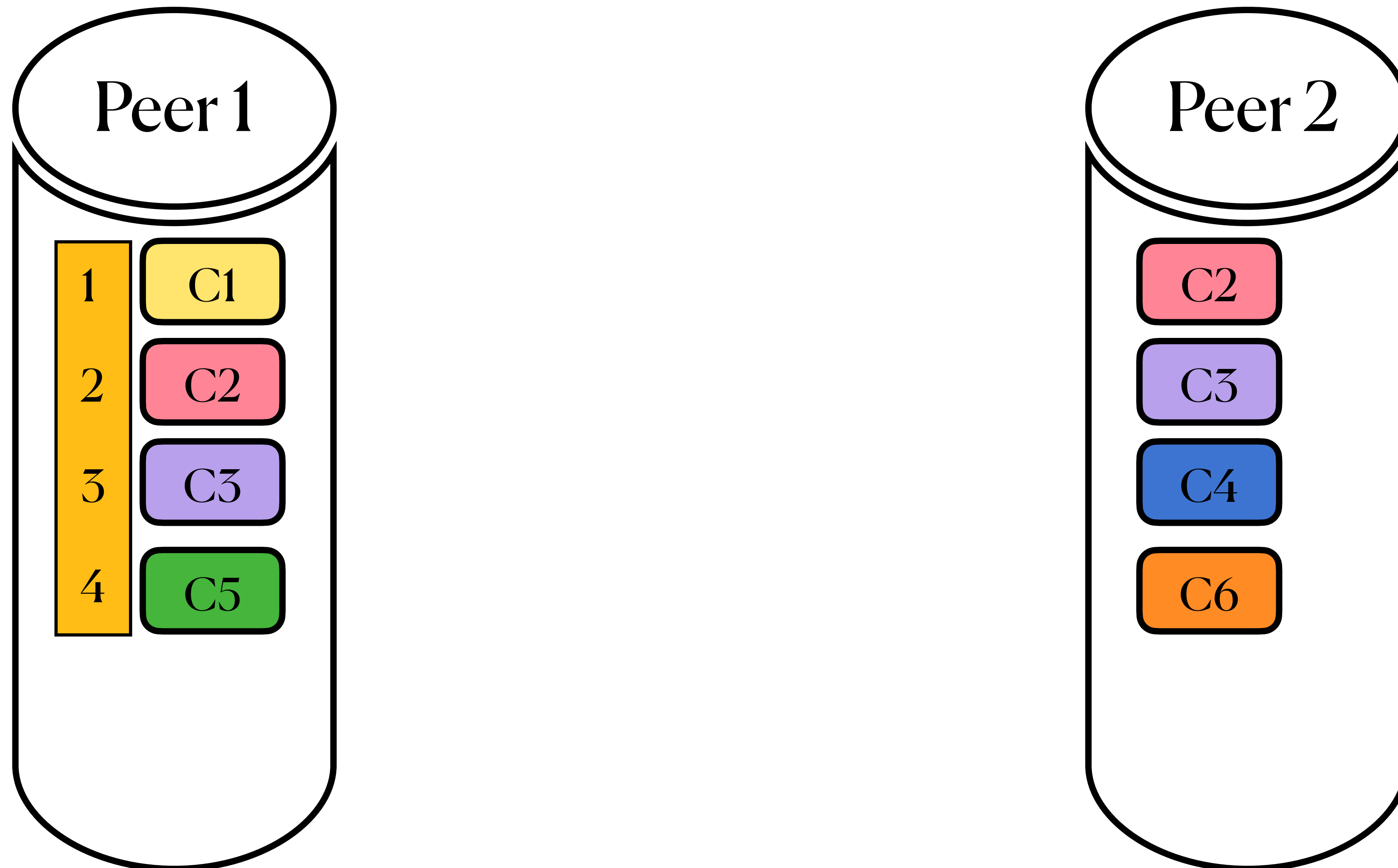


The Sync Protocol

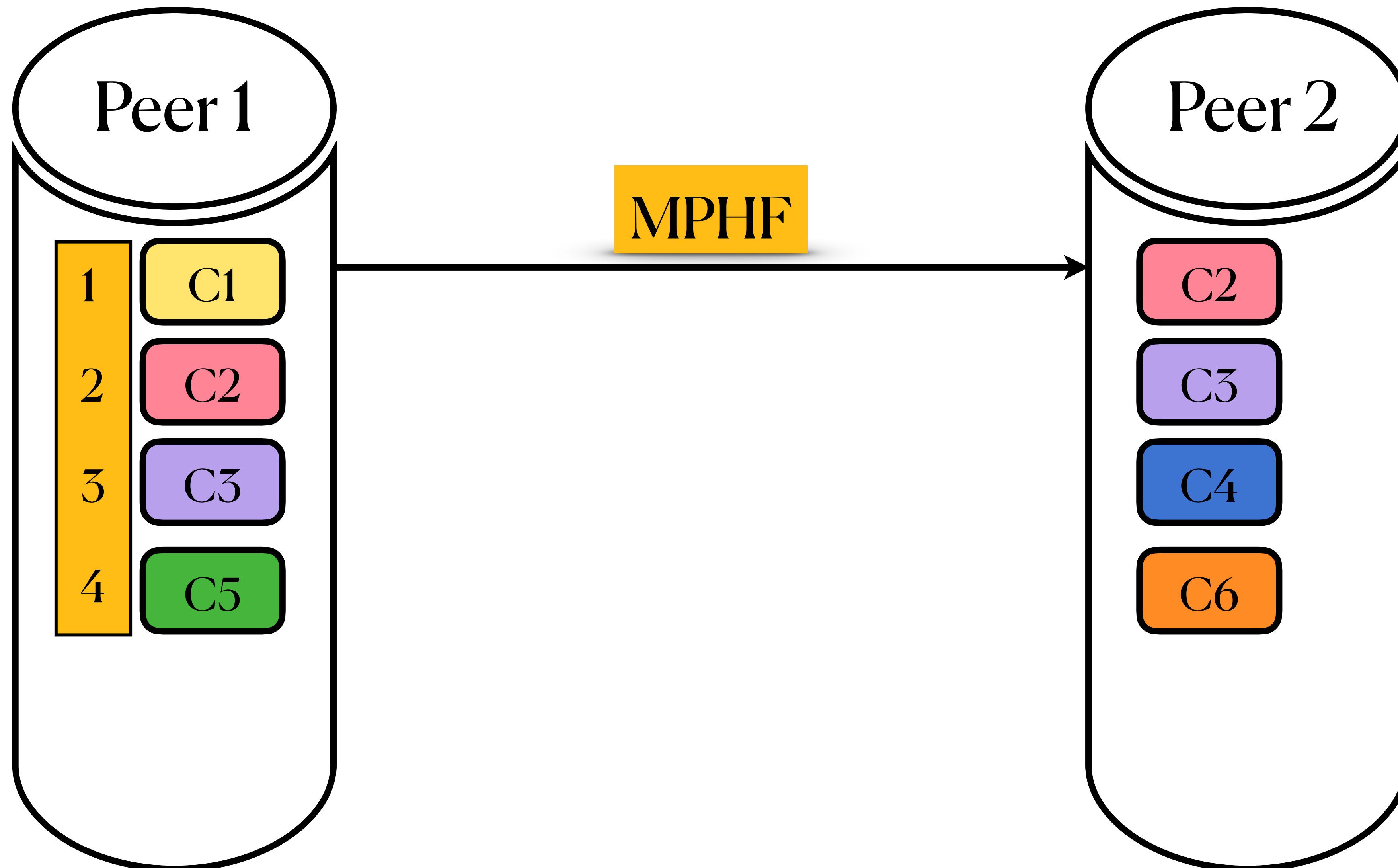
The Sync Protocol



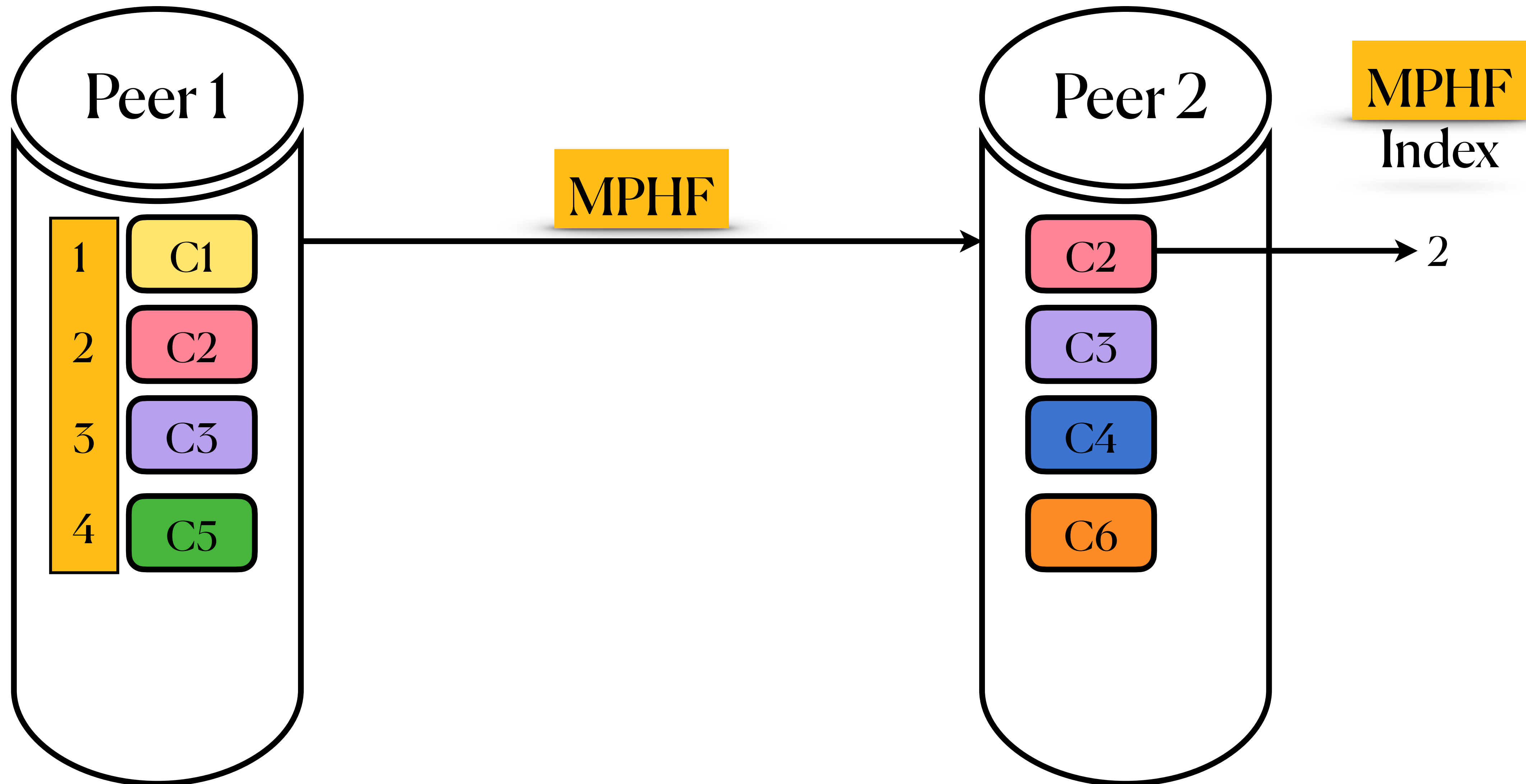
The Sync Protocol



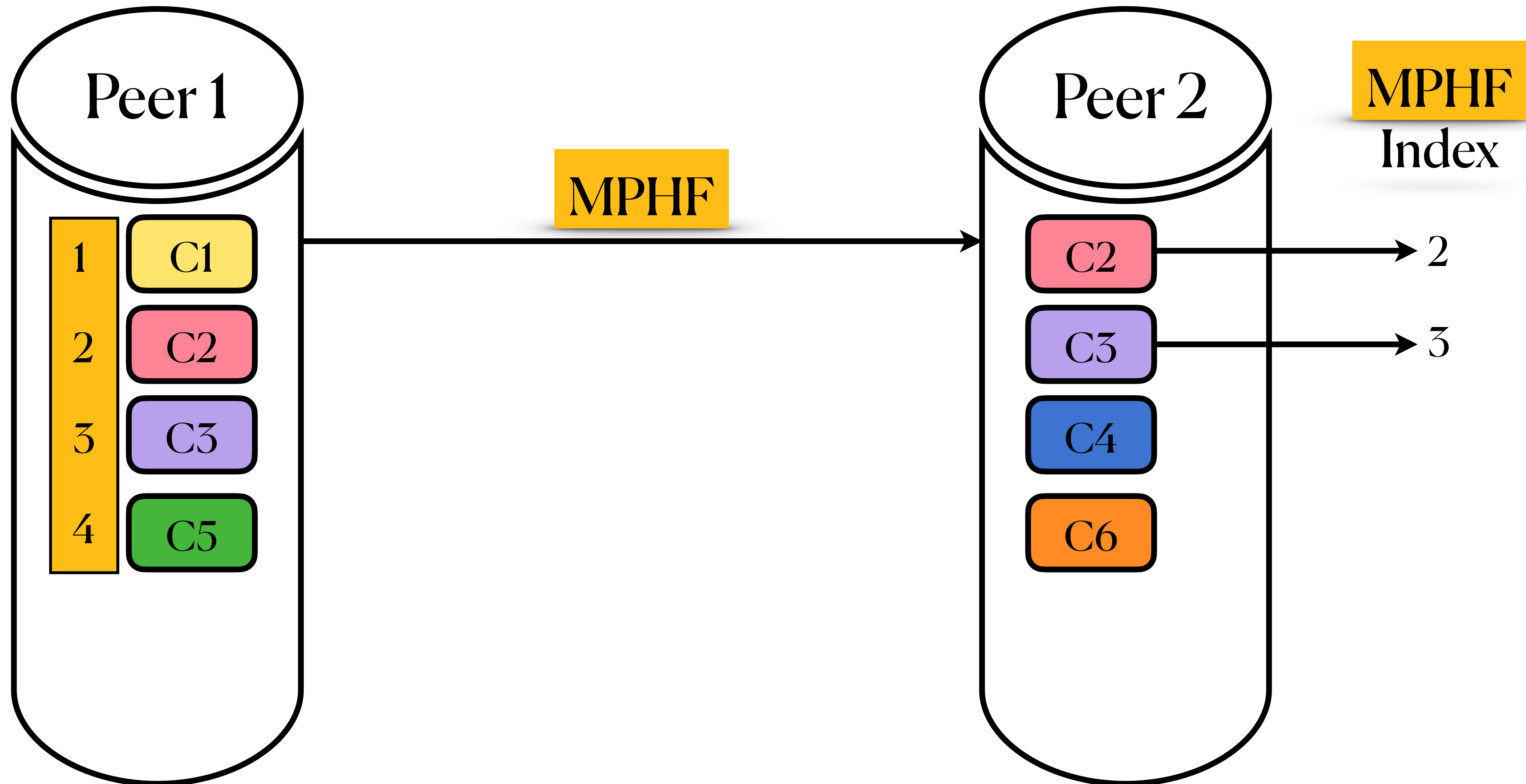
The Sync Protocol



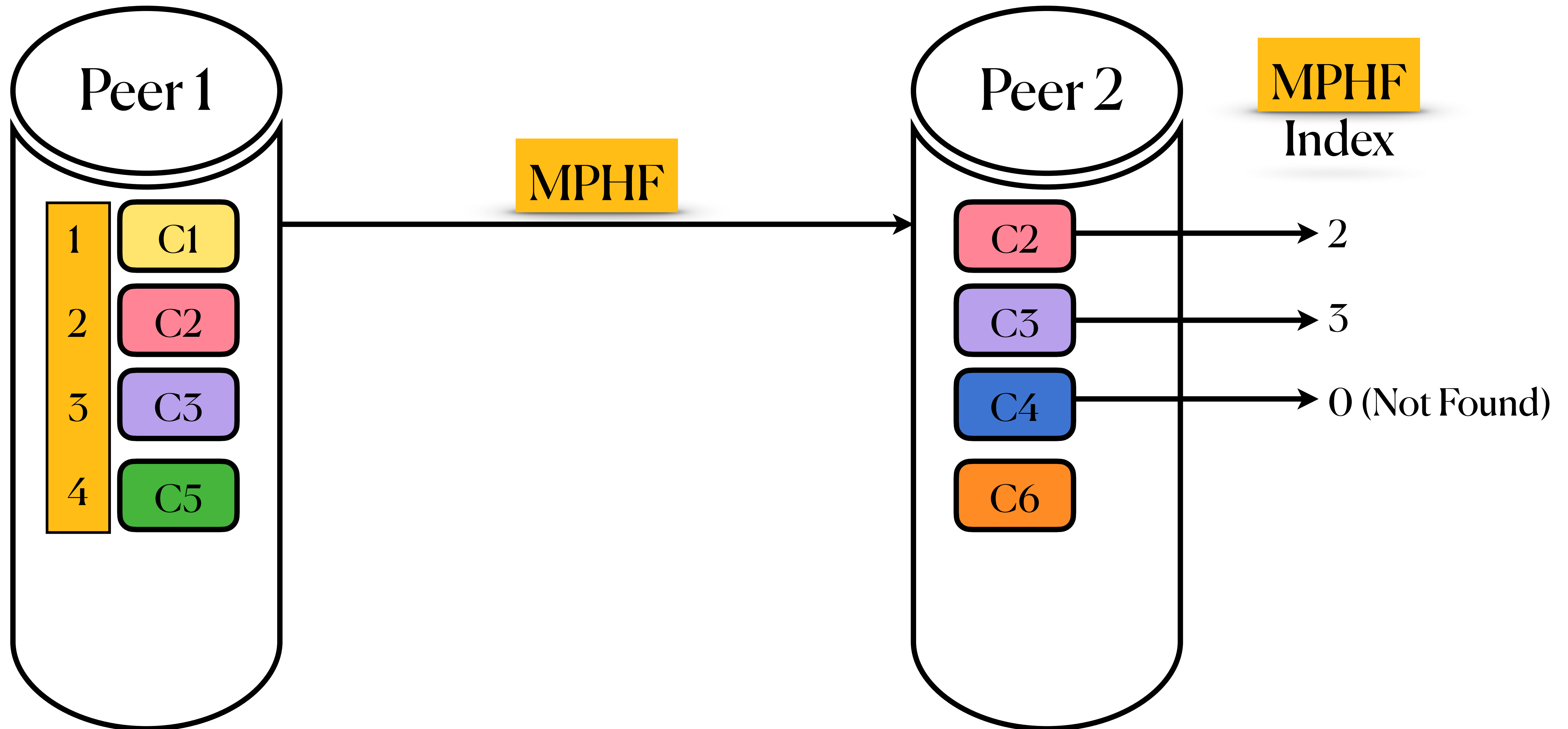
The Sync Protocol



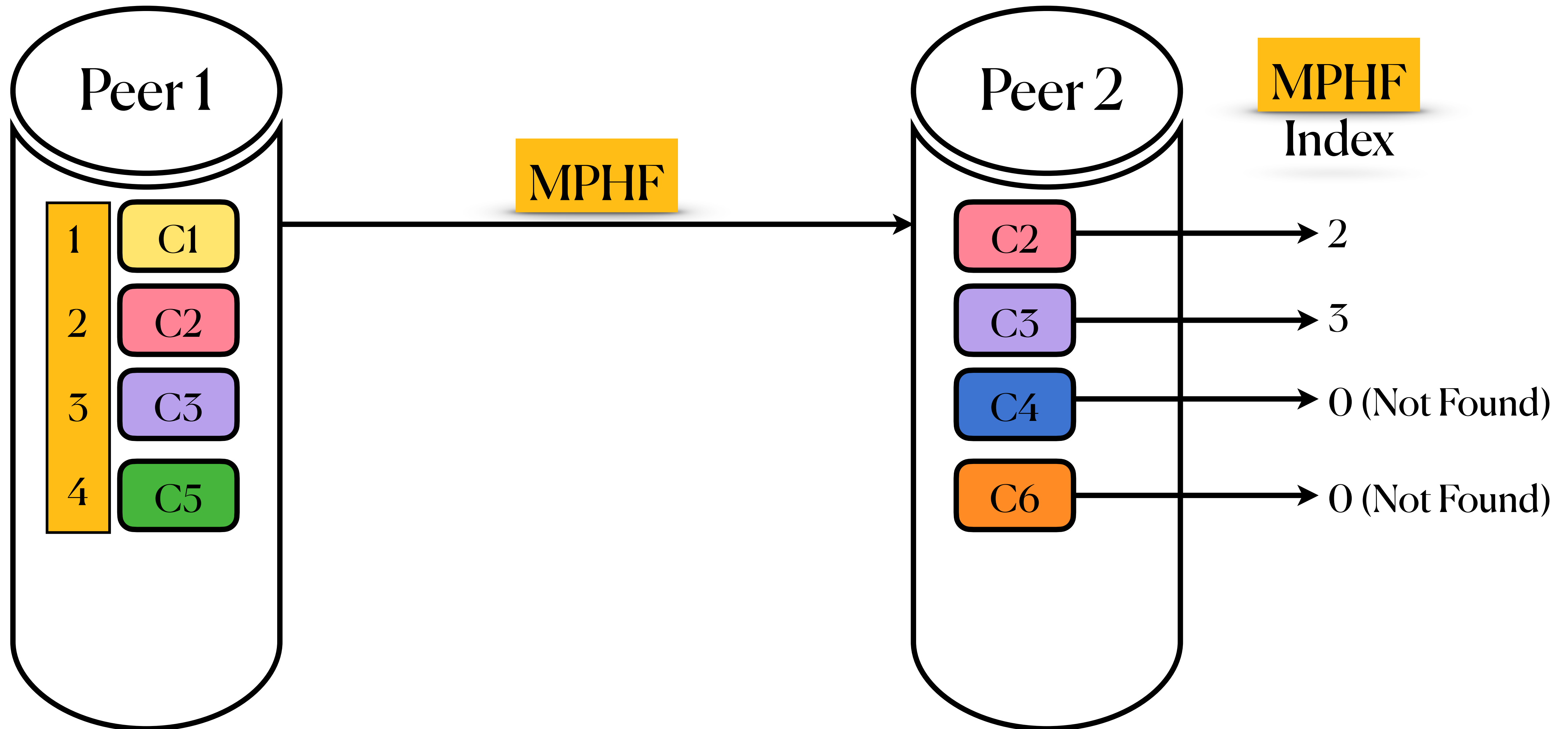
The Sync Protocol



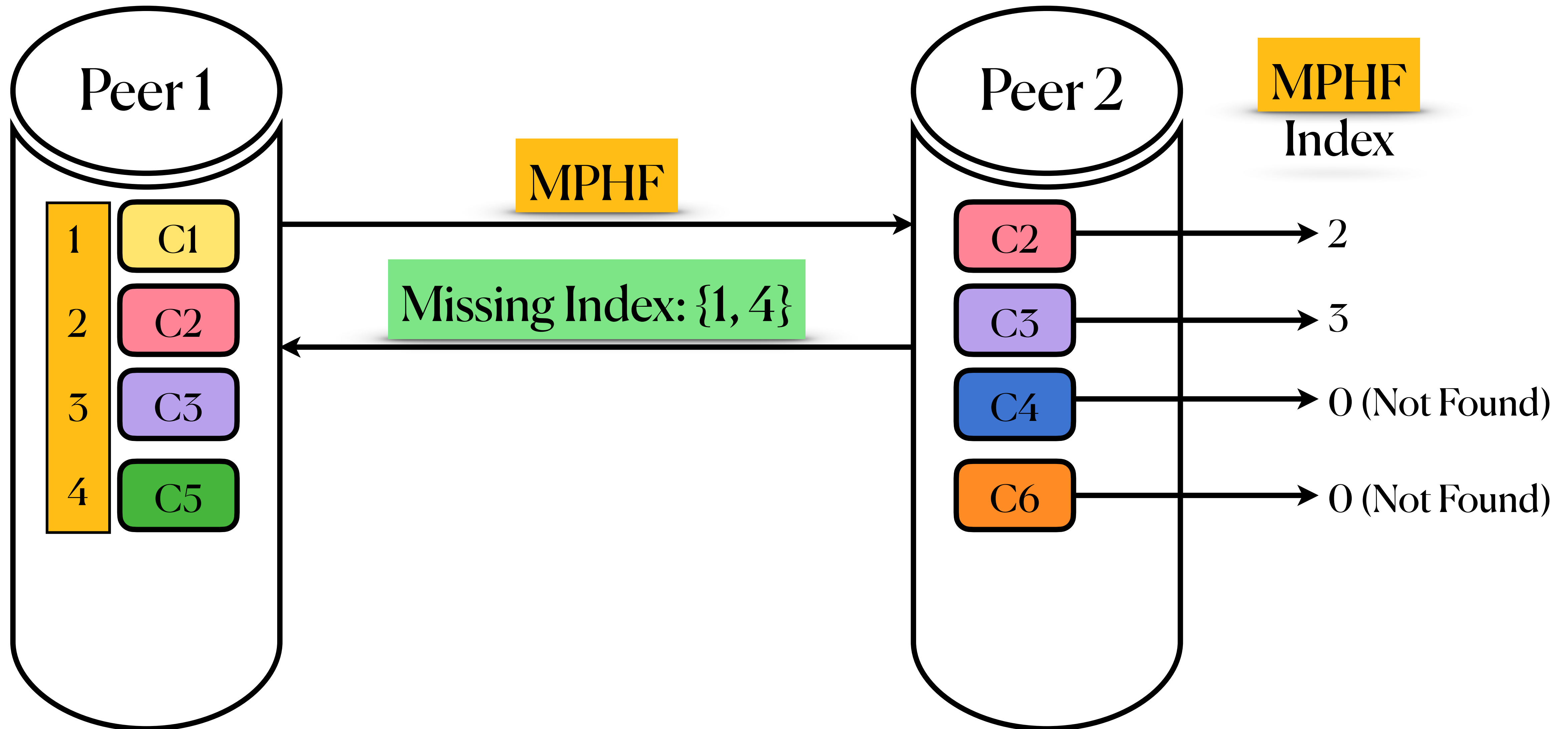
The Sync Protocol



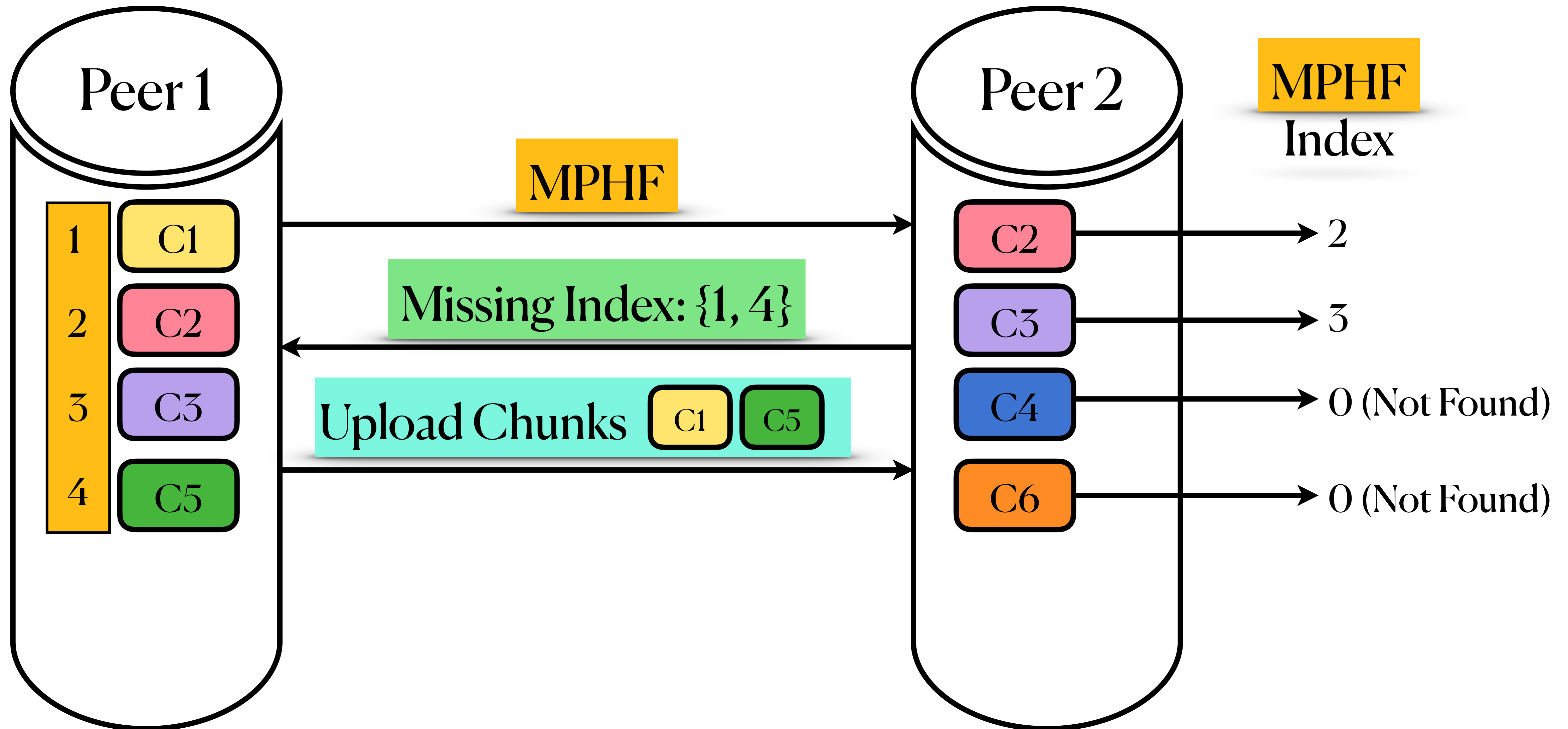
The Sync Protocol



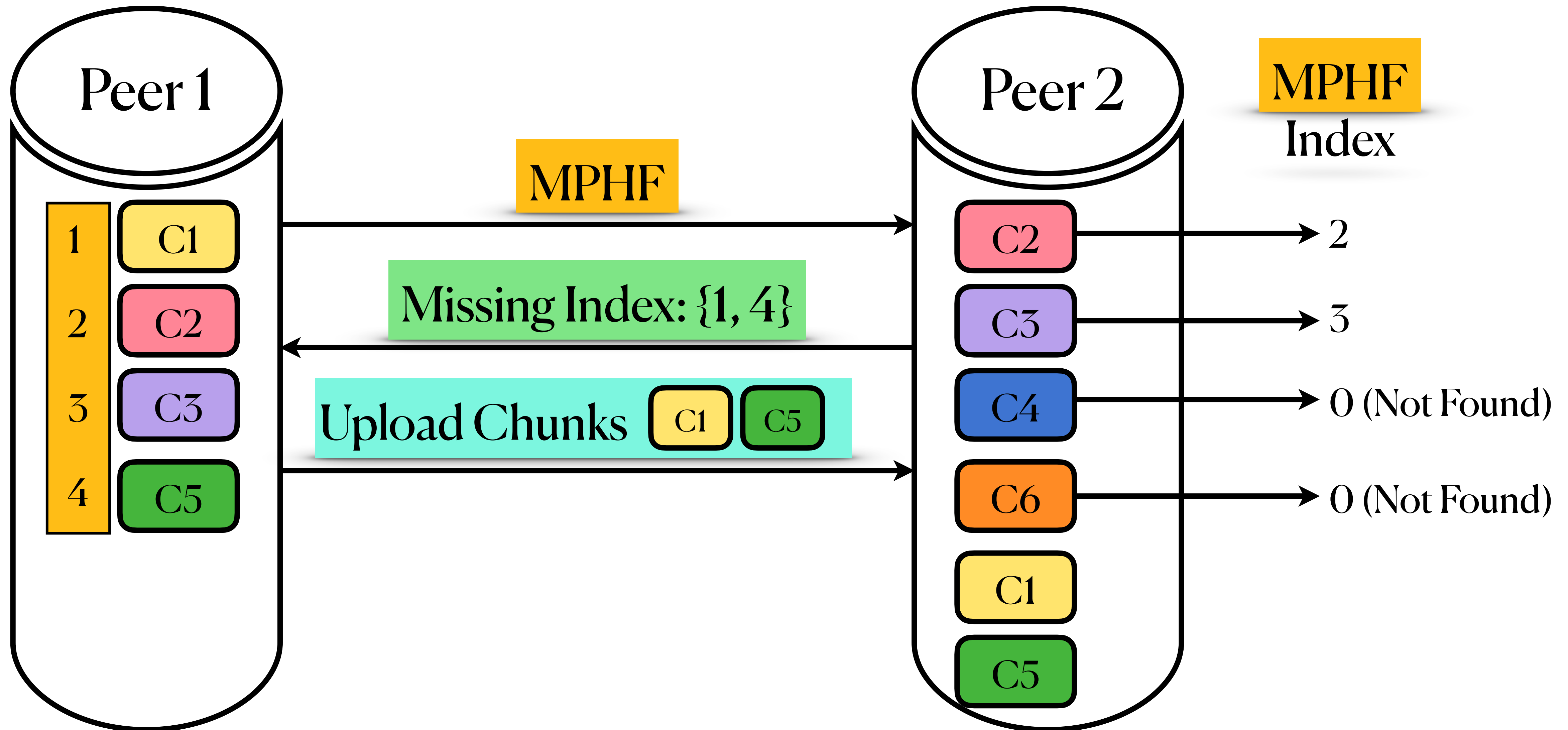
The Sync Protocol



The Sync Protocol

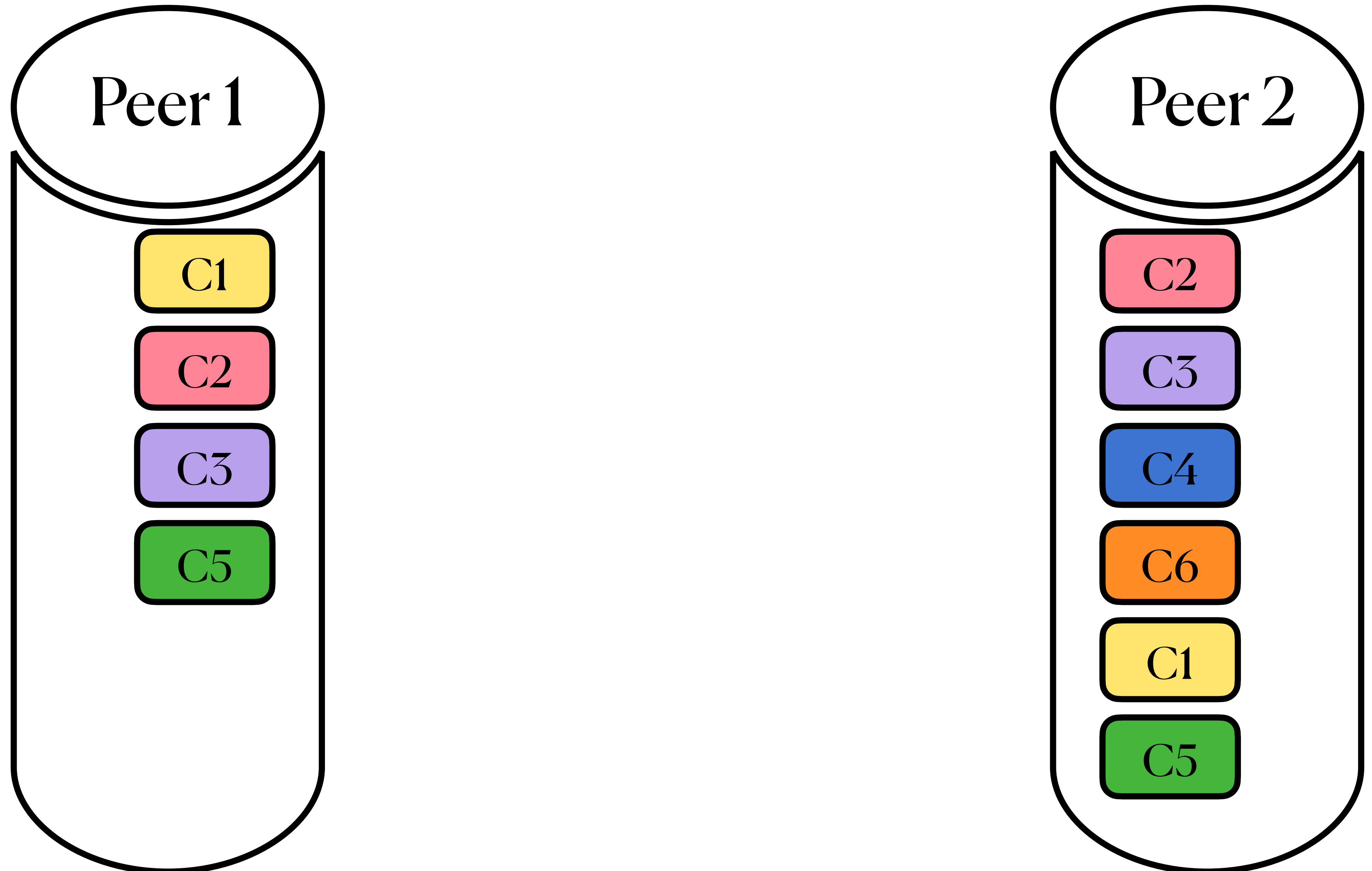


The Sync Protocol

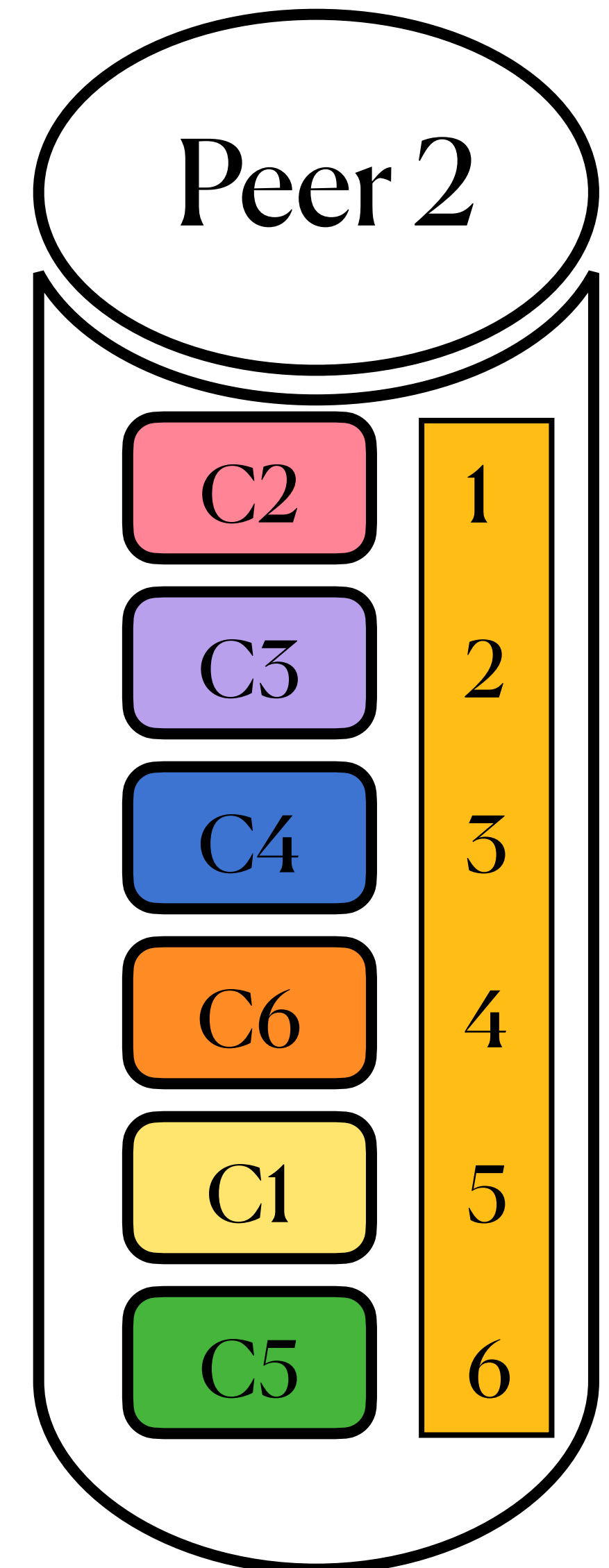
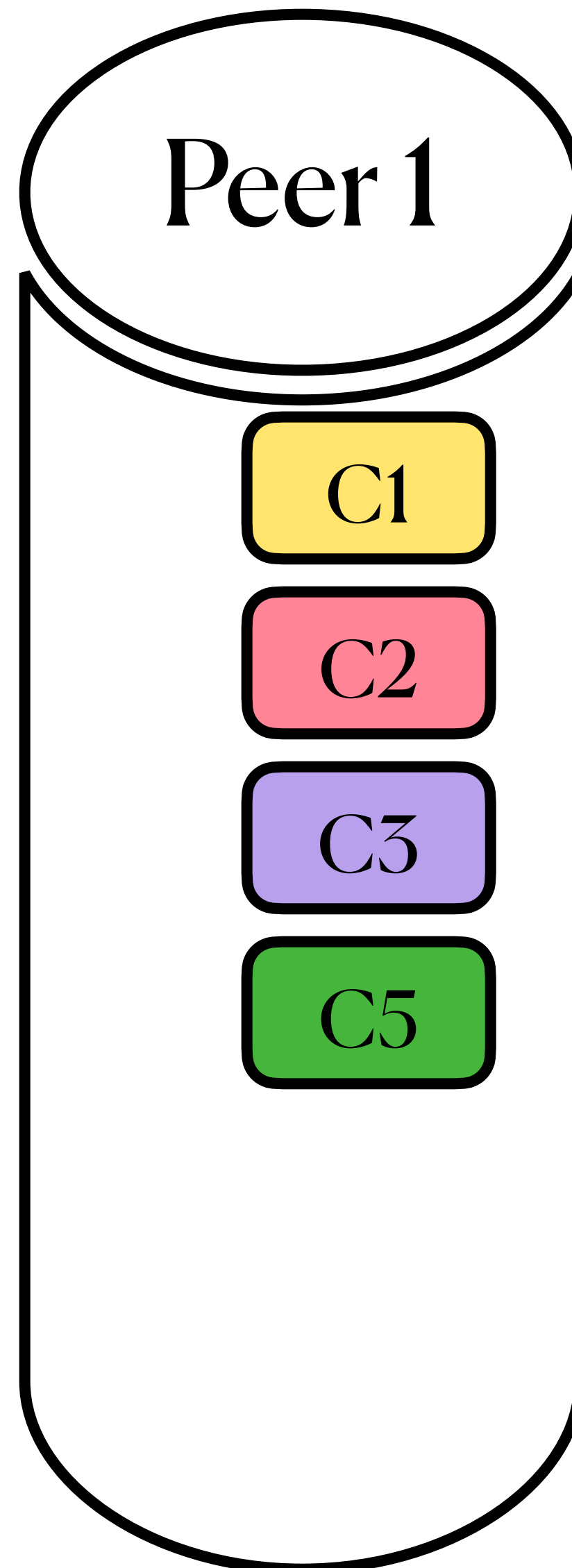


Reverse Sync

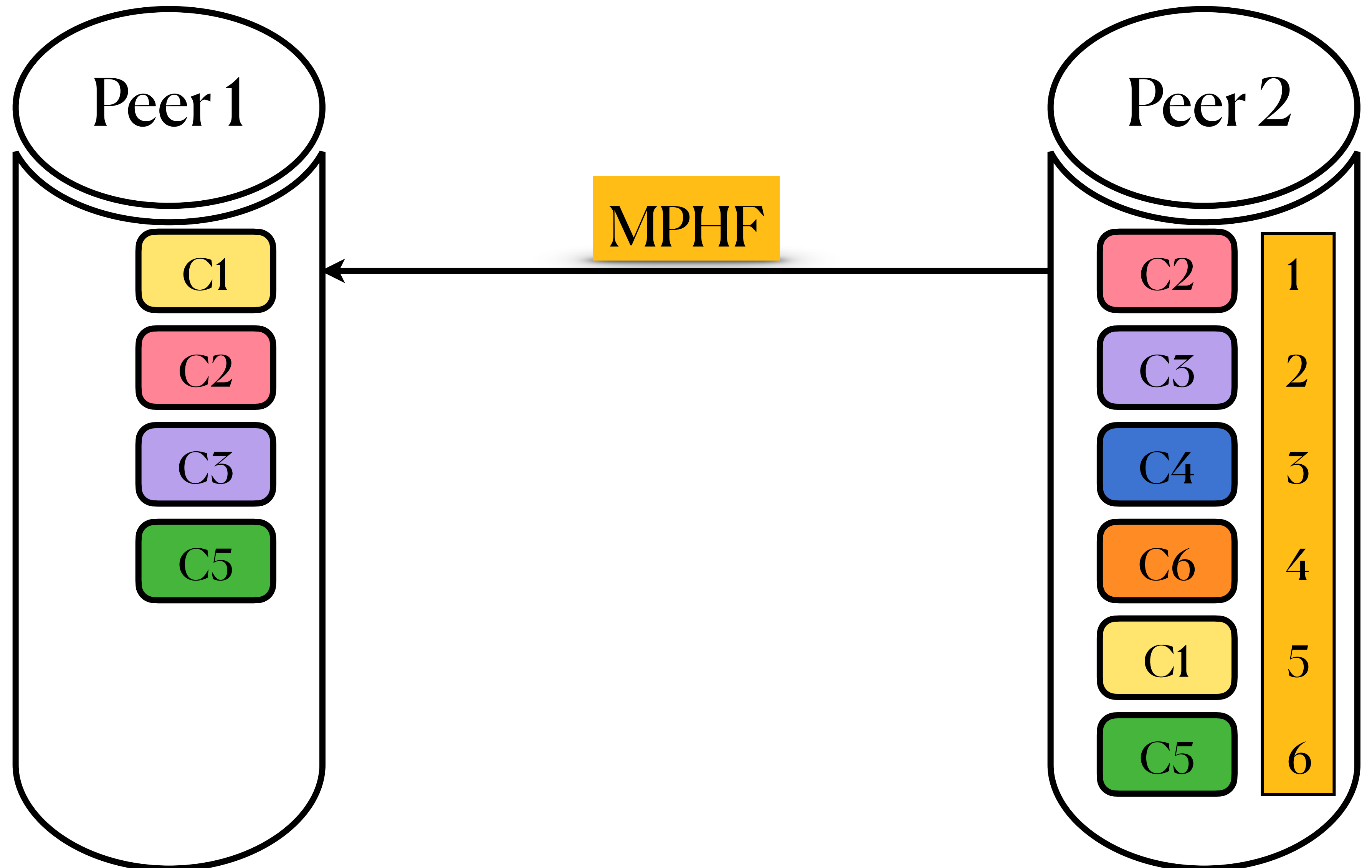
The Sync Protocol



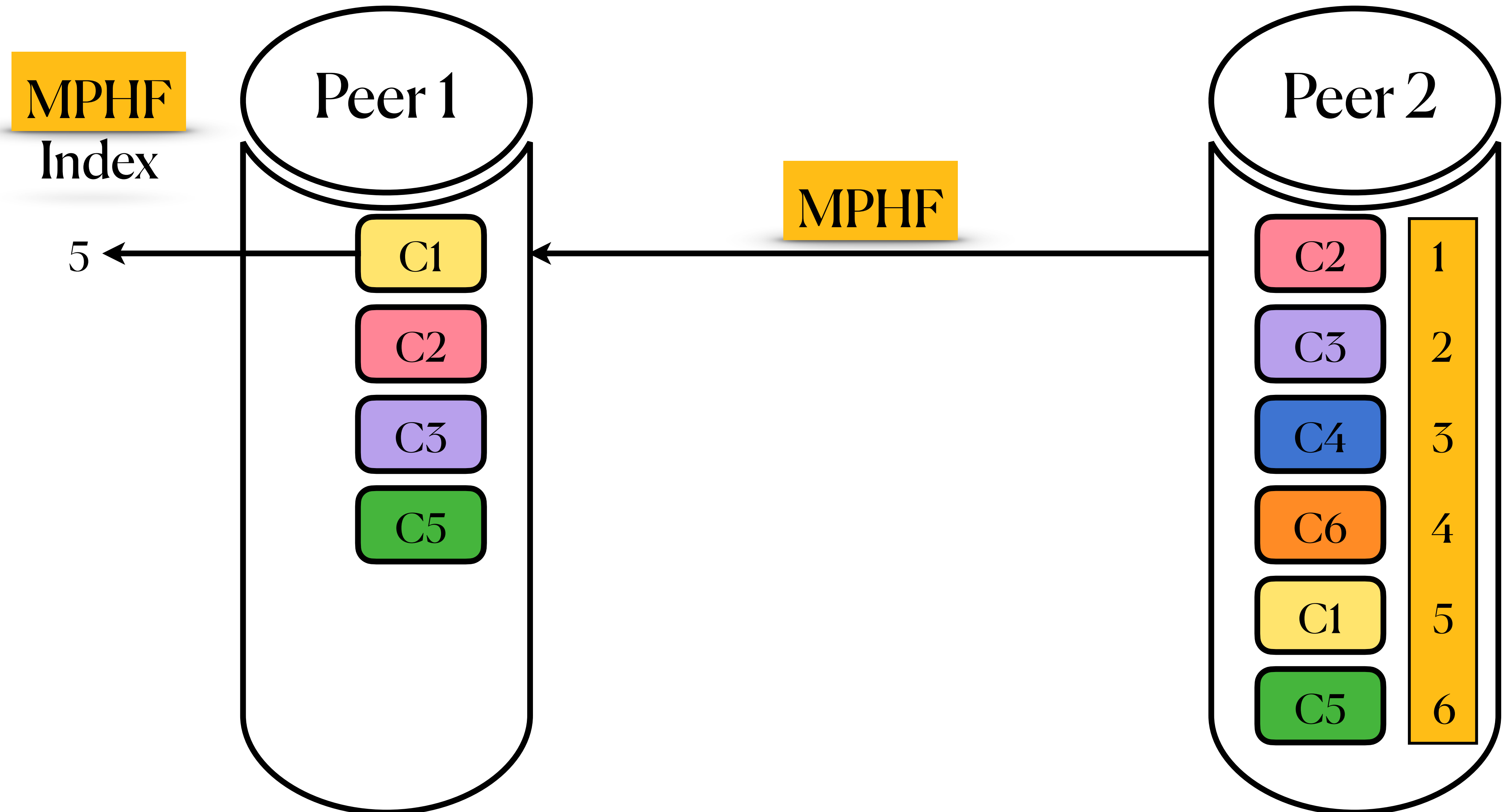
The Sync Protocol



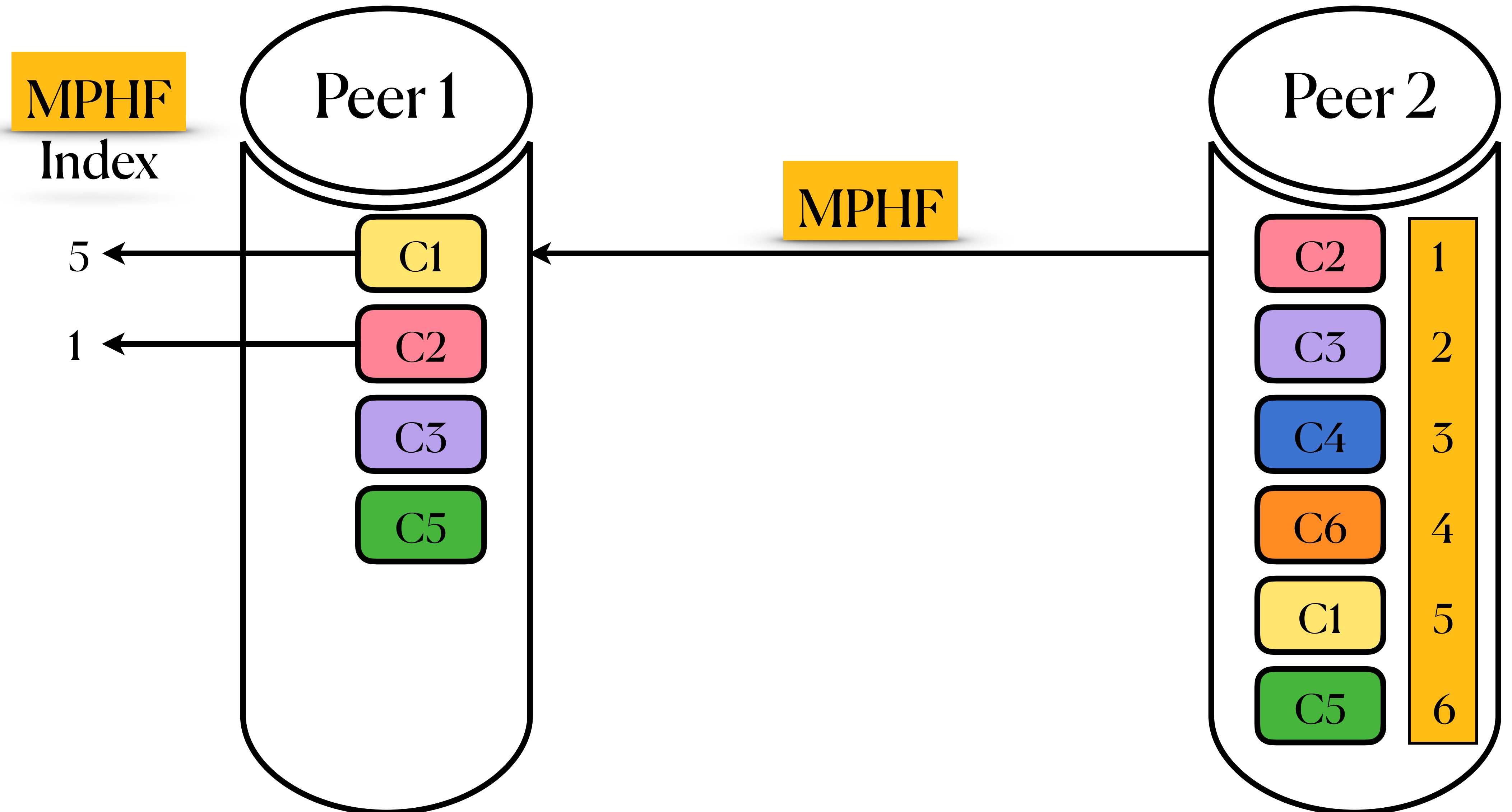
The Sync Protocol



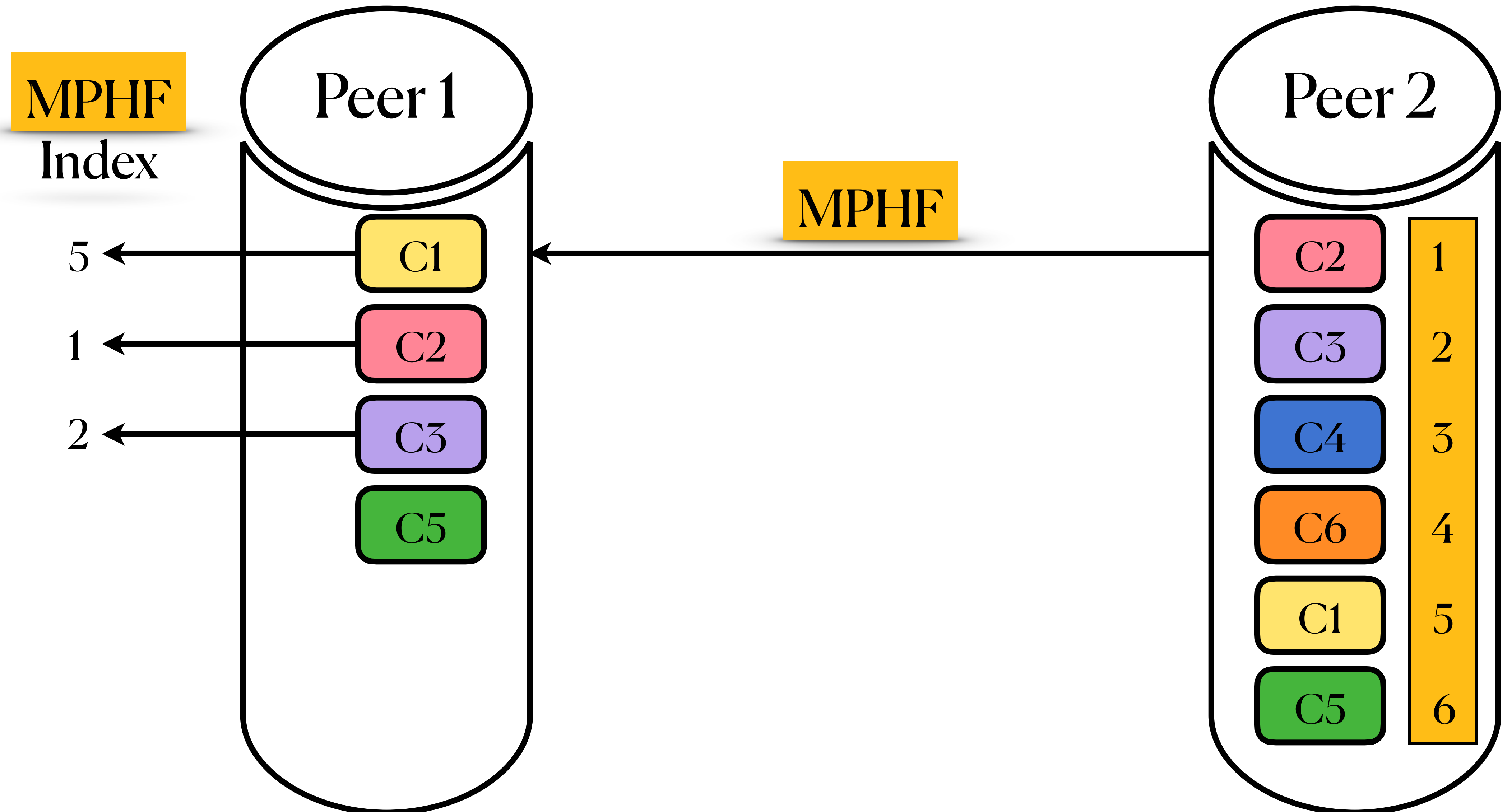
The Sync Protocol



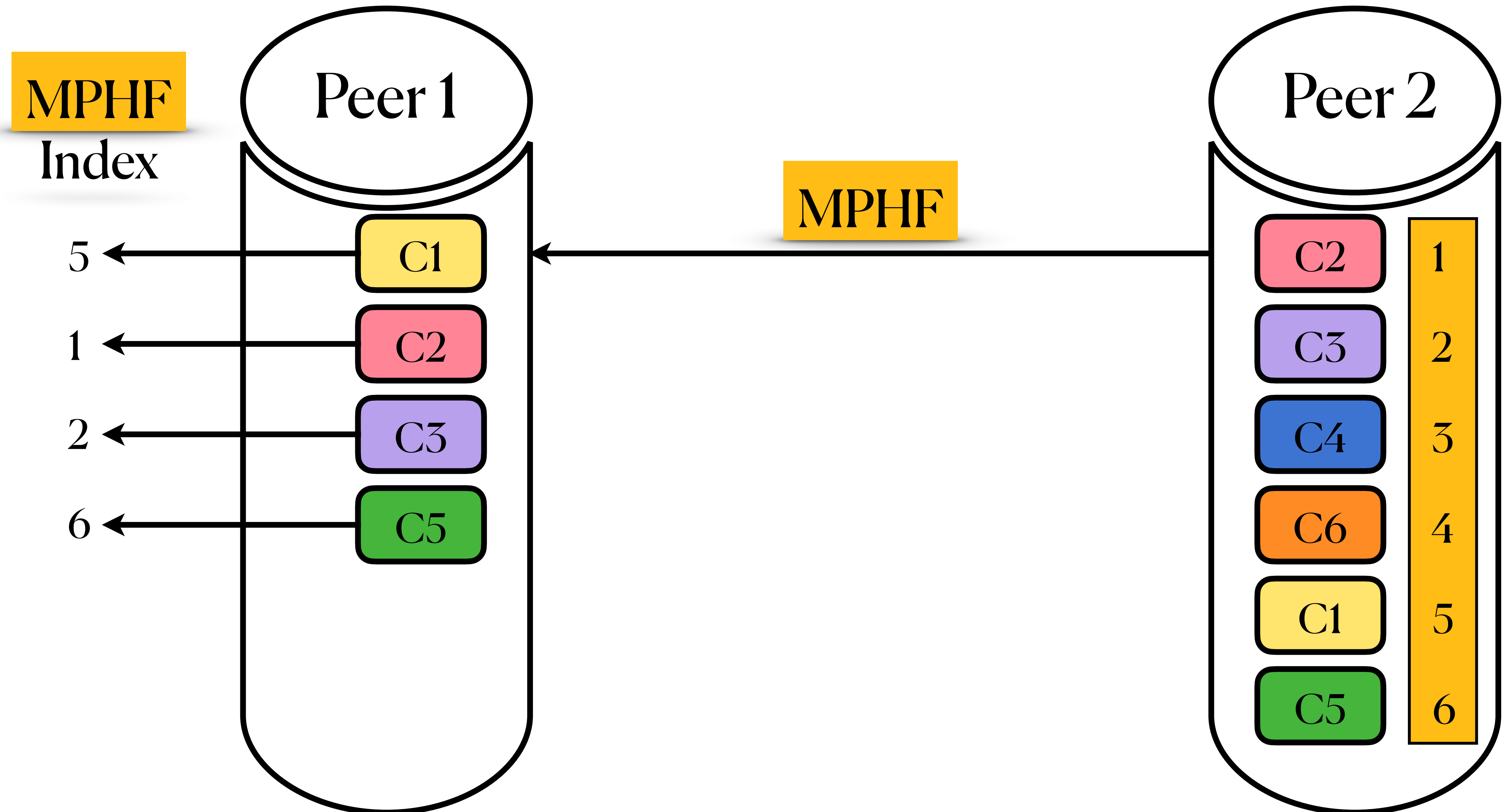
The Sync Protocol



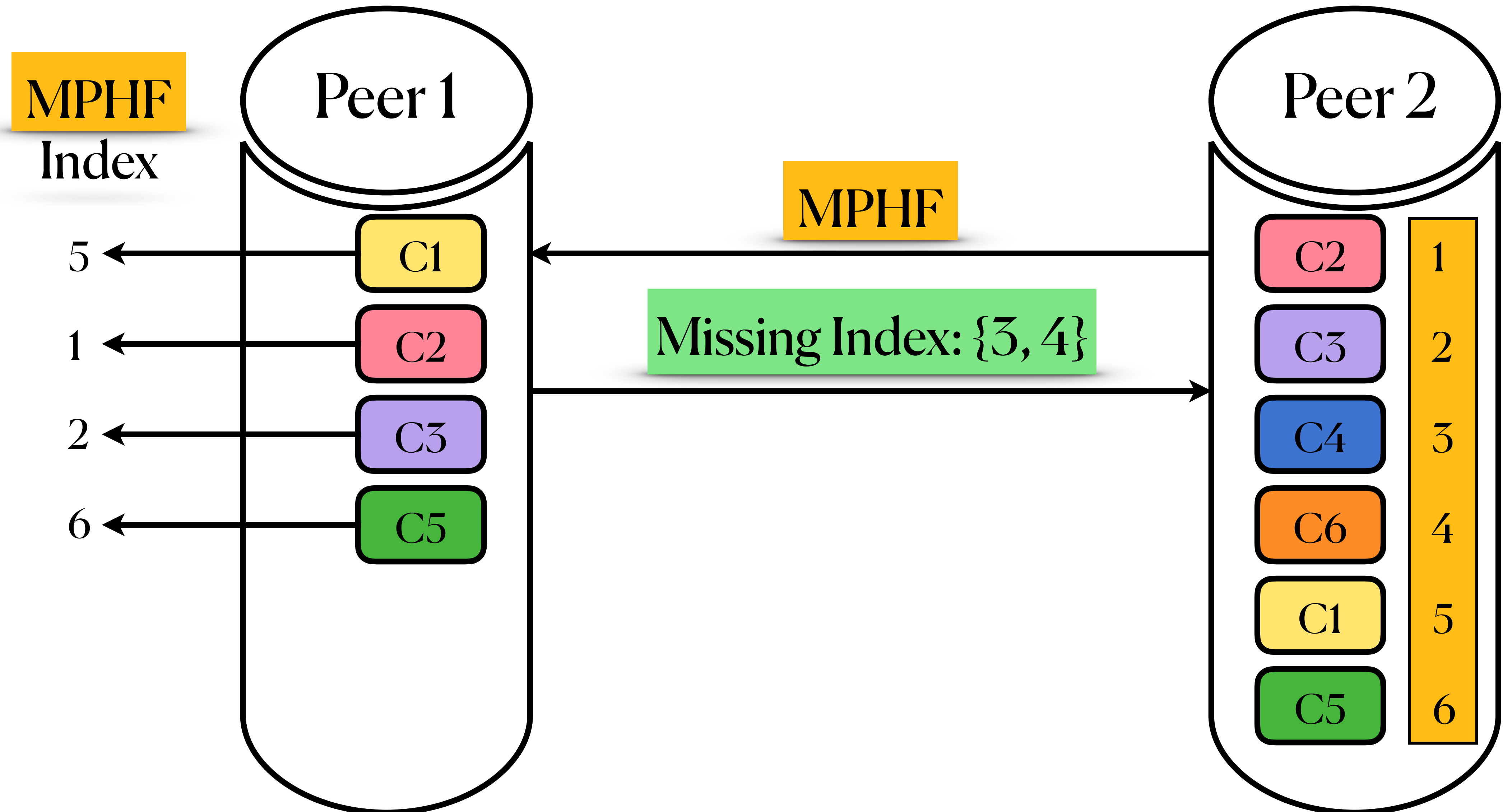
The Sync Protocol



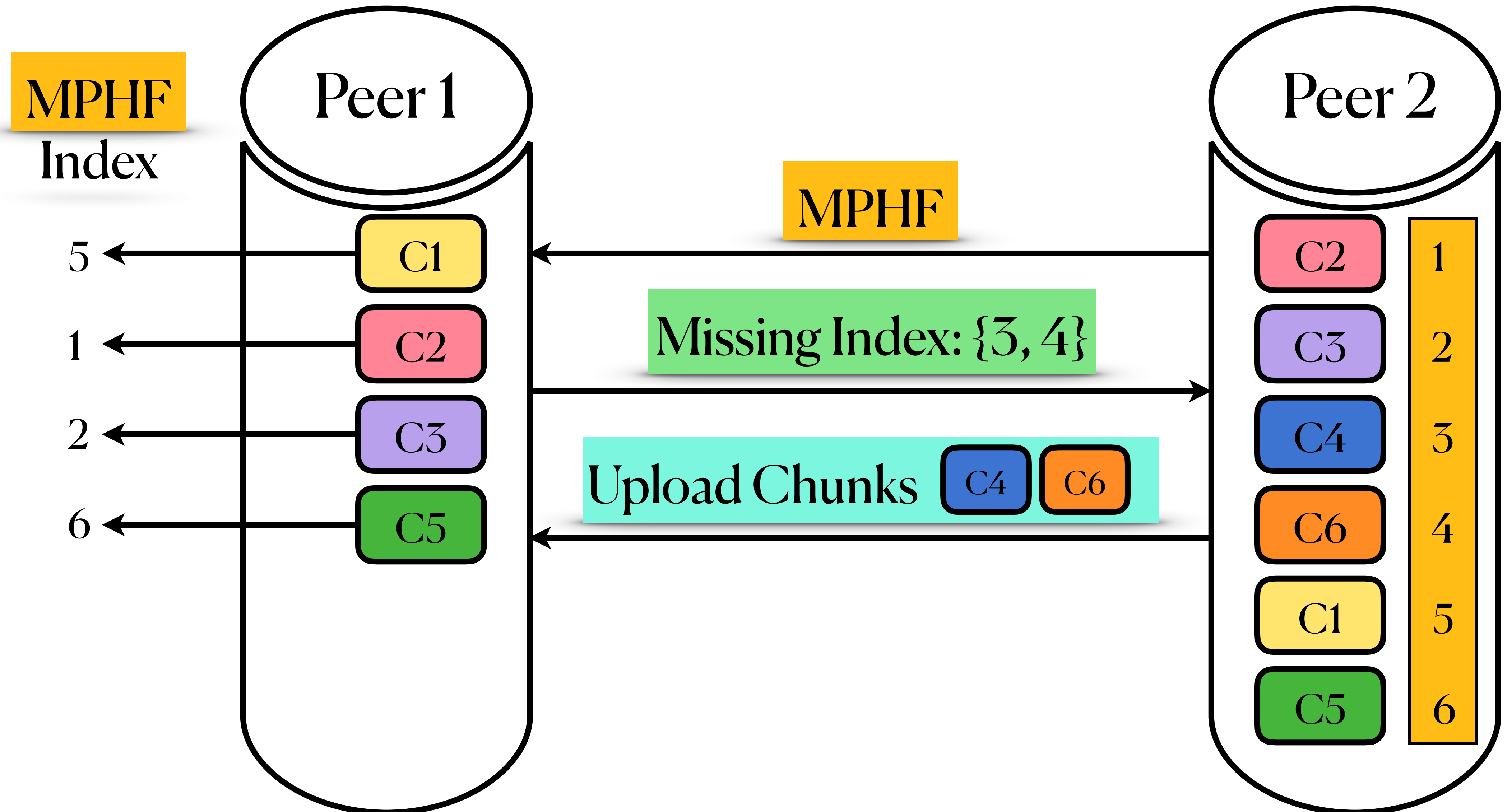
The Sync Protocol



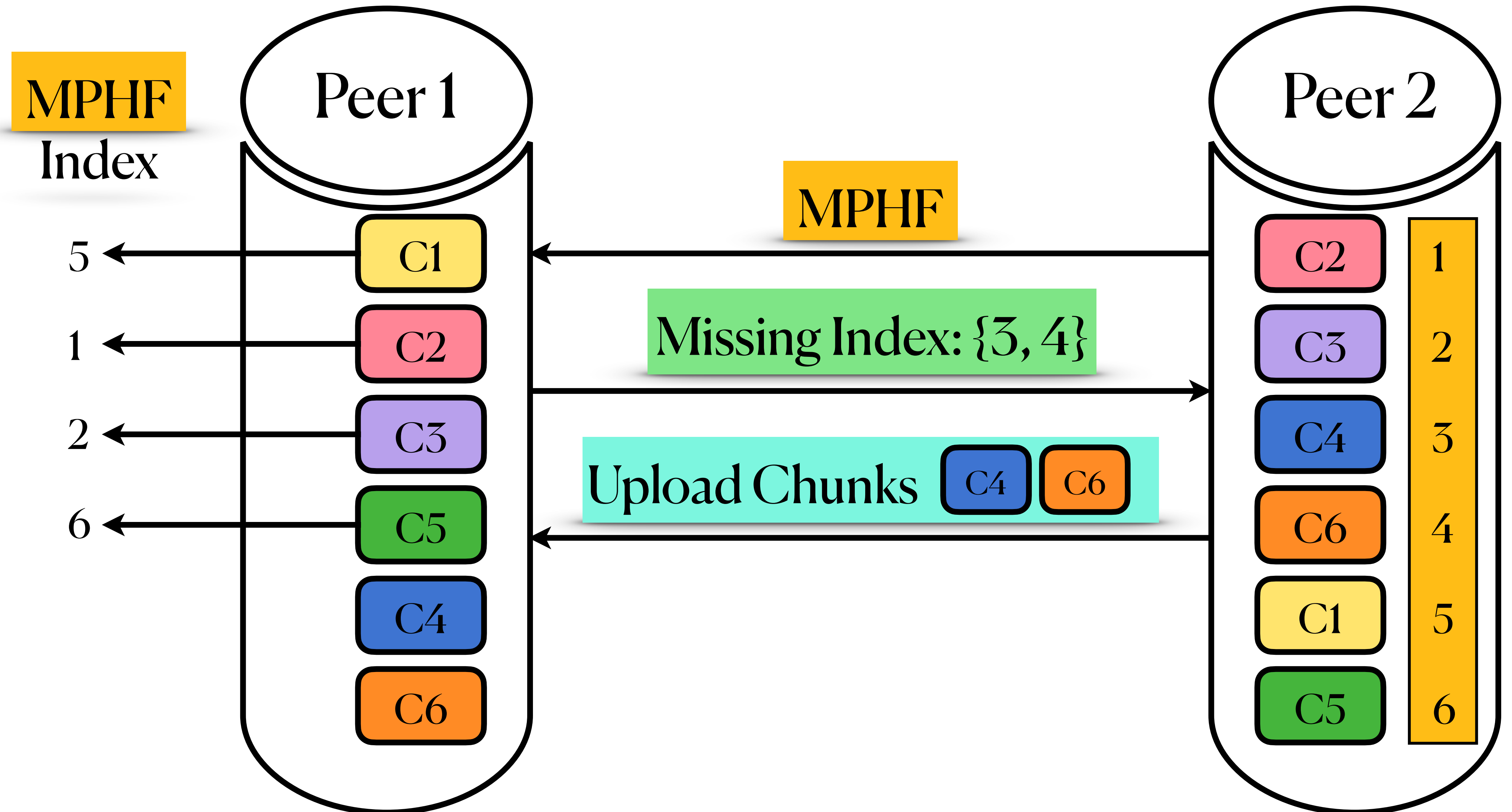
The Sync Protocol



The Sync Protocol

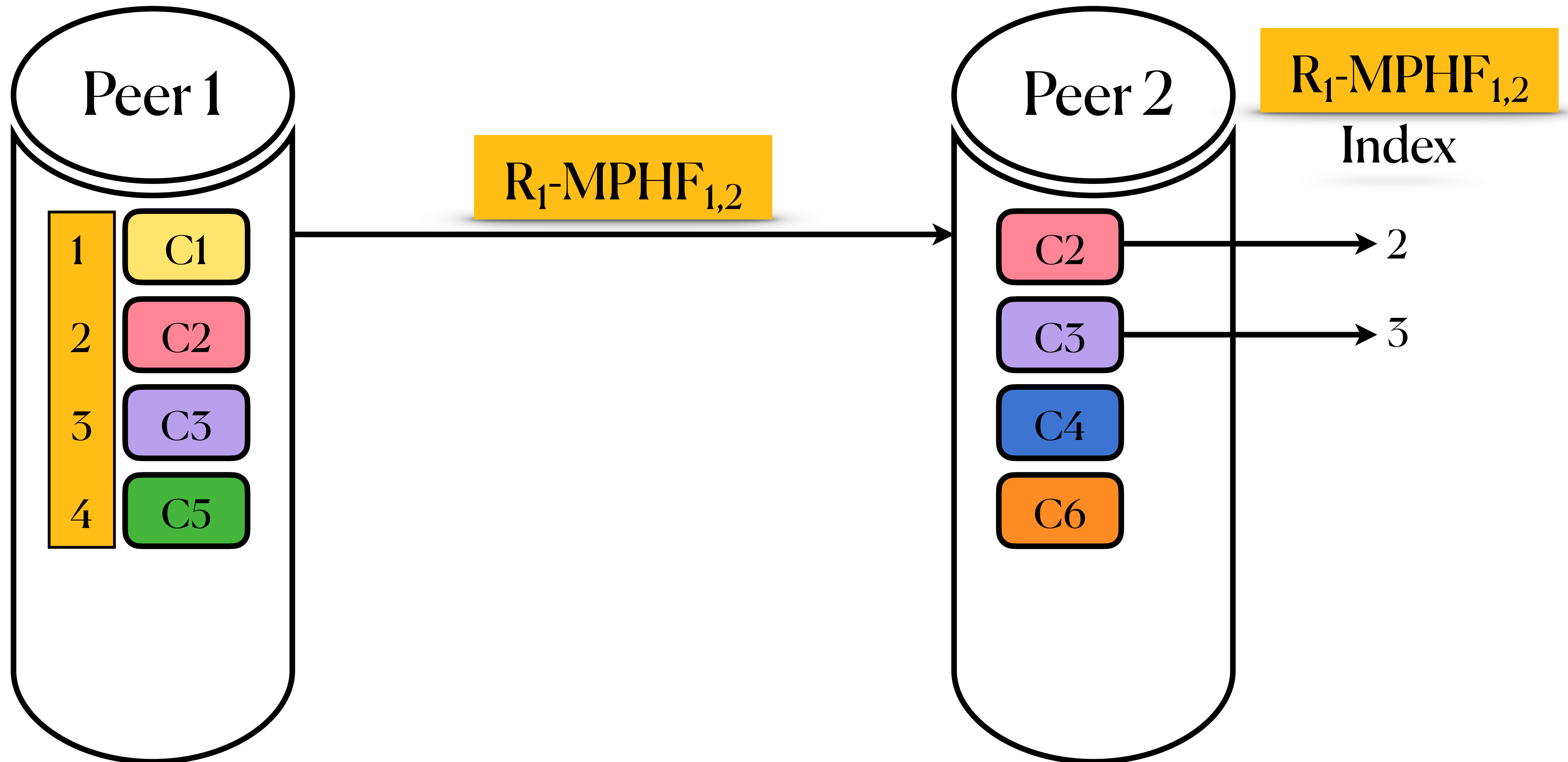


The Sync Protocol

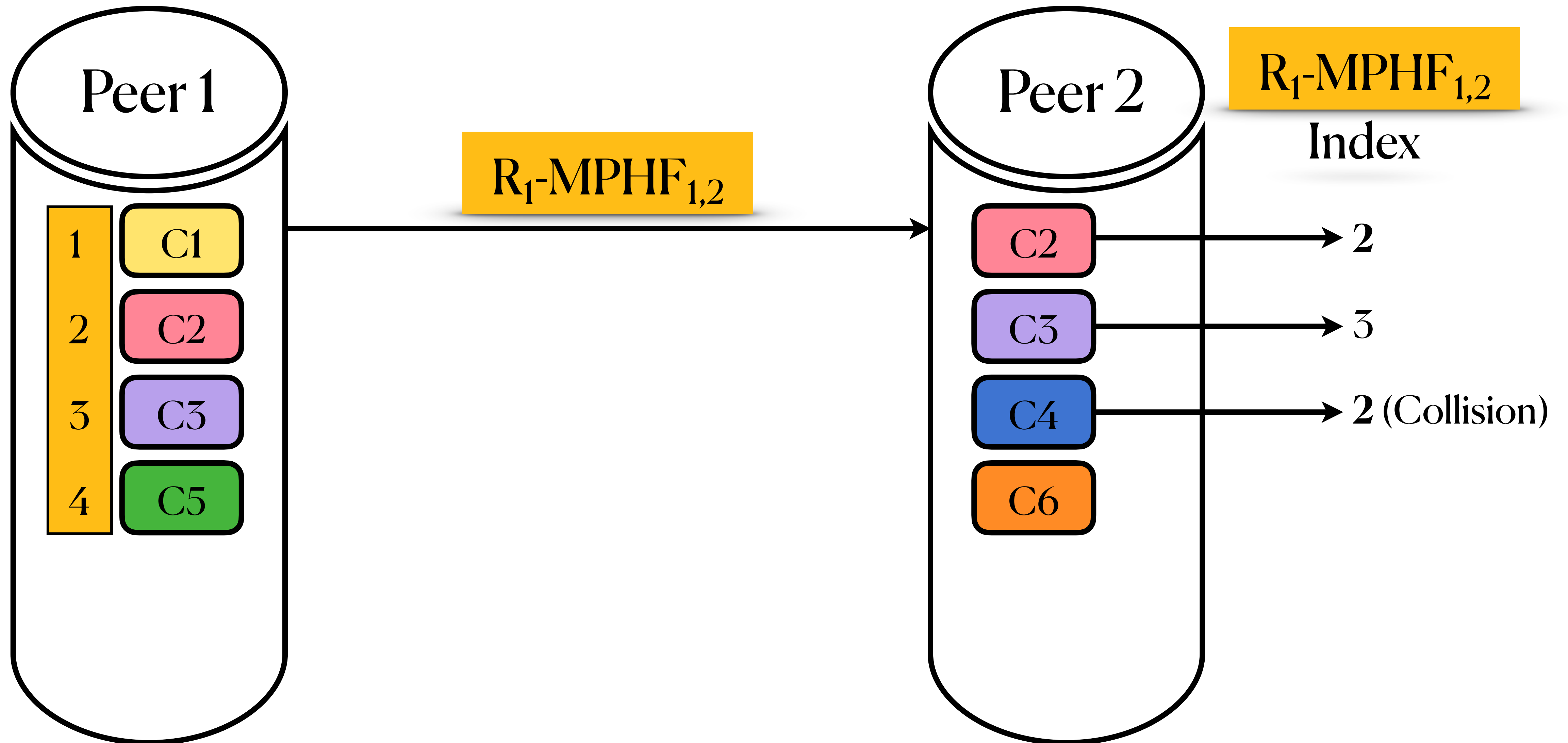


False Positives

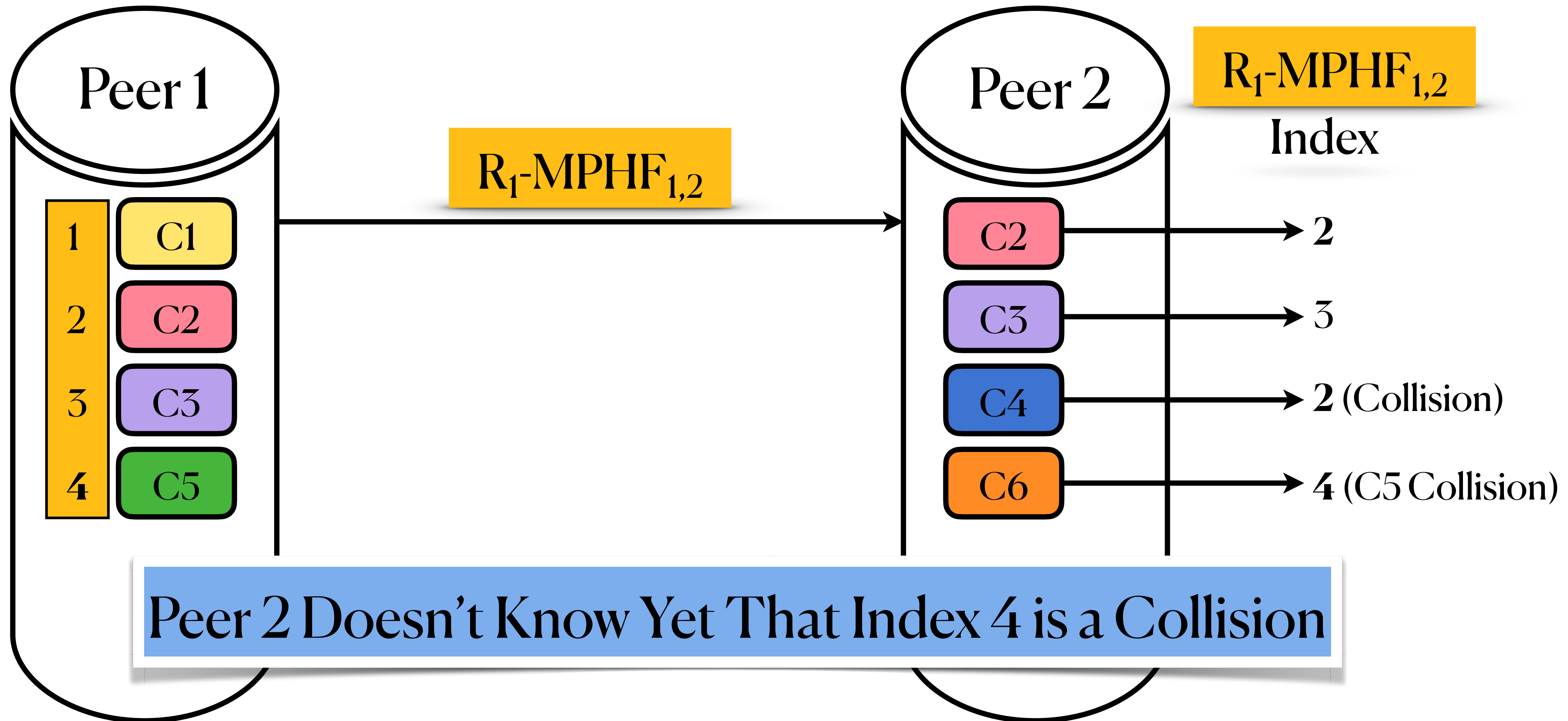
The Sync Protocol — Round 1



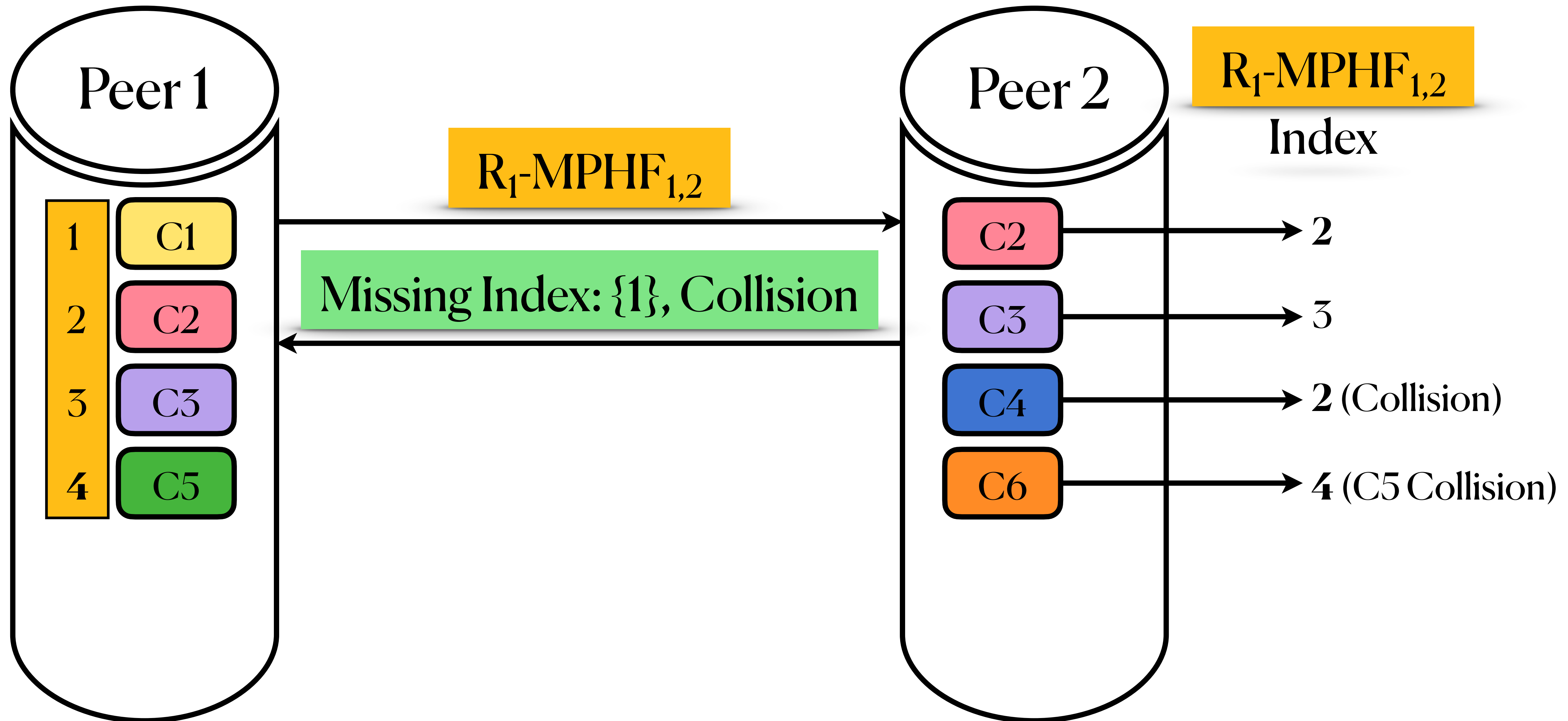
The Sync Protocol — Round 1



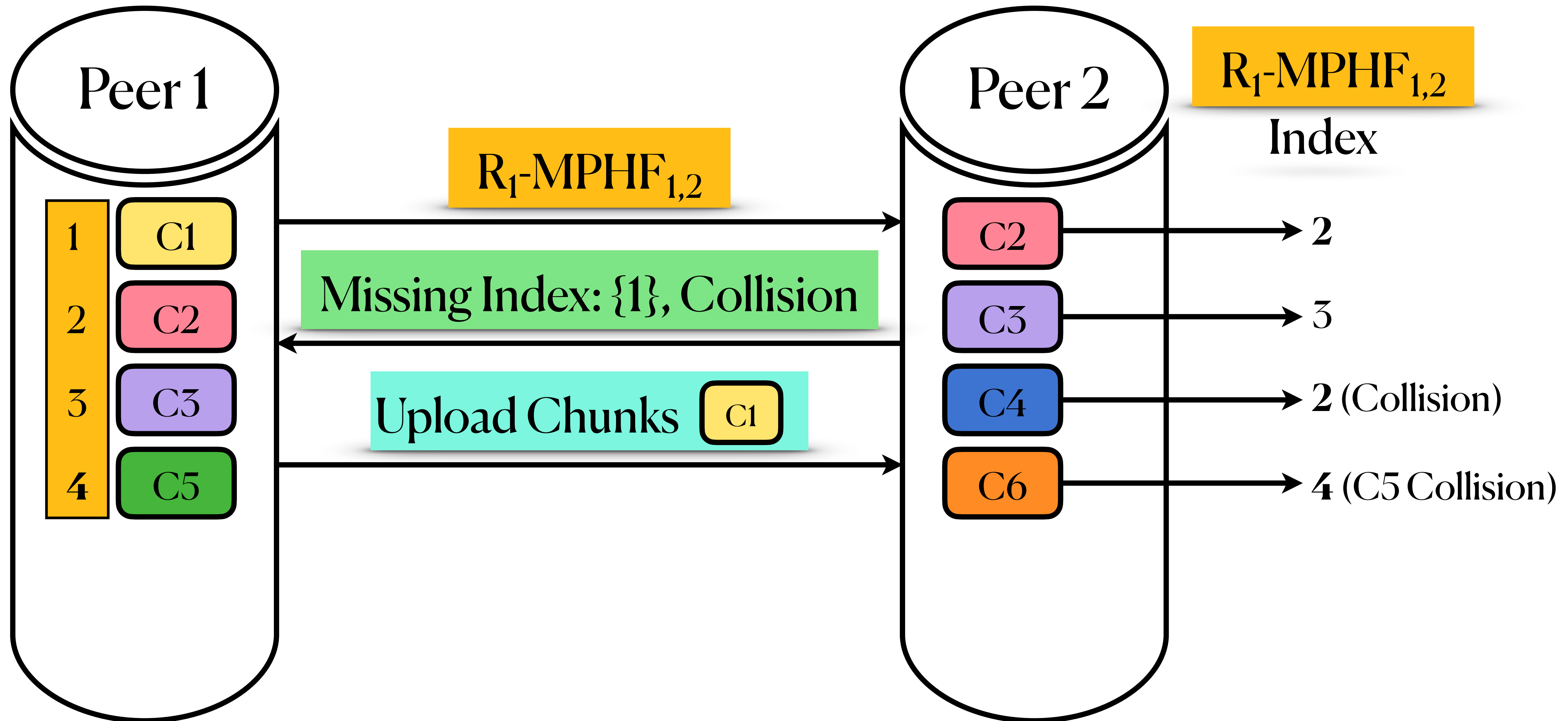
The Sync Protocol — Round 1



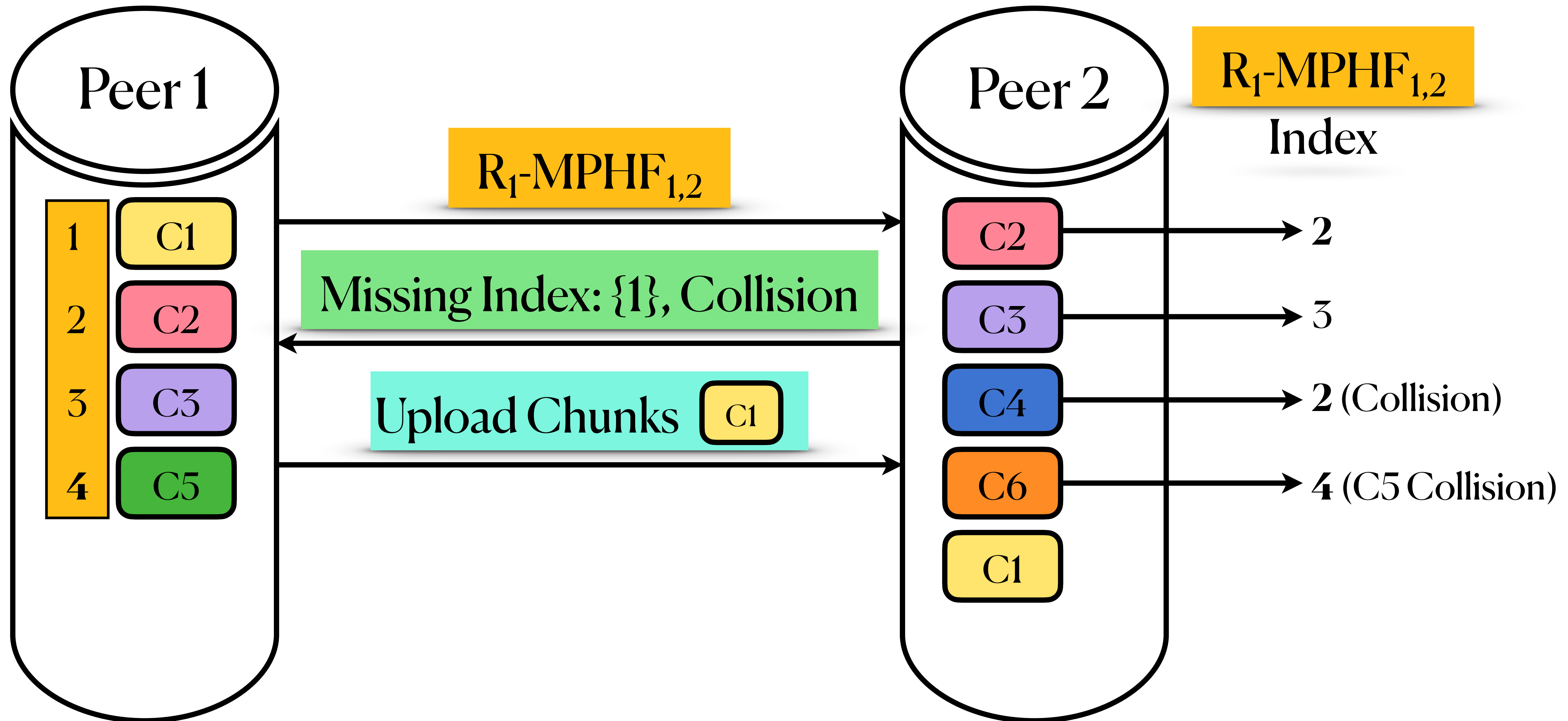
The Sync Protocol — Round 1



The Sync Protocol — Round 1

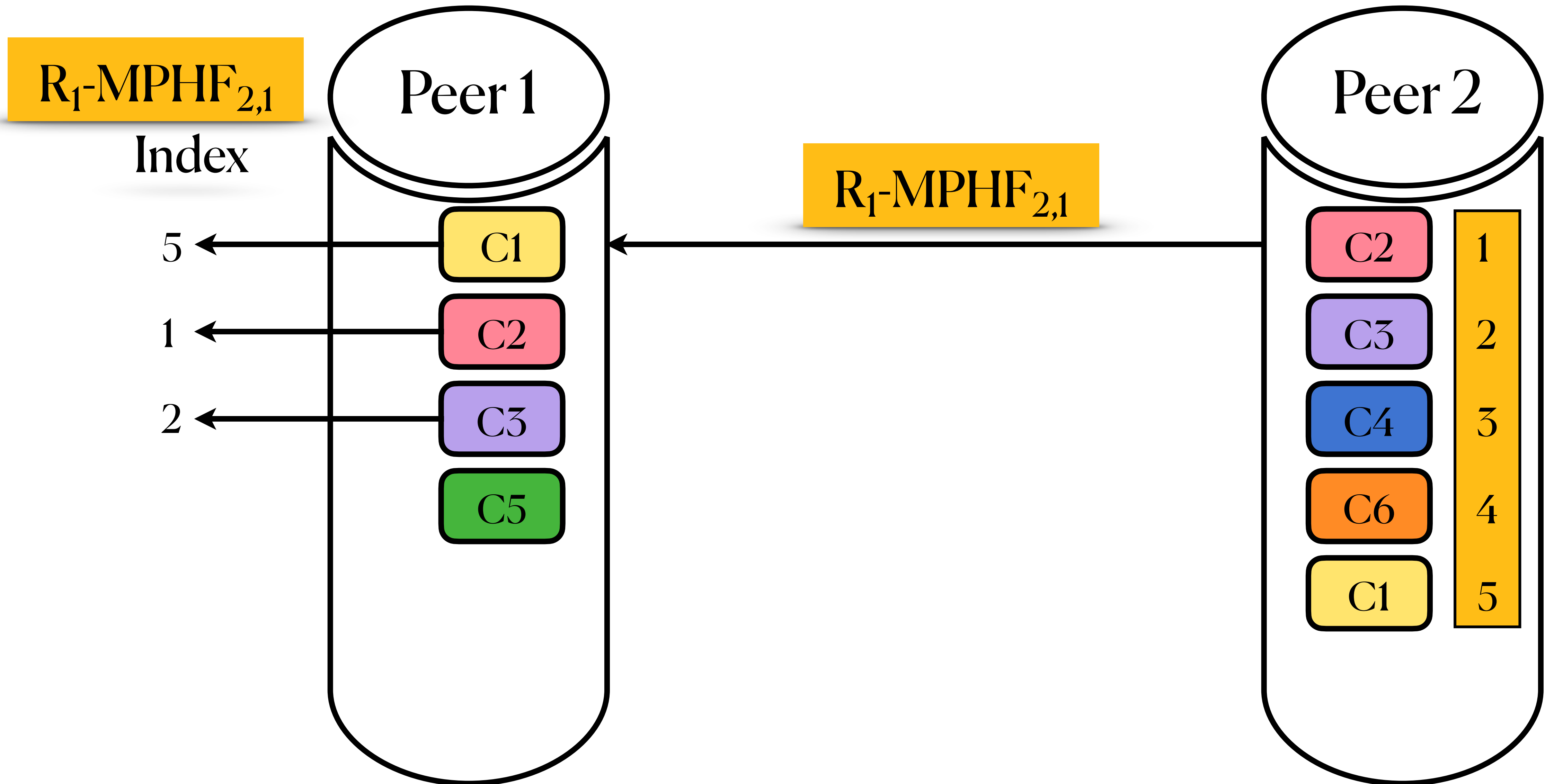


The Sync Protocol — Round 1

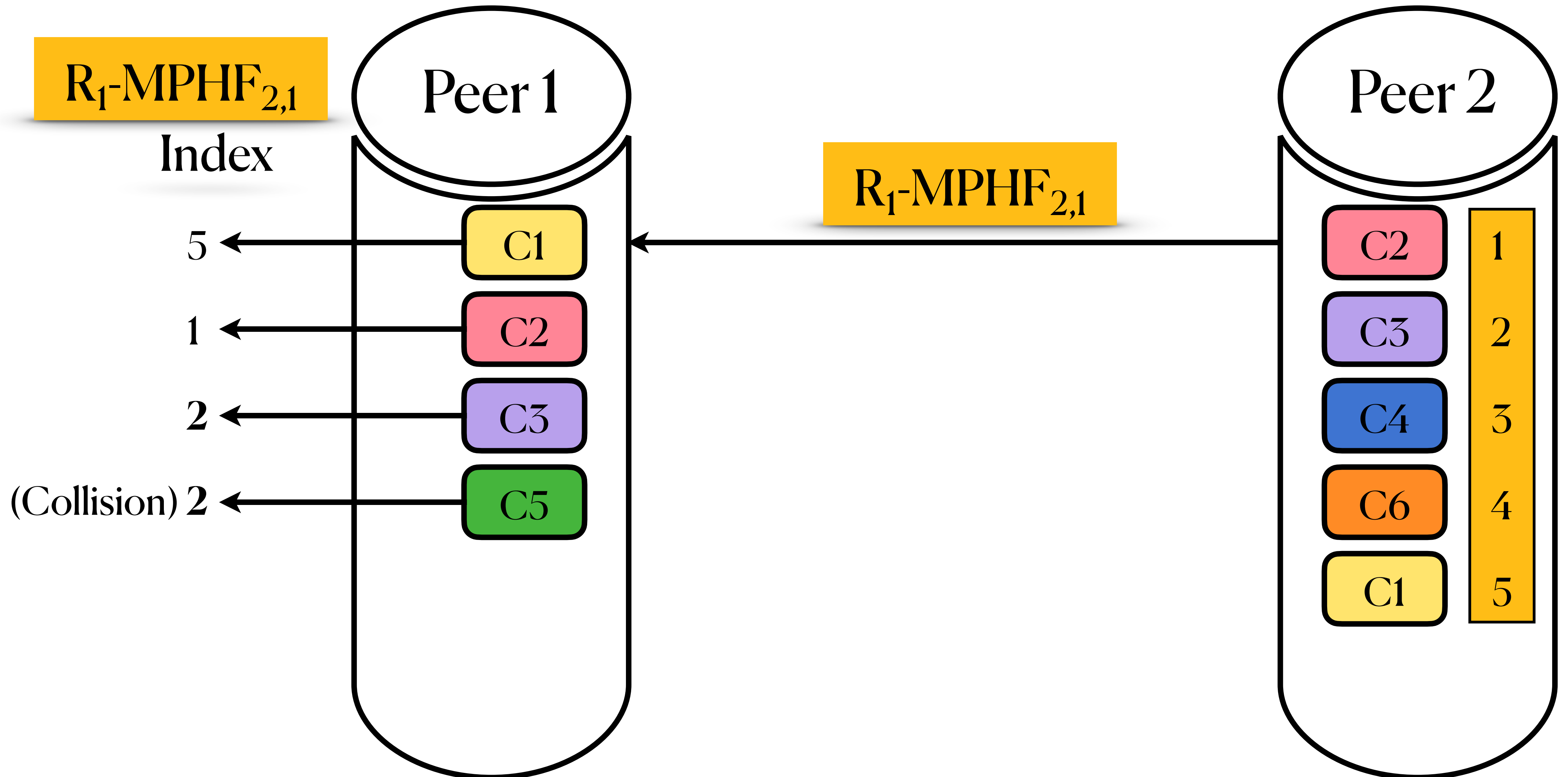


Reverse Sync

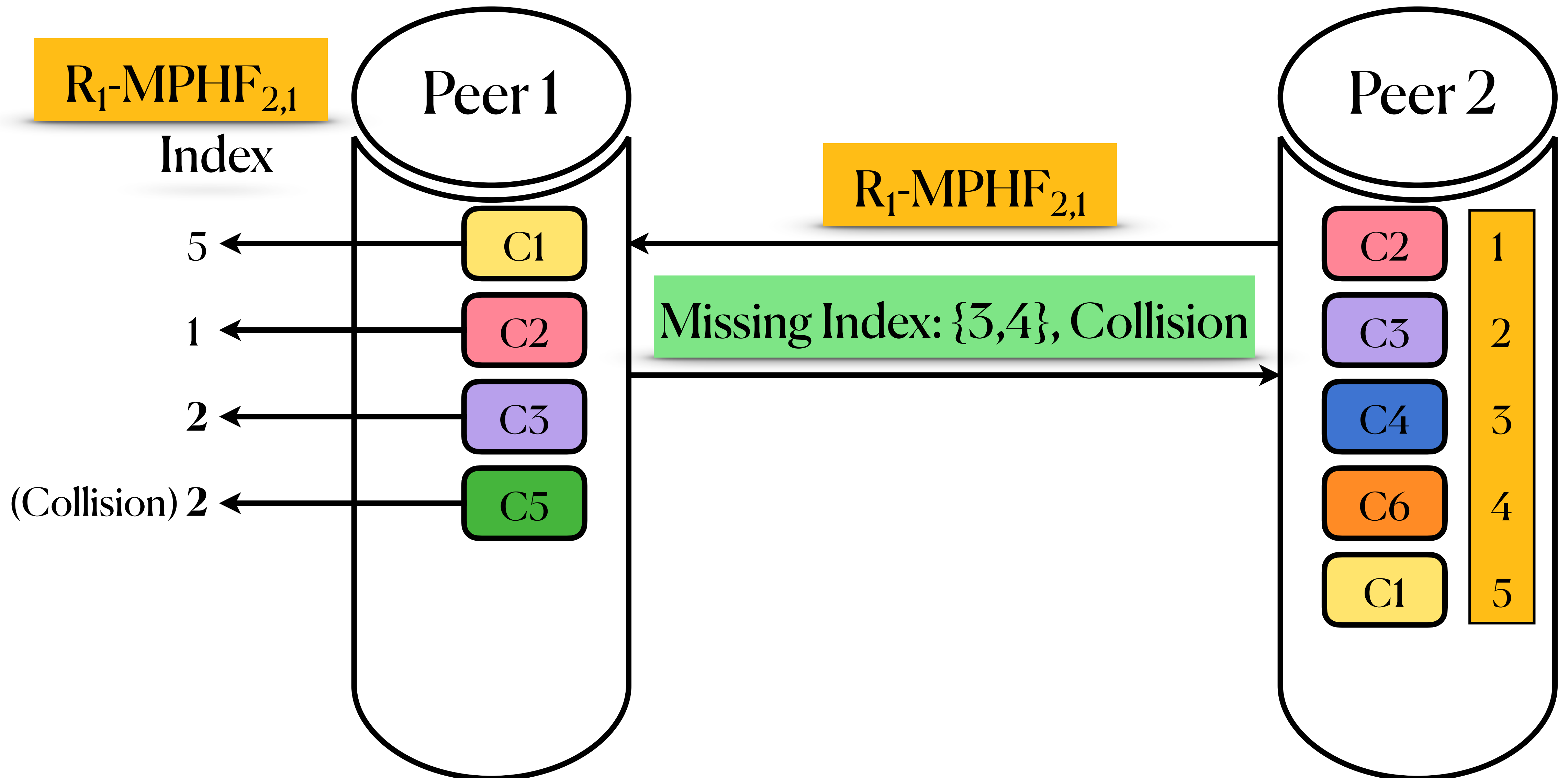
The Sync Protocol — Round 1



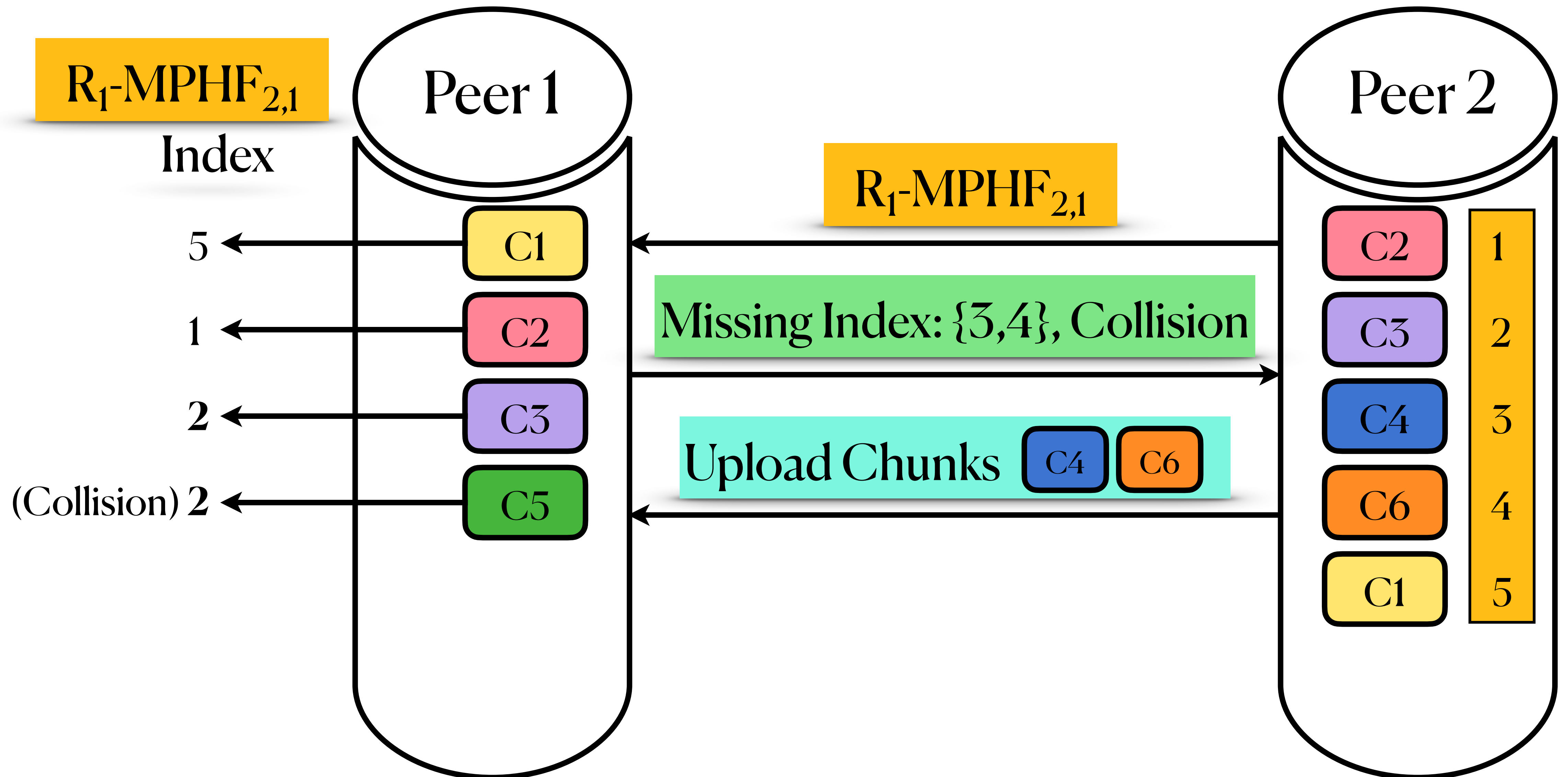
The Sync Protocol — Round 1



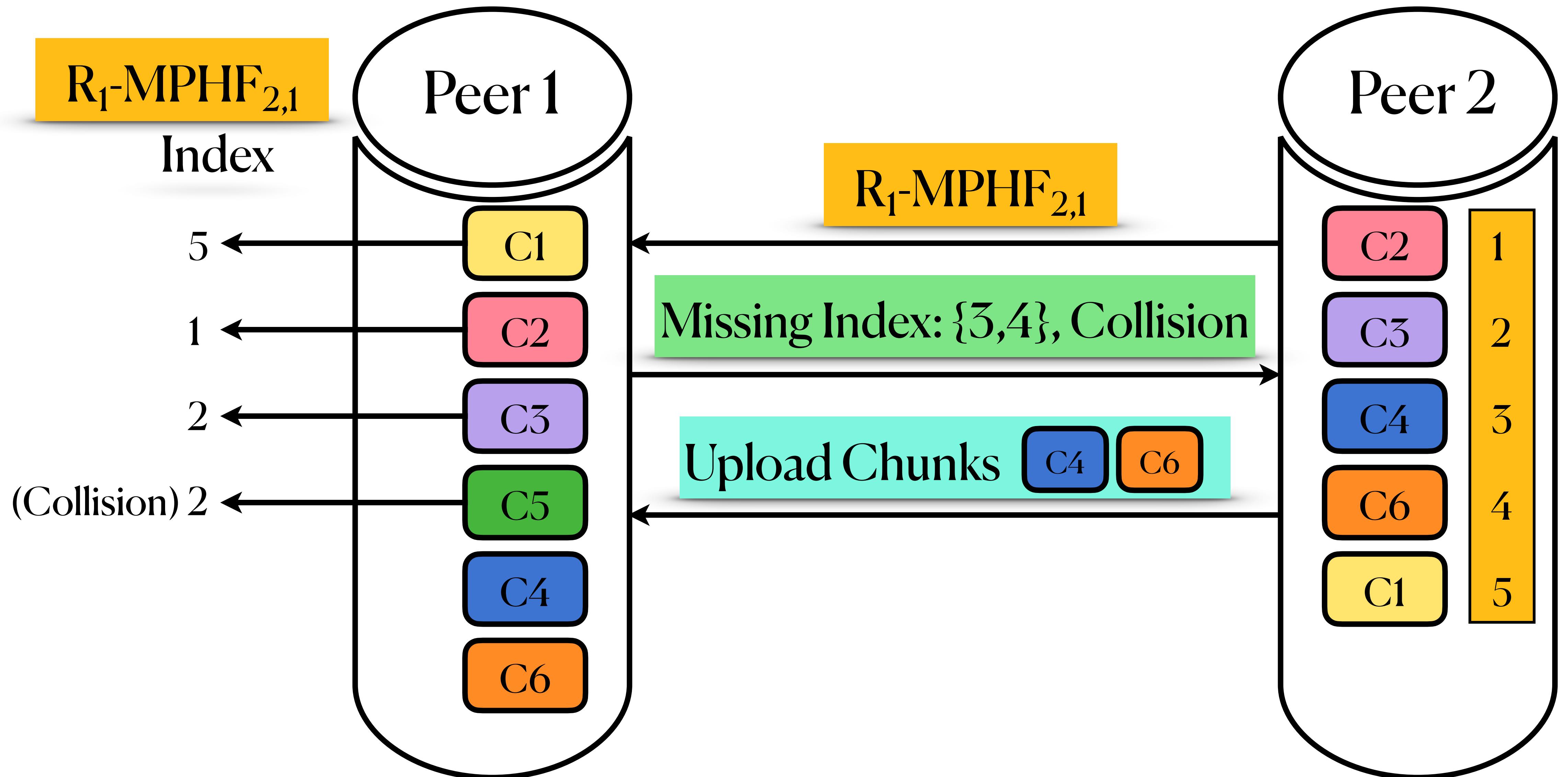
The Sync Protocol — Round 1



The Sync Protocol — Round 1

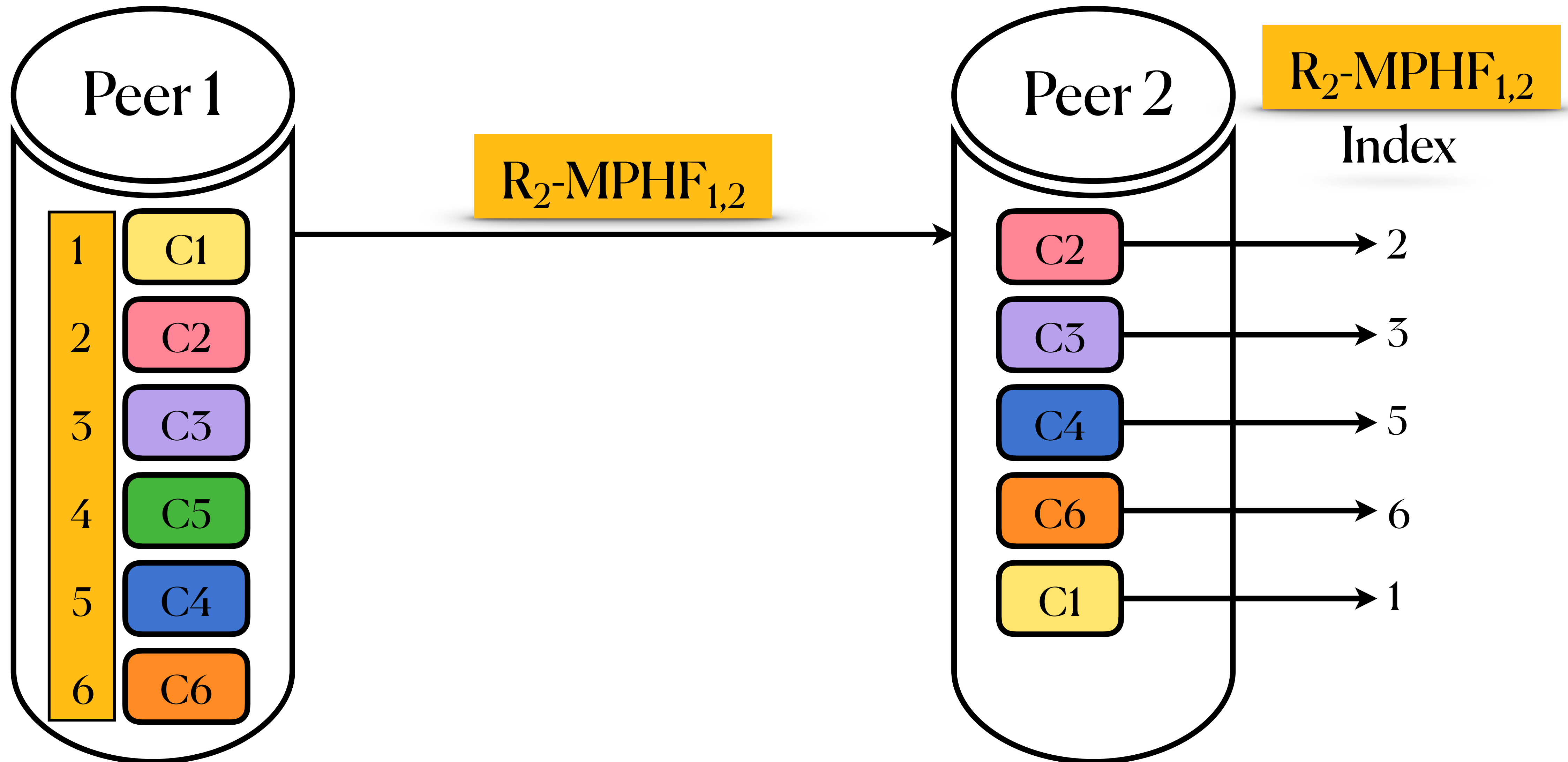


The Sync Protocol — Round 1

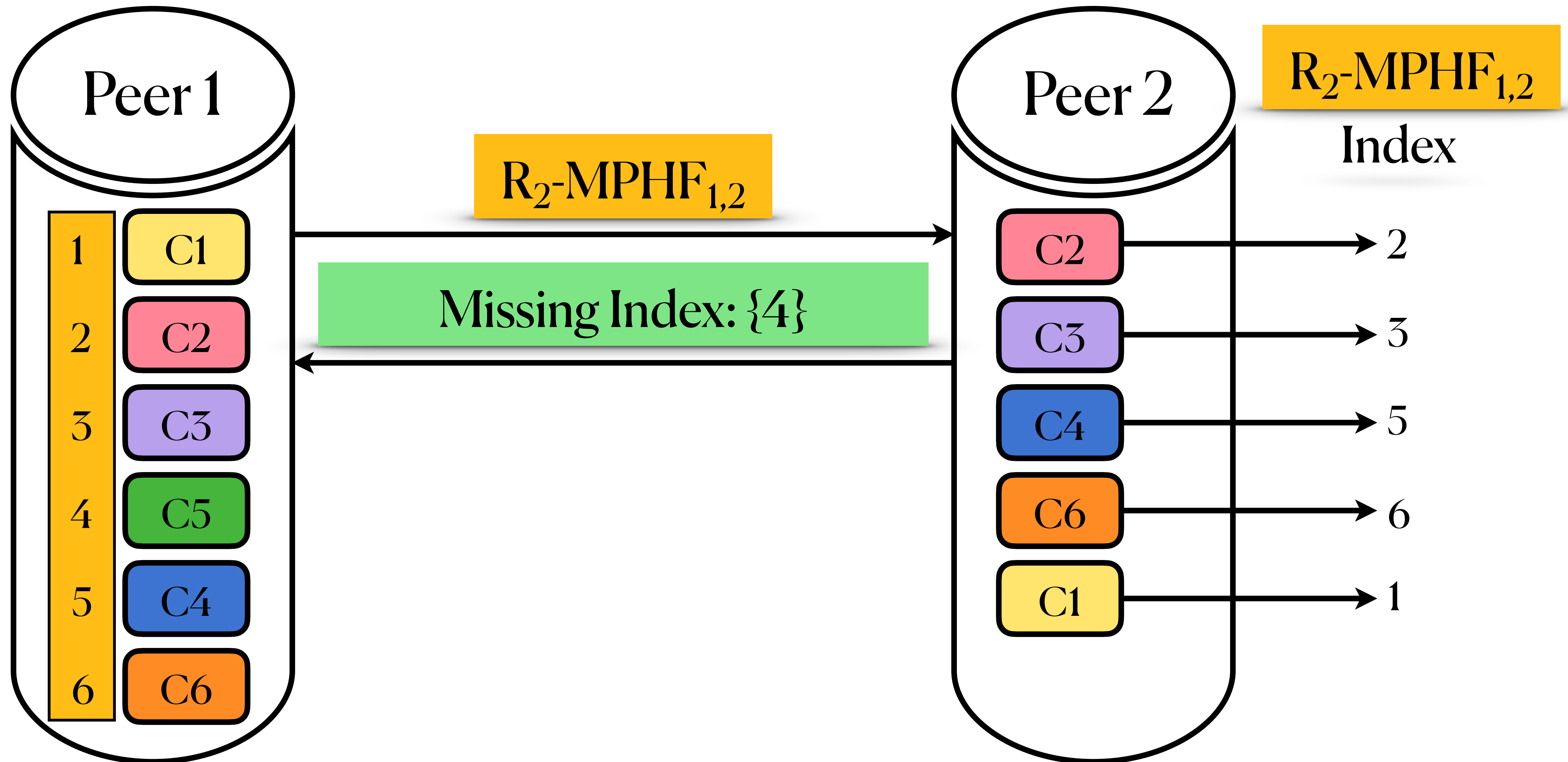


Round 2

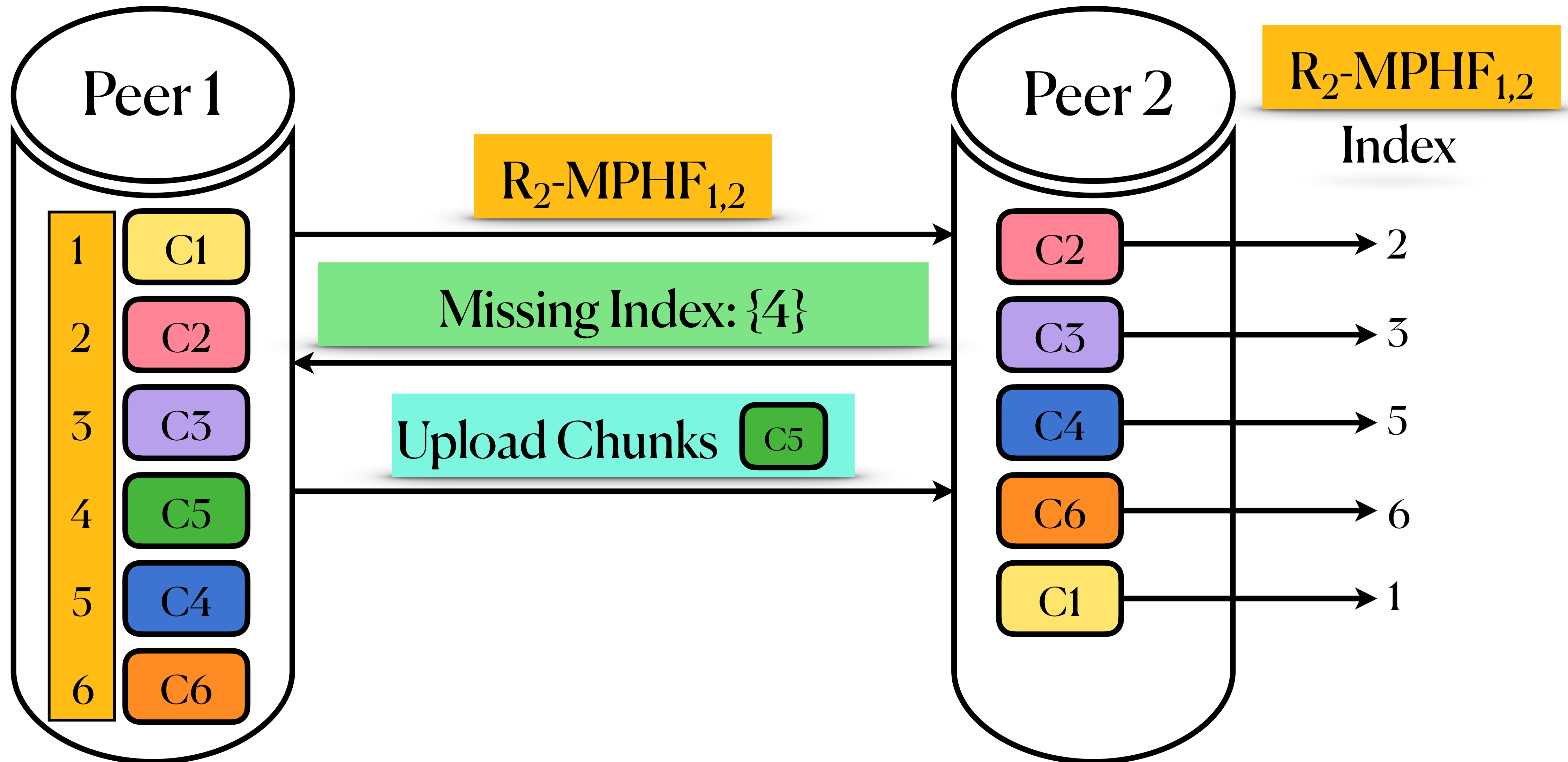
The Sync Protocol — Round 2



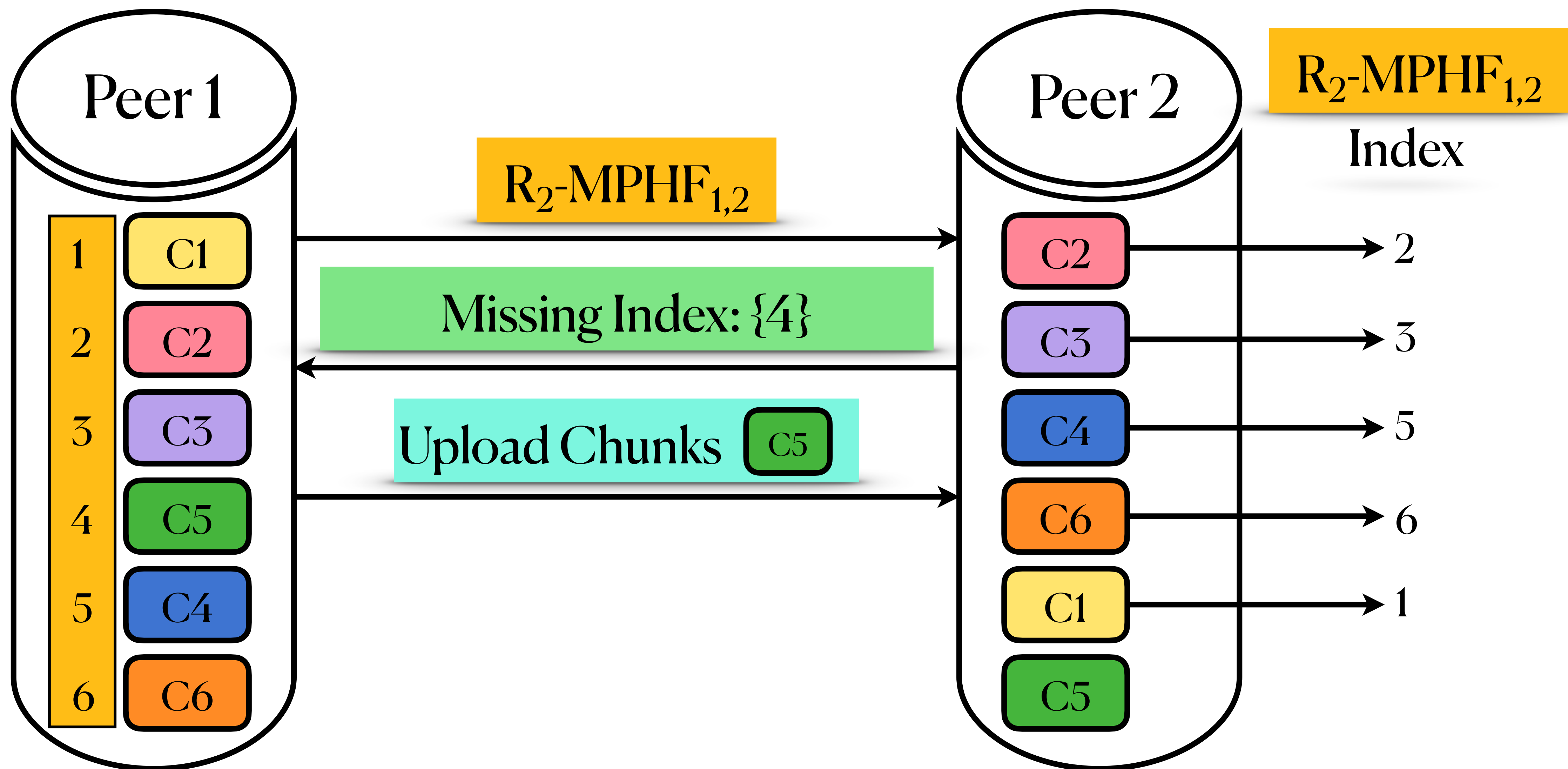
The Sync Protocol — Round 2



The Sync Protocol — Round 2



The Sync Protocol — Round 2



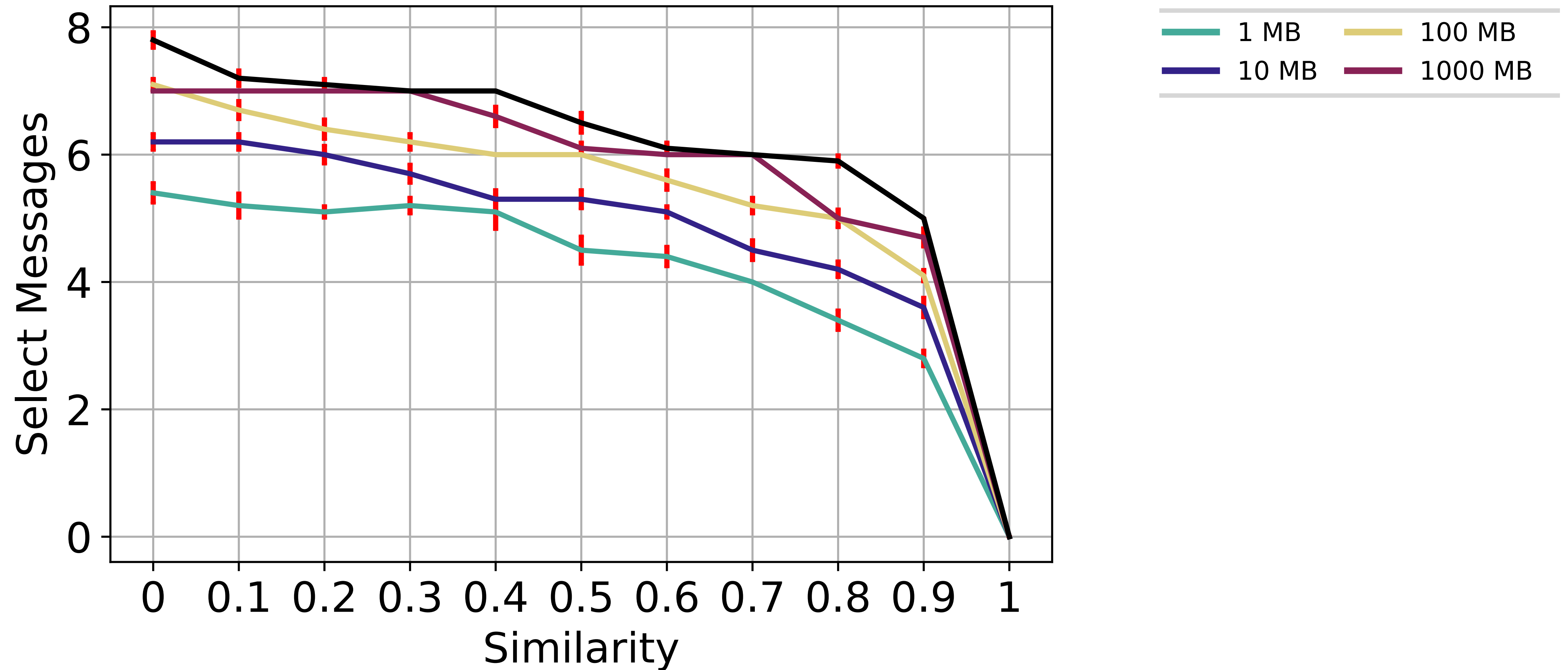
Experimental Results

Similarity Between Peer Replicas

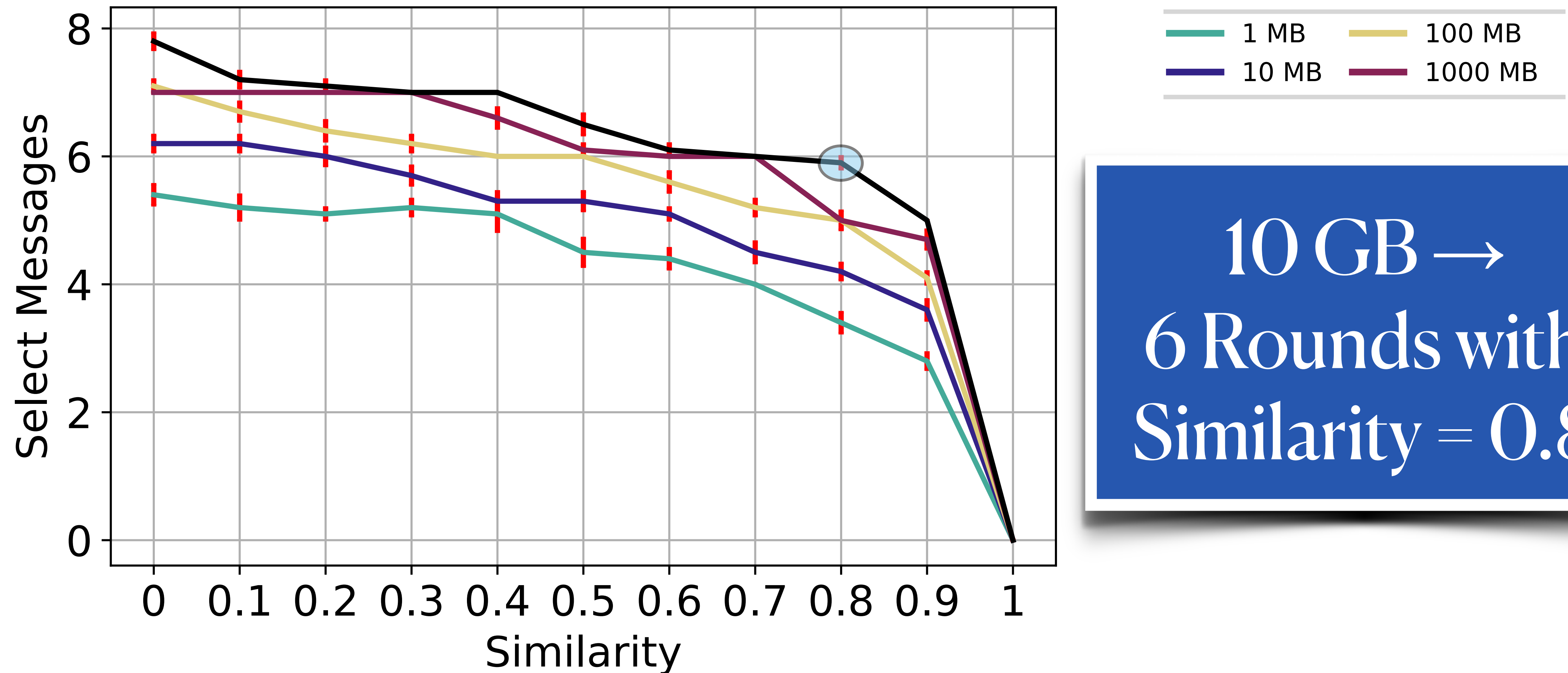
- $\text{Similarity} = |A \cap B| / \min(|A|, |B|)$

Similarity	Chunks are
0	Fully disjoint
1	Fully overlapping

Iterations Needed to Sync

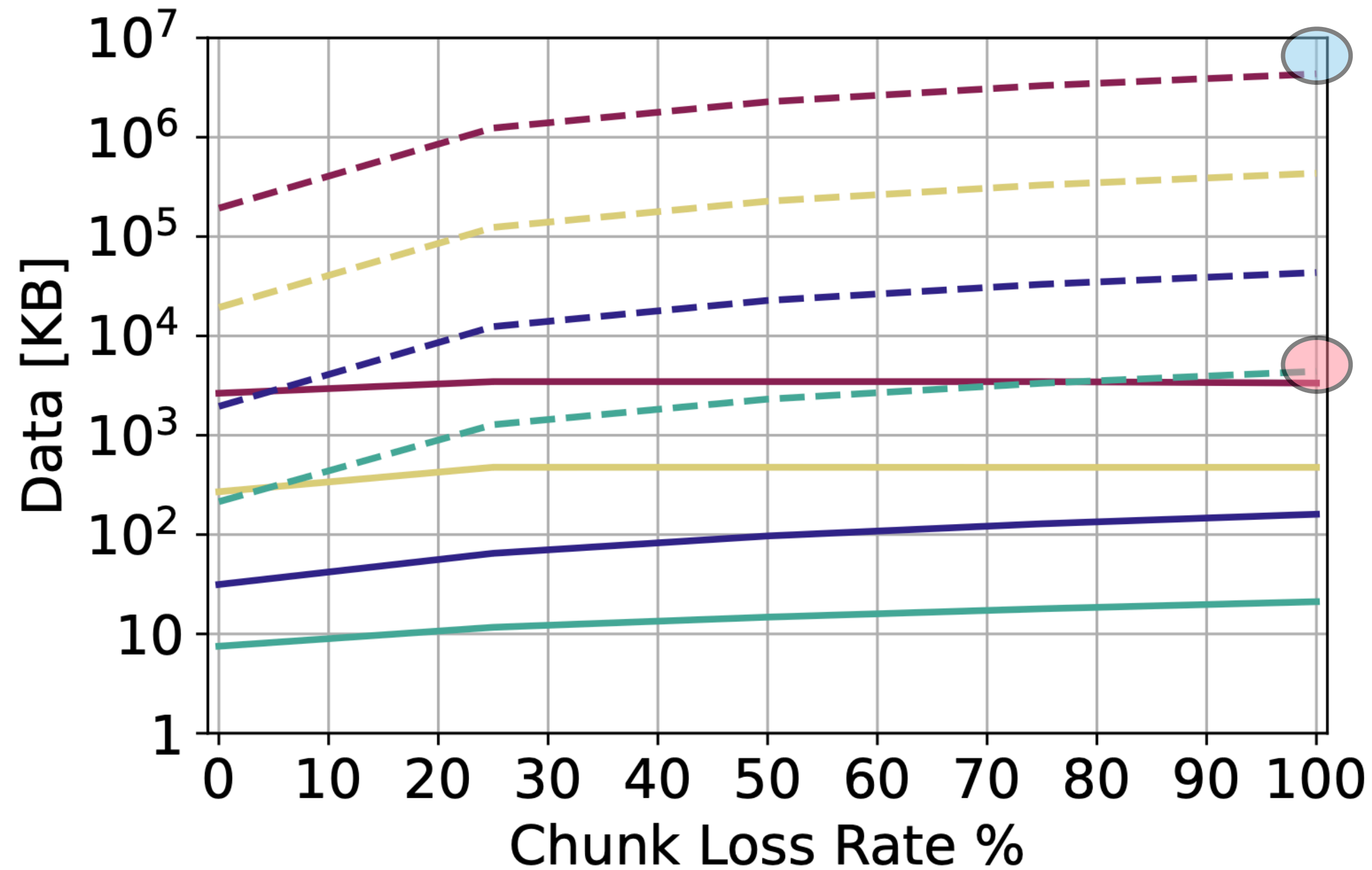


Iterations Needed for Two Peers to Sync



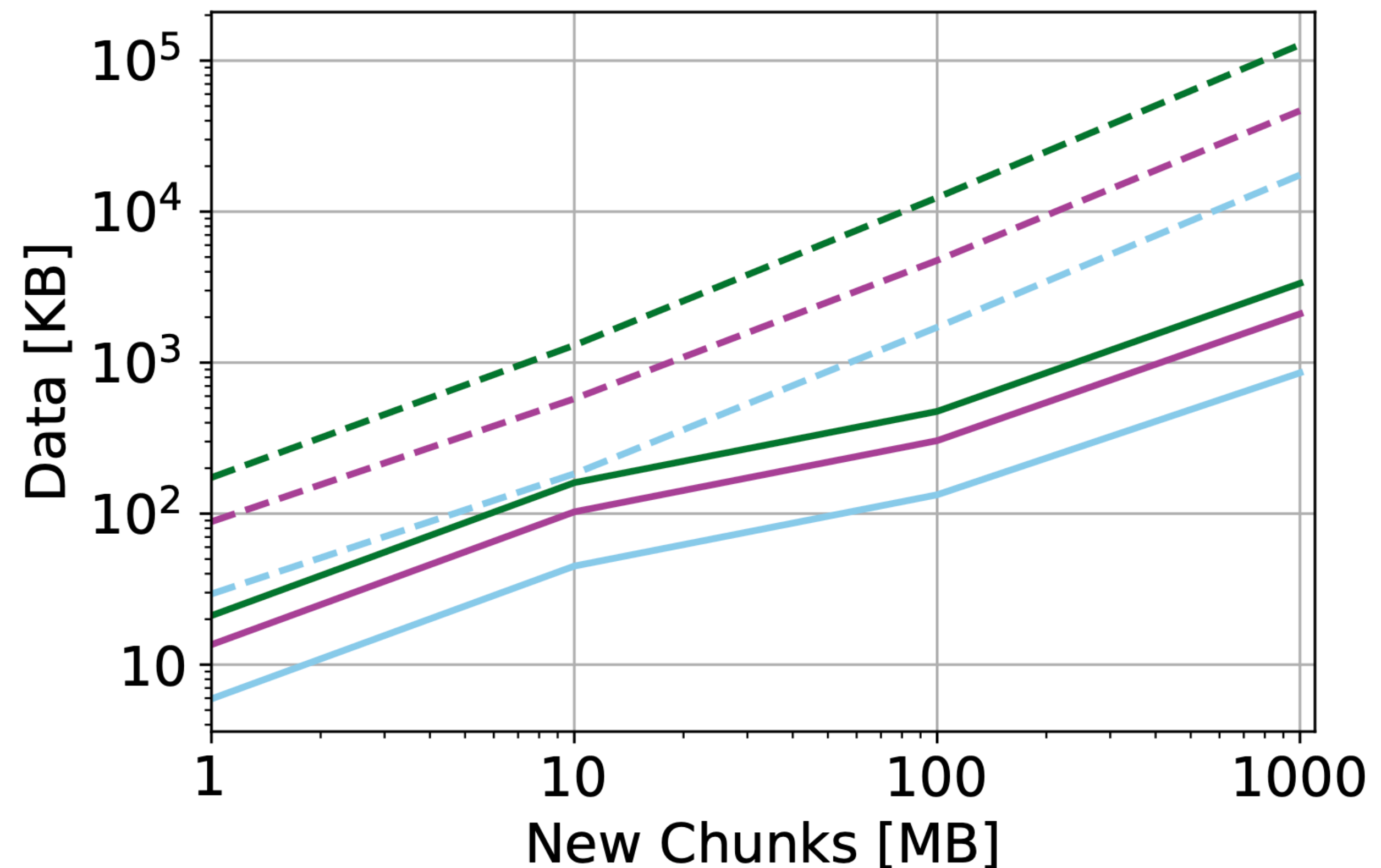
Data Transmitted to Sync

Chunk Loss Rate



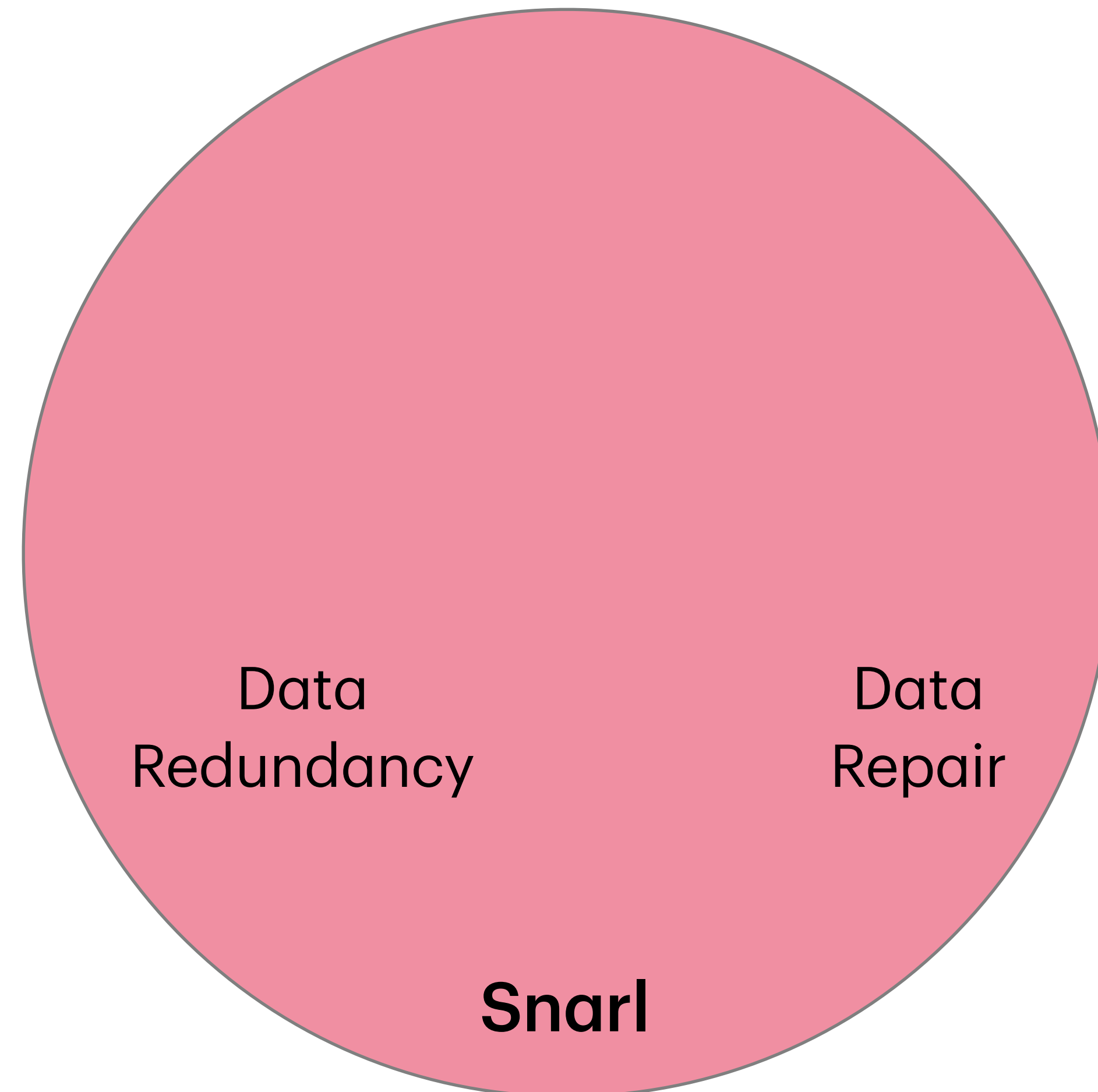
(c) Data transmitted, $n=26$

New Chunk Added

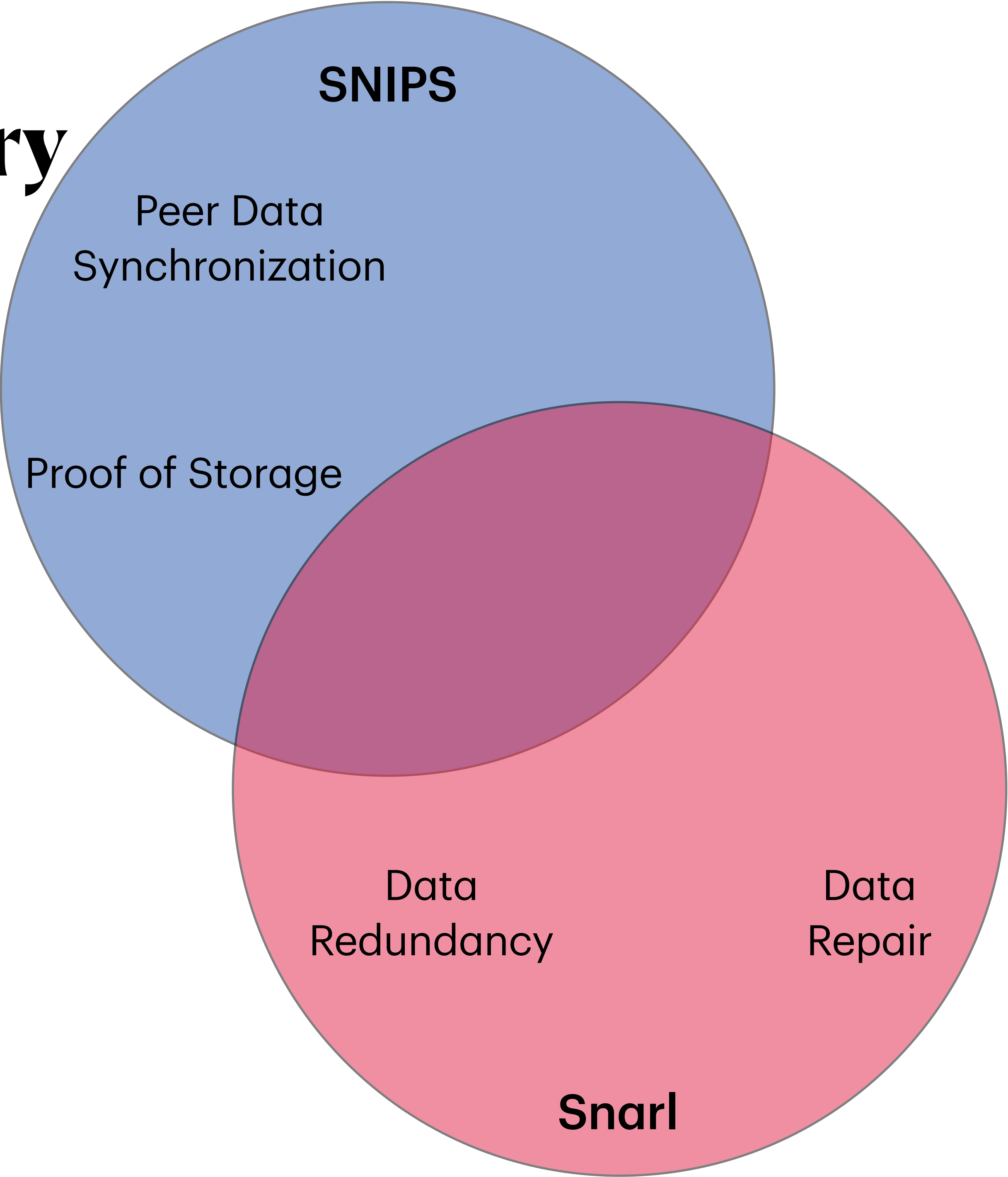


(d) Data transmitted

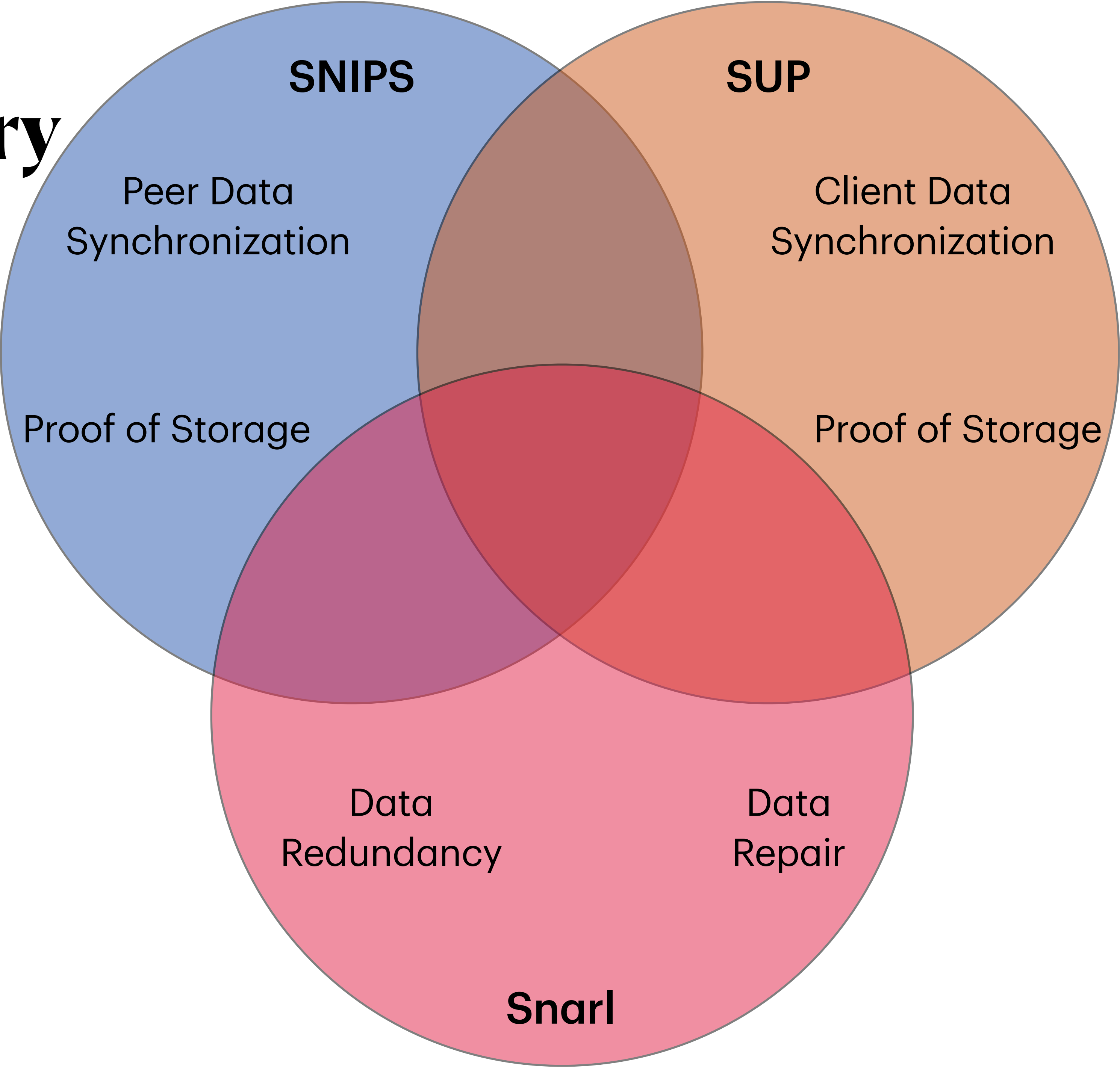
Summary



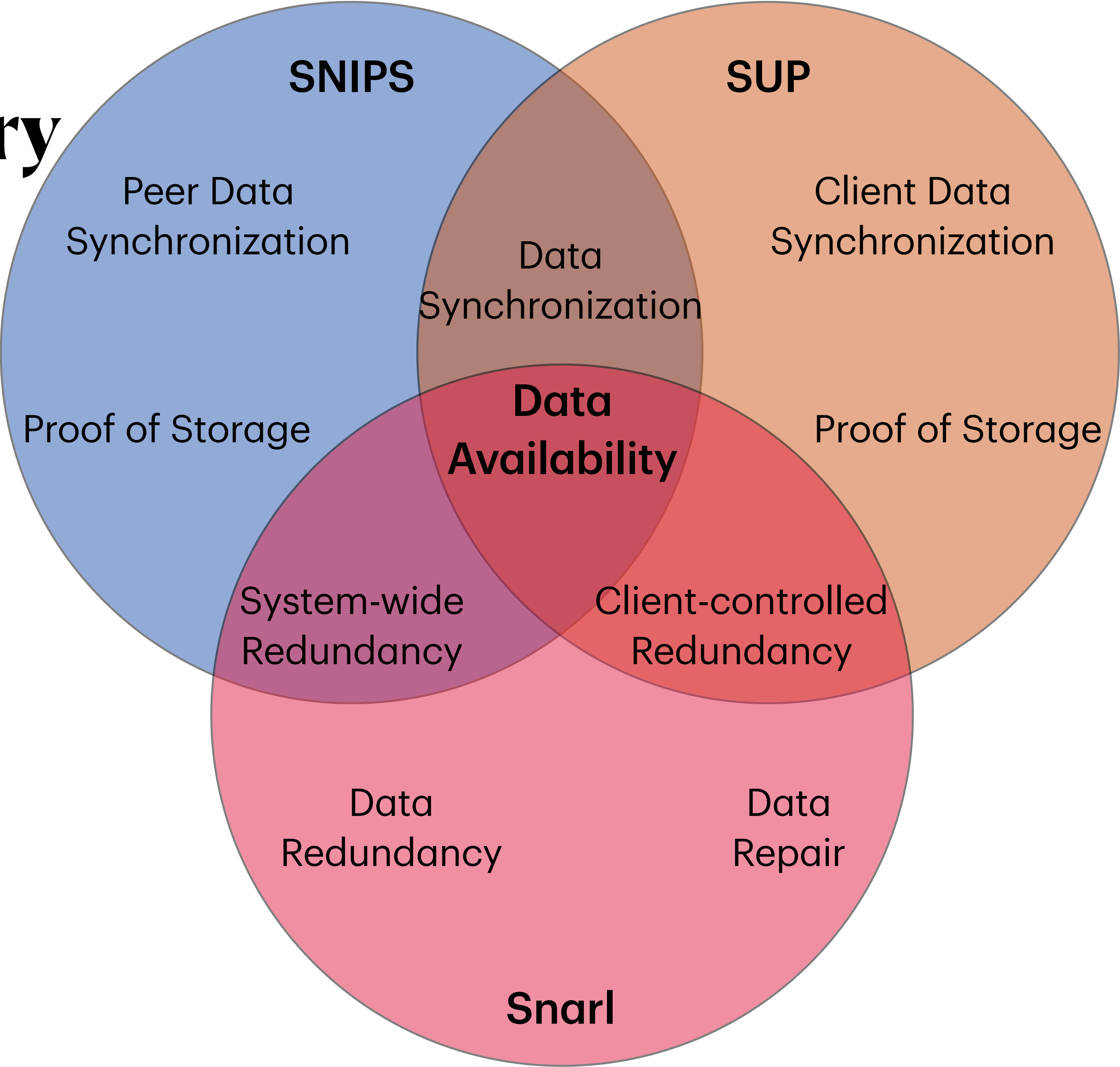
Summary



Summary



Summary



Thanks for Listening to my Talk!

Questions?

Racin Nygaard, Vero Estrada-Galiñanes, and Hein Meling. *Snarl: entangled merkle trees for improved file availability and storage utilization*. In ACM Middleware '21.

Racin Nygaard and Hein Meling. *SNIPS: Succinct Proof of Storage for Efficient Data Synchronization in Decentralized Storage Systems*. In arXiv 2304.04891. 2023.

Racin Nygaard, Hein Meling, and John Ingve Olsen. *Cost-effective Data Upkeep in Decentralized Storage Systems*. In ACM SAC '23.



Racin Nygaard



Vero Estrada-Galiñanes

Proof of Storage

Proof of Storage

- Paper title: Succinct Non-Interactive Proof of Storage
- See the paper for details!

Open Questions

- Concurrent Repair Algorithm
- What's the limit to chunk repairability?
- Scaling the Sync Protocol